达梦数据

全栈数据产品与解决方案提供商

股票代码 688692

达梦技术丛书

DM8 安全管理



前言

概述

本文档主要介绍 DM 的安全体系结构，各安全管理模块的功能、原理及相关的数据库管理员和用户如何利用这些安全管理功能对数据库或自己的数据进行安全保护。

读者对象

本文档主要适用于 DM 数据库的：

- 数据库管理员
- 开发工程师
- 测试工程师
- 技术支持工程师

通用约定

在本文档中可能出现下列标志，它们所代表的含义如下：

表 0.1 标志含义

标志	说明
 警告：	表示可能导致系统损坏、数据丢失或不可预知的结果。
 注意：	表示可能导致性能降低、服务不可用。
 小窍门：	可以帮助您解决某个问题或节省您的时间。

说明：	表示正文的附加信息，是对正文的强调和补充。

在本文档中可能出现下列格式，它们所代表的含义如下：

表 0.2 格式含义

格式	说明
HarmonyOS Sans SC Light	表示正文。
Poppins Light	表示代码或者屏幕显示内容。
粗体	表示命令行中的关键字（命令中保持不变、必须照输的部分）或者正文中强调的内容。标题、警告、注意、小窍门、说明等内容均采用粗体。
< >	语法符号中，表示一个语法对象。
::=	语法符号中，表示定义符，用来定义一个语法对象。定义符左边为语法对象，右边为相应的语法描述。
|	语法符号中，表示或者符，限定的语法选项在实际语句中只能出现一个。
{ }	语法符号中，大括号内的语法选项在实际的语句中可以出现 0…N 次(N 为大于 0 的自然数)，但是大括号本身不能出现在语句中。
[ ]	语法符号中，中括号内的语法选项在实际的语句中可以出现 0…1 次，但是中括号本身不能出现在语句中。
关键字	关键字在 DM_SQL 语言中具有特殊意义，在 SQL 语法描述中，关键字以大写形式出现。但在实际书写 SQL 语句时，关键字既可以大写也可以小写。

访问相关文档

如果您安装了 DM 数据库，可在安装目录的“\doc”子目录中找到 DM 数据库的各种手册与技术丛书。

您也可以通过访问我们的网站 www.dameng.com 阅读或下载 DM 的各种相关文档。

联系我们

如果您有任何疑问或是想了解达梦数据库的最新动态消息，请联系我们：

网址：www.dameng.com

技术服务电话：400-991-6599

技术服务邮箱：dmtech@dameng.com

目录

1 概述.....1

1.1 数据库安全管理.....1

1.2 DM 安全管理.....1

1.2.1 DM 安全管理功能系统结构.....1

1.2.2 安全管理功能简介.....2

1.3 DM 安全版本.....3

1.4 一点说明.....3

2 用户标识与鉴别.....4

2.1 DM 的管理用户.....4

2.1.1 三权分立的管理用户.....4

2.1.2 四权分立的管理用户.....7

2.2 如何创建用户.....8

2.2.1 创建用户.....9

2.2.2 口令策略.....13

2.3 用户身份验证模式.....14

2.3.1 数据库身份验证.....14

2.3.2 基于操作系统的身份验证.....14

2.3.3 LDAP 身份验证.....18

2.3.4 SSL 双因子身份验证.....19

2.3.5 KERBEROS 身份验证.....22

2.3.6 UKEY 身份验证.....22
2.3.7 RADIUS 身份验证.....22
2.4 如何修改用户信息.....24
2.5 如何删除用户.....26
3 自主访问控制.....28
3.1 权限管理.....28
3.1.1 数据库权限.....29
3.1.2 对象权限.....32
3.2 角色管理.....34
3.2.1 角色的创建与删除.....34
3.2.2 角色的启用与禁用.....41
3.2.3 SVI 角色用法.....42
3.3 权限的分配与回收.....44
3.3.1 数据库权限的分配与回收.....44
3.3.2 对象权限的分配与回收.....47
3.3.3 角色权限的分配与回收.....56
3.3.4 模式权限的分配与回收.....58
4 强制访问控制.....65
4.1 如何创建策略.....65
4.1.1 策略的组成.....65
4.1.2 创建、修改与删除策略.....66
4.1.3 为策略添加组件.....68
4.2 如何创建标记.....77

4.2.1 创建标记 78
4.2.2 修改标记 79
4.2.3 删除标记 80
4.2.4 隐式创建标记 81
4.3 如何对表应用策略 83
4.3.1 对表应用策略 83
4.3.2 取消表策略 85
4.4 如何对用户应用策略 86
4.4.1 设置用户等级 86
4.4.2 设置用户范围 88
4.4.3 设置用户组 89
4.4.4 清除用户策略 90
4.5 如何对会话应用策略 91
4.5.1 设置会话默认标记 91
4.5.2 设置会话行标记 92
4.5.3 清除会话标记 93
4.5.4 保存会话标记 93
4.6 读写控制规则 94
4.6.1 读访问规则 94
4.6.2 写访问规则 95
4.6.3 特权 97
4.7 扩展客体标记 98
4.7.1 对客体应用标记 98

4.7.2 修改客体标记 99
4.7.3 删除客体标记 100
4.8 一个强制访问控制的例子 101
4.9 相关数据字典表 106
5 审计 109
5.1 审计开关 109
5.2 审计的设置与取消 110
5.2.1 系统级审计 111
5.2.2 语句级审计 112
5.2.3 对象级审计 118
5.2.4 语句序列审计 122
5.2.5 关于审计设置的一些说明 123
5.3 审计文件管理 124
5.4 审计信息查阅 127
5.5 审计实时侵害检测 133
5.5.1 创建与删除实时侵害检测规则 134
5.5.2 实时侵害检测 136
5.5.3 审计告警工具 dmamon 137
5.6 审计分析 141
5.6.1 审计分析工具 Analyzer 142
5.6.2 审计分析工具 dmaudtool 145
5.7 集群环境审计说明 152
6 通信加密 154

6.1 基于传输层的 SSL 协议加密 154
6.1.1 SSL 模式说明 154
6.1.2 SSL 环境配置 156
6.1.3 SSL 证书配置 156
6.1.4 服务器配置 158
6.1.5 客户端连接 158
6.1.6 举例说明 159
6.1.7 常见问题 161
6.2 基于应用层的消息包加密 161
7 存储加密 163
7.1 透明加密 163
7.1.1 全库加密 164
7.1.2 表空间透明加密 165
7.1.3 表列透明加密 166
7.1.4 其他数据库对象加密 168
7.1.5 重做日志文件加密 168
7.2 半透明加密 169
7.3 非透明加密 175
7.4 加密算法和散列算法 201
8 加密引擎 206
8.1 编程接口介绍 208
8.1.1 算法信息相关接口 208
8.1.2 加密过程相关接口 213

8.1.3 解密过程相关接口 217
8.1.4 散列过程相关接口 220
8.1.5 其他可选相关接口 224
8.2 接口库文件使用说明 235
8.3 UKEY 使用说明 236
8.3.1 使用介绍 236
8.3.2 示例 238
8.4 DM 提供的第三方加密和散列算法 239
8.4.1 OPENSSL 软件库提供 239
8.4.2 渔翁硬件加密卡提供 241
8.5 编程实例 243
8.5.1 加密引擎接口 243
8.5.2 密码机引擎接口 276
9 资源限制 292
10 客体重用 297
11 登录用户名密码增强加密 298
11.1 DMKEY 工具的使用 298
11.1.1 启动 dmkey 298
11.1.2 查看 dmkey 参数 299
11.2 服务器端配置 300
11.3 客户端配置 301
11.3.1 dm_svc.conf 301
11.3.2 DPI 301

11.4 启动数据库服务器 301
11.5 应用实例 301
12 登录用户名密码外部存储 303
12.1 DMMKSTORE 工具的使用 304
12.1.1 启动 dmmkstore 304
12.1.2 查看 dmmkstore 参数 305
12.1.3 dmmkstore 参数详解 306
12.2 DM_SVC.CONF 文件配置 314
12.3 应用实例 314
12.4 DMMKSTORE 错误码汇编 316
附录 1 "三权分立"预设角色权限列表 318
附录 2 "四权分立"预设角色权限列表 335
DM 技术支持 351
手册版本 353

1 概述

1.1 数据库安全管理

数据库安全管理是指采取各种安全措施对数据库及其相关文件和数据进行保护。数据库系统的重要指标之一是确保系统安全，以各种防范措施防止非授权使用数据库，主要通过数据库管理系统进行实现。数据库系统中一般采用用户标识与鉴别、存取控制以及密码存储等技术进行安全控制。

数据库安全的核心和关键是其数据安全。数据安全指以保护措施确保数据的完整性、保密性、可用性、可控性和可审查性。由于数据库存储着大量的重要信息和机密数据，而且在数据库系统中大量数据集中存放，供多用户共享，因此，必须加强对数据库访问的控制和数据安全防护。

1.2 DM 安全管理

1.2.1 DM 安全管理功能系统结构

DM 的安全管理就是为保护存储在 DM 数据库中的各类敏感数据的机密性、完整性和可用性提供必要的技术手段，防止对这些数据的非授权泄露、修改和破坏，并保证被授权用户能按其授权范围访问所需要的数据。

DM 作为安全数据库，提供了包括用户标识与鉴别、自主与强制访问控制、通信与存储加密、审计等丰富的安全功能，且各安全功能都可进行配置，满足各类型用户在安全管理方面不同层次的需求。

DM 的安全管理功能体系结构如图 1.1 所示。

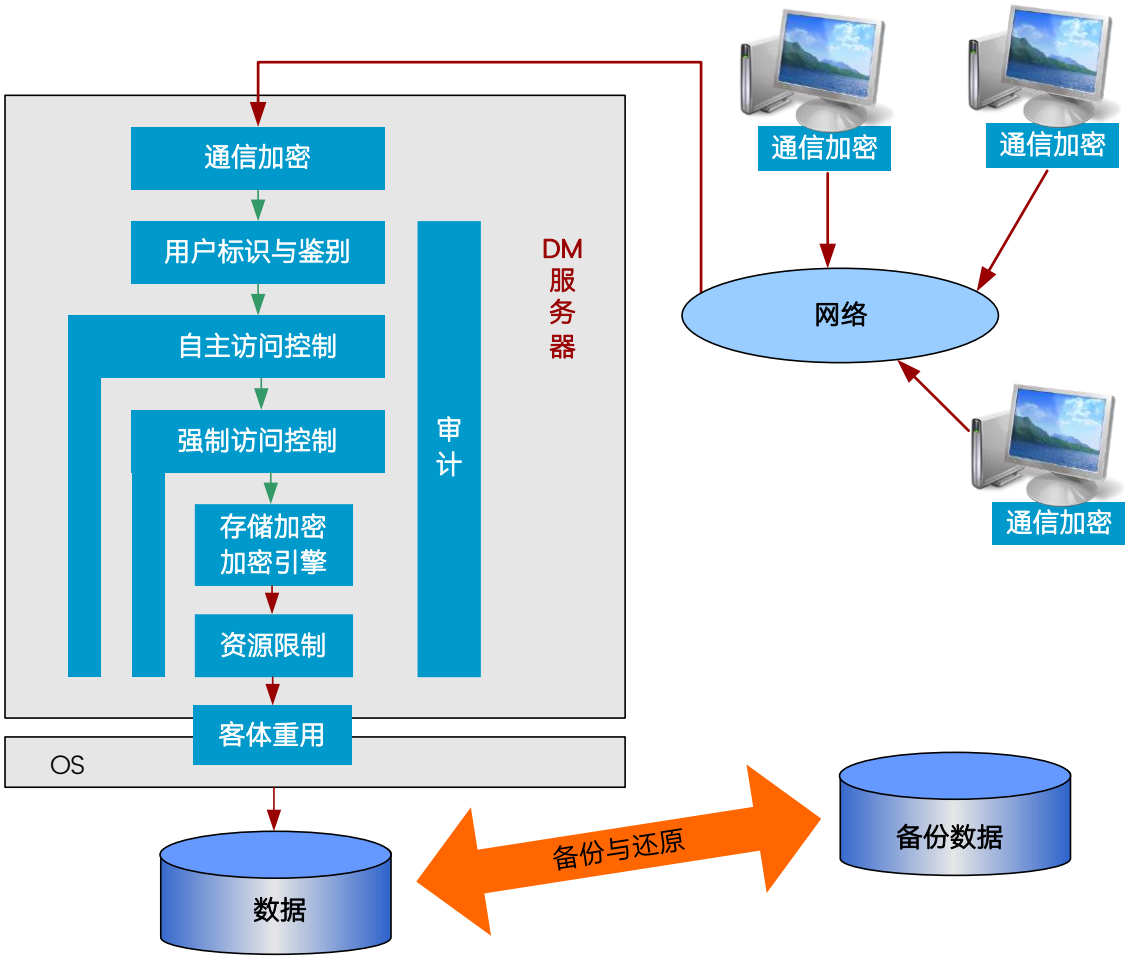


图 1.1 DM 安全功能体系结构

1.2.2 安全管理功能简介

从图 1.1 可以看出 DM 的安全管理功能相当丰富与完善，各安全功能的简介见表 1.1。

表 1.1 DM 安全管理功能简介

安全功能	简介
用户标识与鉴别	可以通过登录帐户区别各用户，并通过口令方式防止用户被冒充
自主访问控制	通过权限管理，使用户只能访问自己权限内的数据对象
强制访问控制	通过安全标记，使用户只能访问与自己安全级别相符的数据对象
审计	审计人员可以查看所有用户的操作记录，为明确事故责任提供证据支持

通信、存储加密	用户可以自主的将数据以密文的形式存储在数据库中。也可以对在网络上传输的数据进行加密
加密引擎	用户可以用自定义的加密算法来加密自己的核心数据
资源限制	可以对网络资源和磁盘资源进行配额设置，防止恶意资源抢占
客体重用	实现了内存与磁盘空间的释放清理，防止信息数据的泄露

1.3 DM 安全版本

DM 数据库版本有标准版、企业版和安全版的区别，不同版本提供的功能有所不同，且需要与版本匹配的 License 才能正常提供服务。标准版和企业版在安全功能上没有区别，因此在本文中，将标准版和企业版统称为普通版本，以与安全版本进行区分。

普通版本提供的安全管理功能有：用户标识与鉴别、自主访问控制、审计、通信加密、存储加密、加密引擎、资源限制与客体重用；安全版本在普通版本的基础上，还提供“四权分立”安全机制和强制访问控制安全管理功能，能满足对安全管理有更高需求的数据库用户的要求。

另外，在普通版本提供的安全功能中，有一些具体的更强的安全功能细节也仅在安全版本提供，将在后面具体介绍的地方一一指出。

1.4 一点说明

本书中的示例，除了具体给出建表语句的除外，其余使用的表均为 DM 示例库中的表。DM 示例库可在安装 DM 数据库时选择安装，其具体说明可参考《DM8_SQL 语言使用手册》。

2 用户标识与鉴别

用户标识与鉴别对试图登录数据库进行数据访问的用户进行身份验证，以确认此用户是否能与某一数据库用户进行关联，并根据关联的数据库用户的权限对此用户在数据库中的数据访问活动进行安全控制。

2.1 DM 的管理用户

在现实生活中，任何一个系统如果将所有的权利都赋予给某一个人，而不加以监督和控制，势必会产生权利滥用的风险。从数据库安全角度出发，一个大型的数据库系统有必要将数据库系统的权限分配给不同的角色来管理，并且各自偏重于不同的工作职责，使之能够互相限制和监督，从而有效保证系统的整体安全。

DM 数据库采用三权分立或四权分立的安全机制，将系统中所有的权限按照类型进行划分，为每个管理员分配相应的权限，管理员之间的权限相互制约又相互协助，从而使整个系统具有较高的安全性和较强的灵活性。

可在创建 DM 数据库时通过建库参数 PRIV_FLAG 设置使用三权分立或四权分立。0 表示三权分立，1 表示四权分立。此参数仅在 DM 安全版本下提供，即仅 DM 安全版本提供四权分立，缺省采用三权分立。

2.1.1 三权分立的管理用户

使用三权分立安全机制时，管理用户分为数据库管理员、数据库安全员和数据库审计员三种类型。

在安装过程中，DM 数据库会预设数据库管理员账号 SYSDBA、数据库安全员账号 SYSSSO、数据库审计员账号 SYSAUDITOR。用户需要基于 DM 默认口令策略自行设置各个管理用户的口令，关于 DM 默认口令策略的介绍请参考 2.2.2 口令策略。

此外，需要说明的是，在 DM 数据库中有一个特殊的预定义用户“SYS”，此用户仅用于保存系统内部对象，并非数据库管理员，也无法进行登录。

2.1.1.1 数据库管理员 DBA

每个数据库至少需要一个 DBA 来管理，DBA 可能是一个团队，也可能是一个人。在不同的数据库系统中，数据库管理员的职责可能也会有比较大的区别，总体而言，数据库管理员的职责主要包括以下任务:

- 评估数据库服务器所需的软、硬件运行环境
- 安装和升级 DM 服务器
- 数据库结构设计
- 监控和优化数据库的性能
- 计划和实施备份与故障恢复

2.1.1.2 数据库安全员 SSO

有些应用对于安全性有着很高的要求，传统的由 SYSDBA 一人拥有所有权限并且承担所有职责的安全机制可能无法满足企业实际需要，此时数据库安全员和数据库审计员两类管理用户就显得异常重要，它们对于限制和监控数据库管理员的所有行为都起着至关重要的作用。

数据库安全员的主要职责是制定并应用安全策略，强化系统安全机制。数据库安全员 SYSSSO 是 DM 数据库初始化的时候就已经创建好的，可以以该用户登录到 DM 数据库来创建新的数据库安全员。

SYSSSO 或者新的数据库安全员都可以制定自己的安全策略，在安全策略中定义安全级别、范围和组，然后基于定义的安全级别、范围和组来创建安全标记，并将安全标记分别应用到主体（用户）和客体（各种数据库对象，如表、索引等），以便启用强制访问控制功能。

数据库安全员不能对用户数据进行增、删、改、查，也不能执行普通的 DDL 操作如创建表、视图等。他们只负责制定安全机制，将合适的安全标记应用到主体和客体，通过这种方式可以有效的对 SYSDBA 的权限进行限制，SYSDBA 此后就不能直接访问添加有安全标记的数据，除非安全员给 SYSDBA 也设定了与之匹配的安全标记，SYSDBA 的权限受到了有效的约束。数据库安全员也可以创建和删除新的安全用户，向这些用户授予和回收安全相关的权限，具体参见 3.3 权限的分配与回收。

2.1.1.3 数据库审计员 AUDITOR

我们可以想象一下，某个企业内部 SYSDBA 非常熟悉公司内部 ERP 系统的数据库设计，该系统包括了员工工资表，里面记录了所有员工的工资，公司的出纳通过查询系统内部员工工资表来发放工资。传统的 SYSDBA 集所有权利于一身，可以很容易修改工资表，从而导致公司工资账务错乱。为了预防该问题，可以采用前面数据库安全员制定安全策略的方法，避免 SYSDBA 或者其他数据库用户具有访问该表的权限。为了能够及时找到 SYSDBA 或者其他用户的非法操作，在 DM 数据库中还可以在系统建设初期，由数据库审计员（SYSAUDITOR 或者其他由 SYSAUDITOR 创建的审计员）来设置审计策略（包括审计对象和操作），在需要时，数据库审计员可以查看审计记录，及时分析并查找出幕后真凶。

从上面的介绍中我们也可以看出，在 DM 数据库中，审计员的主要职责就是创建和删除数据库审计员，设置/取消对数据库对象和操作的审计设置，查看和分析审计记录等。

2.1.2 四权分立的管理用户

使用四权分立安全机制时，管理用户分为数据库管理员、数据库安全员、数据库审计员和数据库对象操作员四种类型。

在安装过程中，DM 数据库会预设数据库管理员账号 SYSDBA、数据库安全员账号 SYSSSO、数据库审计员账号 SYSAUDITOR，数据库对象操作员 SYSDBO。用户需要基于 DM 默认口令策略自行设置各个管理用户的口令，关于 DM 默认口令策略的介绍请参考 2.2.2 口令策略。

此外，需要说明的是，在 DM 数据库中有一个特殊的预定义用户“SYS”，此用户仅用于保存系统内部对象，并非数据库管理员，也无法进行登录。

2.1.2.1 数据库管理员 DBA

三权分立时数据库管理员既可进行系统管理和维护工作，也可对数据内容进行增删查改操作。四权分立的安全机制在三权分立基础上调整了自主访问控制权限。四权分立时数据库管理员只能进行系统管理和维护工作，不能对数据内容进行增删查改操作。数据库操作员可进行增删改查操作，但不能管理和维护系统。

四权分立数据库管理员只具有三权分立数据库管理员的一部分权限。与三权分立的数据库管理员相比，四权分立的数据库管理员的权限变化如下：

1．取消了权限较大的 ANY 特权。ANY 特权是指含有 ANY 关键字的数据库权限，如 CREATE ANY TABLE、SELECT ANY TABLE 等。

2．取消了数据库对象的创建、增删查改、对象权限的管理功能。

3.只保留了与管理与维护相关的且明确的数据库权限。例如：数据库创建、备份、还原和校验等。

2.1.2.2 数据库对象操作员 DBO

可以创建数据库对象，并对自己拥有的数据库对象（表、视图、存储过程、序列、包、外部链接等）具有所有的对象权限并可以授出与回收，但其无法管理与维护数据库对象。

2.1.2.3 数据库安全员 SSO

和三权分立的数据库安全员用法一致。

2.1.2.4 数据库审计员 AUDITOR

和三权分立的数据库审计员用法一致。

2.2 如何创建用户

数据库系统在运行的过程中，往往需要根据实际需求创建用户，然后为用户指定适当的权限。创建用户的操作一般只能由系统预设用户 SYSDBA、SYSSSO、SYSAUDITOR 和 SYSDBO 完成，如果普通用户需要创建用户，必须具有 CREATE USER 的数据库权限。

使用 MANAGER 界面向导创建用户同时具备 PUBLIC 角色（PUBLIC 角色包含 SVI 角色）和 SOI 角色；使用 MANAGER 或 DIsql 执行 CREATE USER 语句创建用户只具备 PUBLIC 角色（PUBLIC 角色包含 SVI 角色），无 SOI 角色。

2.2.1 创建用户

创建用户的命令是 CREATE USER，创建用户所涉及的内容包括为用户指定用户名、认证模式、口令、口令策略、只读属性以及资源限制等。其中用户名是代表用户账号的标识符，长度为 1~128 个字符。用户名可以用双引号括起来，也可以不用，但如果用户名以数字开头，必须用双引号括起来。

在 DM 中使用 CREATE USER 语句创建用户，具体的语法格式如下：

```
CREATE USER [IF NOT EXISTS] <用户名> IDENTIFIED <身份验证模式> [PASSWORD_POLICY <口令策略>][<锁定子句>][<存储加密密钥>][<只读标志>][<资源限制子句>][<密码过期子句>][<允许 IP 子句>][<禁止 IP 子句>][<允许时间子句>][<禁止时间子句>][<TABLESPACE 子句>][<INDEX_TABLESPACE 子句>][<表空间配额子句>];
<身份验证模式> ::=
    <数据库身份验证模式>|
    <外部身份验证模式>
<数据库身份验证模式> ::= BY <口令> [<散列选项>]
<散列选项> ::= HASH WITH [<密码引擎名>.]<散列算法> [<加盐选项>]
<加盐选项> ::= [NO] SALT
<外部身份验证模式> ::=
    EXTERNALLY |
    EXTERNALLY AS <用户 DN>|
    EXTERNALLY RADIUS
<锁定子句> ::=
    ACCOUNT LOCK |
    ACCOUNT UNLOCK
```

<存储加密密钥> ::= ENCRYPT BY <口令>

<只读标志> ::= [NOT] READ ONLY

<资源限制子句> ::=

DROP PROFILE |

PROFILE <profile 名> |

LIMIT <资源设置>

<资源设置> ::=

<资源设置项>{,<资源设置项>} |

<资源设置项>{ <资源设置项>}

<资源设置项> ::=

SESSION_PER_USER <参数设置> |

CONNECT_IDLE_TIME <参数设置> |

CONNECT_TIME <参数设置> |

CPU_PER_CALL <参数设置> |

CPU_PER_SESSION <参数设置> |

MEM_SPACE <参数设置> |

READ_PER_CALL <参数设置> |

READ_PER_SESSION <参数设置> |

FAILED_LOGIN_ATTEMPTS <参数设置> |

PASSWORD_LIFE_TIME <参数设置> |

PASSWORD_REUSE_TIME <参数设置> |

PASSWORD_REUSE_MAX <参数设置> |

PASSWORD_LOCK_TIME <参数设置> |

PASSWORD_GRACE_TIME <参数设置>|

INACTIVE_ACCOUNT_TIME<参数设置>

<参数设置> ::=

<参数值>|

UNLIMITED|

DEFAULT

<密码过期子句> ::= PASSWORD EXPIRE

<允许 IP 子句> ::=

ALLOW_IP NULL |

ALLOW_IP <IP 项>{,<IP 项>} |

ALLOW_IP ADD <IP 项>{,<IP 项>} |

ALLOW_IP DROP <IP 项>{,<IP 项>}

<禁止 IP 子句> ::=

NOT_ALLOW_IP NULL |

NOT_ALLOW_IP <IP 项>{,<IP 项>} |

NOT_ALLOW_IP ADD <IP 项>{,<IP 项>} |

NOT_ALLOW_IP DROP <IP 项>{,<IP 项>}

<IP 项> ::=

<具体 IP>|

<网段>

<允许时间子句> ::= ALLOW_DATETIME <时间项>{,<时间项>}

<禁止时间子句> ::= NOT_ALLOW_DATETIME <时间项>{,<时间项>}

<时间项> ::=

<具体时间段> |

<规则时间段>

<具体时间段> ::= <具体日期> <具体时间> TO <具体日期> <具体时间>

<规则时间段> ::= <规则时间标志> <具体时间> TO <规则时间标志> <具体时间>

<规则时间标志> ::=

MON |

TUE |

WED |

THURS |

FRI |

SAT |

SUN

<TABLESPACE 子句> ::= DEFAULT TABLESPACE <表空间名>

<INDEX_TABLESPACE 子句> ::= DEFAULT INDEX TABLESPACE <表空间名>

<表空间配额子句> ::=

QUOTA UNLIMITED |

QUOTA 0 |

QUOTA <配额大小> ON <表空间名>{ QUOTA <配额大小> ON <表空间名>}

<配额大小> ::=

UNLIMITED |

<空间大小>[K|M|G|T]



注意：

外部身份验证仅在 DM 安全版本中才提供支持。

资源限制子句将在第 9 章进行详细介绍。

例 创建用户名为 BOOKSHOP_USER、口令为 BSpsd_123、会话超时为 3 分钟的用户。

```
CREATE USER BOOKSHOP_USER IDENTIFIED BY "BSpsd_123" LIMIT CONNECT_TIME 3;
```

2.2.2 口令策略

创建用户语句中的 PASSWORD_POLICY 子句用来指定该用户的口令策略，口令策略可以设置为 15 或 31，含义如下:

- 15 口令禁止与用户名相同，口令长度需大于等于 INI 参数 PWD_MIN_LEN 设置的值，口令最长为 48 字节，至少包含一个大写字母（A-Z）和一个小写字母（a-z），至少包含一个数字（0-9）
- 31 口令禁止与用户名相同，口令长度需大于等于 INI 参数 PWD_MIN_LEN 设置的值，口令最长为 48 字节，至少包含一个大写字母（A-Z）和一个小写字母（a-z），至少包含一个数字（0-9），至少包含一个标点符号（英文输入法状态下，除“和空格外的所有符号）

除了在创建用户语句中指定该用户的口令策略，DM 的 INI 参数 PWD_POLICY 可以指定系统的默认口令策略，其参数值的设置规则与 PASSWORD_POLICY <口令策略>子句一致，缺省值为 15。若在创建用户时没有使用 PASSWORD_POLICY <口令策略>子句指定用户的口令策略，则使用系统的默认口令策略。

系统管理员可通过查询 V$PARAMETER 动态视图查询 PWD_POLICY 的当前值。

```
SELECT * FROM V$PARAMETER WHERE NAME= 'PWD_POLICY';
```

PWD_POLICY 参数是比较特殊，属于不包含在 DM.INI 中的 INI 配置项，因此不能使用 Console 工具进行修改。DBA 用户可以使用系统过程 SP_SET_PARA_VALUE 来配置 PWD_POLICY 参数值，关于 SP_SET_PARA_VALUE 的详细介绍请参考《DM8_SQL 语言使用手册》。

例如，将 PWD_POLICY 置为 31，同时修改文件和内存参数，由于 PWD_POLICY 为动态 INI 参数，这样设置后新的参数值可以立即生效。

```
SP_SET_PARA_VALUE(1, 'PWD_POLICY',31);
```


说明：

DM.INI 文件中包含的参数都可以使用 Console 工具或调用系统过程 SP_SET_PARA_VALUE/SP_SET_PARA_DOUBLE_VALUE/ SP_SET_PARA_STRING_VALUE 方法来进行设置。

因为 Console 为脱机工具，对参数值的修改通过修改 dm.ini 文件中的参数值来进行，无论参数类型是静态还是动态，都需要重启 DM 服务器才能使新设置的参数值生效。

2.3 用户身份验证模式

DM 提供多种用户身份验证模式来保护对数据库访问的安全，包括数据库身份验证、基于操作系统的身份验证、LDAP 身份验证、SSL 双因子身份验证、KERBEROS 身份验证、UKEY 身份验证和 RADIUS 身份验证。

本节将对上述身份验证模式进行详细介绍。

2.3.1 数据库身份验证

数据库身份验证需要利用数据库口令进行身份验证，即在创建或修改用户时指定用户口令，用户在登录时输入对应口令进行身份验证。

数据库身份验证是最常用的一种身份验证方式，具体可以参考《DM8_DIsql 使用手册》中有关登录方式的介绍。

2.3.2 基于操作系统的身份验证

基于操作系统的身份验证支持本机验证和远程验证，下面将分别展开介绍。

2.3.2.1 本机验证

本机验证需要将 DM 配置文件 dm.ini 的 ENABLE_LOCAL_OSAUTH 参数设置为 1，表示支持本机验证。该参数为静态参数，数据库管理员可以使用系统过程 SP_SET_PARA_VALUE 进行修改，但修改后需要重新启动 DM 服务器才能生效。

本机验证需要首先将操作系统用户加入到操作系统的 dmdba|dmsso|dmauditor 用户组，分别对应数据库的 SYSDBA|SYSSSO|SYSAUDITOR 用户。也可以将操作系统用户加入到操作系统的 dmusers 用户组，对应数据库的同名用户，即此时数据库中需要存在一个与操作系统用户同名的用户。

举例说明：

例 1 操作系统用户 dameng 以普通用户身份登录数据库。

1. 在数据库中创建一个操作系统的同名用户，并修改 INI 参数。

```
SQL> CREATE USER dameng IDENTIFIED BY "Dameng12345";
SQL> SP_SET_PARA_VALUE(2,'ENABLE_LOCAL_OSAUTH',1);
SQL> COMMIT;
```

重新启动数据库服务器，使 INI 参数生效。

2. 在操作系统中创建 dmusers 用户组，并将操作系统用户 dameng 添加到 dmusers 用户组。

Windows 操作系统下，以管理员身份运行命令提示符工具并执行以下命令：

```
C:\Windows\system32>net localgroup dmusers /add
C:\Windows\system32>net localgroup dmusers dameng /add
```

Linux 操作系统下：

```
[dameng@test163 ~]$ sudo groupadd dmusers
[dameng@test163 ~]$ sudo usermod -G dmusers dameng
```

3. 操作系统用户 dameng 使用本地操作系统身份验证的方式登录数据库。

以 Windows 操作系统为例：

```
D:\dmdbms\bin>disql.exe /@localhost:5236 as users
```

例 2 操作系统用户 dameng_dba 以数据库管理员 SYSDBA 身份登录数据库。

1. 数据库首先修改 INI 参数。

```
SQL> SP_SET_PARA_VALUE(2,'ENABLE_LOCAL_OSAUTH',1);
SQL> COMMIT;
```

重新启动数据库服务器，使 INI 参数生效。

2. 在操作系统中创建 dmdba 用户组，并将 dameng_dba 添加到 dmdba 用户组。

Windows 操作系统下，以管理员身份运行命令提示符工具并执行以下命令：

```
C:\Windows\system32>net localgroup dmdba /add
C:\Windows\system32>net localgroup dmdba dameng_dba /add
```

Linux 操作系统下：

```
[dameng@test163 ~]$ sudo groupadd dmdba
[dameng@test163 ~]$ sudo usermod -G dmdba dameng_dba
```

3. 操作系统用户 dameng_dba 以 SYSDBA 身份登录数据库。

以 Windows 操作系统为例：

```
D:\dmdbms\bin>disql.exe /@localhost:5236 as sysdba
```

2.3.2.2 远程验证

远程验证需要借助 SSL 数字证书来完成，用户需要拥有自己的 SSL 数字证书，并为 DM 服务器以及 DM 客户端或接口均配置好 SSL 环境。

远程验证需要将 DM 配置文件 dm.ini 的 ENABLE_REMOTE_OSAUTH 参数设置为 1，表示支持远程验证。同时需要将 ENABLE_ENCRYPT 参数设置为 1，表示开启 SSL 通信加密。关于 SSL 通信加密的详细介绍请参考 6.1 基于传输层的 SSL 协议加密。

上述参数均为静态参数，数据库管理员可以使用系统过程 SP_SET_PARA_VALUE 进行修改，但修改后需要重新启动 DM 服务器才能生效。由于开启了 SSL 通信加密，因此重启 DM 服务器之前必须保证当前服务器 SSL 环境配置完成，否则服务器无法正常启动。

远程验证创建用户的语法如下：

```
CREATE USER [IF NOT EXISTS] <用户名> IDENTIFIED EXTERNALLY ......;
```



“.....”表示 2.2 节中创建用户语法从“<锁定子句>”开始往后的语法部分。

由于该创建用户的语法仅安全版支持，因此远程验证仅安全版支持。

远程验证仅支持使用交互方式连接 DM 服务器，用户名写为“//username”形式，无需输入密码，需要输入当前登录用户自身的 SSL 数字证书路径以及相应的 SSL 证书密码，如果证书没有密码则可以缺省或使用任意数字代替。

举例说明：

dameng_remote 用户使用当前操作系统上的 DM 客户端，基于远程验证方式连接远程 DM 服务器的操作步骤如下：

1. 执行操作之前需要保证 dameng_remote 用户拥有自己的 SSL 数字证书，并且确认 DM 服务器以及 DM 客户端均已配置好 SSL 环境；

2. 在数据库中创建 dameng_remote 用户，并修改 INI 参数。

```
SQL> CREATE USER dameng_remote IDENTIFIED EXTERNALLY;
SQL> SP_SET_PARA_VALUE(2,'ENABLE_REMOTE_OSAUTH',1);
SQL> SP_SET_PARA_VALUE(2,'ENABLE_ENCRYPT',1);
SQL> COMMIT;
```

重新启动数据库服务器，使 INI 参数生效。

3. 打开 SQL 交互式查询工具，执行 LOGIN 命令登录远程 DM 服务器，需要输入远程 DM 服务器 IP 地址和端口号，用户名写为“//username”形式，无需输入密码，还需输入当前登录用户自身的 SSL 数字证书路径以及相应的 SSL 证书密码。

```
SQL> LOGIN

服务名:192.168.0.1:5236

用户名://dameng_remote

密码:

SSL 路径: E:\dmdbms\bin\client_ssl\dameng_remote

SSL 密码:

UKEY 名称:

UKEY PIN 码:

MPP 类型:

是否读写分离(y/n):

协议类型:



服务器[192.168.0.1:5236]:处于普通打开状态

登录使用时间 : 21.271(ms)

SQL>
```

2.3.3 LDAP 身份验证

DM 提供对 LDAP 的支持，主要利用 LDAP 服务器存储的账户数据信息，验证账户是否是数据库的合法用户。用户需要预先部署 LDAP 服务器和 CA 服务器（或者购买证书认证）。

要使用 LDAP 身份验证，首先要在 DM 配置文件 dm.ini 中添加参数 LDAP_HOST。此参数为 LDAP 数据源所在机器的主库名，假如主库名为 ldapserver.dameng.com，则 LDAP_HOST = ldapserver.dameng.com。

LDAP 身份验证创建用户的语法如下：

```
CREATE USER [IF NOT EXISTS] <用户名> IDENTIFIED EXTERNALLY AS <用户 DN> ……;
```



说明：

“……”表示 2.2 节中创建用户语法从“<锁定子句>”开始往后的语法部分。

由于该创建用户的语法仅安全版支持，因此 LDAP 身份验证仅安全版支持。

语法中的“用户 DN”指 LDAP 数据源中用户 DN，可以唯一标识每一个用户。

例 如果域名为 dameng.com，若以 Administrator 用户登录，按 DN 规范书写则 certificate_dn 为 cn=Administrator,cn=users,dc=dameng,dc=com。则创建用户语法可以写为：

Windows 操作系统下：

```
CREATE USER USER01 IDENTIFIED EXTERNALLY AS 'cn=Administrator,cn=users,
dc=dameng,dc=com';
```

Linux 操作系统下：

```
CREATE USER USER01 IDENTIFIED EXTERNALLY AS 'cn=root, dc=dameng,dc=com';
```

当用户使用 DM 客户端或接口登录 DM 数据库时，将用户名写为“/username”的形式，则表示以 LDAP 验证方式进行登录，登录密码为 LDAP 数据源中创建用户时使用的密码。

2.3.4 SSL 双因子身份验证

SSL 双因子身份验证支持本机验证和远程验证。用户需要拥有自己的 SSL 数字证书，并为 DM 服务器以及 DM 客户端或接口均配置好 SSL 环境。关于 SSL 通信加密的详细介绍请参考 6.1 基于传输层的 SSL 协议加密。

SSL 双因子身份验证需要将 DM 配置文件 dm.ini 的 ENABLE_ENCRYPT 参数设置为 1、2、4 或 5，各个取值具体含义如下：

0：通信数据不经 SSL 加密，通信双方也不进行证书验证。

1：通信数据会被加密，并且通信双方需要交换并验证对方证书是否匹配。

2：通信数据不会被加密，但通信双方会交换并验证对方证书是否匹配。

4：通信数据会被加密，但通信双方不需要验证证书。

5：通信数据会被加密，仅客户端会验证服务器端证书。

6：通信数据基于商密 TLCP 协议加密，通信双方交换并验证对方证书是否匹配。

ENABLE_ENCRYPT 为静态参数，数据库管理员可以使用系统过程 SP_SET_PARA_VALUE 进行修改，但修改后需要重新启动 DM 服务器才能生效。由于开启了 SSL 认证，因此重启 DM 服务器之前必须保证当前服务器 SSL 环境配置完成，否则服务器无法正常启动。

使用 SSL 双因子身份验证方式连接 DM 服务器时，需要输入当前登录用户自身的 SSL 数字证书路径以及相应的 SSL 证书密码，如果证书没有密码则可以缺省或使用任意数字代替。

举例说明：

dameng_ssl 用户使用当前操作系统上的 DM 客户端，基于 SSL 双因子身份验证方式连接远程 DM 服务器的操作步骤如下：

1. 执行操作之前需要保证 dameng_ssl 用户拥有自己的 SSL 数字证书，并且确认 DM 服务器和 DM 客户端均已配置好 SSL 环境；

2. 在数据库中创建 dameng_ssl 用户，并修改 INI 参数。

```
SQL> CREATE USER dameng_ssl IDENTIFIED BY "Dameng12345";
SQL> SP_SET_PARA_VALUE(2,'ENABLE_ENCRYPT',1);
```

```
SQL> COMMIT;
```

重新启动数据库服务器，使 INI 参数生效。

3. dameng_ssl 基于 SSL 双因子身份验证方式连接远程 DM 服务器，需要输入用户名和密码、远程 DM 服务器 IP 地址和端口号以及当前登录用户自身的 SSL 数字证书路径以及相应的 SSL 证书密码。

使用 DIsql 连接：

```
D:\dmdbms\bin>disql.exe
dameng_ssl/Dameng12345@192.168.0.1:5236#"{ssl_path=E:\dmdbms\bin\client_ssl\dameng_ssl,ssl_pwd=12345}"
```

使用 DM 管理工具连接（sslFilesPath 属性和 sslKeystorePass 属性，分别用于指定证书目录和.keystore 文件密码，没有密码时可输入默认密码 changeit）：

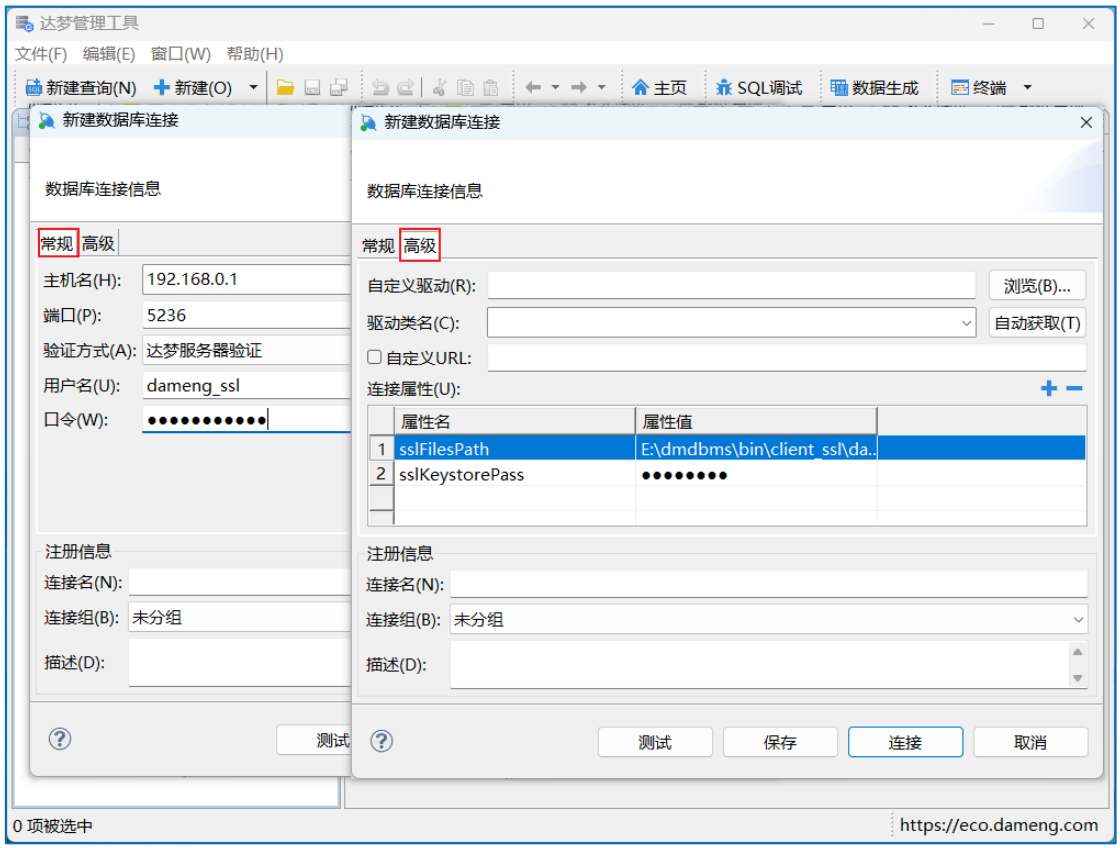


图 2.1 基于 SSL 双因子身份验证连接数据库

2.3.5 KERBEROS 身份验证

KERBEROS 是为 TCP/IP 网络系统设计的可信的第三方认证协议，DM 支持 KERBEROS 身份验证。用户需要预先配置 KERBEROS 使用环境。

KERBEROS 身份验证创建用户的语法如下：

```
CREATE USER [IF NOT EXISTS] <用户名> IDENTIFIED EXTERNALLY ......;
```


说明：

“.....”表示 2.2 节中创建用户语法从“<锁定子句>”开始往后的语法部分。

由于该创建用户的语法仅安全版支持，因此 KERBEROS 身份验证仅安全版支持。

当用户使用 DIsql 或 DPI 接口登录 DM 数据库时，将用户名写为“///username”的形式，则表示以 KERBEROS 验证方式进行登录，不需要输入密码。

2.3.6 UKEY 身份验证

DM 支持 UKEY 身份验证，通过 INI 参数 CLIENT_UKEY 进行 UKEY 验证设置，缺省值为 0：

0：支持所有登录方式，是否采用 UKEY 验证方式登录。当客户端输入 UKEY 名称和 UKEY_PIN 则采用 UKEY 验证，否则不使用 UKEY 验证；

1：客户端强制使用 UKEY 验证。

使用 UKEY 身份验证需要按照 8.1 节的介绍编写动态库，具体请参考 8.3 节的介绍。

2.3.7 RADIUS 身份验证

DM 支持 RADIUS 协议认证机制。DM 服务器作为客户端与 RADIUS 服务器经行交互。RADIUS 用于对于用户名、口令和质询信息的校验。

当对用户名、口令进行校验时，DM 服务器将一条 Access-Request 消息（包含用户名、口令等信息）发送给 RADIUS 服务器，消息会使用共享密钥（DM 和 RADIUS 的共享密钥）进行加密处理，RADIUS 服务器收到该消息后会使用 Access-Accept（表示接受）或 Access-Reject（表示拒绝）进行回复。回复消息为 Access-Reject 时，登录将会失败。

当对质询信息进行校验时，DM 服务器将一条 Access-Request 消息被发送给 RADIUS 服务器后，服务器可能会以 Access-Challenge（表示要求进一步对用户认证）进行回复。此时，DM 服务器将会向用户展示 RADIUS 服务器所提出的问题，并将用户的回答重新发给服务器，而后 RADIUS 服务器使用 Access-Accept 或 Access-Reject 进行回复。

RADIUS 身份验证创建用户的语法如下：

```
CREATE USER [IF NOT EXISTS] <用户名> IDENTIFIED EXTERNALLY RADIUS ......;
```



说明：

“.....”表示 2.2 节中创建用户语法从“<锁定子句>”开始往后的语法部分。

由于该创建用户的语法仅安全版支持，因此 RADIUS 身份验证仅安全版支持。

创建用户后必须使用 ALTER USER 语句将该用户设置为代理用户，语句介绍请参考 2.4 如何修改用户信息。

使用 RADIUS 协议须提前配置文件 dmradius.ini，并将其和 dm.ini 放置在同一目录下。

表 2.1 dmradius.ini 参数

参数	说明
RADIUS_SERVER	RADIUS 验证服务器 IP 地址
RADIUS_PORT	RADIUS 服务器的端口号
RADIUS_SECRET	与 RADIUS 服务器进行通信的共享密钥。DM 和 RADIUS 服务器的共享密钥必须相同，否则无法通信

举例说明：

dameng_radius 用户基于 RADIUS 身份验证方式连接 DM 服务器的操作步骤如下：

1. 执行操作之前用户需要自行配置好 RADIUS 服务器。

2. 配置 dmradius.ini 文件，并将其和 dm.ini 放置在同一目录下。dmradius.ini 文件内容如下：

```
RADIUS_SERVER = 192.168.100.8
RADIUS_PORT = 1812
RADIUS_SECRET = testing123
```

3. 在数据库中创建 dameng_radius 用户，并将其设置为代理用户，本例中将 SYSDBA 用户作为被代理用户。

```
CREATE USER dameng_radius IDENTIFIED EXTERNALLY RADIUS;
ALTER USER SYSDBA GRANT CONNECT THROUGH dameng_radius;
```

4. dameng_radius 基于 RADIUS 身份验证方式连接 DM 服务器，需要输入代理用户名、被代理用户名、RADIUS 服务器中设置的认证请求口令和 DM 服务器端口号。

```
./disql dameng_radius/hello:5236#{proxy_client=SYSDBA}
```

上述示例语句中，“hello”即 RADIUS 服务器中设置的认证请求口令，用户可根据实际配置情况进行修改。

2.4 如何修改用户信息

为了防止不法之徒盗取用户的口令，用户应该经常改变自己的口令。用户的口令不应该是类似 ABCabc123 这样简单的字符串，也不建议指定为自己的姓名和生日，因为这样的口令很容易被破解。一个好的口令应该包含大小写字母、数字、特殊符号在内的混合字符串。统计表明，一个口令中包含的成分越复杂，就越难破译。

修改用户口令的操作一般由用户自己完成，SYSDBA、SYSSSO、SYSAUDITOR 和 SYSDBO 可以无条件修改同类型的用户的口令。普通用户只能修改自己的口令，如果需要

修改其他用户的口令，必须具有 ALTER USER 数据库权限。修改用户口令时，口令策略应符合创建该用户时指定的口令策略。


注意：

具有 ALTER USER 权限的用户可以修改包括管理员在内的所有用户的口令和用户资源限制，因此对 ALTER USER 权限的授予应慎之又慎。

使用 ALTER USER 语句可修改用户口令。除口令外，这个语句还可以修改用户的口令策略、只读属性以及资源限制等。

ALTER USER 的语法与创建用户的语法极为相似，具体语法格式如下：

```
ALTER USER <用户名> [<修改用户子句>] | [<用户代理功能子句>];

<修改用户子句> ::= [IDENTIFIED <身份验证模式>] [PASSWORD_POLICY <口令策略>] [<锁定子句>] [<存储加密密钥>] [<只读标志>][<资源限制子句>][<密码过期子句>] [<允许 IP 子句>][<禁止 IP 子句>][<允许时间子句>][<禁止时间子句>][<TABLESPACE 子句>][<INDEX_TABLESPACE 子句>][<SCHEMA 子句>][<表空间配额子句>]

<SCHEMA 子句> ::= ON SCHEMA <模式名>

<用户代理功能子句> ::= <GRANT | REVOKE> CONNECT THROUGH <代理用户名>
```

<修改用户子句>中的<SCHEMA 子句>用于设置用户的缺省模式，其余子句的具体语法格式请参考 2.2.1 创建用户。

<用户代理功能子句>用于赋予用户 B 能够以代理的身份认证登录用户 A 的权限，即 CONNECT THROUGH 权限。其中，用户 B 表示参数<代理用户名>指定的用户，用户 A 表示参数<用户名>指定的用户，用户 A 也可称作被代理用户。关键字 GRANT 表示赋予权限，关键字 REVOKE 表示收回权限。

例 下面的语句修改用户 BOOKSHOP_USER 的只读属性为只读。

```
ALTER USER BOOKSHOP_USER READ ONLY;
```


注意：

不论 DM 的 INI 参数 DDL_AUTO_COMMIT 设置为自动提交还是非自动提交，ALTER USER 操作都会被自动提交。

2.5 如何删除用户

当一个用户不再需要访问数据库系统时，应将这个用户及时地从数据库中删除，否则可能会有安全隐患。

删除用户的操作一般由 SYSDBA、SYSSSO、SYSAUDITOR 和 SYSDBO 完成，他们可以删除同类型的其他用户。普通用户要删除其他用户，需要具有 DROP USER 权限。

使用 DROP USER 语句删除语句，语法格式为：

```
DROP USER [IF EXISTS] <用户名> [RESTRICT | CASCADE];
```

一个用户被删除后，这个用户本身的信息，以及它所拥有的数据库对象的信息都将从数据字典中被删除。

指定 IF EXISTS 关键字后，删除不存在的用户时不会报错，否则会报错。

如果在删除用户时未使用 CASCADE 选项（缺省使用 RESTRICT 选项），若该用户建立了数据库对象，DM 将返回错误信息，而不删除此用户。

如果在删除用户时使用了 CASCADE 选项，除数据库中该用户及其创建的所有对象被删除外，若其他用户创建的对象引用了该用户的对象，DM 还将自动删除相应的引用完整性约束及依赖关系。

例 假设用户 BOOKSHOP_USER 建立了自己的表或其他数据库对象，执行下面的语句：

```
DROP USER BOOKSHOP_USER;
```

将提示错误信息“试图删除被依赖对象[BOOKSHOP_USER]”。

下面的语句则能成功执行，会将 BOOKSHOP_USER 所建立的数据库对象一并删除。

```
DROP USER BOOKSHOP_USER CASCADE;
```


注意：

正在使用中的用户可以被其他具有 DROP USER 权限的用户删除，被删除的用户继续做操作或尝试重新连接数据库会报错。

3 自主访问控制

自主访问控制（Discretionary Access Control，DAC）是这样一种访问控制方式：由数据库对象的拥有者自主决定是否将自己拥有的对象的部分或全部访问权限授予其他用户。也就是说，在自主访问控制下，用户可以按照自己的意愿，有选择地与其他用户共享他拥有的数据库对象。

3.1 权限管理

DM 数据库对用户的权限管理有着严密的规定，如果没有权限，用户将无法完成任何操作。

用户权限有两类：数据库权限和对象权限。数据库权限主要是指针对数据库对象的创建、删除、修改的权限，对数据库备份等权限。而对象权限主要是指对数据库对象中的数据的访问权限。数据库权限一般由 SYSDBA、SYSAUDITOR 和 SYSSSO 指定，也可以由具有特权的其他用户授予。对象权限一般由数据库对象的所有者授予用户，也可由 SYSDBA 用户指定，或者由具有该对象权限的其他用户授权。

DM 数据库用户可以通过动态视图 V$AUTHORITIES 来查询当前数据库中的所有权限，该视图字段如下：

序号	列	数据类型	说明
1	ID	INTEGER	权限 ID
2	NAME	VARCHAR(256)	权限名称
3	TYPE	INTEGER	权限类型。1 表示数据库权限；2 表示对象权限；3 表示黑名单权限。黑名单权限表示关闭指定的对象权限

3.1.1 数据库权限

数据库权限是与数据库安全相关的非常重要的权限，其权限范围比对象权限更加广泛，因而一般被授予数据库管理员或者一些具有管理功能的角色。数据库权限与 DM 预定义角色有着重要的联系，详细介绍请参考 3.2.1.1 DM 预定义角色。

DM 提供了 100 余种数据库权限，表 3.1 列出了最常用的几种数据库权限。

表 3.1 常用的几种数据库权限

数据库权限	说明
CREATE TABLE	在自己的模式中创建表的权限
CREATE VIEW	在自己的模式中创建视图的权限
CREATE USER	创建用户的权限
CREATE TRIGGER	在自己的模式中创建触发器的权限
ALTER USER	修改用户的权限
ALTER DATABASE	修改数据库的权限
CREATE PROCEDURE	在自己模式中创建存储程序的权限
SELECT ANY DICTIONARY	查询任意系统表或系统视图的记录

不同类型的数据库对象，其相关的数据库权限也不相同。

例如，对于表对象，相关的数据库权限包括：

- CREATE TABLE：在自己的模式中创建表
- CREATE ANY TABLE：在任意模式中创建表
- ALTER TABLE：修改自己模式中的表
- ALTER ANY TABLE：修改任意模式中的表
- DROP TABLE：删除自己的模式下的表
- DROP ANY TABLE：删除任意模式中的表
- INSERT TABLE：向自己的模式中的表插入记录

- INSERT ANY TABLE：向任意模式中的表插入记录
- UPDATE TABLE：更新自己的模式中的表的记录
- UPDATE ANY TABLE：更新任意模式中的表的记录
- DELETE TABLE：删除自己模式中的表的记录
- DELETE ANY TABLE：删除任意模式中的表的记录
- SELECT TABLE：查询自己的模式中的表的记录
- SELECT ANY TABLE：查询任意模式中的表的记录
- REFERENCES TABLE：引用自己的模式中的表
- REFERENCES ANY TABLE：引用任意模式中的表
- DUMP TABLE：导出自己的模式中的表
- DUMP ANY TABLE：导出任意模式中的表
- GRANT TABLE：向其他用户进行自己模式中的表上权限的授权
- GRANT ANY TABLE：向其他用户进行任意模式中表上权限的授权
- FLASHBACK TABLE：闪回自己的模式中的表
- FLASHBACK ANY TABLE：闪回任意模式中的表

而对于存储程序对象，其相关的数据库权限则包括：

- CREATE PROCEDURE：在自己的模式中创建存储程序
- CREATE ANY PROCEDURE：在任意模式中创建存储程序
- DROP PROCEDURE：删除自己的模式中的存储过程
- DROP ANY PROCEDURE：删除任意模式中的存储程序
- EXECUTE PROCEDURE：执行自己模式中的存储程序
- EXECUTE ANY PROCEDURE：执行任意模式中的存储程序
- GRANT PROCEDURE：向其他用户进行自己模式中的存储程序上权限的授权

- GRANT ANY PROCEDURE：向其他用户进行任意模式中的存储程序上权限的授权

需要说明的是，表、视图、触发器、存储程序等对象为模式对象，在默认情况下对这些对象的操作都是在当前用户自己的模式下进行的。如果要在其他用户的模式下操作这些类型的对象，需要具有相应的 ANY 特权。例如，要能够在其他用户的模式下创建表，当前用户必须具有 CREATE ANY TABLE 数据库权限，如果希望能够在其他用户的模式下删除表，必须具有 DROP ANY TABLE 数据库权限。


说明：

DM 中有一个较特殊的数据库权限"CREATE SESSION"，表示创建会话连接数据库的权限。系统预设的管理员用户都具备此权限，新建用户缺省也具备此权限，管理员可根据实际需要回收指定用户的 CREATE SESSION 权限以限制该用户连接数据库。

3.1.2 对象权限

对象权限主要是对数据库对象中的数据的访问权限，主要用来授予需要对某个数据库对象的数据进行操纵的数据库普通用户。表 3.2 列出了主要的对象权限。

表 3.2 常用的对象权限

数据库对象类型对象权限	表	视图	存储程序	包	类	类型	序列	目录	library	域/CHARACTER SET /COLLATION
SELECT	✓	✓					✓			
INSERT	✓	✓								
DELETE	✓	✓								
UPDATE	✓	✓								
REFERENCES	✓	✓								
SELECT FOR DUMP	✓									
EXECUTE			✓	✓	✓	✓		✓	✓	
READ								✓		
WRITE								✓		
USAGE										✓
FLASHBACK	✓	✓								
INDEX	✓									
ALTER	✓									

SELECT、INSERT、DELETE 和 UPDATE 权限分别是针对数据库对象中的数据的查询、插入、删除和修改的权限。对于表和视图来说，删除操作是整行进行的，而查询、插入和修改却可以在一行的某个列上进行，所以在指定权限时，DELETE 权限只要指定所要访问的表就可以了，而 SELECT、INSERT 和 UPDATE 权限还可以进一步指定是对哪个列的权限。

表对象的 REFERENCES 权限是指可以与一个表建立关联关系的权限，如果具有了这个权限，当前用户就可以通过自己的一个表中的外键，与对方的表建立关联。关联关系是通过主键和外键进行的，所以在授予这个权限时，可以指定表中的列，也可以不指定。

表对象的 SELECT FOR DUMP 权限是指导出表对象的权限，需要与同一表对象的 SELECT 的权限一起授予才能成功将该表对象导出。

视图对象语法上支持授权 REFERENCES 和 FLASHBACK 权限，但实际不起作用。

存储程序等对象的 EXECUTE 权限是指可以执行这些对象的权限。有了这个权限，一个用户就可以执行另一个用户的存储程序、包、类等。

目录对象的 READ 和 WRITE 权限指可以读或写访问某个目录对象的权限。

域对象的 USAGE 权限指可以使用某个域对象的权限。拥有某个域的 USAGE 权限的用户可以在定义或修改表时为表列声明使用这个域。

当一个用户获得另一个用户的某个对象的访问权限后，可以以“模式名.对象名”的形式访问这个数据库对象。一个用户所拥有的对象和可以访问的对象是不同的，这一点在数据字典视图中有所反映。在默认情况下用户可以直接访问自己模式中的数据库对象，但是要访问其他用户所拥有的对象，就必须具有相应的对象权限。

对象权限的授予一般由对象的所有者完成，也可由 SYSDBA 或具有某对象权限且具有转授权限的用户授予，但最好由对象的所有者完成。

3.2 角色管理

角色是一组权限的组合，使用角色的目的是使权限管理更加方便。假设有 10 个用户，这些用户为了访问数据库，至少拥有 CREATE TABLE、CREATE VIEW 等权限。如果将这些权限分别授予这些用户，那么需要进行的授权次数是比较多的。但是如果把这些权限事先放在一起，然后作为一个整体授予这些用户，那么每个用户只需一次授权，授权的次数将大大减少，而且用户数越多，需要指定的权限越多，这种授权方式的优越性就越明显。这些事先组合在一起的一组权限就是角色，角色中的权限既可以是数据库权限，也可以是对象权限，还可以是别的角色。

为了使用角色，首先在数据库中创建一个角色，这时角色中没有任何权限。然后向角色中添加权限。最后将这个角色授予用户，这个用户就具有了角色中的所有权限。在使用角色的过程中，可以随时向角色中添加权限，也可以随时从角色中删除权限，用户的权限也随之改变。如果要回收所有权限，只需将角色从用户回收即可。

3.2.1 角色的创建与删除

3.2.1.1DM 预定义角色

在 DM 数据库中有两类角色，一类是 DM 预设定的角色，一类是用户自定义的角色。DM 提供了一系列的预定义角色以帮助用户进行数据库权限的管理。预定义角色在数据库被创建之后即存在，并且已经包含了一些权限，数据库管理员可以将这些角色直接授予用户。

在“三权分立”和“四权分立”机制下，DM 的预定义角色及其所具有的权限是不相同的。表 3.3 和表 3.4 分别列出了“三权分立”和“四权分立”机制下常见的系统角色及其简单介绍，完整的预定义角色权限列表请参见附录 1 和附录 2。

表 3.3 “三权分立”常见的数据库预设定的角色

角色名称	角色简单说明
DBA	DM 数据库系统中对象与数据操作的最高权限集合，拥有构建数据库的全部特权，只有 DBA 才可以创建数据库结构
RESOURCE	可以创建数据库对象，对有权限的数据库对象进行数据操纵，不可以创建数据库结构
PUBLIC	不可以创建数据库对象，只能对有权限的数据库对象进行数据操纵。常规情况下，不建议回收用户的 PUBLIC 角色，一旦用户的 PUBLIC 角色被回收，将导致很多功能因权限不够无法使用
VTI	具有系统动态视图的查询权限。VTI 默认授权给 DBA 角色且可转授，不支持从 DBA 中回收 VTI。可以为 SYSDBA 授予和回收 VTI
SOI	具有非审计/安全系统表的查询权限。SOI 默认授权给 SYSDBA 用户且可转授，不支持从 SYSDBA 回收 SOI
SVI	具有基础 V 视图的查询权限。SVI 默认授权给 PUBLIC 且可转授，不支持从 PUBLIC 中回收 SVI。可以为 SYSDBA 授予和回收 SVI
DB_AUDIT_ADMIN	数据库审计的最高权限集合，可以对数据库进行各种审计操作，并创建新的审计用户
DB_AUDIT_OPER	可以对数据库进行各种审计操作，但不能创建新的审计用户
DB_AUDIT_PUBLIC	无实际权限
DB_AUDIT_VTI	具有系统动态视图的查询权限，DB_AUDIT_VTI 默认授权给 DB_AUDIT_ADMIN 且可转授
DB_AUDIT_SOI	具有基础类系统表和与审计操作相关的系统表的查询权限
DB_AUDIT_SVI	具有基础 V 视图和审计 V 视图的查询权限。DB_AUDIT_SVI 默认授权给 DB_AUDIT_PUBLIC 且可转授
DB_POLICY_ADMIN	数据库强制访问控制的最高权限集合，可以对数据库进行强制访问控制管理，并创建新的安全管理用户
DB_POLICY_OPER	可以对数据库进行强制访问控制管理，但不能创建新的安全管理用户
DB_POLICY_PUBLIC	无实际权限

DB_POLICY_VTI	具有系统动态视图的查询权限，DB_POLICY_VTI 默认授权给 DB_POLICY_ADMIN 且可转授
DB_POLICY_SOI	具有基础类系统表和与安全操作相关的系统表的查询权限
DB_POLICY_SVI	具有基础 V 视图和安全 V 视图的查询权限
SYS_ADMIN	系统预留角色，暂未启用
SELECT_CATALOG_ROLE	METADATA.GET_DDL 专用，查询相同类型其他用户的对象、或 SYSDBO 用户（包含创建的用户）下的对象、或 SYS 模式下的对象。此处的相同类型其他用户是指 SYSDBA 或 SYSDBA 创建的用户。SELECT_CATALOG_ROLE 默认授权给 DBA 角色且可转授
DB_AUDIT_SELECT_CATALOG_ROLE	METADATA.GET_DDL 专用，查询相同类型其他用户或不同类型用户的对象（包含 SYS 模式）。DB_AUDIT_SELECT_CATALOG_ROLE 默认授权给 DB_AUDIT_ADMIN 角色且可转授
DB_POLICY_SELECT_CATALOG_ROLE	METADATA.GET_DDL 专用，查询相同类型其他用户或不同类型用户的对象（包含 SYS 模式）。DB_POLICY_SELECT_CATALOG_ROLE 默认授权给 DB_POLICY_ADMIN 角色且可转授

表 3.4 “四权分立”常见的数据库预设定的角色

角色名称	角色简单说明
DBA	拥有构建数据库的全部特权，只有 DBA 才可以创建数据库结构
RESOURCE	可以创建和删除角色
PUBLIC	只能查询相关的数据字典表
VTI	具有系统动态视图的查询权限。VTI 默认授权给 DBA 角色且可转授，不支持从 DBA 中回收 VTI。可以为 SYSDBA 授予和回收 VTI
SOI	具有非审计/安全系统表的查询权限。SOI 默认授权给 SYSDBA 用户且可转授，不支持从 SYSDBA 回收 SOI
SVI	具有基础 V 视图的查询权限。SVI 默认授权给 PUBLIC 且可转授，不支持从

	PUBLIC 中回收 SVI。可以为 SYSDBA 授予和回收 SVI
DB_AUDIT_ADMIN	数据库审计的最高权限集合，可以对数据库进行各种审计操作，并创建新的审计用户
DB_AUDIT_OPER	可以对数据库进行各种审计操作，但不能创建新的审计用户
DB_AUDIT_PUBLIC	无实际权限
DB_AUDIT_VTI	具有系统动态视图的查询权限，DB_AUDIT_VTI 默认授权给 DB_AUDIT_ADMIN 且可转授
DB_AUDIT_SOI	具有基础类系统表和与审计操作相关的系统表的查询权限
DB_AUDIT_SVI	具有基础 V 视图和审计 V 视图的查询权限。DB_AUDIT_SVI 默认授权给 DB_AUDIT_PUBLIC 且可转授
DB_POLICY_ADMIN	数据库强制访问控制的最高权限集合，可以对数据库进行强制访问控制管理，并创建新的安全管理用户
DB_POLICY_OPER	可以对数据库进行强制访问控制管理，但不能创建新的安全管理用户
DB_POLICY_PUBLIC	无实际权限
DB_POLICY_VTI	具有系统动态视图的查询权限，DB_POLICY_VTI 默认授权给 DB_POLICY_ADMIN 且可转授
DB_POLICY_SOI	具有基础类系统表和与安全操作相关的系统表的查询权限
DB_POLICY_SVI	具有基础 V 视图和安全 V 视图的查询权限
DB_OBJECT_ADMIN	可以在自己的模式下创建各种数据库对象并进行数据操纵，也可以创建和删除非模式对象
DB_OBJECT_OPER	可以在自己的模式下创建数据库对象并进行数据操纵
DB_OBJECT_PUBLIC	不可以创建数据库对象，只能对有权限的数据库对象进行数据操纵
DB_OBJECT_VTI	具有系统动态视图的查询权限，DB_OBJECT_VTI 默认授权给 DB_OBJECT_ADMIN 且可转授
DB_OBJECT_SOI	具有非审计/安全系统表的查询权限
DB_OBJECT_SVI	和 SVI 权限一样

SYS_ADMIN	系统内部角色，具有 CREATE ANY TRIGGER 和 DROP ANY TRIGGER 权限。四权分立时，由于没有 CREATE ANY TRIGGER 和 DROP ANY TRIGGER 权限, 系统自动将 SYS_ADMIN 授权给 SYSDBA，只有具备 SYS_ADMIN 时 SYSDBA 才可以创建库级的事件触发器。SYS_ADMIN 既不可转授，也不可从 SYSDBA 回收
SELECT_CATALOG_ROLE	METADATA.GET_DDL 专用，查询相同类型其他用户的对象、或 SYSDBO 用户（包含创建的用户）下的对象、或 SYS 模式下的对象。此处的相同类型其他用户是指 SYSDBA 或 SYSDBA 创建的用户。SELECT_CATALOG_ROLE 默认授权给 DBA 角色且可转授
DB_AUDIT_SELECT_CATALOG_ROLE	METADATA.GET_DDL 专用，查询相同类型其他用户或不同类型用户的对象（包含 SYS 模式）。DB_AUDIT_SELECT_CATALOG_ROLE 默认授权给 DB_AUDIT_ADMIN 角色且可转授
DB_POLICY_SELECT_CATALOG_ROLE	METADATA.GET_DDL 专用，查询相同类型其他用户或不同类型用户的对象（包含 SYS 模式）。DB_POLICY_SELECT_CATALOG_ROLE 默认授权给 DB_POLICY_ADMIN 角色且可转授
DB_OBJECT_SELECT_CATALOG_ROLE	METADATA.GET_DDL 专用，查询相同类型其他用户的对象、或 SYSDBA 用户（包含创建的用户）下的对象、或 SYS 模式下的对象。此处的相同类型其他用户是指 SYSDBO 或 SYSDBO 创建的用户。DB_OBJECT_SELECT_CATALOG_ROLE 默认授权给 DB_OBJECT _ADMIN 角色且可转授

根据角色拥有的工作权限的不同，可以将预定义角色分为 DBA 类型预定义角色、AUDITOR 类型预定义角色、SSO 类型预定义角色和 DBO 类型预定义角色。下面详细介绍:

- DBA 类型预定义角色拥有部分管理员工作权限，有权利执行一部分管理员工作。DBA 类型预定义角色包括 DBA、RESOURCE、PUBLIC、VTI、SOI、SVI、SYS_ADMIN、SELECT_CATALOG_ROLE;
- AUDITOR 类型预定义角色拥有部分审计员工作权限，有权利执行一部分审计员工作。AUDITOR 类型预定义角色包括 DB_AUDIT_ADMIN、DB_AUDIT_OPER、

DB_AUDIT_PUBLIC、DB_AUDIT_VTI、DB_AUDIT_SOI、DB_AUDIT_SVI、DB_AUDIT_SELECT_CATALOG_ROLE;

- SSO 类型预定义角色拥有部分安全员工作权限，有权利执行一部分安全员工作。SSO 类型预定义角色包括 DB_POLICY_ADMIN、DB_POLICY_OPER、DB_POLICY_PUBLIC、DB_POLICY_VTI、DB_POLICY_SOI、DB_POLICY_SVI、DB_POLICY_SELECT_CATALOG_ROLE;
- DBO 类型预定义角色拥有部分对象操作员工作权限，有权利执行一部分对象操作员工作。DBO 类型预定义角色包括 DB_OBJECT_ADMIN、DB_OBJECT_OPER、DB_OBJECT_PUBLIC、DB_OBJECT_VTI、DB_OBJECT_SOI、DB_OBJECT_SVI、DB_OBJECT_SELECT_CATALOG_ROLE。

初始时仅有管理员具有创建用户的权限，每种类型的管理员创建的用户缺省就拥有这种类型的 PUBLIC 预定义角色，如 SYSAUDITOR 新创建的用户缺省就具有 DB_AUDIT_PUBLIC 角色。之后管理员可根据需要进一步授予新建用户其他预定义角色。

当不使用 SP_RESTRICT_DBA（1）限制 DBA 角色的 ANY 特权时，由于 DBA 角色拥有 ADMIN ANY DATABASE PRIVILEGE 特权，因此 DBA 类型用户可以将 CREATE USER 权限转授给其他用户，非 DBA 类型用户的用户不可以。当使用 SP_RESTRICT_DBA（1）限制 DBA 的 ANY 特权时，因为限制了 ADMIN ANY DATABASE PRIVILEGE，所以此时所有用户都无法转授 CREATE USER 权限。

拥有 CREATE USER 权限的用户可以创建新的用户，他们创建的新用户缺省也具有与其创建者相同类型的 PUBLIC 预定义角色。

■ **限制 DBA 的 ANY 特权**

从附录 1 和附录 2 的表中我们可以看到，缺省情况下，DBA 角色拥有许多的 ANY 特权，如 DROP ANY TABLE、INSERT ANY TABLE 等等。DM 提供了一种限制 DBA 的 ANY 特权的

方法：在非 DM 安全版本中，只有 SYSDBA 用户才能限制 DBA 的 ANY 特权；在 DM 安全版本中，只有具有 DB_POLICY_ADMIN 角色的用户才能限制 DBA 的 ANY 特权。通过执行下面的系统过程来实现，系统过程执行完成后需要重启服务器该限制才能生效。

```
VOID
SP_RESTRICT_DBA(
        FLAG            INT
);
```

参数 FLAG 置为 1 表示限制 DBA 的 ANY 特权，此过程执行后所有的 DBA 用户将不再具有 ANY 特权；FLAG 置为 0 表示不限制 DBA 的 ANY 特权。

3.2.1.2 创建角色

具有“CREATE ROLE”数据库权限的用户也可以创建新的角色，其语法如下：

```
CREATE ROLE [IF NOT EXISTS] <角色名>;
```

使用说明：

1. 创建者必须具有 CREATE ROLE 数据库权限；
2. 角色名的长度不能超过 128 个字符；
3. 角色名不允许和系统已存在的用户名重名；
4. 角色名不允许是 DM 保留字。

例 创建角色 BOOKSHOP_ROLE1，赋予其 PERSON.ADDRESS 表的 SELECT 权限。

```
CREATE ROLE BOOKSHOP_ROLE1;
GRANT SELECT ON PERSON.ADDRESS TO BOOKSHOP_ROLE1;
```

3.2.1.3 删除角色

具有“DROP ROLE”权限的用户可以删除角色，其语法如下：

```
DROP ROLE [IF EXISTS] <角色名>;
```

即使已将角色授予了其他用户，删除这个角色的操作也将成功。此时，那些之前被授予该角色的用户将不再具有这个角色所拥有的权限，除非用户通过其他途径也获得了这个角色所具有的权限。

指定 IF EXISTS 关键字后，删除不存在的角色时不会报错，否则会报错。

例如，接上一小节的例子。将角色 BOOKSHOP_ROLE1 授予用户 BOOKSHOP_USER，则用户 BOOKSHOP_USER 具有了 SELECT 表 PERSON.ADDRESS 的权限。此时删除角色 BOOKSHOP_ROLE1，那么用户 BOOKSHOP_USER 将不再能查询表 PERSON.ADDRESS。但是如果用户 BOOKSHOP_USER 从别的途径（直接授权或通过别的角色授权）重复获得了 SELECT 表 PERSON.ADDRESS 的权限，那么即使删除了角色 BOOKSHOP_ROLE1，用户 BOOKSHOP_USER 仍然具有 SELECT 表 PERSON.ADDRESS 的权限。

3.2.2 角色的启用与禁用

某些时候，用户不愿意删除一个角色，但是却希望这个角色失效，此时可以使用 DM 系统过程 SP_SET_ROLE 来设置这个角色为不可用，将第二参数置为 0 表示禁用角色。

例 下面的语句将角色 BOOKSHOP_ROLE1 禁用。

```
SP_SET_ROLE('BOOKSHOP_ROLE1', 0);
```

使用说明:

1. 只有拥有 ADMIN_ANY_ROLE 权限的用户才能启用和禁用角色，并且设置后立即生效;
2. 凡是包含禁用角色 A 的角色 M，M 中禁用的角色 A 将无效，但是 M 仍有效;
3. 系统预设的角色是不能设置的，如：DBA、PUBLIC、RESOURCE。

当用户希望启用某个角色时，同样可以通过 SP_SET_ROLE 来启用角色，只要将第二个参数置为 1 即可。

例 下面的语句将启用角色 BOOKSHOP_ROLE1。

```
SP_SET_ROLE('BOOKSHOP_ROLE1', 1);
```

3.2.3 SVI 角色用法

在启用了 SVI 环境的情况下，拥有 SVI 角色权限的用户可以查询 V 视图。V 视图请参考《DM8 系统管理员手册》。缺省情况下系统未启用 SVI 环境，因此需要使用 V 视图的用户，须先手动启用 SVI 环境，即执行 SP_INIT_SVI_SYS(1)和 SP_SWITCH_SVI(1)过程。

下面是使用 SVI 角色的相关系统过程:

1. SP_INIT_SVI_SYS

定义:

```
SP_INIT_SVI_SYS(
CREATE_FLAG                int
)
```

功能说明:

可以手动创建所有 V 视图，并授权给 SVI、DB_POLICY_SVI、DB_OBJECT_SVI 和 DB_AUDIT_SVI。只有 DBA 和 DB_OBJECT_ADMIN 可以执行。

参数详解

CREATE_FLAG：1 创建所有 V 视图；0 删除所有 V 视图

返回值

无

举例说明

```
SP_INIT_SVI_SYS(1);
SP_INIT_SVI_SYS(0);
```

2. SP_SWITCH_SVI

定义:

```
SP_SWITCH_SVI
    (
    flag        int
    )
```

功能说明:

SVI 环境启用或禁用。只有 DBA/DB_OBJECT_ADMIN 可以执行。SVI 启用时字典信息来源于 V 视图，SVI 禁用时字典信息来源于 SYS.数据字典表。

SP_INIT_SVI_SYS(1)需和 SP_SWITCH_SVI(1)一起使用，SP_INIT_SVI_SYS(0)需和 SP_SWITCH_SVI(0)一起使用，才有效果。否则报错。

参数说明:

flag：1 启用 SVI 环境；0 禁用 SVI 环境

返回值:

启用返回 1；禁用返回 0

举例说明:

```
SP_SWITCH_SVI(0);
SP_SWITCH_SVI(1);
```

3. SF_GET_SVI

定义:

int

SF_GET_SVI ()

功能说明:

获取当前系统是否启用 SVI 环境。

参数说明:

无

返回值:

启用返回 1；禁用返回 0

举例说明:

```
SP_SWITCH_SVI(0);
SELECT SF_GET_SVI;   //结果为 0
SP_SWITCH_SVI(1);
SELECT SF_GET_SVI;   //结果为 1
```

3.3 权限的分配与回收

可以通过 GRANT 语句将权限（包括数据库权限、对象权限、角色权限和模式权限）分配给用户和角色，之后也可以使用 REVOKE 语句将授出的权限再进行回收。

3.3.1 数据库权限的分配与回收

3.3.1.1 数据库权限的分配

可以使用 GRANT 语句授予用户和角色数据库权限。

数据库权限的授权语句语法为：

```
GRANT <特权> TO <用户或角色>{,<用户或角色>} [WITH ADMIN OPTION];
```

<特权> ::= <数据库权限>{,<数据库权限>};

<用户或角色>::= <用户名> | <角色名>

使用说明:

1. 授权者必须具有对应的数据库权限以及其转授权。四权分立下四类管理员（DBA、SSO、AUDITOR、DBO）均不能转授 PROFILE 权限（CREATE PROFILE、DROP PROFILE、ALTER PROFILE）和 USER 权限（CREATE USER、ALTER USER、DROP USER）；三权分立下只有管理员 DBA 能转授 PROFILE 权限和 USER 权限，管理员 SSO 和 AUDITOR 不能转授 PROFILE 权限和 USER 权限;
2. 接受者必须与授权者用户类型一致;
3. 数据库权限及其与 DM 预定义角色的关系列表可参考附录 1 与附录 2;
4. WITH ADMIN OPTION 表示可以转授，即被授权者可以把这些权限继续授予给其他用户或角色，缺省不能转授。例如：A 将权限转授给 B，B 又将权限转授给 C。那么 A→B→C 之间就形成了一个链式关系。

例 系统管理员 SYSDBA 把建表的权限授予用户 BOOKSHOP_USER1，并允许其转授。BOOKSHOP_USER1 把权限继续转授给 BOOKSHOP_USER2。

```
//SYSDBA 执行。SYSDBA 将权限授予 BOOKSHOP_USER1
GRANT CREATE TABLE TO BOOKSHOP_USER1 WITH ADMIN OPTION; --使用 WITH ADMIN OPTION，BOOKSHOP_USER1 才可转授
//BOOKSHOP_USER1 执行。BOOKSHOP_USER1 将从 SYSDBA 获取的权限转授给 BOOKSHOP_USER2
GRANT CREATE TABLE TO BOOKSHOP_USER2;--BOOKSHOP_USER2 被授权成功，但不可继续转授
GRANT CREATE TABLE TO BOOKSHOP_USER2 WITH ADMIN OPTION;--BOOKSHOP_USER2 被授权成功，且可继续转授
```

3.3.1.2 数据库权限的回收

可以使用 REVOKE 语句回收授出的指定数据库权限。

回收数据库权限的语句语法为：

```
REVOKE [ADMIN OPTION FOR]<特权> FROM <用户或角色>{,<用户或角色>} ;

<特权> ::= <数据库权限>{,<数据库权限>}

<用户或角色>::= <用户名> | <角色名>
```

使用说明：

1. 权限回收者必须是具有回收相应数据库权限以及转授权的用户；
2. ADMIN OPTION FOR 选项的意义是取消用户或角色的转授权限，但是权限不回收。

例 1 接 3.3.1.1 中的例子，现在 SYSDBA 把用户 BOOKSHOP_USER1 的建表权限收回。

```
REVOKE CREATE TABLE FROM BOOKSHOP_USER1;
```

例 2 接 3.3.1.1 中的例子，之前 SYSDBA 把建视图的权限授给了用户 BOOKSHOP_USER1，并且允许转授。现在 SYSDBA 不让用户 BOOKSHOP_USER1 转授 CREATE VIEW 权限。

```
REVOKE ADMIN OPTION FOR CREATE VIEW FROM BOOKSHOP_USER1;
```

BOOKSHOP_USER1 仍有 CREATE VIEW 权限，但是不能将 CREATE VIEW 权限转授给其他用户。

3.3.1.3 限制相关数据库权限的授予与回收

通过 INI 参数 ENABLE_DDL_ANY_PRIV 限制 DDL 相关的 ANY 特权的授予与回收，有 2 个取值：

- 1：可以授予和回收 DDL 相关的 ANY 特权；
- 0：不可以授予和回收 DDL 相关的 ANY 特权。缺省为 0。

例 当 ENABLE_DDL_ANY_PRIV=0 时，SYSDBA 无法授予或回收 create any trigger 权限。

```
create user DBSEC  identified by "Dbsec_123";
GRANT create any trigger TO DBSEC;          //报错
revoke create any trigger from DBSEC; //报错
```

3.3.2 对象权限的分配与回收

3.3.2.1 对象权限的分配

可以使用 GRANT 语句将对象权限授予用户和角色。

对象权限的授权语句语法为：

```
GRANT <特权> [OFF] ON <对象> TO <用户或角色>{,<用户或角色>} [WITH GRANT OPTION];
<特权>::= ALL [PRIVILEGES] |
          <动作> {, <动作>}
<动作>::= SELECT[(<列清单>)] |
          INSERT[(<列清单>)] |
          UPDATE[(<列清单>)] |
          DELETE |
          REFERENCES[(<列清单>)] |
          EXECUTE |
          READ |
          WRITE |
          USAGE |
          INDEX |
          ALTER |
```

SELECT FOR DUMP |

FLASHBACK

<列清单>::= <列名> {,<列名>}

<对象> ::= SCHEMA <模式名> |

DIRECTORY <目录名> |

[<对象类型>] [<模式名>.] <对象名>

<对象类型>::= TABLE | VIEW | PROCEDURE | FUNCTION | PACKAGE | CLASS | TYPE | SEQUENCE | DIRECTORY | DOMAIN | SCHEMA | SYNONYM | LIBRARY

<对象名> ::= <表名> | <视图名> | <存储过程/函数名> |<包名> |<类名> |<类型名> |<序列名> | <域名> | <同义词名>|<LIBRARY 名>

<用户或角色>::= <用户名> | <角色名>

使用说明:

1. 授权者必须是具有对应对象权限以及其转授权的用户;
2. 使用 INI 参数 OBJECT_PRIV_MODE 可控制授权策略是否兼容 ORACLE;
3. OFF 授予黑名单权限，即关闭（黑名单）用户的模式级对象权限。模式级对象权限是指<对象类型>为 SCHEMA 的权限;
4. <对象>为 SCHEMA 时，须提前设置 INI 参数 GRANT_SCHEMA=1 或 2，即开启授予和回收模式权限功能。授权用户需要具有对授权模式的"GRANT ANY OBJECT PRIVILEGE"权限。此时，<动作>为 EXECUTE、READ 、WRITE，虽然可授权成功，但不起作用。设置 INI 参数 GRANT_SCHEMA=1 时，ALL 或 ALL PRIVILEGES 表示将指定模式的所有对象权限都授予被授予者;
5. 如未指定<对象>的<模式名>，模式为授权者所在的模式。DIRECTORY 为非模式对象，没有模式;

6. 如设定了对象类型，则该类型必须与对象的实际类型一致，否则会报错；
7. WITH GRANT OPTION 表示可以转授，即被授权者可以把这些权限继续授予其他用户或角色，缺省不能转授；
8. <特权>为 ALL 或 ALL PRIVILEGES 时，表示将指定的数据库对象上所有的对象权限都授予被授权者。不同类型的数据库对象相关的对象权限见 3.1.2 对象权限的介绍；
9. 不带列清单授权时，如果对象上存在同类型的列权限，会全部自动合并；
10. 对于用户所在的模式的表，用户具有所有权限而不需特别指定；
11. INDEX 动作向其他用户授予指定表的创建和删除索引（包含全文索引）的权限；
12. ALTER 动作仅支持向其他用户授予指定表的修改权限、表（列）注释权限和指定序列的修改权限。

例 1 SYSDBA 把 PERSON.ADDRESS 表的全部权限授给用户 BOOKSHOP_USER1。

```
GRANT ALL PRIVILEGES ON PERSON.ADDRESS TO BOOKSHOP_USER1;
```

例 2 假设用户 BOOKSHOP_USER1 创建了存储过程 BOOKSHOP_USER1_PROC1，数据库管理员 SYSDBA 把该存储过程的执行权 EXECUTE 授给已存在用户 BOOKSHOP_USER2，并使其具有该权限的转授权。

```
GRANT EXECUTE ON PROCEDURE BOOKSHOP_USER1.BOOKSHOP_USER1_PROC1
TO BOOKSHOP_USER2 WITH GRANT OPTION;
```

例 3 假设 SYSDBA 是表 BOOKSHOP_T1 的创建者，用户 BOOKSHOP_USER1、BOOKSHOP_USER2、BOOKSHOP_USER3 存在，且都不是 DBA 权限用户。

```
//以 SYSDBA 身份登录，并执行语句：
GRANT SELECT ON BOOKSHOP_T1 TO BOOKSHOP_USER1 WITH GRANT OPTION;
//以 BOOKSHOP_USER1 身份登录，并执行语句：
GRANT SELECT ON SYSDBA.BOOKSHOP_T1 TO BOOKSHOP_USER2;
//以 BOOKSHOP_USER2 身份登录，并执行语句：
```

```
GRANT SELECT ON SYSDBA.BOOKSHOP_T1 TO BOOKSHOP_USER3;
//错误，用户 BOOKSHOP_USER2 没有 SELECT 权限的转授权
```

例 4 用户 SYSDBA 创建一个名为 VIEW_PRODUCT 的视图，要求对于该视图，允许 BOOKSHOP_USER1 能够进行查询、插入、删除和更新操作，用户 BOOKSHOP_USER2 和 BOOKSHOP_USER3 也可进行同样的工作，但要求其操作权限由用户 BOOKSHOP_USER1 控制。

```
//以 SYSDBA 的身份登录:
CREATE VIEW VIEW_PRODUCT AS
SELECT * FROM PRODUCTION.PRODUCT
WHERE NOWPRICE>=20 AND ORIGINALPRICE<40 WITH CHECK OPTION;
GRANT SELECT, INSERT, DELETE, UPDATE ON VIEW_PRODUCT
TO BOOKSHOP_USER1 WITH GRANT OPTION;
//以用户 BOOKSHOP_USER1 的身份登录:
GRANT SELECT, INSERT, DELETE, UPDATE ON SYSDBA.VIEW_PRODUCT TO BOOKSHOP_USER2,
BOOKSHOP_USER3;
```

例 5 假定表的创建者 SYSDBA 把所创建的 PRODUCTION.PRODUCT 表的部分列的更新权限授予用户 BOOKSHOP_USER1，BOOKSHOP_USER1 再将此更新权限转授给用户 BOOKSHOP_USER2。

```
//首先，以 SYSDBA 的身份登录，并执行语句:
GRANT UPDATE (ORIGINALPRICE, NOWPRICE) ON PRODUCTION.PRODUCT TO BOOKSHOP_USER1
WITH GRANT OPTION;
//其次，以 BOOKSHOP_USER1 的身份登录，并执行语句，把列级更新权限授给用户 BOOKSHOP_USER2:
GRANT UPDATE(ORIGINALPRICE, NOWPRICE)ON PRODUCTION.PRODUCT TO BOOKSHOP_USER2;
```

例 6 使用 SYSDBA 创建用户 USER01 和 USER02，并把模式 USER02 下所有对象的查询权限授予用户 USER01。须提前开启 INI 参数 GRANT_SCHEMA=1 或 2。

```
DROP USER USER01 CASCADE;
```

```
DROP USER USER02 CASCADE;

//创建用户时，系统自动创建了同名模式 USER01，USER02

CREATE USER USER01 IDENTIFIED BY "USER01_psd";

CREATE USER USER02 IDENTIFIED BY "USER02_psd";

//授予用户 USER02 建表，插入的权限

GRANT CREATE TABLE,INSERT TABLE TO USER02;

//将模式 USER02 下所有对象的查询权限授予用户 USER01

GRANT SELECT ON SCHEMA USER02 TO USER01;

//登录用户 USER02

CREATE TABLE T1(C1 INT);

INSERT INTO T1 VALUES(1);

COMMIT;

//登录用户 USER01，可以查询成功

SELECT * FROM USER02.T1;
```

例 7 为黑名单用户 USER02 授予黑名单权限，即取消 USER02 的对象权限。须提前开启 INI 参数 GRANT_SCHEMA=1 或 2。

```
//将 USER01 对象的查询权限授予 USER02。此时 user02 获得 user01 对象的查询权限

GRANT SELECT ON SCHEMA USER01 TO USER02;

//为 USER02 授予黑名单权限。此时 USER02 失去 USER01 的 T1 的查询权限

GRANT SELECT OFF ON USER01.T1 TO USER02;
```

例 8 为用户 USER1 授予 ALTER 对象权限，ALTER 权限包括对表（列）的注释权限。

```
//SYSDBA 登录，先创建用户和表，然后把 ALTER 权限授予 USER1

CREATE USER USER1 IDENTIFIED BY "USER1_psd";

CREATE TABLE TESTT(C1 INT);

GRANT ALTER ON  TESTT TO USER1;

// USER1 登录，对 TESTT 表进行注释，可注释成功
```

```
COMMENT ON TABLE SYSDBA.TESTT IS 'COMMENT IN ALTER';

//SYSDBA 登录，进行查询

SELECT * FROM  SYSTABLECOMMENTS WHERE TVNAME='TESTT';

行号       SCHNAME TVNAME TABLE_TYPE COMMENT$

---------- ------- ------ ---------- --------

1          SYSDBA  TESTT TABLE      COMMENT IN ALTER
```

3.3.2.2 对象权限的回收

可以使用 REVOKE 语句回收授出的指定数据库对象的指定权限。

对象权限的回收语句语法为：

```
REVOKE [GRANT OPTION FOR] <特权>[OFF]  ON <对象> FROM <用户或角色> {,<用户或角色>} [<回

收选项>];

<特权>::= ALL [PRIVILEGES] |

              <动作> {, <动作>}+

<动作>::= SELECT |

              INSERT |

              UPDATE |

              DELETE |

              REFERENCES |

              EXECUTE |

              READ |

              WRITE |

              USAGE |

              INDEX |

              ALTER |

              SELECT FOR DUMP |

              FLASHBACK
```

<对象> ::= SCHEMA <模式名> |

DIRECTORY <目录名> |

[<对象类型>] [<模式名>.] <对象名>

<对象类型>::= TABLE | VIEW | PROCEDURE | FUNCTION | PACKAGE | CLASS | TYPE | SEQUENCE | DOMAIN | SYNONYM | LIBRARY

<对象名> ::= <表名> | <视图名> | <存储过程/函数名> |<包名> |<类名> |<类型名> | <序列名> | <域名> | <同义词名>|<LIBRARY 名>

<用户或角色>::= <用户名> | <角色名>

<回收选项> ::= RESTRICT | CASCADE

使用说明:

1. 权限回收者必须是具有回收相应对象权限以及转授权的用户;
2. 使用 INI 参数 OBJECT_PRIV_MODE 可控制权限回收策略是否兼容 ORACLE;
3. V8.1.4.109 版本之后，拥有特权 GRANT ANY OBJECT PRIVILEGE 的用户，能够回收任意对象权限；降级至 V8.1.4.107 及更早的版本，降级前由拥有特权 GRANT ANY OBJECT PRIVILEGE 或者 GRANT ANY XX（如 GRANT ANY TABLE/VIEW/PROCEDURE 等）或者特例授予的权限，只能由对象所属用户进行回收;
4. <对象类型>为 SCHEMA 时，须提前设置 INI 参数 GRANT_SCHEMA=1 或 2，即开启授予和回收模式权限功能;
5. 回收时不能带列清单，若对象上存在同类型的列权限，则一并被回收;
6. 使用 GRANT OPTION FOR 选项的目的是收回用户或角色权限转授的权利，而不回收用户或角色的权限;

7. <回收选项>分为 CASCADE 和 RESTRICT。不同的回收选项，其意义不同。具体如下：

- CASCADE 表示级联回收，可回收授予时带或不带 WITH GRANT OPTION 的权限，若带 WITH GRANT OPTION 还会引起级联回收。级联回收指不仅收回该用户或角色直接拥有的权限，还同时回收所有因该用户或角色而间接获得这些权限的其他用户或角色的权限。例如：A 将 SELECT 授权（选择了 WITH GRANT OPTION）给 B，B 转授给 C，则使用 CASCADE 选项收回 A 的 SELECT 权限时，会将 B 和 C 的 SELECT 权限一同收回。利用此选项时不会检查权限是否存在依赖等限制。另外，利用此选项进行级联回收时，若被回收对象上存在另一条路径授予同样权限给该对象时，则仅需回收当前权限。
- RESTRICT 表示限制回收，只能回收授予时不带 WITH GRANT OPTION 的权限。无法回收授予时带 WITH GRANT OPTION 的权限，也无法回收存在依赖等限制的权限（如角色上的某权限被别的用户用于创建视图等）。
- 缺省<回收选项>，表示无法回收授予时带 WITH GRANT OPTION 的权限，但也不会检查要回收的权限是否存在依赖等限制。



说明： **用户 A 给用户 B 授权且允许其转授，B 将权限转授给 C。当 A 回收 B 的权限的时候必须加 CASCADE 回收选项。**

例 1 接 3.3.2.1 节例 1，SYSDBA 从用户 BOOKSHOP_USER1 处回收其授出的 PERSON.ADDRESS 表的全部权限。

```
REVOKE ALL PRIVILEGES ON PERSON.ADDRESS FROM BOOKSHOP_USER1 CASCADE;
```

例 2 接 3.3.2.1 节例 2，SYSDBA 从用户 BOOKSHOP_USER2 处回收其授出的存储过程 BOOKSHOP_USER1_PROC1 的 EXECUTE 权限。

```
REVOKE EXECUTE ON PROCEDURE BOOKSHOP_USER1.BOOKSHOP_USER1_PROC1
FROM BOOKSHOP_USER2 CASCADE;
```

例 3 假定系统中存在非 SYSDBA 用户 BOOKSHOP_USER1、BOOKSHOP_USER2、BOOKSHOP_USER3 和 BOOKSHOP_USER4，其中 BOOKSHOP_USER1 具有 CREATE TABLE 数据库权限，BOOKSHOP_USER2 具有 CREATE VIEW 数据库权限，且已成功执行了如下的语句：

```
BOOKSHOP_USER1 登录执行:
CREATE TABLE T1(ID INT, NAME VARCHAR(100));
GRANT SELECT ON T1 TO PUBLIC;
GRANT INSERT(ID) ON T1 TO BOOKSHOP_USER2 WITH GRANT OPTION;
BOOKSHOP_USER2 登录执行:
CREATE VIEW V1 AS SELECT NAME FROM BOOKSHOP_USER1.T1 WHERE ID < 10;
GRANT INSERT(ID) ON BOOKSHOP_USER1.T1 TO BOOKSHOP_USER3 WITH GRANT OPTION;
BOOKSHOP_USER3 登录执行:
GRANT INSERT(ID) ON BOOKSHOP_USER1.T1 TO BOOKSHOP_USER4 WITH GRANT OPTION;
BOOKSHOP_USER4 登录执行:
GRANT INSERT(ID) ON BOOKSHOP_USER1.T1 TO BOOKSHOP_USER2 WITH GRANT OPTION;
```

此时，若 BOOKSHOP_USER1 执行以下语句会引起之前授予的 INSERT(ID)权限被合并。

```
GRANT INSERT ON T1 TO BOOKSHOP_USER2 WITH GRANT OPTION;
```

若 BOOKSHOP_USER1 执行以下语句，会成功，因为这种方式不检查权限是否存在限制。

```
REVOKE SELECT ON T1 FROM PUBLIC;
```

若 BOOKSHOP_USER1 执行以下语句，则会失败，因为这种方式检查权限是否存在限制，即 BOOKSHOP_USER2 利用此权限创建了视图 V1。

```
REVOKE SELECT ON T1 FROM PUBLIC RESTRICT;
```

若 BOOKSHOP_USER1 执行以下语句，会进行级联回收 BOOKSHOP_USER2、BOOKSHOP_USER3、BOOKSHOP_USER4 授出的 INSERT(ID)权限。

```
REVOKE INSERT ON T1 FROM BOOKSHOP_USER2 CASCADE;
```

但若 BOOKSHOP_USER1 连续执行了以下两条语句，则后一条语句仅回收其授予 BOOKSHOP_USER2 的权限，而不会产生级联回收，因为有另一条路经授予了 BOOKSHOP_USER3 同样的权限 INSERT(ID)。

```
GRANT INSERT ON T1 TO BOOKSHOP_USER3 WITH GRANT OPTION;
REVOKE INSERT ON T1 FROM BOOKSHOP_USER2 CASCADE;
```

例 4 接 3.3.2.1 节例 7，回收黑名单权限。

```
//回收黑名单权限。此时 USER02 重新获得 USER01 的 T1 的查询权限
REVOKE SELECT OFF ON USER01.T1 FROM USER02;
```

3.3.3 角色权限的分配与回收

3.3.3.1 角色权限的分配

通常角色包含权限或其他角色，通过使用 GRANT 语句将一个角色授予用户或另一角色可以使得用户和角色继承该角色所具有的权限。

授予角色权限的语句语法如下：

```
GRANT <角色名>{, <角色名>} TO <用户或角色>{,<用户或角色>} [WITH ADMIN OPTION];
<用户名或角色名> ::= <用户名> | <角色名>
```

使用说明：

1. 角色的授予者必须为拥有相应的角色以及其转授权的用户。但有一点例外：四权分立下，SYSDBA、SYSDBO 虽然没有 ADMIN ANY ROLE 权限，但依然可以对其他

用户或角色授予 SVI（由 SYSDBA 授予）、DB_OBJECT_SVI（由 SYSDBO 授予）角色权限；

2. 接受者必须与授权者类型一致(譬如不能把审计角色授予标记角色)；
3. WITH ADMIN OPTION 表示可以转授，即被授权者可以把这些权限继续授予给其他用户或角色，缺省不能转授；
4. 不支持角色的循环转授，如将 BOOKSHOP_ROLE1 授予 BOOKSHOP_ROLE2，BOOKSHOP_ROLE2 不能再授予 BOOKSHOP_ROLE1；
5. 支持对系统角色授权系统权限。但是不支持再为 PUBLIC 系统角色授予在 dminit 初始化时已经被授予的系统权限（INSERT TABLE、UPDATE TABLE、DELETE TABLE、SELECT TABLE、DUMP TABLE、REFERENCE TABLE、GRANT TABLE、INSERT VIEW、UPDATE VIEW、DELETE VIEW、SELECT VIEW、GRANT VIEW、EXECUTE PROCEDURE、GRANT PROCEDURE、SELECT SEQUENCE、GRANT SEQUENCE、EXECUTE PACKAGE、GRANT PACKAGE、SELECT MATERIALIZED VIEW、GRANT DOMAIN、USAGE DOMAIN），也不支持为 PUBLIC 角色授予 SVI 角色。不支持为 DBA 角色授予 VTI 角色。

例假设存在角色 BOOKSHOP_ROLE1，角色 BOOKSHOP_ROLE2，用户 BOOKSHOP_USER1。

```
//让用户 BOOKSHOP_USER1 继承角色 BOOKSHOP_ROLE1 的权限
GRANT BOOKSHOP_ROLE1 TO BOOKSHOP_USER1;
//让角色 BOOKSHOP_ROLE2 继承角色 BOOKSHOP_ROLE1 的权限
GRANT BOOKSHOP_ROLE1 TO BOOKSHOP_ROLE2;
```

3.3.3.2 角色权限的回收

可以使用 REVOKE 语句回收用户或其它角色从指定角色继承过来的权限。

回收角色权限的语句语法为：

```
REVOKE [ADMIN OPTION FOR] <角色名>{,<角色名>} FROM <角色名或用户名>;
<角色名或用户名> ::= <用户名> | <角色名>
```

使用说明：

1. 权限回收者必须是具有回收相应角色以及转授权的用户。对于角色授权时使用 WITH ADMIN OPTION 先给用户 1 授权，然后又转授用户 2 的情况，回收用户 1 的角色时不会级联回收用户 2 的角色；
2. 使用 ADMIN OPTION FOR 选项的目的是收回用户或角色权限转授的权利，而不回收用户或角色的权限；
3. 支持对系统角色回收系统权限。但是不支持回收 PUBLIC 系统角色中在 dminit 初始化时已经被授予的系统权限，也不支持回收 PUBLIC 角色的 SVI 角色。不支持回收 DBA 角色的 VTI 角色。

例 接 3.3.3.1 节的例子，回收用户 BOOKSHOP_USER1 和角色 BOOKSHOP_ROLE2 的 BOOKSHOP_ROLE1 角色。

```
REVOKE BOOKSHOP_ROLE1 FROM BOOKSHOP_USER1;
REVOKE BOOKSHOP_ROLE1 FROM BOOKSHOP_ROLE2;
```

3.3.4 模式权限的分配与回收

模式权限是模式级别的对象权限，又称为模式级别对象的 ANY 特权。

若要使用模式权限的分配与回收功能，须先开启 INI 参数 GRANT_SCHEMA=1 或 2。

3.3.4.1 模式权限的分配

为用户赋予模式级别的对象 ANY 特权后，该用户拥有对该模式用户下该类对象的 ANY 权限。

模式级别对象的 ANY 特权授予语法如下：

```
GRANT <模式权限> ON SCHEMA <模式> TO <用户或角色>{,<用户或角色>} [WITH ADMIN OPTION];

<模式权限>::=ALL [PRIVILEGES] | <ANY 特权>

<ANY 特权>::= SELECT ANY TABLE |
              INSERT ANY TABLE |
              UPDATE ANY TABLE |
              DELETE ANY TABLE |
              REFERENCES ANY TABLE |
              DUMP ANY TABLE |
              FLASHBACK ANY TABLE
              CREATE ANY TABLE |
              ALTER ANY TABLE |
              DROP ANY TABLE |
              SELECT ANY VIEW |
              INSERT ANY VIEW |
              UPDATE ANY VIEW |
              DELETE ANY VIEW |
              CREATE ANY VIEW |
              ALTER ANY VIEW |
              DROP ANY VIEW |
              SELECT ANY MATERIALIZED VIEW |
              CREATE ANY MATERIALIZED VIEW |
              ALTER ANY MATERIALIZED VIEW |
              DROP ANY MATERIALIZED VIEW |
```

CREATE ANY INDEX |

ALTER ANY INDEX |

DROP ANY INDEX |

CREATE ANY CONTEXT INDEX |

ALTER ANY CONTEXT INDEX |

DROP ANY CONTEXT INDEX |

SELECT ANY SEQUENCE |

CREATE ANY SEQUENCE |

ALTER ANY SEQUENCE |

DROP ANY SEQUENCE |

EXECUTE ANY PROCEDURE |

CREATE ANY PROCEDURE |

DROP ANY PROCEDURE |

EXECUTE ANY PACKAGE |

CREATE ANY PACKAGE |

DROP ANY PACKAGE |

USAGE ANY DOMAIN |

CREATE ANY DOMAIN |

DROP ANY DOMAIN |

CREATE ANY TRIGGER |

ALTER ANY TRIGGER |

DROP ANY TRIGGER |

CREATE ANY SYNONYM |

DROP ANY SYNONYM

<用户名或角色名> ::= <用户名> | <角色名>

使用说明:

1. 授权者必须是具有对应对象权限以及其转授权的用户;
2. 授权用户需要具有对授权模式的"GRANT ANY OBJECT PRIVILEGE"权限;

3. 设置 INI 参数 GRANT_SCHEMA=2 时，ALL 或 ALL PRIVILEGES 表示将指定模式的上述所有 ANY 特权都授予被授予者；
4. WITH ADMIN OPTION 表示可以转授，即被授权者可以把这些权限继续授予给其他用户或角色，缺省不能转授。

例 1 通过 SCHEMA 级别授权，将用户 USER227 查询任何表的权限给到新的用户 USER228。

```
--新建用户 USER227
CREATE USER USER227 IDENTIFIED BY Dmsys_123;
GRANT "PUBLIC","RESOURCE","SOI","SVI","VTI" TO USER227;
--新建另外一个新的用户 USER228
CREATE USER USER228 IDENTIFIED BY Dmsys_456;
GRANT "PUBLIC","RESOURCE","SOI","SVI","VTI" TO USER228;
--通过 SCHEMA 级别授权，将用户 USER227 查询任何表的权限给到新的用户 USER228。
GRANT SELECT ANY TABLE ON SCHEMA USER227 TO USER228;
--登录 USER227 用户创建表
CREATE TABLE TEST1(C1 INT);
INSERT INTO TEST1 VALUES(1);
COMMIT;
--登录 USER228 用户查询到 user227 上的表
SELECT * FROM USER227.TEST1;
```

3.3.4.2 模式权限的回收

可以使用 REVOKE 语句回收授出的指定模式级别对象的 ANY 特权。

模式级别对象 ANY 特权的回收语句语法为：

REVOKE [GRANT ADMIN FOR] <模式权限> ON SCHEMA <模式名> FROM <用户或角色> {,<用户或角色>} [<回收选项>];

<模式权限>::=ALL [PRIVILEGES] | <ANY 特权>

<ANY 特权>::= SELECT ANY TABLE |

INSERT ANY TABLE |

UPDATE ANY TABLE |

DELETE ANY TABLE |

REFERENCES ANY TABLE |

DUMP ANY TABLE |

FLASHBACK ANY TABLE

CREATE ANY TABLE |

ALTER ANY TABLE |

DROP ANY TABLE |

SELECT ANY VIEW |

INSERT ANY VIEW |

UPDATE ANY VIEW |

DELETE ANY VIEW |

CREATE ANY VIEW |

ALTER ANY VIEW |

DROP ANY VIEW |

SELECT ANY MATERIALIZED VIEW |

CREATE ANY MATERIALIZED VIEW |

ALTER ANY MATERIALIZED VIEW |

DROP ANY MATERIALIZED VIEW |

CREATE ANY INDEX |

ALTER ANY INDEX |

DROP ANY INDEX |

CREATE ANY CONTEXT INDEX |

ALTER ANY CONTEXT INDEX |

```
                DROP ANY CONTEXT INDEX |
                SELECT ANY SEQUENCE |
                CREATE ANY SEQUENCE |
                ALTER ANY SEQUENCE |
                DROP ANY SEQUENCE |
                EXECUTE ANY PROCEDURE |
                CREATE ANY PROCEDURE |
                DROP ANY PROCEDURE |
                EXECUTE ANY PACKAGE |
                CREATE ANY PACKAGE |
                DROP ANY PACKAGE |
                USAGE ANY DOMAIN |
                CREATE ANY DOMAIN |
                DROP ANY DOMAIN |
                CREATE ANY TRIGGER |
                ALTER ANY TRIGGER |
                DROP ANY TRIGGER |
                CREATE ANY SYNONYM |
                DROP ANY SYNONYM
<用户名或角色名> ::= <用户名> | <角色名>
<回收选项> ::= RESTRICT | CASCADE
```

使用说明:

1. 权限回收者必须是具有回收相应角色以及转授权的用户;
2. 使用 ADMIN OPTION FOR 选项的目的是收回用户或角色权限转授的权利，而不回收用户或角色的权限;
3. 设置 INI 参数 GRANT_SCHEMA=2 时，ALL 或 ALL PRIVILEGES 表示将指定模式的上述所有 ANY 特权都授予被授予者;

4. <回收选项>分为 CASCADE 和 RESTRICT。不同的回收选项，其意义不同。具体如下：

- CASCADE 表示级联回收，可回收授予时带或不带 WITH ADMIN OPTION 的权限，若带 WITH ADMIN OPTION 还会引起级联回收。利用此选项时不会检查权限是否存在依赖等限制。另外，利用此选项进行级联回收时，若被回收对象上存在另一条路径授予同样权限给该对象时，则仅需回收当前权限。
- RESTRICT 表示限制回收，只能回收授予时不带 WITH ADMIN OPTION 的权限。无法回收授予时带 WITH ADMIN OPTION 的权限，也无法回收存在依赖等限制的权限（如角色上的某权限被别的用户用于创建视图等）。
- 缺省<回收选项>，表示无法回收授予时带 WITH ADMIN OPTION 的权限，但也不会检查要回收的权限是否存在依赖等限制。

例 1 接 3.3.4.1 节例 1，SYSDBA 从用户 USER228 处回收其授出的 SELECT ANY TABLE 权限。

```
REVOKE SELECT ANY TABLE ON SCHEMA USER227 FROM USER228 CASCADE;
```

4 强制访问控制

强制访问控制(Mandatory Access Control, MAC)是根据客体的敏感标记和主体的访问标记对客体访问实行限制的一种方法。在强制访问控制中，系统给主体和客体都分配一个特殊的安全标记，主体的安全标记反映了该主体可信的程度，客体的安全标记则与其包含信息的敏感度一致，且主体不能改变他自己及任何其它客体的安全标记，主体是否可以对客体执行特定的操作取决于主体和客体的安全标记之间的支配关系。因此，强制访问控制可以控制系统中信息流动的轨迹，能有效地抵抗特洛伊木马的攻击，这在一些对安全要求很高的数据库应用中是非常必要的。

DM 利用策略和标记来实现 DM 数据库的强制访问控制。执行强制访问控制的用户必须具有 LABEL DATABASE 数据库权限，在新初始化的数据库中，只有 SYSSSO 具有这个权限。

强制访问控制功能仅在 DM 安全版中提供。只有初始化库时指定 ENABLE_MAC=1 或启动 DM 实例时指定过 ENABLE_MAC 参数的情况下才支持强制访问控制。DM 数据共享集群 DMDSC、DM 数据守护和读写分离集群都支持强制访问控制，但 DM 大规模并行处理集群 MPP 暂不支持强制访问控制。

4.1 如何创建策略

4.1.1 策略的组成

策略是一组预定义的标记组件，包括等级、范围和组这三种组件，三种组件分别从不同的维度对数据进行了描述。一个策略最多只能包含这三种组件，其中必须包含等级，范围和组可以缺省。

- **等级（Level）**

等级是线性有序的名称序列，用 L=（l1，l2，…，lp）表示。其中 li（1≤i≤p）表示第 i 个名称，任意两个名称 li、lj 之间，若 i≤j，则 li≤lj，于是有 l1≤l2≤…≤lp，其中 l1，l2，…，lp 称为等级分类（以下简称等级）。

在 DM 中，一个策略最大可定义 10000 个等级。用户在定义策略中的等级时，需要为其指定编号，其编号在 0—9999 之间（编号小的意味着级别较低）。

- **范围（Compartment）**

范围是集合类型，设集合 C={c1，c2，…，cm}中每一元素都是一名称，c1，c2，…，cm 间彼此独立，无序，则集合 C 及其任意子集称为非等级类别集合，其中 c1，c2，…，cm 称为非等级类别（以下简称为范围）。

在 DM 中，最大可定义 10000 个范围，需要用户设置编号，且编号在一个策略里面须是唯一的，编号之间没有级别高低之分。

- **组（Group）**

组为树形结构，有父子之分，可以用来描述组织结构。

设树 G={g1，g2，…，gm}，其中每一元素都是一名称，g1，g2，…，gm 间有父子之分，则 g1，g2，…，gm 称为组。

在 DM 中，最多能定义 10000 个组，也就是说组织结构的层次最多为 10000。组中只能有一个根组，除根组外，每个组有且仅有一个父组。

4.1.2 创建、修改与删除策略

4.1.2.1 创建策略

使用如下系统过程创建一个策略。

```
VOID
MAC_CREATE_POLICY(
```

```
    POLICY_NAME             VARCHAR(128)
);
```

参数说明:

POLICY_NAME: 新创建的策略名称

使用说明:

只有具有 LABEL DATABASE 数据库权限的用户才能执行此操作。

例 创建策略 P_01。

```
MAC_CREATE_POLICY('P_01');
```

4.1.2.2 修改策略

可以使用下面的系统过程修改一个策略的策略名。

```
VOID
MAC_ALTER_POLICY(
    POLICY_NAME             VARCHAR(128),
    NEW_NAME                VARCHAR(128)
);
```

参数说明:

POLICY_NAME: 待修改的策略名称

NEW_NAME : 将修改成的策略名称

使用说明:

只有具有 LABEL DATABASE 数据库权限的用户才能执行此操作。

例 将策略 P_01 改名为 P_02。

```
MAC_ALTER_POLICY('P_01', 'P_02');
```

4.1.2.3 删除策略

可以使用下面的系统过程删除一个已存在的策略。

```
VOID
MAC_DROP_POLICY(
        POLICY_NAME             VARCHAR(128),
        DROP_COLUMN                     INT
);
```

参数说明:

POLICY_NAME: 待删除的策略名称

DROP_COLUMN: 对应用此策略的表的标记列的处理方式，取值为 0 或 1

0: 不删除应用此策略的所有表对应的标记列

1: 删除应用此策略的所有表对应的标记列

若 DROP_COLUMN 为 NULL，则按默认为 0 进行处理

使用说明:

1. 只有具有 LABEL DATABASE 数据库权限的用户才能执行此操作;
2. 指定待删除策略必须存在。

例 删除策略 P_02。

```
MAC_DROP_POLICY('P_02');
```

4.1.3 为策略添加组件

创建一个策略后，需要对策略添加组件，这样策略才可以被应用到表和用户上。

4.1.3.1 为策略添加等级

使用下面的系统过程为指定的策略添加等级。

```
VOID
MAC_CREATE_LEVEL(
        POLICY_NAME             VARCHAR(128),
        LEVEL_NUM               INT,
        LEVEL_NAME              VARCHAR(128)
);
```

参数说明：

POLICY_NAME：要添加等级的策略名

LEVEL_NUM：创建的等级编号，在 0—9999 之间的整数

LEVEL_NAME：创建的等级名称

使用说明：

1. LEVEL_NAME 不能包含":"和",";
2. 该过程只能由具有 LABEL DATABASE 的用户调用；
3. 指定策略必须存在；
4. 同一个策略中，等级 ID 和等级名称唯一；
5. 一个策略最多可以定义 10000 个等级；
6. 每个等级都要有一个等级 ID，ID 越小表示安全等级越低。

例 创建策略 P_03，并给策略 P_03 添加等级 L_01，等级编号为 10。

```
MAC_CREATE_POLICY('P_03');
MAC_CREATE_LEVEL('P_03', 10, 'L_01');
```

在为策略添加等级后，还可以通过系统过程修改等级名称和删除等级。

■ **修改等级**

修改等级名称的系统过程如下：

```
VOID
MAC_ALTER_LEVEL(
```

```
    POLICY_NAME          VARCHAR(128),
    LEVEL_NAME           VARCHAR(128),
    NEW_NAME             VARCHAR(128)
);
```

参数说明:

POLICY_NAME: 要修改等级名的策略名

LEVEL_NAME: 待修改的等级名称

NEW_NAME : 要修改成的等级名称

使用说明:

1. NEW_LEVEL 不能包含":"和",";
2. 该过程只能由具有 LABEL DATABASE 的用户调用;
3. 待修改等级名必须存在。

例 将策略 P_03 中的等级 L_01，更名为 L_02。

```
MAC_ALTER_LEVEL('P_03', 'L_01', 'L_02');
```

- **删除等级**

删除等级的系统过程如下:

```
VOID
MAC_DROP_LEVEL(
    POLICY_NAME          VARCHAR(128),
    LEVEL_NAME                   VARCHAR(128)
);
```

参数说明:

POLICY_NAME: 待删除等级所在策略名

LEVEL_NAME: 待删除的等级名称

使用说明:

1. 该过程只能由具有 LABEL DATABASE 的用户调用;
2. 指定等级必须存在;
3. 如果待删除等级被某个标记使用，则拒绝删除。

例 删除策略 P_03 中的等级 L_02。

```
MAC_DROP_LEVEL('P_03', 'L_02');
```

4.1.3.2 为策略添加范围

使用下面的系统过程为指定的策略添加范围。

```
VOID
MAC_CREATE_COMPARTMENT(
        POLICY_NAME             VARCHAR(128),
        COMPART_NUM             INT,
        COMPART_NAME                    VARCHAR(128)
);
```

参数说明:

POLICY_NAME: 要添加范围的策略名

COMPART_NUM: 创建的范围编号，在 0—9999 之间的整数

COMPART_NAME: 创建的范围名称

使用说明:

1. COMPART_NAME 不能包含":"和",";
2. 该过程只能由具有 LABEL DATABASE 的用户调用;
3. 指定策略必须存在;
4. 同一个策略中，范围 ID 和范围名称唯一;

5. 一个策略中最多可以定义 10000 个范围；

6. 范围独立无序，范围之间是平等关系，没有等级高低之分，范围之间的比较运算采用集合间的包含关系。

例 给策略 P_03 添加范围 C_01。

```
MAC_CREATE_COMPARTMENT('P_03',10, 'C_01');
```

在为策略添加范围后，还可以通过系统过程修改范围名称和删除范围。

■ **修改范围**

修改范围名称的系统过程如下：

```
VOID
MAC_ALTER_COMPARTMENT(
        POLICY_NAME             VARCHAR(128),
        COMPART_NAME                    VARCHAR(128),
        NEW_NAME                VARCHAR(128)
);
```

参数说明：

POLICY_NAME：要修改范围名的策略名

COMPART_NAME：待修改的范围名称

NEW_NAME　　：要修改成的范围名称

使用说明：

1. NEW_NAME 不能包含":"和","；

2. 该过程只能由具有 LABEL DATABASE 的用户调用；

3. 待修改范围名必须存在。

例 将策略 P_03 中的范围 C_01，更名为 C_02。

```
MAC_ALTER_COMPARTMENT('P_03', 'C_01', 'C_02');
```

■ **删除范围**

删除范围的系统过程如下：

```
VOID
MAC_DROP_COMPARTMENT(
        POLICY_NAME             VARCHAR(128),
        COMPART_NAME                    VARCHAR(128)
);
```

参数说明：

POLICY_NAME：待删除范围所在策略名

COMPART_NAME：待删除的范围名称

使用说明：

1. 该过程只能由具有 LABEL DATABASE 的用户调用；
2. 指定范围必须存在；
3. 如果待删除范围被某个标记使用，则拒绝删除。

例 删除策略 P_03 中的范围 C_02。

```
MAC_DROP_COMPARTMENT('P_03', 'C_02');
```

4.1.3.3 为策略添加组

使用下面的系统过程为指定的策略添加组。

```
VOID
MAC_CREATE_GROUP(
        POLICY_NAME             VARCHAR(128),
        GROUP_NUM               INT,
        GROUP_NAME              VARCHAR(128),
        PARENT_NAME             VARCHAR(128)
```

```
);
```

参数说明：

POLICY_NAME：要添加组的策略名

GROUP_NUM：创建的组编号，在 0—9999 之间的整数

GROUP_NAME：创建的组名称

PARENT_NAME：新创建组的父组的名称

使用说明：

1. GROUP_NAME 不能包含":"和","；
2. 该过程只能由具有 LABEL DATABASE 的用户调用；
3. 指定策略必须存在；
4. 同一个策略中，组 ID 和组名称唯一；
5. 一个策略最多可以定义 10000 个组；
6. 同一个策略中，只能有一个根组，如果 PARENT_NAME 为 NULL，则创建根组；
7. 组之间的比较运算采用树形结构间的从属关系。

例 给策略 P_03 创建根组 G_01；再以 G_01 为父组，创建组 G_02；以 G_02 为父组，创建组 G_03。

```
MAC_CREATE_GROUP ('P_03',10, 'G_01',NULL);
MAC_CREATE_GROUP ('P_03',20, 'G_02', 'G_01');
MAC_CREATE_GROUP ('P_03',30, 'G_03', 'G_02');
```

在为策略添加组后，还可以通过系统过程修改组名称、更新父组和删除组。

■ **修改组**

修改组名称的系统过程如下：

```
VOID
```

```
MAC_ALTER_GROUP(
        POLICY_NAME             VARCHAR(128),
        GROUP_NAME              VARCHAR(128),
        NEW_NAME                VARCHAR(128)
);
```

参数说明:

POLICY_NAME: 要修改组名的策略名

GROUP_NAME: 待修改的组名称

NEW_NAME : 要修改成的组名称

使用说明:

1. NEW_NAME 不能包含":"和",";
2. 该过程只能由具有 LABEL DATABASE 的用户调用;
3. 待修改组名必须存在。

例 将策略 P_03 中的组 G_03，更名为 G_04。

```
MAC_ALTER_GROUP('P_03', 'G_03', 'G_04');
```

■ **更新父组**

更新父组的系统过程如下:

```
VOID
MAC_ALTER_GROUP_PARENT(
        POLICY_NAME             VARCHAR(128),
        GROUP_NAME              VARCHAR(128),
        PARENT_NAME             VARCHAR(128)
);
```

参数说明:

POLICY_NAME: 要更新父组的组所在的策略名

GROUP_NAME：待更新父组的组名称

PARENT_NAME：待修改成的父组名称

使用说明：

1. 该过程只能由具有 LABEL DATABASE 的用户调用；
2. 待修改组和待修改成的父组必须存在；
3. 父组不能是自身，同时不能是自己的子节点。

例 将策略 P_03 中的组 G_04，更换父组为 G_01。

```
MAC_ALTER_GROUP_PARENT ('P_03', 'G_04', 'G_01');
```

- **删除组**

删除组的系统过程如下：

```
VOID
MAC_DROP_GROUP (
        POLICY_NAME             VARCHAR(128),
        GROUP_NAME              VARCHAR(128),
);
```

参数说明：

POLICY_NAME：待删除组所在策略名

GROUP_NAME：待删除的组名称

使用说明：

1. 该过程只能由具有 LABEL DATABASE 的用户调用；
2. 指定组必须存在；
3. 待删除的组不能有子节点存在，否则删除失败。

例 删除策略 P_03 中的组 G_04。

```
MAC_DROP_GROUP('P_03', 'G_04');
```

4.2 如何创建标记

当把策略应用于用户或表时，那么该用户或表就获得了一个安全标记。一个安全标记由多个组件组成，其组件包括等级、范围和组。每个标记必须包含一个等级，范围和组则是可选的。

在 DM 中，标记用字符串表示，其最大长度为 4000。格式如下：

```
<等级>:[<范围>{,<范围>}]:[<组>{,<组>}];
```

假定系统中存在策略 P1，其包含等级 L1,L2,L3,范围 C1,C2,C3,C4,组 G1,G2,G3。则下列标记为合法标记：

```
L1::
L1:C1:
L1:C1,C2:
L1::G1
L1:C1:G1,G2
```

而下列标记为不合法标记：

```
:
:C1,C2
C1,C2
:C1:G1
::G1,G2
G1,G2
```

标记以二进制位的形式存储于数据库中。当一个策略被应用到表上时，需要指定标记的存储列名，此列的类型为 INT，此时表上每条元组的标记均存在于该列中。

在应用策略时，还可设置标记列是否被隐藏。当标记列被隐藏时，若用户在插入数据时不指定列清单，则在值列表中可以不设置该列的值，另外，在查询时也不显示该列数据。

标记列上可以建索引，设置列约束、改列名等，但不能修改列类型、设置缺省值等。标记列不能被用户显式删除，除非在表上取消该策略。

4.2.1 创建标记

可使用下面的系统过程创建一个标记。

```
VOID
MAC_CREATE_LABEL(
        POLICY_NAME             VARCHAR(128),
        LABEL_TAG               INT,
        LABEL_VALUE             VARCHAR(4000)
);
```

参数说明：

POLICY_NAME：要创建的标记所在的策略名

LABEL_TAG：标记值，有效范围为 0~999999999

LABEL_VALUE：标记串

使用说明：

1. 该过程只能由具有 LABEL DATABASE 的用户调用；
2. 指定策略必须存在；
3. 根据标记值和标记串，生成一个标记，若此标记已经存在，则返回错误。标记值由用户自己指定，且标记值在系统中应是唯一的，不同的策略中也不能使用相同的标记值。

例 创建标记 L_01:C_01:G_02。

```
MAC_CREATE_POLICY('P_04');
MAC_CREATE_LEVEL ('P_04',100, 'L_01');
MAC_CREATE_COMPARTMENT ('P_04',100, 'C_01');
```

```
MAC_CREATE_GROUP ('P_04',100, 'G_01',NULL);
MAC_CREATE_GROUP ('P_04',200, 'G_02', 'G_01');
MAC_CREATE_LABEL('P_04',11, 'L_01:C_01:G_02');
```

上述例子执行后，将在 SYSMACLABELS 系统表中记录 TAG 值以及标记字符串'L1:C1:G1'的内部格式，这个格式是对字符串'L1:C1:G1'进行处理后的格式。

如果要查看系统中某个标记对应的具体的值，例如，要查看刚刚创建的 ID 为 11 的标记的具体值，可以使用如下系统函数：

```
SF_MAC_LABEL_TO_CHAR(11);
```

此系统函数的参数为标记的标记值，返回该标记对应的标记串。

4.2.2 修改标记

可根据标记的标记值或标记串对标记修改其标记串，使用如下的系统过程：

```
VOID
MAC_ALTER_LABEL(
        POLICY_NAME             VARCHAR(128),
        LABEL_TAG               INT,
        NEW_LABEL_VALUE         VARCHAR(4000)
);
VOID
MAC_ALTER_LABEL(
        POLICY_NAME             VARCHAR(128),
        LABEL_VALUE             VARCHAR(4000),
        NEW_LABEL_VALUE         VARCHAR(4000)
);
```

参数说明：

POLICY_NAME：待修改标记所在策略名

LABEL_TAG：待修改标记的标记值

LABEL_VALUE：待修改标记的标记串

NEW_LABEL_VALUE：要修改为的标记串

使用说明：

1. 该过程只能由具有 LABEL DATABASE 的用户调用；
2. 指定策略必须存在；
3. 新的标记串必须是不存在的标记串，如果存在则报错；
4. 修改标记的用处主要在于可以不用更新表中的标记列，而直接修改标记串，来改变原始数据的安全级别；
5. 修改标记的标记串，如果新的标记串为 NULL，则保持原始值；
6. 如果修改的标记同时应用在用户上，那么用户的标记合法性可能会遭到破坏。

例 更改标记 L_01:C_01:G_02。

```
MAC_ALTER_LABEL('P_04',11, 'L_01:C_01:G_01,G_02');
MAC_ALTER_LABEL('P_04', 'L_01:C_01:G_01,G_02', 'L_01:C_01:');
```

4.2.3 删除标记

可根据标记的标记值或标记串删除标记，使用如下的系统过程：

```
VOID
MAC_DROP_LABEL(
        POLICY_NAME             VARCHAR(128),
        LABEL_TAG               INT
);
VOID
MAC_DROP_LABEL(
        POLICY_NAME             VARCHAR(128),
        LABEL_VALUE             VARCHAR(4000)
);
```

参数说明：

POLICY_NAME：待删除标记所在策略名

LABEL_TAG：待删除标记的标记值

LABEL_VALUE：待删除标记的标记串

使用说明：

1. 该过程只能由具有 LABEL DATABASE 的用户调用；
2. 指定策略必须存在；
3. 待删除的标记必须是已存在的标记，否则报错；
4. 标记可以进行删除，即使标记被应用在表或用户上，这样会导致表或用户的标记失效，使用时需注意。

例 创建标记之后删除。

```
MAC_CREATE_LABEL('P_04',12, 'L_01::');
MAC_DROP_LABEL('P_04',12);
MAC_CREATE_LABEL('P_04',13, 'L_01::G_01');
MAC_DROP_LABEL('P_04', 'L_01::G_01');
```

4.2.4 隐式创建标记

可以使用下面的系统过程隐式创建标记。

```
VOID
SP_MAC_LABEL_FROM_CHAR(
        POLICY_NAME             VARCHAR(128),
        LABEL_VALUE             VARCHAR(4000),
        TAG                     INT
);
```

参数说明：

POLICY_NAME：标记所在的策略名

LABEL_VALUE：标记串

标记值：输出参数，标记串对应的标记值

使用说明：

1. 指定策略必须存在；

这个系统过程首先会去系统表 SYSMACLABELS 查找是否具有指定标记串的标记，如果存在，则返回相应的标记值，如果不存在，系统会自动创建一个对应的标记，并返回给用户相应的标记值，标记值由系统自动生成。

DM 的 INI 参数 MAC_LABEL_OPTION 用于控制 SP_MAC_LABEL_FROM_CHAR 过程的使用范围，有三种取值：

- 0：只有 SSO 可以调用
- 1：所有用户都可以调用
- 2：所有用户可以调用，但是非 SSO 用户不会主动创建新的 LABEL

例 1 隐式创建标记，标记串不存在的情况。

```
DECLARE
TAG INT;
BEGIN
SP_MAC_LABEL_FROM_CHAR ('P_04', 'L_01::G_02', TAG);
PRINT TAG ;
END;
/
```

由于之前系统中没有指定标记串的标记，此时会为这个标记串创建一个标记，返回系统自动生成的 TAG 值。

例 2 隐式创建标记，标记值已存在的情况。

```
MAC_CREATE_LABEL('P_04',21, 'L_01:C_01:G_02');
```

```
DECLARE
TAG INT;
BEGIN
SP_MAC_LABEL_FROM_CHAR ('P_04', 'L_01:C_01:G_02', TAG);
PRINT TAG ;
END;
/
```

由于先创建了标记串为 L_01:C_01:G_02 的标记，指定标记值为 21，因此后面的 SP_MAC_LABEL_FROM_CHAR 过程调用不会创建新的标记，直接返回标记值 21。

4.3 如何对表应用策略

4.3.1 对表应用策略

将策略应用在一个表上时，就使该表处于一定的等级、范围和组内。一个表上可以应用多个策略，但一个策略对表只能应用一次。

对表应用策略时，会在表上新建一个标记列，用于记录标记，新建的标记列名必须和表中已有列名不同，同时还可以指定初始的标记值，以及是否将标记列设置为隐藏列。

使用下述系统过程对表应用策略:

```
VOID
MAC_APPLY_TABLE_POLICY（
        POLICY_NAME             VARCHAR(128),
        SCHEMANAME              VARCHAR(128),
        TABLENAME               VARCHAR(128),
        COLNAME                 VARCHAR(128),
        LABELVALUE              VARCHAR(4000),
        OPTION                  INT
```

);

参数说明:

POLICY_NAME: 应用于指定表的策略名

SCHEMANAME: 表所属模式名称

TABLENAME: 策略所应用的表名称

COLNAME : 用于记录标记的列名称

LABELVALUE: 用于说明被应用了策略的表中，已有元组的等级、范围和组

OPTION: 1 代表隐藏标记列，0 代表不隐藏标记列，缺省为 0

使用说明:

1. 该过程只能由具有 LABEL DATABASE 的用户调用;
2. 指定策略必须存在;
3. 策略不能应用在系统表、临时表、HUGE 表、水平分区表、物化视图、含有聚集主键的表、含有位图连接索引的表、含有位图索引的表上。

例 将策略 P_04 应用于 PRODUCTION 模式中 PRODUCT 表，指定标记列为 LABEL_COL，不隐藏标记列。

```
MAC_APPLY_TABLE_POLICY ('P_04', 'PRODUCTION', 'PRODUCT', 'LABEL_COL', 'L_01::',0);
```

过程中的 OPTION 参数用来指定该标记列是否隐藏。若对表应用策略时指定新增的这一列隐藏，那么对表进行 INSERT 数据时，如果未指定标记列的列名，就不能对这一列插入数据，如果对其插入数据，则会出错；如果指明具体的列来插入数据，这一列是允许插入数据的。当执行 SELECT * 来查询该表数据时，标记列被隐藏不予显示；但也允许通过指明列名来查询该列。若对表应用策略时指定新增的这一列为 0，那么该列就可以被视为一个普通列。

当一个新的策略被应用于表上时，过程的 LABELVALUE 参数用于初始化表中已有的数据关于该策略的标记。当某个策略已被应用于表时，则其已有的等级、范围和组均不能被删除，除非从所有的用户和表上取消对该策略的应用。

4.3.2 取消表策略

使用下述系统过程取消表上的指定策略：

```
VOID
MAC_REMOVE_TABLE_POLICY（
        POLICY_NAME    VARCHAR(128),
        SCHEMANAME     VARCHAR(128),
        TABLENAME      VARCHAR(128),
        DROP_COLUMN INT
）；
```

参数说明：

POLICY_NAME：表上待取消应用的策略

SCHEMANAME：表所属模式名称

TABLENAME：表名称

DROP_COLUMN：1 代表删除标记列，0 代表不删除标记列，缺省为 0

使用说明：

1. 该过程只能由具有 LABEL DATABASE 的用户调用；
2. 指定策略必须存在。

例 取消应用在 PRODUCTION 模式中 PRODUCT 表上的策略，删除标记列。

```
MAC_REMOVE_TABLE_POLICY ('P_04', 'PRODUCTION', 'PRODUCT' ,1);
```

4.4 如何对用户应用策略

用户的标记来源于策略。对用户应用一个策略，要指定一个最高等级、最低等级、默认等级和行等级，其中默认等级介于最高等级和最低等级之间，最高等级限制了用户最高的读写权限，最低等级限制了用户的写权限，默认等级表示用户登录时会话标记的等级，行等级表示插入一行数据的标记使用的等级；指定范围和组，对范围需要指定读的范围、写的范围、默认范围和行级范围，其中写范围必须是读范围的子集，默认范围是读范围的子集，行级范围是写范围和默认范围的交集的子集。组的规则与范围相同。用户标记属性如下表所示：

表 4.1 用户标记属性表

组成	备注
max_level	最大读写级别
min_level	最小写级别
def_level	默认级别，登录时级别
row_level	列级别，用于默认插入
categories	包含的范围，可设置属性
groups	包含的组，可设置属性

4.4.1 设置用户等级

对用户应用策略，应先设置用户的等级，使用如下系统过程：

```
VOID
MAC_USER_SET_LEVELS(
        POLICY_NAME             VARCHAR(128),
        USER_NAME               VARCHAR(128),
        MAX_LEVEL               VARCHAR(128),
        MIN_LEVEL               VARCHAR(128),
        DEF_LEVEL               VARCHAR(128),
```

```
    ROW_LEVEL               VARCHAR(128)
);
```

参数说明：

POLICY_NAME：应用于用户的策略名称

USER_NAME：策略所应用的用户名称

MAX_LEVEL：应用于用户的最大等级

MIN_LEVEL：应用于用户的最小等级

DEF_LEVEL：应用于用户的默认等级

ROW_LEVEL：应用于用户的行等级

使用说明：

1. 该过程只能由具有 LABEL DATABASE 的用户调用；
2. 指定策略必须存在；
3. MAX_LEVEL 不能为空；
4. MIN_LEVEL 不指定时，设置为策略的最小级别；
5. DEF_LEVEL 不指定时，设置为 MAX_LEVEL；
6. ROW_LEVEL 不指定时，设置为 DEF_LEVEL；
7. 合法的 LEVEL 规则如下：

   MAX_LEVEL >= MIN_LEVEL

   MAX_LEVEL >= DEF_LEVEL >= MIN_LEVEL

   DEF_LEVEL >= ROW_LEVEL >= MIN_LEVEL

例 设置 BOOKSHOP_USER 用户的等级。等级 L_01，L_02，L_03，L_04 已存在，且等级级别相等，或依次增加。

```
MAC_USER_SET_LEVELS('P_04', 'BOOKSHOP_USER', 'L_04', 'L_01', 'L_03', 'L_02');
```

4.4.2 设置用户范围

对用户设置了等级后，可以对用户设置相应的范围和组。

使用下面的系统过程为用户设置范围：

```
VOID
MAC_USER_SET_COMPARTMENTS(
        POLICY_NAME             VARCHAR(128),
        USER_NAME               VARCHAR(128),
        READ                    VARCHAR(128),
        WRITE                   VARCHAR(128),
        DEFAULT                 VARCHAR(128),
        ROW                     VARCHAR(128)
);
```

参数说明：

POLICY_NAME：应用于用户的策略名称

USER_NAME：策略所应用的用户名称

READ：应用于用户的可读范围

WRITE：应用于用户的可写范围

DEFAULE：应用于用户的默认范围

ROW：应用于用户的行范围

使用说明：

1. 该过程只能由具有 LABEL DATABASE 的用户调用；
2. 用户范围具有 READ、WRITE、DEFAULT 和 ROW 四个属性，说明如下：
    1) READ：用户的可读范围，READ 不能为空；

2） WRITE：用户的可写范围，进行 UPDATE，DELETE 时需要使用，进行判断权限。WRITE 属于 READ，为空时使用 READ;

3） DEFAULT：会话默认使用的范围。DEFAULT 属于 READ，为空时使用 READ;

4） ROW：插入时不指定标记时使用。ROW 属于 DEFAULT 和 WRITE 的交集，为空时使用 DEFAULT 和 WRITE 的交集。

例 假设范围 C_01、C_02、C_03 已存在，设置 BOOKSHOP_USER 用户的范围。

```
MAC_USER_SET_COMPARTMENTS('P_04','BOOKSHOP_USER','C_01,C_02,C_03','C_01,C_02','C_01,C_03','C_01');
```

4.4.3 设置用户组

使用下面的系统过程为用户设置组：

```
VOID
MAC_USER_SET_GROUPS (
        POLICY_NAME             VARCHAR(128),
        USER_NAME               VARCHAR(128),
        READ_GROUP              VARCHAR(128),
        WRITE_GROUP             VARCHAR(128),
        DEF_GROUP               VARCHAR(128),
        ROW_GROUP               VARCHAR(128)
);
```

参数说明：

POLICY_NAME：应用于用户的策略名称

USER_NAME：策略所应用的用户名称

READ_GROUP：应用于用户的可读组

WRITE_GROUP：应用于用户的可写组

DEF_GROUP：应用于用户的默认组

ROW_GROUP：应用于用户的行组

使用说明：

1. 该过程只能由具有 LABEL DATABASE 的用户调用；
2. 用户的组具有 READ、WRITE、DEFAULT 和 ROW 四个属性，说明如下：
   1） READ：用户的可读组；
   2） DEFAULT：会话默认使用的组；
   3） WRITE：进行 UPDATE，DELETE 时需要使用，进行判断权限；
   4） ROW：插入时不指定标记时使用；
   5） 合法的用户组规则如下：
      - WRITE 属于 READ
      - DEFAULT 属于 READ
      - ROW 属于 DEFAULT 和 WRITE 的交集

例 假设组 G_01、G_02、G_03 已存在，设置 BOOKSHOP_USER 用户的组。

```
MAC_USER_SET_GROUPS ('P_04', 'BOOKSHOP_USER', 'G_01,G_02,G_03', 'G_02,G_03',
'G_01,G_03', 'G_03');
```

4.4.4 清除用户策略

使用下面的系统过程清除应用于某用户的指定策略。

```
VOID
MAC_USER_REMOVE_POLICY(
        POLICY_NAME             VARCHAR(128),
        USER_NAME               VARCHAR(128)
);
```

参数说明：

POLICY_NAME：要清除的策略名

USER_NAME：用户名

使用说明：

1. 该过程只能由具有 LABEL DATABASE 的用户调用；

2. 指定策略必须存在。

例 清除用户 BOOKSHOP_USER 的策略 P_04。

```
MAC_USER_REMOVE_POLICY ('P_04', 'BOOKSHOP_USER');
```

4.5 如何对会话应用策略

为了方便用户在执行时进行 MAC 权限的设置，系统提供了针对会话的标记设置，这些设置会挂载在会话上，一旦会话结束，这些信息就完全被抛弃，而不会影响系统表。

用户可以设置自身当前会话的默认标记，但其必须在合法的范围内，等级必须在用户的 MIN_LEVEL 和 MAX_LEVEL 之间，范围必须是用户 READ_COMP 的子集，组必须是用户 READ_GROUP 的子集。用户还可以设置会话的行级标记，同样必须在合法的范围内。

4.5.1 设置会话默认标记

使用如下的系统过程设置会话的默认标记：

```
VOID
MAC_SET_SESSION_LABEL(
        POLICY_NAME             VARCHAR(128),
        LABELVALUE              VARCHAR(4000)
);
```

参数说明：

POLICY_NAME：策略名

LABELVALUE：为会话设置的默认标记

使用说明：

1. 设置的等级必须在用户的 MIN_LEVEL 和 MAX_LEVEL 之间，范围必须是用户 READ_COMP 的子集，组必须是用户 READ_GROUP 的子集；
2. 由于行标记的范围来源于会话标记，故重置会话标记后，需对行标记进行调整。

例 接 4.4.1、4.4.2、4.4.3 节的例子，BOOKSHOP_USER 用户设置会话默认标记。

```
MAC_SET_SESSION_LABEL('P_04', 'L_03:C_01,C_02:G_01,G_03');
```

4.5.2 设置会话行标记

使用如下的系统过程设置会话的行标记：

```
VOID
MAC_SET_SESSION_ROW_LABEL(
        POLICY_NAME             VARCHAR(128),
        LABELVALUE              VARCHAR(4000)
);
```

参数说明：

POLICY_NAME：策略名

LABELVALUE：为会话设置的行标记

使用说明：

1. 该过程只能由具有 LABEL DATABASE 的用户调用；
2. 设置的行标记必须是合法值。

例 设置当前会话行标记。

```
MAC_SET_SESSION_ROW_LABEL('P_04', 'L_01:C_02:G_01');
```

4.5.3 清除会话标记

使用如下的系统过程清除会话上对应某策略的标记：

```
VOID
MAC_RESTORE_DEFAULT_LABELS(
        POLICY_NAME             VARCHAR(128)
);
```

参数说明：

POLICY_NAME：指定策略名

使用说明：

清除当前会话对应指定策略的所有的标记，仅用户的标记可用。

例 清除当前会话对应策略 P_04 的所有标记。

```
MAC_RESTORE_DEFAULT_LABELS('P_04');
```

4.5.4 保存会话标记

会话上的标记只在会话生存期间存在，并没有保存到数据库的数据字典中，一旦会话结束，会话标记就被丢弃。DM 提供以下的系统过程允许用户将会话上指定策略刷入相应的数据字典，这时，用户标记会被会话标记覆盖，会话标记与用户标记一致。

```
VOID
MAC_SAVE_DEFAULT_LABELS (
        POLICY_NAME     VARCHAR(128)
);
```

参数说明：

POLICY_NAME：要保存的会话标记所在的策略名

例 将当前会话上策略 P_04 的标记刷入相应的数据字典。

```
MAC_SAVE_DEFAULT_LABELS('P_04');
```

4.6 读写控制规则

DM 支持同时使用自主访问控制与强制访问控制策略。自主访问控制策略优先于强制访问控制策略。为了能访问一条元组，用户不仅首先要满足自主访问控制条件，还必须满足强制访问控制的条件。

用户访问表时，必须保证应用于表上的所有策略均适用于该用户，否则访问被拒绝。若表上未应用任何策略，则用户只需满足自主访问控制条件即可。

4.6.1 读访问规则

强制访问控制的读访问规则为：

1. 用户的等级必须大于等于数据的等级；
2. 用户的标记必须包含至少一个数据的组（或者是其某一个的父亲组）；
3. 用户的标记必须包含数据的所有的范围。

如果用户满足以上三个规则，即可以对数据进行读取。比较的先后顺序是等级，组，范围。DM 中实现读访问规则的流程如图 4.1 所示。

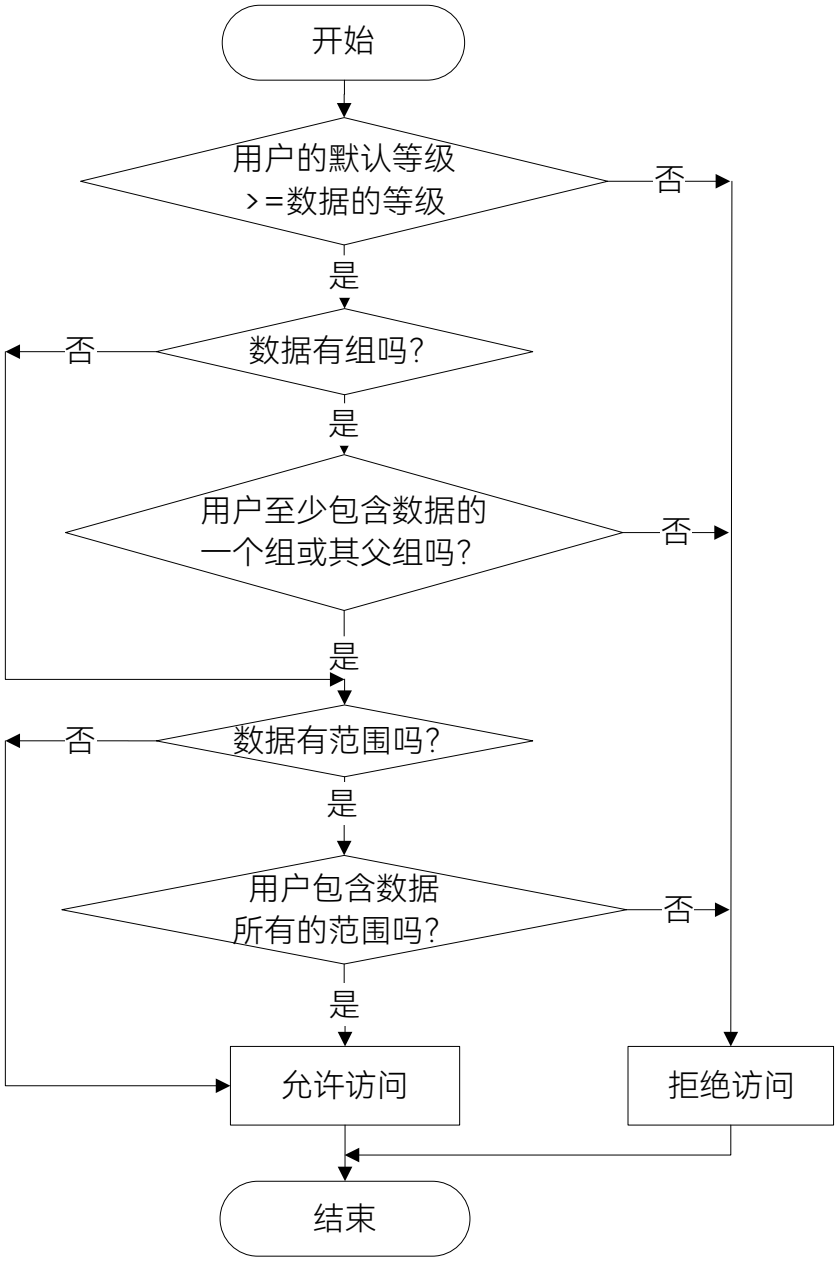


图 4.1 强制访问控制读访问规则

4.6.2 写访问规则

强制访问控制的写访问规则为：

1. 数据标记的等级必须大于等于用户标记的最小等级，小于或等于用户的会话标记的等级；
2. 用户的标记必须包含至少一个数据的组（或者是其某一个的父亲组）的写权限；

3. 用户的标记必须包含数据的所有的范围上的写权限。

如果用户满足以上三个规则，即可以对数据进行写操作。比较的先后顺序是等级，组，范围。DM 中实现写访问规则的流程如图 4.2 所示。

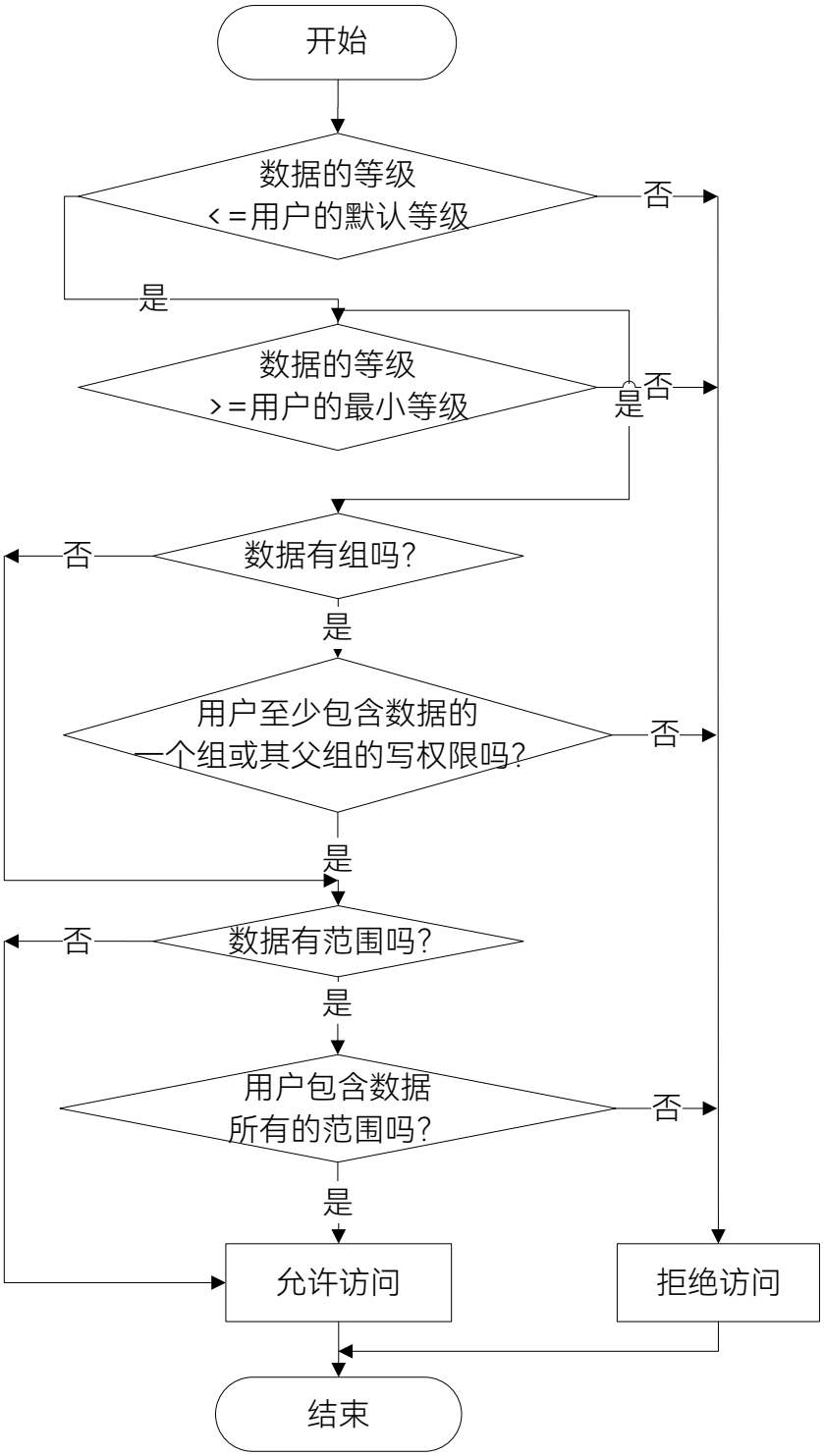


图 4.2 强制访问控制写访问规则

4.6.3 特权

在给用户应用策略时，可以同时授予策略特权。策略特权分为访问特权和行标记特权。

访问特权分为两种：

- ✓ READ：读数据时不受策略影响，但写数据访问控制仍然受到强制访问控制
- ✓ FULL：可以读写任何数据，不受策略影响

一旦一个行的标记设定后，就需要行标记特权才能 UPDATE 其标记列。行标记特权有如下三种：

- ✓ WRITE UP：用户可以利用该特权提升一个行的等级，同时不改变范围和组。这个等级可以提高到用户的最高等级，而该行的原始等级可能比用户的最低等级还低
- ✓ WRITE DOWN：用户可以利用该特权降低一个行的等级，同时不改变范围和组。这个等级可以降低到用户的最低等级，而该行的原始等级可能比用户的最低等级还低
- ✓ WRITE ACROSS：用户可以利用该特权修改一个行的范围和组，同时不改变等级。新的范围和组只要满足在策略中是合理的就可以了，不必限于用户拥有访问权的范围和组

使用下面的系统过程为用户设置特权：

```
VOID
MAC_USER_SET_USER_PRIVS(
    POLICY_NAME         VARCHAR(128),
    USER_NAME           VARCHAR(128),
    PRIVS               VARCHAR(128)
);
```

参数说明：

POLICY_NAME：特权对应的策略名

USER_NAME：为其设置特权的用户名

PRIVS：特权类型

使用说明：

1. 该过程只能由具有 LABEL DATABASE 的用户调用；
2. PRIVS 取值可以是 READ、FULL、WRITEUP、WRITEDOWN、WRITEACCROSS 中的一种或几种的组合。

例 设置 BOOKSHOP_USER 用户的特权。

```
MAC_USER_SET_USER_PRIVS('P1', 'BOOKSHOP_USER', 'READ,WRITEUP');
```

4.7 扩展客体标记

除了行标记，DM 支持对数据库中的表、视图、索引等对象扩展客体标记，一旦一个对象被应用了扩展客体标记，则用户只有在支配相应标记的情况下，才能访问客体。

创建的新对象不会含有默认的标记，需要安全员进行手工设置。

4.7.1 对客体应用标记

使用下面的系统过程可以对客体应用标记：

```
VOID
MAC_APPLY_OBJ_POLICY(
        POLICY_NAME             VARCHAR(128),
        OBJ_TYPE                VARCHAR(128),
        SCH_NAME                VARCHAR(128),
        OBJ_NAME                VARCHAR(128),
        COL_NAME                VARCHAR(128),
        LABEL                   VARCHAR(4000)
);
```

参数说明:

POLICY_NAME: 策略名称

OBJ_TYPE: 客体对象类型，必须是下面几个选项之一，“TABLE”、“VIEW”、“INDEX”、“PROCEDURE”、“FUNCTION”、“PACKAGE”、“SEQUENCE”、“TRIGGER”、“COLUMN”、“SYNONYM”、“DOMAIN”、“CONTEXT INDEX”、“CONTEXT”、“LIBRARY”。

SCH_NAME: 客体所在的模式名，为 NULL 时代表对库级的对象应用策略，如公用同义词。不支持对系统模式下的客体应用标记。

OBJ_NAME: 应用标记的客体名称

COL_NAME: 应用标记的列名，只有在 OBJ_TYPE='COLUMN'时，此列才有效

LABEL: 应用的标记值

使用说明:

1. 该过程只能由具有 LABEL DATABASE 的用户调用;
2. 指定策略必须存在。

例 对模式 PRODUCTION 中的表 PRODUCT 应用策略 P1 的标记（L1:C1:G1）。

```
MAC_APPLY_OBJ_POLICY('P1', 'TABLE', 'PRODUCTION', 'PRODUCT', NULL, 'L1:C1:G1');
```

4.7.2 修改客体标记

使用下面的系统过程修改客体的标记:

```
VOID
MAC_ALTER_OBJ_POLICY(
        POLICY_NAME         VARCHAR(128),
        OBJ_TYPE            VARCHAR(128),
        SCH_NAME            VARCHAR(128),
        OBJ_NAME            VARCHAR(128),
        COL_NAME            VARCHAR(128),
```

```
    LABEL                   VARCHAR(4000)
);
```

参数说明:

POLICY_NAME: 策略名称

OBJ_TYPE: 客体对象类型，必须是下面几个选项之一，“TABLE”、“VIEW”、“INDEX”、“PROCEDURE”、“FUNCTION”、“PACKAGE”、“SEQUENCE”、“TRIGGER”、“COLUMN”、“SYNONYM”、“DOMAIN”、“CONTEXT INDEX”、“CONTEXT”

SCH_NAME: 客体所在的模式名，为 NULL 时代表对库级的对象应用策略，如公用同义词

OBJ_NAME: 修改标记的客体名称

COL_NAME: 修改标记的列名，只有在 OBJ_TYPE='COLUMN'时，此列才有效

LABEL: 修改的标记值

使用说明:

1. 该过程只能由具有 LABEL DATABASE 的用户调用;
2. 指定策略必须存在。

例 将表 PRODUCT 的标记修改为策略 P1 的标记（L1:C1,C2:G1）。

```
MAC_ALTER_OBJ_POLICY ('P1', 'TABLE', 'PRODUCTION', 'PRODUCT', NULL, 'L1:C1,C2:G1');
```

4.7.3 删除客体标记

使用下面的系统过程删除指定客体上对应指定策略的标记:

```
VOID
MAC_DROP_OBJ_POLICY(
    POLICY_NAME         VARCHAR(128),
    OBJ_TYPE            VARCHAR(128),
    SCH_NAME            VARCHAR(128),
```

```
    OBJ_NAME            VARCHAR(128),
    COL_NAME            VARCHAR(128)
);
```

参数说明:

POLICY_NAME: 策略名称

OBJ_TYPE: 客体对象类型，必须是下面几个选项之一，“TABLE”、“VIEW”、“INDEX”、“PROCEDURE”、“FUNCTION”、“PACKAGE”、“SEQUENCE”、“TRIGGER”、“COLUMN”、“SYNONYM”、“DOMAIN”、“CONTEXT INDEX”、“CONTEXT”

SCH_NAME: 客体所在的模式名，为 NULL 时代表对库级的对象应用策略，如公用同义词

OBJ_NAME: 删除标记的客体名称

COL_NAME: 删除标记的列名，只有在 OBJ_TYPE='COLUMN'时，此列才有效

使用说明:

1. 该过程只能由具有 LABEL DATABASE 的用户调用;
2. 指定策略必须存在。

例 删除表 PRODUCT 与策略 P1 相关的标记。

```
MAC_DROP_OBJ_POLICY('P1', 'TABLE', 'PRODUCTION', 'PRODUCT',NULL);
```

4.8 一个强制访问控制的例子

下面给出一个强制访问控制的例子，综合运用了本章介绍的一些主要概念和方法。强制访问控制在真实应用中的使用会复杂得多，需要安全管理员事先进行全面的策略设计。

强制访问控制功能与 INI 参数 ENABLE_ENCRYPT 无关。但是如果用户配置了 ENABLE_ENCRYPT>0，需要提前配置好 SSL 环境；如果 ENABLE_ENCRYPT=0，不用配置 SSL 环境。

1. SYSSSO 登录，创建一个策略 P_TEST。

```
//以 SYSSSO 身份登录，创建策略
MAC_CREATE_POLICY('P_TEST');
MAC_CREATE_LEVEL('P_TEST', 11, 'L_01');
MAC_CREATE_LEVEL('P_TEST', 12, 'L_02');
MAC_CREATE_LEVEL('P_TEST', 13, 'L_03');
MAC_CREATE_LEVEL('P_TEST', 14, 'L_04');
MAC_CREATE_COMPARTMENT('P_TEST',11, 'C_01');
MAC_CREATE_COMPARTMENT('P_TEST',12, 'C_02');
MAC_CREATE_COMPARTMENT('P_TEST',13, 'C_03');
MAC_CREATE_COMPARTMENT('P_TEST',14, 'C_04');
MAC_CREATE_GROUP ('P_TEST',11, 'G_01',NULL);
MAC_CREATE_GROUP ('P_TEST',12, 'G_02','G_01');
MAC_CREATE_GROUP ('P_TEST',13, 'G_03','G_02');
```

2. SYSDBA 登录，创建一个用户 USER_TEST，并授予其 RESOURCE 角色。

```
//以 SYSDBA 身份登录，创建用户 USER_TEST，并授予其 RESOURCE 角色
CREATE USER USER_TEST IDENTIFIED BY "Utest_123";
GRANT RESOURCE TO USER_TEST;
```

3. USER_TEST 登录，创建一个表 TEST，插入两行数据，并查询，此时由于没有对表应用策略，表中没有标记列。

```
//USER_TEST/Utest_123，创建表 TEST，插入数据
CREATE TABLE TEST(C1 INT, C2 INT);
INSERT INTO TEST VALUES(1,1);
INSERT INTO TEST VALUES(2,2);
COMMIT;
SELECT * FROM TEST;
行号      C1          C2
---------- ----------- -----------
```

```
1        1        1
2        2        2
```

4. SYSSSO 登录，为用户 USER_TEST 和表 USER_TEST.TEST 应用策略。

```
// 以 SYSSSO 身份登录，为用户 USER_TEST 应用策略，为表 USER_TEST.TEST 应用策略
MAC_USER_SET_LEVELS('P_TEST', 'USER_TEST', 'L_04', 'L_01', 'L_03', 'L_02');
MAC_USER_SET_COMPARTMENTS('P_TEST', 'USER_TEST', 'C_01,C_02,C_03', 'C_01,C_02',
'C_01,C_03', 'C_01');
MAC_USER_SET_GROUPS ('P_TEST', 'USER_TEST', 'G_01,G_02,G_03', 'G_02,G_03', 'G_01,G_03',
'G_03');
MAC_APPLY_TABLE_POLICY ('P_TEST', 'USER_TEST', 'TEST', 'LABEL_COL', 'L_01::',0);
```

5. USER_TEST 登录，此时查询表 TEST，可看到表中已有行的 LABEL_COL 值为应用表策略时指定的值。

```
// USER_TEST/Utest_123
SELECT * FROM TEST;
行号      C1        C2        LABEL_COL
---------- ----------- ----------- -----------
1        1        1        2
2        2        2        2
```

6. USER_TEST 继续操作，向表 TEST 插入一行新的记录，可以看到这行记录的标记为 USER_TEST 的行标记。

```
INSERT INTO TEST(C1,C2) VALUES(3,3);
COMMIT;
SELECT * FROM TEST;
行号      C1        C2        LABEL_COL
---------- ----------- ----------- -----------
1        1        1        2
2        2        2        2
3        3        3        12
```

```
SELECT SF_MAC_LABEL_TO_CHAR(12);

行号     SF_MAC_LABEL_TO_CHAR(12)

---------- ------------------------

1        L_02:C_01:G_03
```

7. USER_TEST 继续操作，设置会话默认标记和会话行标记，再向表 TEST 中插入一行数据，可看到这一行的标记为会话的行标记。

```
MAC_SET_SESSION_LABEL('P_TEST', 'L_03:C_01,C_02:G_01,G_03');

MAC_SET_SESSION_ROW_LABEL('P_TEST', 'L_01:C_01:G_03');

INSERT INTO TEST(C1, C2) VALUES(4,4);

COMMIT;

SELECT * FROM TEST;

行号     C1          C2          LABEL_COL

---------- ----------- ----------- -----------

1        1           1           2

2        2           2           2

3        3           3           12

4        4           4           14

SELECT SF_MAC_LABEL_TO_CHAR(14);

行号     SF_MAC_LABEL_TO_CHAR(14)

---------- ------------------------

1        L_01:C_01:G_03
```

8. SYSDBA 登录，查询表 USER_TEST.TEST，由于此时 SYSDBA 没有被应用策略 P_TEST，因此一行数据也不能访问。

```
// 以 SYSDBA 身份登录

SELECT * FROM USER_TEST.TEST;

未选定行
```

9. SYSSSO 登录，为 SYSDBA 应用策略，设置 SYSDBA 的标记。

```
// 以 SYSSSO 身份登录，为 SYSDBA 应用策略
MAC_USER_SET_LEVELS('P_TEST', 'SYSDBA', 'L_01', 'L_01', 'L_01', 'L_01');
MAC_USER_SET_COMPARTMENTS('P_TEST', 'SYSDBA', 'C_01,C_02,C_03', 'C_01,C_02',
'C_01,C_03', 'C_01');
MAC_USER_SET_GROUPS ('P_TEST', 'SYSDBA', 'G_01,G_02,G_03', 'G_02,G_03', 'G_01,G_03',
'G_03');
```

10. SYSDBA 登录，再次查询表 USER_TEST.TEST，根据读访问规则，可以查询到三条记录。

```
//以 SYSDBA 身份登录
SELECT * FROM USER_TEST.TEST;
行号      C1          C2          LABEL_COL
---------- ----------- ----------- -----------
1         1           1           2
2         2           2           2
3         4           4           14
```

11. SYSSSO 登录，为 SYSDBA 设置 READ 特权。

```
//以 SYSSSO 身份登录，为 SYSDBA 设置特权
MAC_USER_SET_USER_PRIVS('P_TEST', 'SYSDBA', 'READ');
```

12. SYSDBA 登录，再次查询表 USER_TEST.TEST，由于拥有 READ 特权，可以查询到全部记录。

```
// 以 SYSDBA 身份登录
SELECT * FROM USER_TEST.TEST;
行号      C1          C2          LABEL_COL
---------- ----------- ----------- -----------
1         1           1           2
2         2           2           2
3         3           3           12
4         4           4           14
```

4.9 相关数据字典表

DM 中与强制访问控制相关的数据字典表有以下这些，这些字典表属于模式“SYSSSO”，只有具有 SSO 类型的用户才能查询。

1. SYSMACPLYS

记录策略定义。

序号	列	数据类型	说明
1	ID	INTEGER	策略 ID
2	NAME	VARCHAR(128)	策略名

2. SYSMACLVLS

记录策略的等级。

序号	列	数据类型	说明
1	PID	INTEGER	策略 ID
2	ID	SMALLINT	等级 ID
3	NAME	VARCHAR(128)	等级名

3. SYSMACCOMPS

记录策略的范围。

序号	列	数据类型	说明
1	PID	INTEGER	策略 ID
2	ID	SMALLINT	范围 ID
3	NAME	VARCHAR(128)	范围名

4. SYSMACGRPS

记录策略所在组的信息。

序号	列	数据类型	说明
1	PID	INTEGER	策略 ID

2	ID	SMALLINT	组 ID
3	PARENTID	SMALLINT	父节点 ID
4	NAME	VARCHAR(128)	组名

5. SYSMACLABELS

记录策略的标记信息。

序号	列	数据类型	说明
1	PID	INTEGER	策略 ID
2	ID	INTEGER	标记 ID
3	LABEL	VARCHAR(4000)	标记信息

6. SYSMACTABPLY

记录表策略信息。

序号	列	数据类型	说明
1	TID	INTEGER	表 ID
2	PID	INTEGER	策略 ID
3	COLID	SMALLINT	列 ID
4	OPTIONS	TINYINT	可见性

7. SYSMACUSRPLY

记录用户的策略信息。

序号	列	数据类型	说明
1	UID	INTEGER	用户 ID
2	PID	INTEGER	策略 ID
3	MAXREAD	INTEGER	最大读标记 ID
4	MINWRITE	INTEGER	最小写标记 ID
5	DEFTAG	INTEGER	默认标记 ID
6	ROWTAG	INTEGER	行级标记 ID
7	PRIVS	TINYINT	特权

8. SYSMACOBJ

记录扩展客体标记信息。

序号	列	数据类型	说明
1	OBJID	INTEGER	对象 ID
2	COLID	SMALLINT	列 ID
3	PID	INTEGER	策略 ID
4	TAG	INTEGER	标记 ID

5 审计

审计机制是 DM 数据库管理系统安全管理的重要组成部分之一。DM 数据库除了提供数据安全保护措施外，还提供对日常事件的事后审计监督。DM 具有一个灵活的审计子系统，可以通过它来记录系统级事件、个别用户的行为以及对数据库对象的访问。通过考察、跟踪审计信息，数据库审计员可以查看用户访问的形式以及曾试图对该系统进行的操作，从而采取积极、有效的应对措施。

5.1 审计开关

在 DM 系统中，专门为审计设置了开关，要使用审计功能首先要打开审计开关。审计开关由过程 VOID SP_SET_ENABLE_AUDIT（param int）;控制，过程执行完后会立即生效，param 有三种取值：

- ✓ 0：关闭审计
- ✓ 1：打开普通审计
- ✓ 2：打开普通审计和实时审计

缺省值为 0。

例 打开普通审计开关。

```
SP_SET_ENABLE_AUDIT (1);
```


注意：

审计开关必须由具有数据库审计员权限的管理员进行设置。

数据库审计员可通过查询 V$DM_INI 动态视图查询 ENABLE_AUDIT 的当前值。数据库审计员可通过查询 V$DM_INI 动态视图查询审计开关（ENABLE_AUDIT）的当前值。因为

ENABLE_AUDIT 值是通过 SP_SET_ENABLE_AUDIT 设置的，因此未在 DM.INI 中提供 ENABLE_AUDIT 参数进行配置。

例 查询审计开关的当前值。

```
SELECT * FROM V$DM_INI WHERE PARA_NAME='ENABLE_AUDIT';
```

5.2 审计的设置与取消

数据库审计员指定被审计对象的活动称为审计设置，只有具有 AUDIT DATABASE 权限的审计员才能进行审计设置。DM 提供审计设置系统过程来实现这种设置，被审计的对象可以是某类操作，也可以是某些用户在数据库中的全部行踪。只有预先设置的操作和用户才能被 DM 系统自动进行审计。

DM 允许在三个级别上进行审计设置，如表 5.1 所示。

表 5.1 审计级别

审计级别	说明
系统级	系统的启动、关闭、部分系统事件以及一些系统过程和函数的调用，此级别的审计无法也无需由用户进行设置，只要审计开关打开就会自动生成对应审计记录
语句级	导致影响特定类型数据库对象的特殊 SQL 或语句组的审计。如 AUDIT TABLE 将审计 CREATE TABLE、ALTER TABLE 和 DROP TABLE 等语句
对象级	审计作用在特殊对象上的语句。如 test 表上的 INSERT 语句

审计设置存放于 DM 字典表 SYSAUDIT 中，进行一次审计设置就在 SYSAUDIT 中增加一条对应的记录，取消审计则删除 SYSAUDIT 中相应的记录。

审计记录的刷盘方式由动态系统级的 INI 参数 AUDIT_FLUSH_LEVEL 进行控制。当设置 AUDIT_FLUSH_LEVEL 为 0 时，表示设置为语句级刷盘，一条语句执行结束进行一次审计记录刷盘操作；当设置 AUDIT_FLUSH_LEVEL 为 1 时，表示设置为事务级刷盘，一个事务

执行结束执行一次审计日志刷盘操作；当设置 AUDIT_FLUSH_LEVEL 为 2 时，表示设置为延迟刷盘，当审计记录缓存满时（缓存大小固定为 8MB）自动刷盘。

5.2.1 系统级审计

系统级审计无法也无需由用户进行设置，只要审计开关打开就会自动生成对应的审计记录。表 5.2 记录了系统级审计的审计操作及对应说明。

表 5.2 系统级审计选项

审计的数据库操作	说明
数据库系统启动关闭	区分第一次启动、正常退出后启动、异常退出后启动和还原库后第一次启动
审计空间不足告警	系统定时检测如果审计空间不足就会生成审计记录
数据库版本升级	包括数据字典版本、回滚段版本等
主备之间 redo 日志传输故障和恢复	包括主备之间 redo 日志传输异常和恢复两种
SP_SET_ENABLE_AUDIT	打开/关闭审计开关
SP_AUDIT_STMT/SP_NOAUDIT_STMT	设置/取消语句级审计 SP_AUDIT_XX/SP_NOAUDIT_XX 作为系统过程，本身不触发语句级审计
SP_AUDIT_OBJECT/SP_NOAUDIT_OBJECT	设置对象级审计/取消对象级审计
SP_AUDIT_SET_ENC	设置审计加密
SP_AUDIT_SQLSEQ_START	审计序列设置开始
SP_AUDIT_SQLSEQ_ADD	审计序列添加 SQL
SP_AUDIT_SQLSEQ_END	审计序列设置结束
SP_AUDIT_SQLSEQ_DEL	删除审计序列
SP_DROP_AUDIT_FILE	删除审计文件
SP_SWITCH_AUDIT_FILE	切换审计文件

SP_SET_PARA_VALUE	设置 DM.INI 文件中整型的参数值
SP_SET_PARA_STRING_VALUE	设置 DM.INI 文件中字符型的参数值
SP_SET_PARA_DOUBLE_VALUE	设置 DM.INI 文件中浮点型的参数值
SF_SET_SYSTEM_PARA_VALUE	修改整型、double、varchar 的静态配置参数或动态配置参数
SF_SET_SESSION_PARA_VALUE	设置会话级 INI 参数在当前会话上的值
SP_RESET_SESSION_PARA_VALUE	重置会话级 INI 参数在当前会话上的值，使得当前会话的参数值和全局值一致
SP_SET_PARAM_IN_SESSION	设置指定会话的会话级 INI 参数的值
SP_SET_SQLLOG_INI	修改 SQLLOG.INI 文件的内容
SP_DELETE_SQLLOG_INI_MODE	删除 SQLLOG.INI 文件中的模式

5.2.2 语句级审计

语句级审计的动作是全局的，不对应具体的数据库对象。其审计选项如表 5.3 所示。

表 5.3 语句级审计选项

审计选项	审计的数据库操作	说明
ALL	所有的语句级审计选项	所有可审计操作
USER	CREATE USER ALTER USER DROP USER	创建 / 修改 / 删除用户操作
ROLE	CREATE ROLE DROP ROLE	创建 / 删除角色操作
TABLESPACE	CREATE TABLESPACE ALTER TABLESPACE DROP TABLESPACE	创建 / 修改 / 删除表空间操作
SCHEMA	CREATE SCHEMA DROP SCHEMA SET SCHEMA	创建 / 删除 / 设置当前模式操作

TABLE	CREATE TABLE ALTER TABLE DROP TABLE TRUNCATE TABLE	创建 / 修改 / 删除 / 清空基表操作
VIEW	CREATE VIEW ALTER VIEW DROP VIEW	创建 / 修改 / 删除视图操作
INDEX	CREATE INDEX DROP INDEX	创建 / 删除索引操作
PROCEDURE	CREATE PROCEDURE ALTER PROCEDURE DROP PROCEDURE	创建 / 修改 / 删除存储模块操作
TRIGGER	CREATE TRIGGER ALTER TRIGGER DROP TRIGGER	创建 / 修改 / 删除触发器操作
SEQUENCE	CREATE SEQUENCE ALTER SEQUENCE DROP SEQUENCE	创建 / 修改/删除序列操作
CONTEXT	CREATE CONTEXT INDEX ALTER CONTEXT INDEX DROP CONTEXT INDEX	创建 / 修改 / 删除全文索引操作
SYNONYM	CREATE SYNONYM DROP SYNONYM	创建 / 删除同义词
OPERATOR	CREATE OPERATOR DROP OPERATOR	创建 / 删除自定义运算符
GRANT	GRANT	授予权限操作
REVOKE	REVOKE	回收权限操作
AUDIT	AUDIT	设置审计操作
NOAUDIT	NOAUDIT	取消审计操作
INSERT TABLE	INSERT INTO TABLE	表上的插入操作

UPDATE TABLE	UPDATE TABLE	表上的修改操作
DELETE TABLE	DELETE FROM TABLE	表上的删除操作
SELECT TABLE	SELECT FROM TABLE	表上的查询操作
EXECUTE PROCEDURE	CALL PROCEDURE	调用存储过程或函数操作
PACKAGE	CREATE PACKAGE DROP PACKAGE	创建 / 删除包规范
PACKAGE BODY	CREATE PACKAGE BODY DROP PACKAGE BODY	创建 / 删除包体
MAC POLICY	CREATE POLICY ALTER POLICY DROP POLICY	创建 / 修改 / 删除策略
MAC LEVEL	CREATE LEVEL ALTER LEVEL DROP LEVEL	创建 / 修改 / 删除等级
MAC COMPARTMENT	CREATE COMPARTMENT ALTER COMPARTMENT DROP COMPARTMENT	创建 / 修改 / 删除范围
MAC GROUP	CREATE GROUP ALTER GROUP DROP GROUP ALTER GROUP PARENT	创建 / 修改 / 删除组，更新父组
MAC LABEL	CREATE LABEL ALTER LABEL DROP LABEL	创建 / 修改 / 删除标记
MAC USER	USER SET LEVELS USER SET COMPARTMENTS USER SET GROUPS USER SET PRIVS	设置用户等级 / 范围 / 组 / 特权
MAC TABLE	REMOVE TABLE POLICY APPLY TABLE POLICY	取消 / 应用表标记

MAC SESSION	SESSION LABEL SESSION ROW LABEL RESTORE DEFAULT LABELS SAVE DEFAULT LABELS	保存 / 取消会话标记 设置会话默认标记 设置会话行标记
CHECKPOINT	CHECKPOINT	检查点（checkpoint）
SAVEPOINT	SAVEPOINT	设置保存点
RELEASE SAVEPOINT	RELEASE SAVEPOINT	销毁保存点
EXPLAIN	EXPLAIN	显示执行计划
NOT EXIST		分析对象不存在导致的错误
DATABASE	ALTER DATABASE	修改当前数据库操作
CONNECT	LOGIN LOGOUT	登录 / 退出操作
COMMIT	COMMIT	提交操作
ROLLBACK	ROLLBACK	回滚操作
SET TRANSACTION	SET TRX ISOLATION SET TRX READ WRITE	设置事务的读写属性和隔离级别
BACKUP	BACKUP DATABASE BACKUP TABLESPACE BACKUP TABLE BACKUP ARCHIVE	库/表空间/表/归档备份操作
RESTORE	RESTORE TABLESPACE RESTORE TABLE	表空间/表还原操作
DIMP	DIMP FULL DIMP OWNER DIMP SCHEMA DIMP TABLE	逻辑导入：库级/用户级/模式级/表级
DEXP	DEXP FULL DEXP OWNER DEXP SCHEMA DEXP TABLE	逻辑导出：库级/用户级/模式级/表级

FLDR	FLDR IN FLDR OUT	FLDR 工具导入/导出
WARNING	AUD SPACE WARNING	审计剩余可用空间不足
KEY	CREATE KEY DESTROY KEY	生成/销毁密钥
CRYPT	ENCRYPT DECRYPT	数据加密/解密
DTS	DTS IN DTS OUT	DTS 工具迁入/迁出
PROFILE	CREATE PROFILE ALTER PROFILE DROP PROFILE	创建/修改/删除 PROFILE
ARCH	主备之间 redo 日志传输故障和恢复	包括主备之间 redo 日志传输异常和恢复两种
MANAGER	MANAGER EXP FULL MANAGER EXP LOADED MANAGER EXP SELECTED	MANAGER 工具导出：导出全部数据/导出已加载数据/导出选中数据

设置语句级审计的系统过程如下：

```
VOID
SP_AUDIT_STMT(
        TYPE            VARCHAR(30),
        USERNAME        VARCHAR (128),
        WHENEVER        VARCHAR (20)
)
```

参数说明：

TYPE：语句级审计选项，即上表中的第一列

USERNAME　　：用户名，NULL 表示不限制

WHENEVER ：审计时机，可选的取值为：

- ALL：所有的
- SUCCESSFUL：操作成功时
- FAIL：操作失败时

例 1 审计表的创建、修改、删除和清空。

```
SP_AUDIT_STMT('TABLE', 'NULL', 'ALL');
```

例 2 对 SYSDBA 创建、修改、删除用户成功进行审计。

```
SP_AUDIT_STMT('USER', 'SYSDBA', 'SUCCESSFUL');
```

例 3 对用户 USER2 进行的表的修改和删除进行审计，不管失败和成功。

```
SP_AUDIT_STMT('UPDATE TABLE', 'USER2', 'ALL');
SP_AUDIT_STMT('DELETE TABLE', 'USER2', 'ALL');
```

取消语句级审计的系统过程如下：

```
VOID
SP_NOAUDIT_STMT(
        TYPE            VARCHAR(30),
        USERNAME        VARCHAR (128),
        WHENEVER        VARCHAR (20)
)
```

参数说明：

TYPE：语句级审计选项，即上表中的第一列

USERNAME ：用户名，NULL 表示不限制

WHENEVER ：审计时机，可选的取值为：

- ALL：所有的
- SUCCESSFUL：操作成功时

- FAIL：操作失败时

使用说明：

取消审计语句和设置审计语句进行匹配，只有完全匹配的才可以取消审计，否则无法取消审计。

例 1 取消对表的创建、修改、删除和清空的审计。

```
SP_NOAUDIT_STMT('TABLE', 'NULL', 'ALL');
```

例 2 取消对 SYSDBA 创建、修改、删除用户成功进行审计。

```
SP_NOAUDIT_STMT('USER', 'SYSDBA', 'SUCCESSFUL');
```

例 3 取消对用户 USER2 进行的表的修改和删除的审计。

```
SP_NOAUDIT_STMT('UPDATE TABLE', 'USER2', 'ALL');
SP_NOAUDIT_STMT('DELETE TABLE', 'USER2', 'ALL');
```

5.2.3 对象级审计

对象级审计发生在具体的对象上，需要指定模式名以及对象名。其审计选项如表 5.4 所示。

表 5.4 对象级审计选项

审计选项(SYSAUDITRECORDS 表中 operation 字段对应的内容)	TABLE	VIEW	COL	PROCEDURE FUNCTION	TRIGGER
INSERT	✓	✓	✓		
UPDATE	✓	✓	✓		
DELETE	✓	✓	✓		
SELECT	✓	✓	✓		
EXECUTE				✓	
MERGE INTO	✓	✓			
EXECUTE TRIGGER					✓

LOCK TABLE	✓				
BACKUP TABLE	✓				
RESTORE TABLE	✓				
ALL(所有对象级审计选项)	✓	✓	✓	✓	✓

其中，对于 UPDATE 和 DELETE 操作，因为也需要做 SELECT 操作，所以只要设置审计 SELECT 操作时，UPDATE 和 DELETE 也会作为 SELECT 操作被审计。

设置对象级审计的系统过程如下:

```
VOID
SP_AUDIT_OBJECT (
        TYPE            VARCHAR(30),
        USERNAME        VARCHAR (128),
        SCHNAME         VARCHAR (128),
        TVNAME          VARCHAR (128),
        WHENEVER        VARCHAR (20)
)

VOID
SP_AUDIT_OBJECT (
        TYPE            VARCHAR(30),
        USERNAME        VARCHAR (128),
        SCHNAME         VARCHAR (128),
        TVNAME          VARCHAR (128),
        COLNAME      VARCHAR (128),
        WHENEVER        VARCHAR (20)
)
```

参数说明:

TYPE：对象级审计选项，即上表中的第一列

USERNAME：用户名。为空串或 NULL 时表示不限定用户

SCHNAME：模式名。为空串或 NULL 时表示不限定模式

TVNAME：表名、视图名、存储过程名。不支持取值为空串或 NULL

COLNAME：列名。为空串或 NULL 时表示不限定列

WHENEVER：审计时机，可选的取值为：

- ALL：所有的
- SUCCESSFUL：操作成功时
- FAIL：操作失败时

例 1 对 SYSDBA 对表 PERSON.ADDRESS 进行的添加和修改的成功操作进行审计。

```
SP_AUDIT_OBJECT('INSERT', 'SYSDBA', 'PERSON', 'ADDRESS', 'SUCCESSFUL');
SP_AUDIT_OBJECT('UPDATE', 'SYSDBA', 'PERSON', 'ADDRESS', 'SUCCESSFUL');
```

例 2 对 SYSDBA 对表 PERSON.ADDRESS 的 ADDRESS1 列进行的修改成功的操作进行审计。

```
SP_AUDIT_OBJECT('UPDATE','SYSDBA','PERSON','ADDRESS','ADDRESS1','SUCCESSFUL');
```

取消对象级审计的系统过程如下：

```
VOID
SP_NOAUDIT_OBJECT (
        TYPE            VARCHAR(30),
        USERNAME        VARCHAR (128),
        SCHNAME         VARCHAR (128),
        TVNAME          VARCHAR (128),
        WHENEVER        VARCHAR (20)
)

VOID
SP_NOAUDIT_OBJECT (
```

```
    TYPE            VARCHAR(30),
    USERNAME        VARCHAR (128),
    SCHNAME         VARCHAR (128),
    TVNAME          VARCHAR (128),
    COLNAME     VARCHAR (128),
    WHENEVER        VARCHAR (20)
)
```

参数说明：

TYPE：对象级审计选项，即上表中的第一列

USERNAME：用户名。为空串或 NULL 时表示不限定用户

SCHNAME：模式名。为空串或 NULL 时表示不限定模式

TVNAME：表名、视图名、存储过程名。不支持取值为空串或 NULL

COLNAME：列名。为空串或 NULL 时表示不限定列

WHENEVER：审计时机，可选的取值为：

- ALL：所有的
- SUCCESSFUL：操作成功时
- FAIL：操作失败时

使用说明：

取消审计语句和设置审计语句进行匹配，只有完全匹配的才可以取消审计，否则无法取消审计。

例 1 取消对 SYSDBA 对表 PERSON.ADDRESS 进行的添加和修改的成功操作的审计。

```
SP_NOAUDIT_OBJECT('INSERT', 'SYSDBA', 'PERSON', 'ADDRESS', 'SUCCESSFUL');
SP_NOAUDIT_OBJECT('UPDATE', 'SYSDBA', 'PERSON', 'ADDRESS', 'SUCCESSFUL');
```

例 2 取消对 SYSDBA 对表 PERSON.ADDRESS 的 ADDRESS1 列进行的修改成功操作的审计。

```
SP_NOAUDIT_OBJECT('UPDATE','SYSDBA','PERSON','ADDRESS','ADDRESS1','SUCCESSFUL');
```

5.2.4 语句序列审计

DM 还提供了语句序列审计功能，作为语句级审计和对象级审计的补充。语句序列审计需要审计员预先建立一个审计规则，包含 N 条 SQL 语句（SQL1,SQL2……），如果某个会话依次执行了这些 SQL 语句，就会触发审计。

建立语句序列审计规则的过程包括下面三个系统过程。

```
VOID
SP_AUDIT_SQLSEQ_START(
        NAME    VARCHAR (128)
)

VOID
SP_AUDIT_SQLSEQ_ADD(
        NAME    VARCHAR (128),
        SQL     VARCHAR (8188)
)

VOID
SP_AUDIT_SQLSEQ_END(
        NAME    VARCHAR (128)
)
```

参数说明：

NAME：语句序列审计规则名

SQL：需要审计的语句序列中的 SQL 语句

使用说明：

建立语句序列审计规则需要先调用 SP_AUDIT_SQLSEQ_START，之后调用若干次 SP_AUDIT_SQLSEQ_ADD，每次加入一条 SQL 语句，审计规则中的 SQL 语句顺序根据加入 SQL 语句的顺序确定，最后调用 SP_AUDIT_SQLSEQ_END 完成规则的建立。

例 建立一个语句序列审计规则 AUDIT_SQL1。

```
SP_AUDIT_SQLSEQ_START('AUDIT_SQL1');
SP_AUDIT_SQLSEQ_ADD('AUDIT_SQL1', 'SELECT NAME FROM TEST1;');
SP_AUDIT_SQLSEQ_ADD('AUDIT_SQL1', 'SELECT ID FROM TEST2;');
SP_AUDIT_SQLSEQ_ADD('AUDIT_SQL1', 'SELECT * FROM TEST3;');
SP_AUDIT_SQLSEQ_END('AUDIT_SQL1');
```

可使用下面的系统过程删除指定的语句序列审计规则。

```
VOID
SP_AUDIT_SQLSEQ_DEL(
        NAME    VARCHAR (128)
)
```

参数说明:

NAME：语句序列审计规则名

例 删除语句序列审计规则 AUDIT_SQL1。

```
SP_AUDIT_SQLSEQ_DEL('AUDIT_SQL1');
```

5.2.5 关于审计设置的一些说明

- 只要审计功能被启用，系统级的审计记录就会产生;
- 在进行数据库审计时，审计员之间没有区别，可以审计所有数据库对象，也可取消其他审计员的审计设置;
- 语句级审计不针对特定的对象，只针对用户;
- 对象级审计针对指定的用户与指定的对象进行审计;

- 在设置审计时，审计选项不区分包含关系，都可以设置；
- 在设置审计时，审计时机不区分包含关系，都可以进行设置；
- 如果用户执行的一条语句与设置的若干审计项都匹配，只会在审计文件中生成一条审计记录。

5.3 审计文件管理

DM 审计信息存储在审计文件中。审计文件默认存放在数据库的 SYSTEM_PATH 指定的路径（即数据库所在路径）下的 audit 目录中。用户也可在 dm.ini 文件中添加参数 AUD_PATH 来指定审计文件的存放路径，例如：

```
#file location of dm.ctl
CTL_PATH        = D:\dmdbms\data\DAMENG\dm.ctl    #ctl file path
CTL_BAK_PATH =D:\dmdbms\data\DAMENG\ctl_bak   #dm.ctl backup path
CTL_BAK_NUM  = 10   #backup number of dm.ctl, allowed to keep one more backup file
besides specified number.
SYSTEM_PATH    = D:\dmdbms\data\DAMENG       #system path
CONFIG_PATH    = D:\dmdbms\data\DAMENG       #config path
TEMP_PATH      = D:\dmdbms\data\DAMENG      #temporary file path
BAK_PATH       = D:\dmdbms\data\DAMENG\bak   #backup file path
AUD_PATH       =D:\dmdbms\data\DAMENG\aud
```

审计文件命名格式为“AUDIT_GUID_创建时间.log”，其中“GUID”为 DM 给定的一个唯一值。

DM 的审计文件总存储空间大小由 INI 参数 AUDIT_SPACE_LIMIT 进行限制，该参数取值范围为 204800~1024*1024M，0 表示不限制，缺省为 0。AUDIT_SPACE_LIMIT 为静态参数，可通过系统过程 SP_SET_PARA_VALUE 进行修改，但是修改需要重新启动 DM 数据库服务器才能生效。

单个审计文件的大小可以通过 DM 的 INI 参数 AUDIT_MAX_FILE_SIZE 指定。当单个审计文件超过指定大小时，系统会自动切换审计文件，自动创建新的审计文件，审计记录将写入新的审计文件中。AUDIT_MAX_FILE_SIZE 为动态系统级参数，有效值范围为 8~4096M，缺省值为 100M，DBA 用户可通过系统过程 SP_SET_PARA_VALUE 对其进行动态修改。

DM 支持通过设置 INI 参数 AUDIT_KEEP_DAYS 来指定审计文件最小保留天数，该参数取值范围为 0~3660 天，即使达到审计文件总存储空间大小限制（即 AUDIT_SPACE_LIMIT 设置的值）也不会删除在指定保留天数内的审计文件。只有最老审计日志文件的保留天数超过 AUDIT_KEEP_DAYS 才会根据 AUDIT_FILE_FULL_MODE 设定值来处理老的审计文件。

随着系统的运行，审计记录将会不断增加，审计文件需要更多的磁盘空间。在极限情况下，审计记录可能会因为磁盘空间不足而无法写入审计文件，最终导致系统无法正常运行。对这种情况的处理有两种策略，通过设置 DM 的 INI 参数 AUDIT_FILE_FULL_MODE 进行配置。当将 AUDIT_FILE_FULL_MODE 置为 1 时，将删除最老的审计文件，直至有足够的空间创建新审计文件，若将所有可以删除的审计文件都删除后空间仍旧不够，则数据库会挂起不再处理任何请求，直至磁盘空间被清理出足够创建新审计文件的空间；当将 AUDIT_FILE_FULL_MODE 置为 2 时，将不再写审计记录；当将 AUDIT_FILE_FULL_MODE 置为 3 时，结合 1 和 2，先尝试删老审计文件，再创建审计文件，若空间仍不足则不再审计，系统会定时检测是否有空余空间，如果有则恢复审计；缺省值为 3。AUDIT_FILE_FULL_MODE 为静态参数，可通过系统过程 SP_SET_PARA_VALUE 进行修改，但是修改需要重新启动 DM 数据库服务器才能生效。


注意：

这三种策略都可能会导致审计记录的缺失，因此，管理员应该及时对审计文件进行备份。

若系统审计人员已对历史审计信息进行了充分分析，不再需要某个时间点之前的历史审计信息，可使用下面的系统过程删除指定时间点之前的审计文件，但是不会删除 DM 当前正在使用的审计文件。若设置了 ini 参数 AUDIT_KEEP_DAYS，指定了审计文件最小保留天数，使用 SP_DROP_AUDIT_FILE 时也不会删除还在指定保留天数内的审计文件。

```
VOID
SP_DROP_AUDIT_FILE(
        TIME_STR        VARCHAR(128),
        TYPE            INT
);
```

参数说明：

TIME_STR：指定的时间字符串

TYPE：审计文件类型，0 表示删除普通审计文件，1 表示删除实时审计文件

例 指定删除 2023-12-6 16:30:00 以前的普通审计文件。

```
SP_DROP_AUDIT_FILE('2023-12-6 16:30:00',0);
```

DM 审计文件支持文件加密，审计管理员可使用下面的系统过程设置审计文件加密：

```
VOID
SP_AUDIT_SET_ENC(
        NAME            VARCHAR(128),
        KEY             VARCHAR(128)
)
```

参数说明：

NAME：加密算法名，可使用 DM 支持的加密算法，见表 7.1；也支持用户自定义加密算法，见第 8 章加密引擎介绍

KEY：加密密钥

DM 审计文件支持手动进行切换。切换是指生成一个新的审计文件，并将新生成的审计信息存储在新生成的审计文件中。一般用在审计文件被删除无法存储审计信息，或任何需要生成新审计文件的情况下。审计管理员可用以下语句切换审计文件：

```
VOID
SP_SWITCH_AUDIT_FILE(
    AUDIT_SWITCH                INT
)
```

参数说明：

AUDIT_SWITCH：切换开关值，取值 1 或 2。1 表示切换普通审计文件，2 表示同时切换普通和实时审计文件。对于 DMDSC 环境仅切换当前节点审计文件。

例 切换普通审计文件。

```
SP_SWITCH_AUDIT_FILE(1);
```

5.4 审计信息查阅

当使用 DM 提供的审计机制进行了审计设置后，除语句序列审计设置外的审计设置信息都记录在数据字典表 SYSAUDITOR.SYSAUDIT 中，结构如下表所示：

表 5.5 SYSAUDITOR.SYSAUDIT 表结构

序号	列	数据类型	说明
1	LEVEL	SMALLINT	审计级别
2	UID	INTEGER	用户 ID
3	TVPID	INTEGER	表/视图/触发器/存储过程函数 ID
4	COLID	SMALLINT	列 ID
5	TYPE	SMALLINT	审计类型
6	WHENEVER	SMALLINT	审计情况

语句序列审计设置信息记录在数据字典表 SYSAUDITOR.SYSAUDITSQLSEQ 中，结构如下表所示：

表 5.6 SYSAUDITOR. SYSAUDITSQLSEQ 表结构

序号	列	数据类型	说明
1	NAME	VARCHAR(128)	语句序列审计规则名
2	SQLSEQ	INTEGER	语句序列中的 SQL 语句序号
3	SQLSTR	VARCHAR(8188)	语句序列中的 SQL 语句

审计类型用户可以查询上述数据字典表查看审计设置信息。

只要 DM 系统处于审计活动状态，系统按审计设置进行审计活动，并将审计信息写入审计文件。审计用户可通过查询 V$AUDIT_SPACE 动态视图获取审计文件存储空间的相关信息，动态视图的结构如下表所示：

表 5.7 V$AUDIT_SPACE 视图结构

序号	列	数据类型	说明
1	AUD_PATH	VARCHAR(256)	审计文件存放路径
2	AUDIT_FILE_FULL_MODE	INTEGER	剩余空间不足时的处理方式： 1：审计文件创建失败，卡住； 2：审计文件创建失败，不再审计； 3：结合 1 和 2，先尝试删老审计文件，再创建审计文件失败则不再审计
3	AUDIT_MAX_FILE_SIZE	INTEGER	单个审计文件的最大大小，单位 MB
4	AUD_SPACE_LIMIT	INTEGER	审计文件总存储空间大小限制，单位 MB
5	AUD_SPACE_FREE	INTEGER	可用审计文件存储空间大小，单位 MB
6	AUDIT_IS_NORMAL	CHAR(1)	审计是否正常。Y:是；N:否，不再审计

审计用户可通过查询 V$AUDIT_FILE 动态视图获取审计文件存放路径下审计文件的具体信息，动态视图的结构如下表所示：

表 5.8 V$AUDIT_FILE 视图结构

序号	列	数据类型	说明
1	FILEPATH	VARCHAR(256)	审计文件的完整路径
2	SGUID	INTEGER	服务器的唯一标识，仅用于判断审计文件与库是否匹配
3	RT_FLAG	INTEGER	是否实时审计
4	HPC_SEQNO	INTEGER	DSC 节点号
5	VERSION	INTEGER	版本号
6	DBNAME	VARCHAR(128)	库名称
7	IS_FULL	INTEGER	审计文件是否已满。1：已满，0：未满。
8	BEGIN_TIME	DATETIME(6)	审计文件的创建时间
9	LENGTH	BIGINT	审计文件的长度，单位 BYTE

审计用户可通过查询 V$AUDITRECORDS 动态视图获取审计文件存放路径下审计文件中审计记录的具体信息，审计记录内容包括操作者的用户名、所在站点、所进行的操作、操作的对象、操作时间、当前审计条件等。动态视图的结构如下表所示：

表 5.9 V$AUDITRECORDS 结构

序号	列	数据类型	说明
1	USERID	INTEGER	用户 ID
2	USERNAME	VARCHAR(128)	用户名
3	ROLEID	INTEGER	角色 ID。 没有具体角色的用户和 SQL 序列审计，没有角色信息，置为-1。
4	ROLENAME	VARCHAR(128)	角色名。 没有具体角色的用户和 SQL 序列审计，没有角色信息，内容为空。
5	IP	VARCHAR(64)	IP 地址
6	SCHID	INTEGER	模式 ID
7	SCHNAME	VARCHAR(128)	模式名

8	OBJID	INTEGER	对象 ID
9	OBJNAME	VARCHAR(128)	对象名
10	OPERATION	VARCHAR(128)	操作类型名
11	SUCC_FLAG	CHAR(1)	成功标记
12	SQL_TEXT	VARCHAR(8188)	SQL 文本
13	DESCRIPTION	VARCHAR(8188)	描述信息
14	OPTIME	DATETIME(6)	操作时间
15	MAC	VARCHAR(25)	操作对应的 MAC 地址
16	SEQNO	TINYINT	DMDSC 环境下表示生成审计记录的节点号，非 DMDSC 环境下始终 0
17	BIND_INFO	VARCHAR(8188)	绑定参数具体值，如果绑定参数是复杂类型，仅记录类型名，不记录具体值

其中审计记录中的 IP 地址的显示格式由 DM 的 INI 参数 AUDIT_IP_STYLE 进行控制，设置为 0 时，仅显示 IP；设置为 1 时，显示 IP 和主机名；缺省值为 0。AUDIT_IP_STYLE 为静态参数，可通过系统过程 SP_SET_PARA_VALUE 进行修改，但是修改需要重新启动 DM 数据库服务器才能生效。

例 SYSAUDITOR 进行如下的审计设置：

```
//以 SYSAUDITOR 身份登录
SP_AUDIT_OBJECT('INSERT', 'SYSDBA', 'PERSON', 'ADDRESS', 'SUCCESSFUL');
SP_AUDIT_OBJECT('UPDATE', 'SYSDBA', 'PERSON', 'ADDRESS', 'SUCCESSFUL');
```

之后查询 SYSAUDITOR.SYSAUDIT 数据字典表，可以看到对应的审计设置记录：

```
SELECT * FROM SYSAUDITOR.SYSAUDIT;
行号      LEVEL      UID        TVPID      COLID      TYPE    WHENEVER
---------- ----------- ----------- ----------- ----------- -----------
```

```
1       2       50331649    1196       -1       50       1

2       2       50331649    1196       -1       53       1
```

以 SYSDBA 登录，对表 PERSON.ADDRESS 进行插入和删除操作：

```
// 以 SYSDBA 身份登录
INSERT INTO PERSON.ADDRESS(ADDRESS1,ADDRESS2,CITY,POSTALCODE) VALUES('AAA','','BBB','111111');
DELETE FROM PERSON.ADDRESS WHERE ADDRESS1='AAA';
COMMIT;
```

再以 SYSAUDITOR 登录，查询 SYSAUDITOR.V$AUDITRECORDS，可以看到 SYSDBA 刚才的插入操作生成的审计记录：

```
// 以 SYSAUDITOR 身份登录
SELECT * FROM SYSAUDITOR.V$AUDITRECORDS WHERE USERNAME='SYSDBA';
行号      USERID      USERNAME ROLEID      ROLENAME IP  SCHID       SCHNAME
---------- ----------- -------- ----------- -------- --- ----------- -------
        OBJID       OBJNAME OPERATION SUCC_FLAG
        ----------- ------- --------- ---------
        SQL_TEXT
        ------------------------------------------------------------------
        DESCRIPTION OPTIME                    MAC              SEQNO
        ----------- --------------------------- ----------------- ----
            BIND_INFO
            ---------
1       50331649    SYSDBA   67108864    DBA      ::1 150995945   PERSON
        1196        ADDRESS INSERT    Y
        INSERT INTO PERSON.ADDRESS(ADDRESS1,ADDRESS2,CITY,POSTALCODE)
VALUES('AAA','','BBB','111111');
                2016-03-15 15:07:49.000000E0-3F-49-AE-86-5F 0
            NULL
```

由于并没有相关的审计设置，SYSDBA 之前执行的 DELETE 操作没有生成审计记录。

DM 还提供了以下系统函数供查看审计设置中涉及的审计类型 TYPE、审计级别 LEVEL、以及生成时机 WHENEVER 常量对应的具体信息，作为上述动态视图的补充。

- **SF_GET_AUDIT_TYPENAME**

```
VARCHAR
SF_GET_AUDIT_TYPENAME(
        TYPE  INT
);
```

参数说明:

TYPE：审计类型，对应 SYSAUDIT 中 TYPE 字段值

返回值:

参数 TYPE 值的意义描述

例 查看 SYSAUDIT 中 TYPE 值 50 的意义描述。

```
SELECT SF_GET_AUDIT_TYPENAME(50);
```

返回值为：INSERT

- **SF_GET_AUDIT_LEVELNAME**

```
VARCHAR
SF_GET_AUDIT_LEVELNAME(
        LEVEL  INT
);
```

参数说明:

LEVEL：审计级别，对应 SYSAUDIT 中 LEVEL 字段值

返回值:

参数 LEVEL 值的意义描述

例 查看 SYSAUDIT 中 LEVEL 值为 2 的意义描述。

```
SELECT SF_GET_AUDIT_LEVELNAME(2);
```

返回值为：OBJECT

- **SF_GET_AUDIT_WHENEVERNAME**

```
VARCHAR
SF_GET_AUDIT_WHENEVERNAME(
        WHENEVER INT
);
```

参数说明：

WHENEVER ：审计记录生成时机，对应 SYSAUDIT 中 WHENEVER 字段值

返回值：

参数 WHENEVER 值的意义描述

例 查看 SYSAUDIT 中 WHENEVER 值为 1 的意义描述。

```
SELECT SF_GET_AUDIT_WHENEVERNAME(1);
```

返回值为：SUCCESSFUL

5.5 审计实时侵害检测

当执行 SP_SET_ENABLE_AUDIT (2);时，开启审计实时侵害检测功能。

实时侵害检测系统用于实时分析当前用户的操作，并查找与该操作相匹配的实时审计分析规则，如果规则存在，则判断该用户的行为是否是侵害行为，确定侵害等级，并根据侵害等级采取相应的响应措施。

5.5.1 创建与删除实时侵害检测规则

当执行 SP_SET_ENABLE_AUDIT (2);时，具有 AUDIT DATABASE 权限的用户可以使用下面的系统过程创建实时侵害检测规则。

```
VOID
SP_CREATE_AUDIT_RULE(
        RULENAME        VARCHAR(128),
        OPERATION       VARCHAR(30),
        USERNAME        VARCHAR(128),
        SCHNAME         VARCHAR(128),
        OBJNAME         VARCHAR(128),
        WHENEVER        VARCHAR(20),
        ALLOW_IP        VARCHAR(1024),
        ALLOW_DT        VARCHAR(1024),
        INTERVAL        INTEGER,
        TIMES           INTERGER
);
```

参数说明:

RULENAME：创建的审计实时侵害检测规则名。

OPERATION：审计操作名，可取值包括：

- 对象级审计中的全部可选操作；
- 语句级审计中的部分可选操作，其中暂不支持 DIMP、DEXP、CONNECT、INSERT TABLE、UPDATE TABLE、DELETE TABLE、SELECT TABLE、EXECUTE PROCEDURE 审计选项对应的审计操作；
- SHUTDOWN、STARTUP、UNLOCK USER、CONNECT 和 DISCONNECT 操作。

USERNAME：用户名，没有指定或'NULL'表示所有用户。

SCHNAME：模式名，没有时指定为'NULL'。

OBJNAME：对象名，没有时指定为'NULL'。

WHENEVER：审计时机，取值 ALL/SUCCESSFUL/FAIL。

ALLOW_IP：IP 列表，以','隔开。例如'"192.168.0.1","127.0.0.1"'。

ALLOW_DT：时间串，格式如下：

```
ALLOW_DT::= <时间段项>{,<时间段项>}

<时间段项> ::= <具体时间段> | <规则时间段>

<具体时间段> ::= <具体日期><具体时间> TO <具体日期><具体时间>

<规则时间段> ::= <规则时间标志><具体时间> TO <规则时间标志><具体时间>

<规则时间标志> ::= MON | TUE | WED | THURS | FRI | SAT | SUN
```

INTERVAL：时间间隔，单位为分钟。

TIMES：次数。

例 1 创建一个审计实时侵害检测规则 DANGEROUS_SESSION，该规则检测每个星期一 8：00 至 9：00 的所有非本地 SYSDBA 的登录动作。

```
SP_CREATE_AUDIT_RULE ('DANGEROUS_SESSION','CONNECT', 'SYSDBA', 'NULL', 'NULL', 'ALL',
'"127.0.0.1"','MON "8:00:00" TO MON "9:00:00"',0, 0);
```

例 2 创建一个审计实时侵害检测规则 PWD_CRACK，该规则检测可能的口令暴力破解行为。

```
SP_CREATE_AUDIT_RULE ('PWD_CRACK','CONNECT', 'NULL', 'NULL', 'NULL', 'FAIL', 'NULL','NULL',1, 50);
```

当不再需要某个实时侵害检测规则时，可使用下面的系统过程进行删除。

```
VOID
SP_DROP_AUDIT_RULE(
```

```
    RULENAME        VARCHAR(128)
);
```

参数说明：

RULENAME：待删除的审计实时侵害检测规则名

例 删除已创建的实时侵害检测规则 DANGEROUS_SESSION。

```
SP_DROP_AUDIT_RULE ('DANGEROUS_SESSION');
```

5.5.2 实时侵害检测

当执行 SP_SET_ENABLE_AUDIT (2);时，实时侵害检测系统会实时分析用户的操作，将其与系统中创建的实时侵害检测规则进行匹配，判断该用户操作是否是侵害行为。

为了界定不同侵害行为对系统的危害情况，实时侵害检测系统定义了四种级别的侵害等级，从四级到一级侵害行为严重程度递增。这样根据用户操作匹配的实时侵害检测规则来可以判断其属于哪一级的侵害行为。

四种侵害等级如下：

- 四级：操作只违反了 IP 或者时间段设置之一，且对应累计次数在规定时间间隔内没有达到门限值，或者没有门限值设置；
- 三级：操作同时违反了 IP 和时间段设置，且累计次数在规定时间间隔内没有达到门限值，或者没有门限值设置；
- 二级：操作的 IP 和时间段都正常，且累计次数在规定时间间隔内达到了门限值；
- 一级：操作只违反了 IP 或时间段设置之一，且对应累计次数在规定时间间隔内达到了门限值，或者操作同时违反了 IP 和时间段设置，且累计次数在规定时间间隔内达到了门限值。

实时侵害检测规则的设置，使得某些操作可能会触发多条审计分析规则，也就是说可能存在多条审计分析规则同时匹配的情况。在这种情况下，需要把操作记录保存到所有匹配的、

同时规定了操作频率门限值的审计分析规则下，并且使用这些审计分析规则对该操作进行分析，从所有满足的侵害等级中选择一个最高的侵害等级进行响应。

DM 实时侵害检测系统根据侵害检测结果做出相应的安全审计响应，由低到高四个等级分别为：

- 四级响应：实时报警生成，对四级侵害行为进行响应。当系统检测到四级侵害行为时，生成报警信息；
- 三级响应：违例进程终止，对三级侵害行为进行响应。当系统检测到三级侵害行为时，终止当前操作（用户当前连接仍然保持）。同时生成报警信息；
- 二级响应：服务取消，对二级侵害行为进行响应。当系统检测到二级侵害行为时，强制断开用户当前连接，退出登录。同时生成报警信息；
- 一级响应：账号锁定或失效，对一级侵害行为进行响应。当系统检测到一级侵害行为时，强制断开用户当前连接，退出登录，并且锁定账号或使账号失效。同时生成报警信息。

这些安全审计相应动作中除了生成报警信息，其余的都由 DM 数据库服务器即可完成。生成报警信息的实现包括 DM 数据库服务器将报警信息写入一个专门的日志文件，由 DM 的审计告警工具 dmamon 实时分析这个日志文件，发现报警信息并将报警信息以邮件的形式发送给指定的邮箱。

5.5.3 审计告警工具 dmamon

审计告警工具 dmamon 用来在 DM 实时侵害检测系统检测到侵害事件且需要进行报警时可将报警信息以邮件的形式发送给指定邮箱。

dmamon 的运行需要指定对应的配置文件 dmamon.ini，此配置文件只能通过工具 dmamon_ctl 进行创建或修改。

5.5.3.1 创建或修改 dmamon.ini

使用 dmamon_ctl 工具创建或修改 dmamon.ini。

启动 dmamon_ctl 工具的命令格式为：

```
dmamon_ctl [FILE =value]

或

dmamon_ctl [FILE =value] DCR_INI= value
```

参数说明：

FILE：用来指定需要编辑的 dmamon.ini 的路径。创建 dmamon.ini 时，可以不指定 FILE 参数。

DCR_INI：在 DMASM 环境下，用来指定 dmdcr.ini 路径。非 DMASM 环境不用指定。

例如：

```
dmamon_ctl FILE=c:\dmamon.ini
```

当需要创建 dmamon.ini 时，可以不指定 FILE 参数，则 dmamon_ctl 工具会一步步引导用户配置 dm.ini 所在目录、发送和接收邮件的电子邮箱等信息，创建一个新的 dmamon.ini 文件。配置项如下表所示。

表 5.10 dmamon.ini 配置项

项目	项目意义	字段	字段意义
[PATH]	实时审计警报记录文件所在的文件目录	PATH	实时审计警报记录文件所在的文件目录
[SENDER]	发件人	EMAIL	发件人邮箱
		SMTP	发件人邮箱 SMTP 服务器
		USERNAME	发件人用户名
		PASSWORD	发件人密码
[RECEIVER]	收件人	EMAIL	收件人邮箱

[INTERVAL]	时间间隔	INTERVAL	检查审计警报记录文件的时间间隔，单位秒

dmamon_ctl 工具支持的命令如下表。

表 5.11 dmamon_ctl 命令

命令	说明
EDIT	修改 INI 文件信息
ADD	增加接收者的邮件地址信息
REMOVE	删除接收者的邮件地址信息
SHOW	显示当前控制文件的内容
SAVE	保存，保存路径必须为 dmamon.ini 的绝对路径。退出之前须保存，否则修改无效
EXIT	退出当前程序
HELP	显示帮助信息

5.5.3.2 使用 dmamon

配置好 dmamon.ini 后，就可以使用 dmamon 工具进行实时审计告警了。启动 dmamon 工具的命令格式为：

```
dmamon PATH=<dmamon.ini 路径> USERID=<userid>

或

dmamon PATH=<dmamon.ini 路径> USERID=<userid> dcr_ini=<dmdcr.ini 路径>

<userid>::=<username>/<password> [@<connect_identifier>][<option>]

<connect_identifier> ::=<svc_name> | {<host>[:<port>]} | <unixsocket_file>

<option>::= #{ <extend_option>=<value>{,<extend_option>=<value>} } //此行外层{}是为了封装参数之用，书写时需要保留
```

参数说明：

PATH：指定 dmamon.ini 文件的路径，字符串类型

USERID：指定登录数据库的信息

<username>/<password>：指定用户名和密码，仅支持具有审计权限的用户使用 dmamon 工具

<svc_name>：服务名

<host>[:<port>]：服务器 IP 地址和端口号。缺省情况下默认为本地服务器和端口号 LOCALHOST:5236。当服务器为本机时，SERVER:PORT 可直接写 LOCALHOST。当连接其他服务器时，SERVER:PORT 需写上 IP 地址和 PORTNUM，例如：192.168.0.248:8888

<unixsocket_file>：专门用于在 LINUX 系统中，当服务器与客户端之间使用 UNIXSOCKET（UNIX DOMAIN SOCKET - IPC）协议通信时，指定客户端连接的 socket 文件路径。例如：

```
./dmamon PATH=/home/dmamon.ini
USERID=SYSAUDITOR/DMauditor_123@/home/test/foo.sock#{inet_type=UNIXSOCKET}
```

<option>为扩展选项，用法为<extend_option>=<value>。所有 value 值不能包含空格，不能包含特殊的符号，如引号等。书写扩展选项时需要用引号#"{ }"进行封装，例如：#"{INET_TYPE=tcp,mpp_type=local}"。

现支持的扩展选项如下：

extend_option	value
mpp_type	MPP 登录属性，此属性的设置对非 MPP 系统没有影响。取值 GLOBAL 和 LOCAL，默认为 GLOBAL。GLOBAL 表示 MPP 环境下建立的会话为全局会话，对数据库的导入导出操作在所有节点进行；LOCAL 表示 MPP 环境下建立的会话为本地会话，对数据库的导入导出操作只在本地节点进行
inet_type	网络通信协议类型。取值 TCP/UDP/IPC/UNIXSOCKET/RDMA，缺省为 TCP。 使用 UDP 协议时，服务器需要设置 INI 参数 ENABLE_UDP 非 0；使用 IPC 协议时，会忽略 IP 地址和端口号，直接连接本机上 INI 参数 ENABLE_IPC=1 的服务器，一台机

	器上只能有一个服务器设置 ENABLE_IPC=1；使用 UNIXSOCKET 协议时，无需指定 IP 地址和端口号，但服务器需要配置 INI 参数 UNIX_SOCKET_PATHNAME 指定 socket 文件路径，客户端连接服务器时也需要指定 socket 文件路径，二者必须一致，仅 LINUX 环境下支持 UNIXSOCKET；使用 RDMA 协议时，需要安装并配置 RDMA 网卡，连接数据库时需要指定 RDMA 网卡配置的 IP 地址以及服务器的端口号
ssl_path	通信加密的 SSL 数字证书路径，缺省为不使用加密。数字证书路径由用户自己创建，将相应的证书需放入该文件夹中。其中服务器证书必须与 dmserver 目录同级，客户端目录可以任意设置。和 ssl_pwd 一起使用。 各用户只能使用自己的 SSL 数字证书，例如 SYSAUDITOR 账户只能使用 /home/dmdbms/bin/client_ssl/SYSAUDITOR 下的证书和密码，如果证书没有密码可以用缺省或任意数字代替。 例如：./dmamon PATH=/home/dmamon.ini USERID=SYSAUDITOR/DMauditor_123@192.168.1.64:5236#"{ssl_path=/home/dmdbms/bin/client_ssl/SYSAUDITOR,ssl_pwd=12345}"
ssl_pwd	通信加密的 SSL 数字证书密码。和 ssl_path 一起使用。缺省为不加密

DCR_INI：在 DMASM 环境下，用来指定 dmdcr.ini 路径。非 DMASM 环境不用指定。

5.6 审计分析

DM 提供了图形界面的审计分析工具 Analyzer 以及通过命令行启动的审计分析工具 dmaudtool。

Analyzer 实现对审计记录的分析功能，能够根据所制定的分析规则，对审计记录进行分析，判断系统中是否存在对系统安全构成危险的活动。只有审计用户才能使用审计分析工具 Analyzer。

dmaudtool 实现对审计记录的解析、筛选以及导出功能。审计文件的内容是不可视的，dmaudtool 可解析审计文件中的审计记录并导出至指定输出文件当中以供用户查看。同时

dmaudtool 也提供筛选功能，可以针对审计记录中的用户名、模式名、对象名或者操作时间有选择性地筛选导出。审计分析工具 dmaudtool 的使用对用户无限制。

下面将详细介绍审计分析工具 Analyzer 和 dmaudtool 的使用方法。

5.6.1 审计分析工具 Analyzer

审计用户登录审计分析工具后，通过 Analyzer 可以创建和删除审计规则，并可以指定对某些审计文件应用某些规则。并将审计结果以表格的方式展现出来。需要注意的是，普通审计用户登录 Analyzer 工具至少需要具备 DB_AUDIT_SOI 和 DB_AUDIT_VTI 角色。

审计用户登录 Analyzer 后可看到工具主界面如图 5.1 所示。

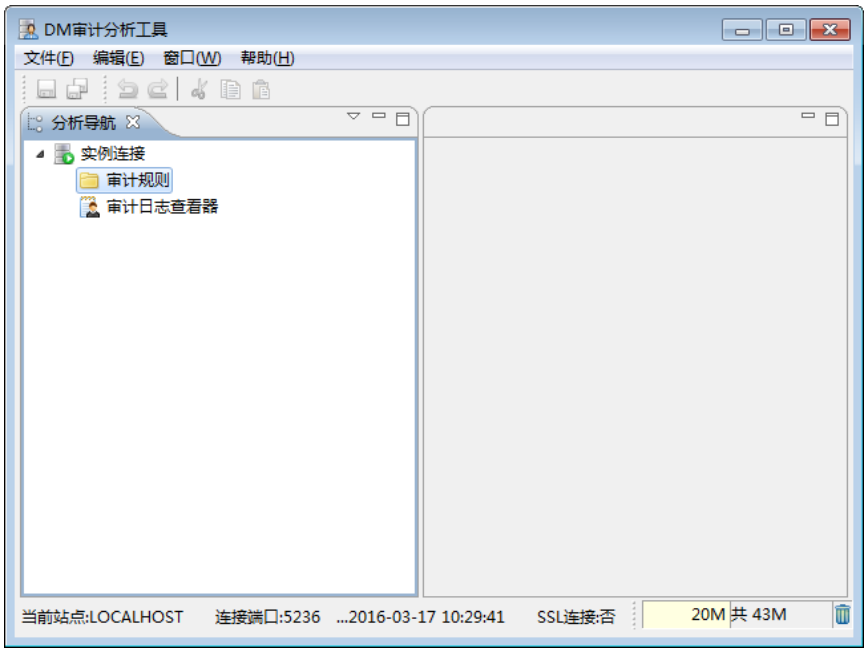


图 5.1 Analyzer 工具主界面

5.6.1.1 审计规则

右键单击导航树中的“审计规则”节点，在弹出的菜单中选择“新建审计分析规则”，可打开如下图所示的新建审计分析规则窗口。

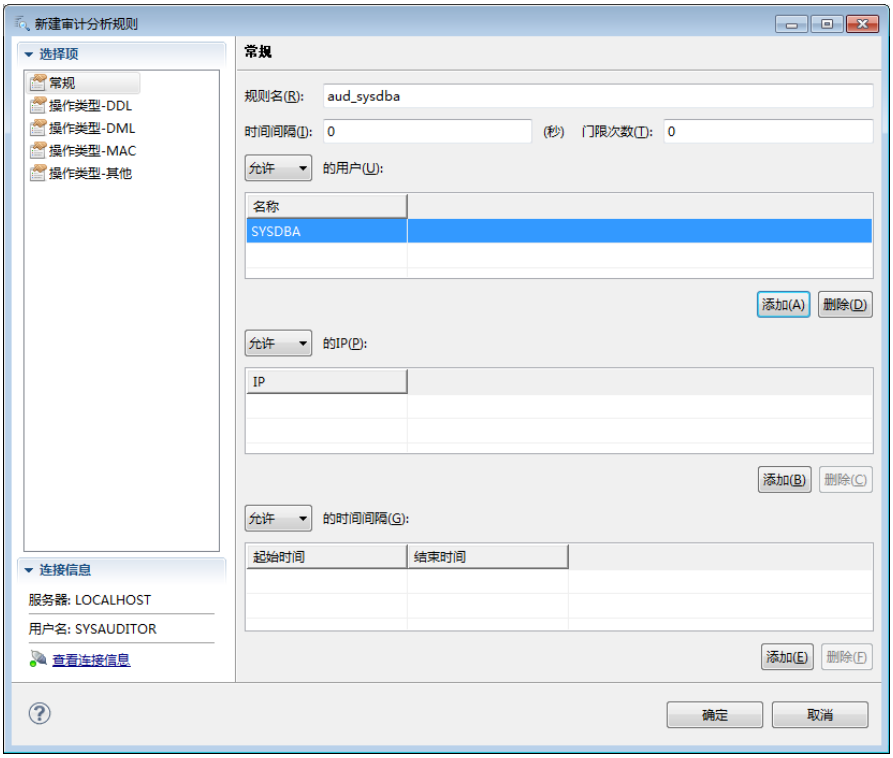


图 5.2 新建审计分析规则窗口

如上图中创建了一个名为 aud_sysdba 的审计分析规则，对 SYSDBA 的所有审计记录进行分析。之后我们就可以将这个审计分析规则应用于对审计记录文件的分析。右键点击“审计规则”或一个具体的审计分析规则节点，在弹出的菜单中选择“审计规则分析”，弹出如下图所示审计规则分析窗口。

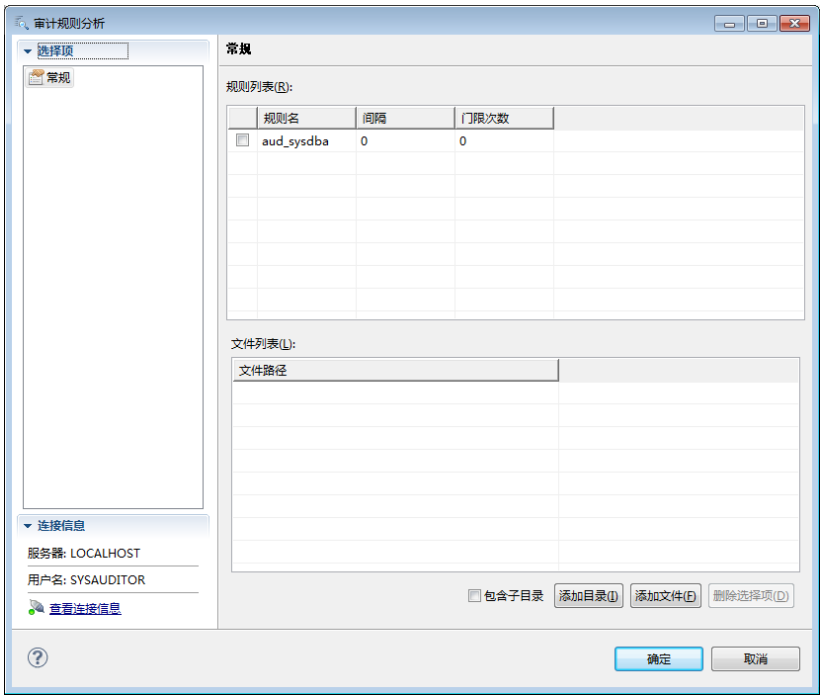


图 5.3 审计规则分析窗口

选择需要应用的审计分析规则，可以同时选择多条，在下面的文件列表中添加需要进行分析的审计记录文件，Analyzer 工具就会根据审计分析规则对文件中的审计记录进行分析，将满足规则的审计记录以表格的形式显示出来，如下图所示。

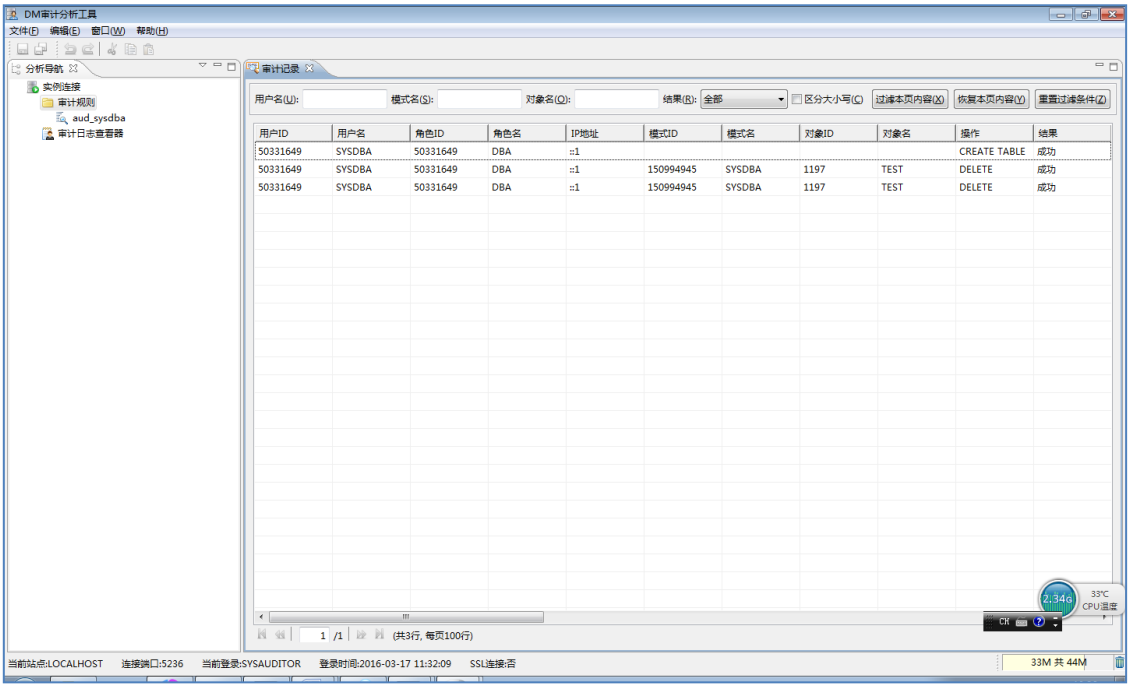


图 5.4 审计规则分析结果

在审计规则分析结果显示窗口中，还可以对显示的审计记录根据需要进行进一步过滤，进一步分析出需要的审计记录。

5.6.1.2 审计日志查看器

双击“审计日志查看器”节点，可打开如下图所示的条件设置窗口。

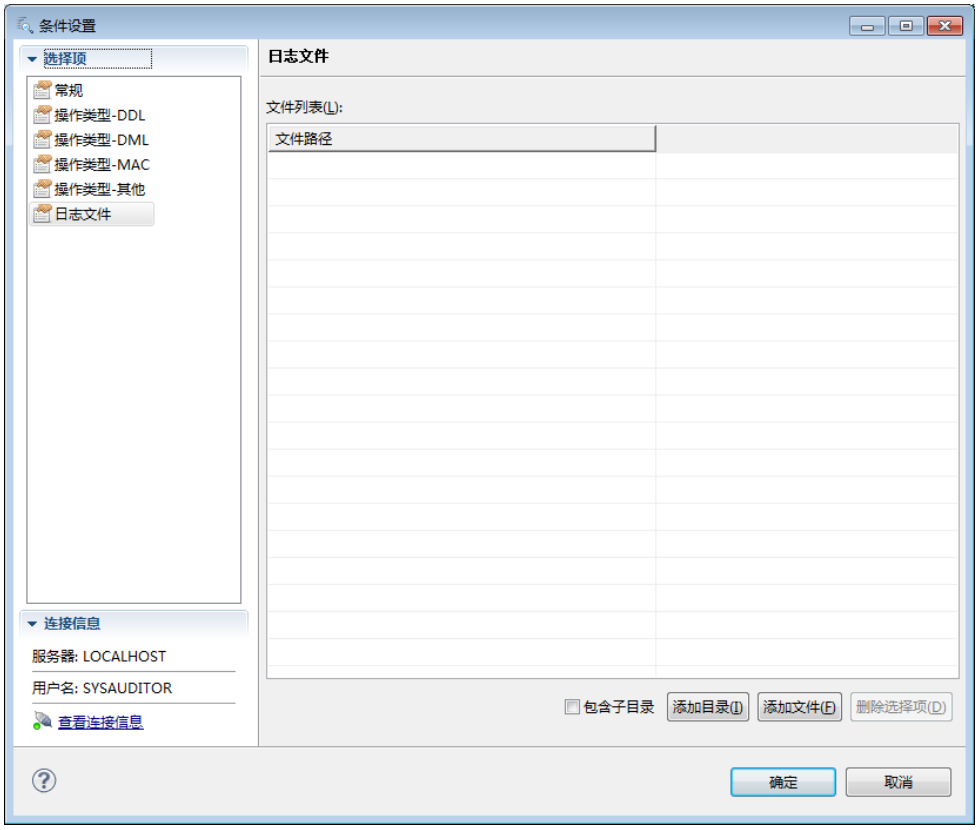


图 5.5 审计日志查看条件设置窗口

添加需要查看的审计记录文件，还可在窗口左侧的“选择项”中设置各种过滤条件，之后点击“确定”，将满足规则的审计记录以表格的形式显示出来，与图 5.5 的显示相同。

5.6.2 审计分析工具 dmaudtool

审计信息以审计记录的形式存放在 DM 数据库审计文件中，关于审计文件的存放路径等信息请参见 5.3 审计文件管理。审计文件的内容是不可视的，审计分析工具 dmaudtool

可以筛选解析审计文件中的审计记录并且导出至指定输出文件当中以供用户查看。只有具有 DB_AUDIT_SOI 角色权限的用户可以使用审计分析工具 dmaudtool。

分析本地磁盘上的审计文件时，要求 dmaudtool 工具与审计文件在同一台主机上。分析 DMASM 文件系统中的审计文件时，不要求 dmaudtool 工具与审计文件在同一台主机上。

5.6.2.1 使用 dmaudtool 工具

dmaudtool 工具位于 DM 安装目录的 bin 目录下。启动 dmaudtool 工具的命令格式为：

```
dmaudtool USERID=<userid> AFIL_PATH=<afil_path> OUT_PATH=<out_path> {PARAMETER=<value>}
```

USERID：指定数据库的连接信息，必选参数。详细参数介绍请参考 5.5.3.2 使用 dmamon。

AFIL_PATH：指定审计文件所在的路径，必选参数。

OUT_PATH：指定审计分析结果的输出文件完整路径，必选参数。若指定的输出文件已存在，则报错。

PARAMETER：其他可选参数，多个参数之间排列顺序无影响，参数之间使用空格间隔。参数将在下一节进行详细介绍。

<value>：参数取值。

例如：分析 E:\dmdbms\data\DAMENG 目录下的所有审计文件，并将分析结果输出至 E:\dmdbms\data\DAMENG\out\aud_out.txt 文件中，行间隔符设为 AAAA，输出文件大小不限。命令行操作如下：

```
dmaudtool USERID=SYSAUDITOR/DMauditor_123@192.168.1.60:5236
AFIL_PATH=E:\dmdbms\data\DAMENG
```

```
OUT_PATH=E:\dmdbms\data\DAMENG\out\aud_out R_SEP=AAAA
```

5.6.2.2 dmaudtool 参数一览表

本节提供 dmaudtool 的参数一览表，供用户快速参考。

表 5.12 dmaudtool 参数一览表

参数	含义	备注
USERID	指定数据库的连接信息	必选。 仅支持本地连接
AFIL_PATH	指定审计文件所在的路径	必选
OUT_PATH	指定审计分析结果的输出文件完整路径	必选。与 OUT_SIZE 配合使用，若文件大小超过 OUT_SIZE 指定值，则会在目录下生成同名+序号的新输出文件
UPDATE_KEY	更新审计日志文件密钥	可选。专门用于使用更新审计日志密钥功能的场景中
OUT_SIZE	指定审计分析结果输出文件的大小，单位为兆（M）。指定范围为 500~65534（M）	可选。缺省为 0，表示不限制文件大小
DCR_INI	DCR_INI 路径，分析存储在 DMASM 上的审计文件时指定	分析存储在 DMASM 上的审计文件时必选，否则可选，此时系统将不考虑该参数值
USERNAME	对应审计记录中的用户名，表示收集该用户名的审计记录	可选。若未指定则不考虑该参数值，若均未指定则默认分析指定目录下所有审计文件
SCHNAME	对应审计记录中模式名，表示收集该模式名的审计记录	
OBJNAME	对应审计记录中对象名，表示收集该对象名的审计记录	
TIME_FROM	对应审计记录中操作时间，用于指定收集审计记录的起始时间	

TIME_TO	对应审计记录中操作时间，用于指定收集审计记录的结束时间	
R_SEP	指定输出文件中的行分隔符，用于间隔输出文件中行与行之间的数据	可选。缺省为回车符。考虑到记录中可能存在的字符，间隔符需要设置相对复杂，避免解析出错
C_SEP	指定输出文件中的列分割符，用于间隔输出文件中列与列之间的数据	可选。缺省为'|'
HELP	打印帮助信息	可选

5.6.2.3 dmaudtool 工具使用示例

5.6.2.3.1 帮助 HELP

使用 help 参数即可查看帮助信息。帮助信息中会显示 dmaudtool 版本信息以及所有参数的大致信息，供用户快速参考。

命令行操作如下：

```
dmaudtool help
```

5.6.2.3.2 分析指定目录下所有审计文件

分析 E:\dmdbms\data\DAMENG 目录下的所有审计文件，将分析结果保存至 E:\dmdbms\data\DAMENG\out\aud_out.txt 文件中，行间隔符设为 AAAA，输出文件大小不限。

命令行操作如下：

```
dmaudtool USERID=SYSAUDITOR/DMauditor_123@192.168.1.60:5236
AFIL_PATH=E:\dmdbms\data\DAMENG
OUT_PATH=E:\dmdbms\data\DAMENG\out\aud_out_1.txt R_SEP=AAAA
```

5.6.2.3.3 分析指定用户的审计记录

使用 USERNAME 参数指定用户名，分析审计文件中指定用户的所有审计记录。

命令行操作如下：

```
dmaudtool USERID=SYSAUDITOR/DMauditor_123@192.168.1.60:5236
AFIL_PATH=E:\dmdbms\data\DAMENG
OUT_PATH=E:\dmdbms\data\DAMENG\out\aud_out_2.txt R_SEP=AAAA USERNAME=USER01
```

5.6.2.3.4 分析指定时间之后的审计文件

使用 TIME_FROM 参数指定起始时间，分析指定时间之后的所有审计文件。

命令行操作如下：

```
dmaudtool USERID=SYSAUDITOR/DMauditor_123@192.168.1.60:5236
AFIL_PATH=E:\dmdbms\data\DAMENG
OUT_PATH=E:\dmdbms\data\DAMENG\out\aud_out_3.txt R_SEP=AAAA
TIME_FROM="2022/9/1 21:00:00"
```

5.6.2.3.5 分析存储在 DMASM 上的审计文件

若审计文件存储在 DMASM 文件系统中，则需要设置 DCR_INI 参数，指定 dmdcr.ini 文件的路径。

命令行操作如下：

```
./dmaudtool USERID=SYSAUDITOR/DMauditor_123@192.168.100.121:5425
AFIL_PATH=+DMDATA/home/data/DAMENG OUT_PATH=/home/test/out/aud_out.txt
DCR_INI=/home/data/DAMENG/DSC01/dmdcr.ini R_SEP=AAAA
```

5.6.2.3.6 更新审计日志文件密钥

通过指定 UPDATE_KEY 参数更新审计日志文件密钥。

使用说明：

1. 审计日志文件默认不使用加密算法进行加密，对于不加密文件则不更新密钥。

2. dmaudtool 密钥更新功能不更新当前最新审计日志文件的密钥。使用密钥更新前需要先通过审计文件加密系统过程 sp_audit_set_enc 来设置更新密钥，切换新的审计日志文件，然后使用 dmaudtool 工具更新之前审计日志密钥。

3. USERID 需要指定具有审计员权限用户，否则无法进行密钥更换。

4. UPDATE_KEY 指定为 NULL，实际没有用到。

命令行操作如下：

```
dmaudtool.exe USERID=SYSAUDITOR/DMauditor_123@192.168.0.33:4356
AFIL_PATH=c:\dm7data\dameng\aud UPDATE_KEY=NULL
```

5.6.2.4 审计分析结果导入到目标库中

将审计分析结果导入到目标库中需要利用 DM 自带的导入导出工具 dmfldr。将 dmaudtool 的分析结果输出文件作为 dmfldr 的输入文件，并配置与 dmaudtool 分隔符相匹配的控制文件即可完成导入工作。dmaudtool 按照动态视图 V$AUDITRECORDS 的行列顺序输出审计结果，因此需要在目标库中创建一个与动态视图 V$AUDITRECORDS 结构一样的目标表，用以存放审计记录。

以章节 5.6.2.3.4 分析指定时间之后的审计文件中的分析结果输出文件为例，操作步骤如下：

1. 在目标库中创建一个与动态视图 V$AUDITRECORDS 结构相同的目标表；

SQL 语句如下：

```
CREATE TABLE aud_test
(
"USERID" INTEGER,
```

```
"USERNAME" VARCHAR(128),
"ROLEID" INTEGER,
"ROLENAME" VARCHAR(128),
"IP" VARCHAR(64),
"SCHID" INTEGER,
"SCHNAME" VARCHAR(128),
"OBJID" INTEGER,
"OBJNAME" VARCHAR(128),
"OPERATION" VARCHAR(128) NOT NULL,
"SUCC_FLAG" CHAR(1) NOT NULL,
"SQL_TEXT" VARCHAR(8188),
"DESCRIPTION" VARCHAR(8188),
"OPTIME" DATETIME(6) NOT NULL,
"MAC" VARCHAR(25),
"SEQNO" BYTE,
"BIND_INFO" VARCHAR(8188)
);
```

2. 创建控制文件 out.ctrl，并存放到 E:\dmdbms\data\DAMENG 目录下；

out.ctrl 中内容如下：

```
LOAD DATA
INFILE 'E:\dmdbms\data\DAMENG\out\aud_out_3.txt' STR 'AAAA'
INTO TABLE aud_test
FIELDS '|'
(
"USERID",
"USERNAME",
"ROLEID",
"ROLENAME",
"IP",
"SCHID",
```

```
"SCHNAME",
"OBJID",
"OBJNAME",
"OPERATION",
"SUCC_FLAG",
"SQL_TEXT",
"DESCRIPTION",
"OPTIME",
"MAC",
"SEQNO",
"BIND_INFO"
)
```

3. 打开命令行窗口，进入 DM 安装目录的 bin 目录，运行 dmfldr，将分析结果导入到目标表中。

命令行操作如下：

```
dmfldr userid=SYSDBA/DMdba_123@192.168.1.60:5236
control='e:\dmdbms\data\DAMENG\out.ctrl'
```



注意： **审计记录中可能存在过长的执行语句，所以在导入目标表时可能出现"记录超长"的错误，可以使用新库作为目标库，初始化数据库时调大建库参数 page_size。例如，存在一个执行语句长度为 3900 字节，可以设置 page_size=32。**

5.7 集群环境审计说明

本节主要介绍主备集群、DMDSC 集群以及 DMDPC 集群环境下的审计文件生成规则。

- 主备集群环境

主库和备库各自生成审计文件，并且只能查询自身审计文件中的信息。

主库节点审计规则与单机一致，可以根据审计设置正常审计所有内容，并查询相关所有审计信息。

备库节点不会加载审计系统表，除了系统启动、关闭或者 OPEN 时会强制生成审计信息外，不会生成其他审计信息。

- DMDSC 集群环境

DMDSC 集群环境下，每个节点各自生成审计文件，可以正常审计所有内容，在任意节点上均可查询到所有节点的审计信息。

- DMDPC 集群环境

DMDPC 集群环境下，仅 MP 节点生成审计文件。其他节点会从 MP 节点获取审计设置信息，生成相应审计内容，最终都由 MP 节点写入审计文件。在任意节点上均可查询到所有审计信息，但使用 Analyzer 工具进行审计日志解析时，只能连接 MP 节点。

6 通信加密

DM 提供两种通信加密方式：一基于传输层的 SSL 协议加密；二基于应用层的消息包加密。两个层次的加密互不影响，可以同时使用。因为任何一种加密方式都足够安全，而且双重加密会占用更多的数据库资源，所以通常情况下，没必要同时开启这两种加密。

当未采用通信加密时，为了提高安全性，数据库系统自动对登录消息中的用户名密码使用内部算法进行了简单的加解密操作，这里的简单加解密操作为使用 AES256_CFB 算法加解密。如果用户需要对用户名密码进行增强加密，请参考 11 登录用户名密码增强加密。

下面对基于传输层的 SSL 协议加密和基于应用层的消息包加密进行详细介绍。

6.1 基于传输层的 SSL 协议加密

6.1.1 SSL 模式说明

达梦数据库为用户提供了不同的 SSL 加密认证机制，具体内容和配置需求如下：

1. 基于 SSL 协议

SSL 仅加密：仅对通信数据进行 SSL 加密。客户端和服务器需要配置 SSL 环境，但不需要加载证书。

SSL 仅认证：仅对通信双方进行 SSL 证书验证。客户端和服务器需要配置 SSL 环境，双方都需要加载 CA 证书和自己的证书，并验证对端证书的有效性。

SSL 认证加密：对通信双方进行 SSL 证书验证并对通信数据进行加密。客户端和服务器需要配置 SSL 环境，双方都需要加载 CA 证书和自己的证书，并验证对端证书的有效性。

SSL 单向认证和加密：客户端单向校验服务器端证书的有效性并对通信数据进行加密。客户端和服务器需要配置 SSL 环境，服务器端需要加载 CA 证书和服务器证书，客户端需要加载 CA 证书并验证服务器端证书的有效性。

2. 基于商密 TLCP 协议

商密 TLCP 认证加密：OpenEuler 操作系统在其发布的开源版本基础上增加了对商密 TLCP 协议支持的 OpenSSL，基于商密 TLCP 协议对通信双方进行证书验证和通信数据的加密。客户端和服务器需要配置 SSL 环境，双方都需要加载签名证书和加密证书，并验证对端证书的有效性。

SSL 参数介绍如下：

用户可通过设置服务器配置文件 DM.INI 中的 ENABLE_ENCRYPT 参数来指定 SSL 协议加密的模式，客户端以服务器采用的通信方式与其进行通信，具体如下：

ENABLE_ENCRYPT 取值 0、1、2、3、4 或 5，缺省值为 0。不论设置为 1、2、4 或 5，它内部所使用的密码套件是 SSL 协议自动协商的，达梦未进行任何干涉。

0：通信数据不经 SSL 加密，通信双方也不进行证书验证。

1：通信数据会被加密，并且通信双方需要交换并验证对方证书是否匹配。

2：通信数据不会被加密，但通信双方会交换并验证对方证书是否匹配。

4：通信数据会被加密，但通信双方不需要验证证书。

5：通信数据会被加密，仅客户端会验证服务器端证书。

6：通信数据基于商密 TLCP 协议加密，通信双方交换并验证对方证书是否匹配。

另外，DM.INI 中的 MIN_SSL_VERSION 参数指定了允许连接的 SSL 最低版本，此参数设置仅当 ENABLE_ENCRYPT 不为 0 时有效。MIN_SSL_VERSION 的可取值包括：

0：所有版本

0x0301：TLSv1

0x0302：TLSv1.1

0x0303：TLSv1.2

0x0304：TLSv1.3

MIN_SSL_VERSION 缺省为 0，为静态 INI 参数，修改后需要重启 DM 数据库服务器才能生效。

6.1.2 SSL 环境配置

当 ENABLE_ENCRYPT 配置为 0 以外的值时，需要在服务器和客户端分别配置 SSL 环境。若服务器未配置 SSL 环境则服务器无法正常启动，若客户端未配置 SSL 环境则无法连接服务器。

安装达梦数据库时会自动配置 SSL 环境，即在客户端/服务器端配置 Openssl 的核心加密库和 SSL/TLS 协议库。

当服务器/客户端启动时，按照如下顺序在机器上查找动态库：

1. 在当前目录下查找动态库文件。

2. 在当前目录下的 dependencies 文件夹下查找动态库文件。

3. 在 DM 安装目录下 bin 文件夹、即$DM_HOME/bin 目录下查找。

4. $DM_HOME/bin/dependencies 目录下查找。

5. 在系统上查找。

6.1.3 SSL 证书配置

- ENABLE_ENCRYPT 取值为 0 或 4。

通信双方不进行证书验证，不需要进行证书配置。

- ENABLE_ENCRYPT 取值为 1 或 2。

通信双方需要交换并验证对方证书是否匹配，需要先联系证书颁发机构获取 SSL 证书及证书的密码，具体如下：

服务器证书：CA 证书（ca-cert.pem）、服务器证书（server-cert.pem）、服务器私钥（server-key.pem）。

客户端证书：.keystore 文件、CA 证书（ca-cert.pem）、客户端证书（client-cert.pem）、客户端私钥（client-key.pem）。

- ENABLE_ENCRYPT 取值为 5。

仅客户端验证服务器端证书，在客户端目录中仅需存放 CA 证书，用于对服务器证书的校验。需要联系证书颁发机构获取相关证书及密码，具体如下：

服务器证书：CA 证书（ca-cert.pem）、服务器证书（server-cert.pem）、服务器私钥（server-key.pem）。

客户端证书：.keystore 文件、CA 证书（ca-cert.pem）。

- ENABLE_ENCRYPT 取值为 6。

通信数据基于商密 TLCP 协议进行加密，通信双方交换并验证对方证书是否匹配。需要联系证书颁发机构获取相关证书及密码，具体如下：

服务器证书：服务器加密证书（server-enc.crt），服务器加密 key（server-enc.key），服务器签名证书（server-sign.crt），服务器签名 key（server-sign.key）

客户端证书：客户端加密证书（client-enc.crt），客户端加密 key（client-enc.key），客户端签名证书（client-sign.crt），客户端签名 key（client-sign.key）

证书放置目录：

证书获取后，需要将准备好的证书放到数据库服务器和客户端的指定目录下。

服务器目录：$DM_HOME/bin/server_ssl

客户端目录：$DM_HOME/bin/client_ssl/username，或者在客户端的配置文件中设置 ssl_path，指定客户端证书存放目录。其中，当用户名中含有具有大小写概念的多字节字符时，连接服务器时用户名需要与客户端证书存放目录 $DM_HOME/bin/client_ssl/username 中文件名以及证书中的通用名保持一致，否则无法连接。

6.1.4 服务器配置

系统管理员登录数据库。

```
D:\dmdbms\bin>disql.exe SYSDBA/DMsys_123@localhost:5236
```

通过查询 V$PARAMETER 动态视图查询 ENABLE_ENCRYPT 的当前值。

```
SQL>SELECT * FROM V$PARAMETER WHERE NAME='ENABLE_ENCRYPT';
```

调用系统过程 SP_SET_PARA_VALUE 系统过程修改 ENABLE_ENCRYPT 参数的取值。

```
--取值 1 表示开启 SSL 加密认证
SQL>SP_SET_PARA_VALUE(2,'ENABLE_ENCRYPT',1);
```

指定 TLS 协议版本。

```
--TLSv1.2
SQL>SP_SET_PARA_VALUE(2,'MIN_SSL_VERSION', 0x0303);
```

重新启动数据库服务器，使 INI 参数生效。

6.1.5 客户端连接

使用 DISql 登录时可以在连接串中指定用户的 SSL 证书路径(ssl_path)。若证书私钥使用密码加密，需要在连接串中指定 ssl_pwd 参数，如果证书没有密码则可以缺省或使用任意数字代替。

```
D:\dmdbms\bin>disql.exe SYSDBA/DMsys_123
@localhost:5236#"{ssl_path=D:\dmdbms\bin\client_ssl\SYSDBA,ssl_pwd=12345}"
```

可以通过查询 V$DM_INI 视图中的 ENABLE_ENCRYPT 参数确定是否开启 SSL 加密通信。

```
SQL> select para_name, para_value FROM V$DM_INI where para_name='ENABLE_ENCRYPT';

行号     PARA_NAME     PARA_VALUE

---------- -------------- ----------

1        ENABLE_ENCRYPT 1
```

使用 DM 管理工具连接数据库时，可在“高级”选项卡的连接属性新增 sslFilesPath 属性和 sslKeystorePass 属性，分别用于指定证书目录和.keystore 文件密码，没有密码时可输入默认密码 changeit。

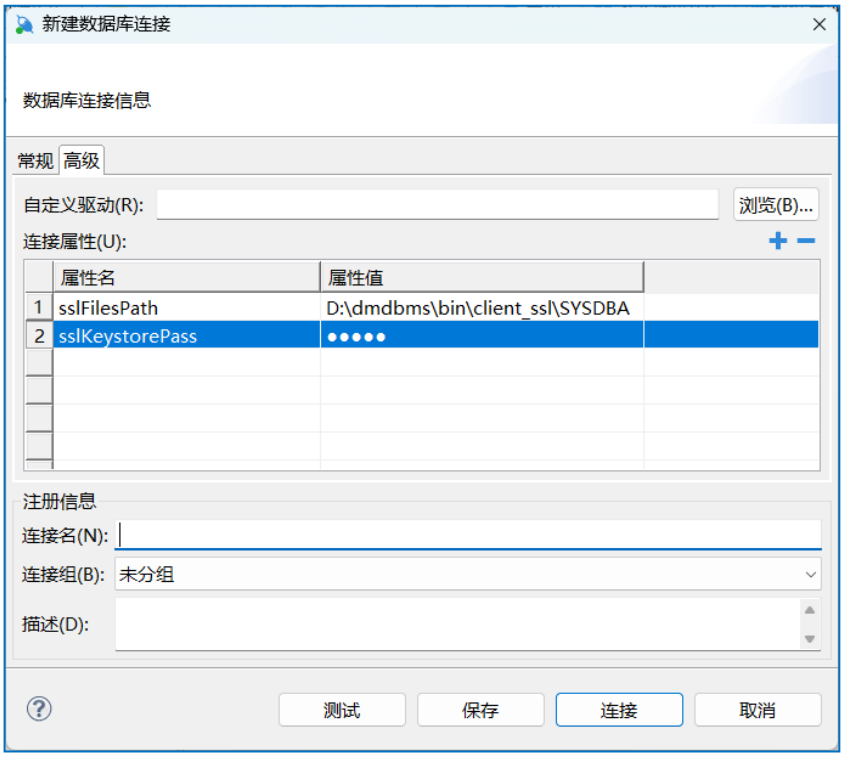


图 6.1 DM 管理工具基于 SSL 协议连接数据库

6.1.6 举例说明

以 ENABLE_ENCRYPT=5，进行 SSL 单向认证和加密的情形为例进行说明。

第一步：获取 SSL 证书，保证用户拥有自己的 SSL 数字证书，并且确认 DM 服务器和 DM 客户端均已配置好 SSL 环境。

第二步：修改 ENABLE_ENCRYPT 参数取值。

系统管理员使用 DISql 连接服务器。

```
D:\dmdbms\bin>disql.exe SYSDBA/DMsys_123@localhost:5236
```

在数据库中创建 DM_ssl 用户。

```
SQL>CREATE USER DM_ssl IDENTIFIED BY "Dm_12345";
```

使用 SP_SET_PARA_VALUE 系统过程将 ENABLE_ENCRYPT 参数的取值修改为 5。

```
SQL> SP_SET_PARA_VALUE(2,'ENABLE_ENCRYPT',5);
```

重新启动数据库服务器，使 INI 参数生效。

第三步：配置 SSL 证书。

当 ENABLE_ENCRYPT 取值为 5 时，客户端对服务器端进行单向的证书校验。因此在 server_ssl 目录下，需要存放 CA 证书（ca-cert.pem）、服务器证书（server-cert.pem）与服务器私钥（server-key.pem），在 client_ssl 目录下的 DM_ssl 用户目录中，仅需存放 CA 证书，用于对服务器证书的校验。

第四步：数据库连接。

DM_ssl 用户连接 DM 服务器，输入用户名和密码、DM 服务器 IP 地址和端口号以及当前登录用户自身的 SSL 数字证书路径以及相应的 SSL 证书密码。

```
D:\dmdbms\bin>disql.exe
DM_ssl/Dm_12345@localhost:5236#"{ssl_path=D:\dmdbms\bin\client_ssl\DM_ssl,ssl_pwd
=12345}"
```

第五步：确定是否通信加密开启成功。

系统管理员通过查询 V$DM_INI 视图中的 ENABLE_ENCRYPT 参数确定是否开启 SSL 加密通信。

```
SQL> select para_name, para_value FROM V$DM_INI where para_name='ENABLE_ENCRYPT';

行号     PARA_NAME     PARA_VALUE

---------- -------------- ----------

1        ENABLE_ENCRYPT 5
```

6.1.7 常见问题

1. 认证方式：

通过设置 INI 参数 ENABLE_ENCRYPT 来指定认证方式，调用系统过程修改该参数后需要重启数据库服务器使参数生效。

2. 客户端登录失败的原因：

1）环境未正确配置：

INI 参数 ENABLE_ENCRYPT 大于 0 时，根据具体取值，客户端需要配置 SSL 环境。否则报错初始化 SSL 环境失败。

2）证书加载失败：

INI 参数 ENABLE_ENCRYPT 设置为客户端需要加载证书的取值时，客户端需要在指定位置配置 SSL 证书，或在配置信息中指定证书文件地址。否则报错“初始化 SSL 环境失败”。

3）建立连接失败：

可能是证书校验失败等原因，报错“网络通讯失败”。

6.2 基于应用层的消息包加密

选择是否使用基于应用层的消息包加密以 DM 数据库服务器端的设置为准，即通过设置服务器配置文件 DM.INI 中的 COMM_ENCRYPT_NAME 参数来指定，客户端以服务器采用的通信方式与其进行通信。

COMM_ENCRYPT_NAME 取值空串、算法名分别代表不加密、加密。缺省为空串。算法名可以通过查询动态视图 V$CIPHERS 获取。本节的算法名必须为分组加密算法或流加密算法(即 CYT_TYPE 为 1 或 2)，且不为非填充模式（即 WORK_MODE 不能是 ECB_NOPAD 或 CBC_NOPAD）。当算法名写错，则使用加密算法 AES256_CFB。应用层加密的密钥是通过 DH 密钥交换算法协商生成。

用户可通过 V$CIPHERS 动态视图查询达梦支持的 COMM_ENCRYPT_NAME 算法名。

```
SELECT CYT_NAME FROM V$CIPHERS WHERE (CYT_TYPE=1 OR CYT_TYPE=2 ) AND
(WORK_MODE!='ECB_NOPAD' AND WORK_MODE!='CBC_NOPAD');
```

用户可通过 V$PARAMETER 动态视图查询 COMM_ENCRYPT_NAME 的当前值。

```
SELECT * FROM V$PARAMETER WHERE NAME='COMM_ENCRYPT_NAME';
```

也可以通过使用客户端工具 Console 或调用系统过程 SP_SET_PARA_STRING_VALUE 重新设置 COMM_ENCRYPT_NAME 的值。不过由于这个参数都为静态 INI 参数，修改后需要重启 DM 数据库服务器才能生效。

```
SP_SET_PARA_STRING_VALUE(2,'COMM_ENCRYPT_NAME','DES_OFB');
```

7 存储加密

为了防止用户直接通过数据文件获取用户信息，DM 提供了全面的数据加密的功能，包括:

- ✓ 透明加密
- ✓ 半透明加密
- ✓ 非透明加密

存储加密在保证数据文件安全性的同时，也会带来一定的性能影响，不同的加密算法对性能的影响各有不同，用户需要根据自己的需求来决定是否进行加密以及加密算法的选择。

DM 使用服务器主密钥 SVR_KEY 和数据库主密钥 DB_KEY 进行分级存储加密。SVR_KEY 为服务器主密钥，用于加密数据库主密钥；DB_KEY 用于加密库内密钥，如用户加密密钥、列加密密钥等。

DM 安全版提供了两个系统过程 SP_UPDATE_SVRKEY() 和 SP_UPDATE_DBKEY()，分别用于更新密钥 SVR_KEY 和 DB_KEY。

例 更新服务器主密钥 SVR_KEY。

```
SQL> SP_UPDATE_SVRKEY();
```

7.1 透明加密

在透明加密中，密钥生成、密钥管理和加解密过程由数据库管理系统自动完成，用户不可见。透明加密的目的主要是保证存储在数据文件中的敏感数据的安全，并不能保护合法用户的个人私密数据。

7.1.1 全库加密

对 DM 数据库进行全库加密需要在初始化数据库时通过 ENCRYPT_NAME 参数指定全库加密算法，加密密钥由 DM 自动生成。若初始化数据库时不指定 ENCRYPT_NAME 参数，则不进行全库加密。

对全库加密使用的算法存在以下限制：

1. 加密算法的类型应为 CYT_TYPE_SYM_BLOCK_ENCRYPT 或 CYT_TYPE_SYM_STREAM_ENCRYPT，加密算法类型介绍请参考 8.1.1 算法信息相关接口中的表 8.2；
2. 与数据库初始化参数 PAGE_ENC_SLICE_SIZE 相关

   若 PAGE_ENC_SLICE_SIZE 为 0：

   a) 不支持算法模式为 WORK_MODE_EXTKEY 的加密算法，算法模式介绍请参考 8.1.1 算法信息相关接口中的表 8.3，下文中相同；

   b) 不支持块大小（blk_size）为 0，但扩展长度（EXTEND_SIZE）>4 的加密算法；

   c) 不支持块大小（blk_size）不为 0，但扩展长度（EXTEND_SIZE）> blk_size 的加密算法；

   d) 不支持块大小（blk_size）>16 的加密算法。

   若 PAGE_ENC_SLICE_SIZE 为非 0：

   a) 不支持扩展长度（EXTEND_SIZE）超过分片大小的加密算法；

   b) 不支持块大小（blk_size）>64 的算法模式为 WORK_MODE_EXTKEY 的加密算法；

   c) 不支持块大小（blk_size）>16 的算法模式非 WORK_MODE_EXTKEY 的加密算法；

d) 不支持扩展长度（EXTEND_SIZE）为 0，但算法模式为 WORK_MODE_ECB 或 WORK_MODE_CBC 的加密算法；

e) 不支持扩展长度为（EXTEND_SIZE）0，但通过 cipher_get_cipher_text_size()计算得到密文长度与明文不同的加密算法。

指定全库加密时，在 DM 数据库服务器启动及运行的过程中，需要对处理的所有数据页通过指定的加密算法和 DM 自动生成的密钥进行加解密处理。如果数据页读入缓存，需要进行解密后才能使用，在缓存中的数据页进行刷盘时，需要对数据页进行加密后存储到数据文件中。

系统存储函数 SF_GET_ENCRYPT_NAME() 用于获取全库加密算法名。未使用全库加密时返回 NULL。

例 使用了全库加密算法 xor1。

```
SQL> select SF_GET_ENCRYPT_NAME();

行号      SF_GET_ENCRYPT_NAME()

---------- ----------------------

1         xor1
```

7.1.2 表空间透明加密

DM 支持在创建表空间时指定进行透明加密，语法格式为：

```
CREATE TABLESPACE [IF NOT EXISTS] <表空间名> <数据文件子句> [<数据页缓冲池子句>] [<存储加密子句>]

<存储加密子句> ::= ENCRYPT WITH <加密算法> [BY <加密密码>]
```

说明： 此处主要说明存储加密相关的语法，其余语法的详细描述请参考

《DM8_SQL 语言使用手册》，后续小节中相同，不再说明。

使用说明：

1. <加密密码>必须满足长度大于等于 INI 参数 PWD_MIN_LEN 设置的值，同时不超过 32，且包含大写、小写、数字。若未指定，由 DM 随机生成；
2. 如果已指定全库加密，则不再支持表空间加密；
3. <加密算法>的限制与全库加密的基本相同，另外，在 PAGE_ENC_SLICE_SIZE 非 0 时，表空间加密还不支持进行加密扩展的算法，如算法模式为 WORK_MODE_ECB 或 WORK_MODE_CBC 的加密算法、扩展长度>0 的加密算法、通过 cipher_get_cipher_text_size()计算得到密文长度与明文不同的加密算法，算法模式介绍请参考 8.1.1 算法信息相关接口中的表 8.3。

7.1.3 表列透明加密

DM 支持对表的列进行透明加密，支持建表时设置加密列，以及修改表定义时设置加密列。存储加密支持所有的列类型，包括大字段类型。用透明加密的方式加密列上的数据时，在数据库中保存加密该列的密钥，执行 DML 语句的过程中系统能自动获取密钥。

建表或修改表时指定对列进行透明加密的语法格式为：

```
……
<列定义> ::= <不同类型列定义> [<列定义子句>] [<STORAGE 子句>][<透明存储加密子句>]
<透明存储加密子句> ::= <透明存储加密子句 1>|<透明存储加密子句 2>
<透明存储加密子句 1>::= ENCRYPT [<透明加密用法>]
<透明存储加密子句 2> ::= ENCRYPT <透明加密用法><散列选项>
<透明加密用法> ::= WITH <加密算法> [<透明加密选项>]|
                   <透明加密选项>
```

<散列选项> ::= HASH WITH<散列算法> [<加盐选项>]

<加盐选项> ::= [NO] SALT

<透明加密选项> ::= <透明加密选项 1> |<透明加密选项 2> |<透明加密选项 3>

<透明加密选项 1> ::= AUTO

<透明加密选项 2> ::= AUTO BY <列存储密钥>

<透明加密选项 3> ::= AUTO BY WRAPPED <列存储密钥的密文>

上述语法格式不完整，主要说明与表列透明加密相关的部分，其他建表相关语法请参考《DM8_SQL 语言使用手册》。

使用说明：

1. 当使用列加密但没有指定加密算法时，缺省使用 AES256_CBC 加密算法；
2. 指定列加密算法时，算法类型应为 CYT_TYPE_SYM_BLOCK_ENCRYPT 或 CYT_TYPE_SYM_STREAM_ENCRYPT，且算法模式不为 WORK_MODE_EXTKEY、WORK_MODE_ECB_NOPAD 或 WORK_MODE_CBC_NOPAD（加密算法类型和算法模式介绍请参考 8.1.1 算法信息相关接口）；虽 DM 支持指定列加密算法为 RC4，但由于该算法安全性较低，并不推荐使用；
3. 散列算法支持 MD5、SHA1 等（可以通过查询动态视图 V$CIPHERS 获取更多支持的散列算法：SELECT CYT_NAME FROM V$CIPHERS WHERE CYT_TYPE=4;），用于保证用户数据的完整性，若用户数据被非法修改，则能判断该数据不合法；
4. <透明加密选项>用于指定对当前列进行按列加密。具体规则如下：
   1) <透明加密选项>用于指定对该列进行按列加密。<透明加密选项>缺省和<透明加密选项 1>均使用系统自动生成的秘钥进行列加密。<透明加密选项 2>和<透明加密选项 3>使用各自指定的密钥进行列加密。透明加密中按列加密的好处是用户可根据需要灵活设置不同的列秘钥。其中<透明加密选项 2>使用

之前需先设置 INI 参数 AUTO_ENCRYPT=1，打开支持列透明加密时用户手动输入<列存储密钥>开关；

2） <列存储密钥>必须满足长度大于等于 INI 参数 PWD_MIN_LEN 设置的值，小于等于 128 个字符，且包含大写、小写、数字。

例 以下是一些对表列进行透明加密的例子。

```
CREATE TABLE TEST_ENCRYPT1(C1 INT, C2 INT ENCRYPT);
CREATE TABLE TEST_ENCRYPT2(C1 INT, C2 INT ENCRYPT WITH DES_ECB);
CREATE TABLE TEST_ENCRYPT5(C1 INT, C2 INT ENCRYPT WITH DES_ECB  HASH WITH MD5 SALT);
```

7.1.4 其他数据库对象加密

DM 还支持对存储过程、存储函数、触发器、包、类、自定义类型等的定义进行加密，创建时在对象名称后加上“WITH ENCRYPTION ”即可。

例 创建存储过程 proc_arg，对其定义进行加密。

```
CREATE OR REPLACE PROCEDURE proc_arg WITH ENCRYPTION(a IN INT, b INT)
AS
BEGIN
        a:=0;
        b:=A+1;
END;
```

7.1.5 重做日志文件加密

DM 支持对联机日志文件和归档日志文件进行加密，可在创建数据库时通过建库参数 RLOG_ENCRYPT_NAME 指定重做日志文件的加密算法，若未指定则不加密。

RLOG_ENCRYPT_NAME 支持使用第三方加密算法，但不支持算法模式为 WORK_MODE_CBC_NOPAD 、 WORK_MODE_ECB_NOPAD 或

WORK_MODE_EXTKEY 的加密算法，加密算法模式介绍请参考 8.1.1 算法信息相关接口中的表 8.3。

7.2 半透明加密

在 2.2.1 创建用户章节中可以看到，创建用户时可以指定存储加密密钥，这个密钥就是为了进行半透明加密时使用的。如果在创建用户时并没有指定存储加密密钥，系统也会自动为用户生成一个默认的存储加密密钥。

创建表或修改表时可以对表列指定半透明加密，指定后 DM 会使用用户的存储加密密钥对数据进行加密。半透明加密列的密文后追加了 4 个字节的 UID，用户查询数据时，若当前会话用户 ID 与列密文数据中存储的 UID 相同，则返回明文，否则返回 NULL。

创建表或修改表时对表列指定半透明加密的语法格式为：

```
……
<列定义> ::= <不同类型列定义> [<列定义子句>] [<STORAGE 子句>][<半透明存储加密子句>]
<半透明存储加密子句> ::= ENCRYPT [WITH <加密算法>] MANUAL [<半透明加密选项>][<散列选项>]
<半透明加密选项> ::= <半透明加密选项 1> | <半透明加密选项 2> | <半透明加密选项 3>
<半透明加密选项 1> ::= <可见用户列表>
<半透明加密选项 2> ::= BY <列存储密钥> [<可见用户列表>]
<半透明加密选项 3> ::= BY WRAPPED <列存储密钥的密文> [<可见用户列表>]
<可见用户列表> ::= USER ([<用户名> {,<用户名>}])
<散列选项> ::= HASH WITH <散列算法> [<加盐选项>]
<加盐选项> ::= [NO] SALT
```

上述语法格式不完整，主要说明与半透明加密相关的部分，其他建表相关语法请参考《DM8_SQL 语言使用手册》。


说明：

使用半透明加密时，不同的用户可看见的数据不同。具体如下：

如果未使用<半透明加密选项>，则用户仅能查看到自己插入的数据；

如果使用了<半透明加密选项>，则只有<可见用户列表>中的用户可看见数据。

使用说明：

1. 表列半透明加密不支持使用 WORK_MODE_KID 模式的加密算法，除此之外对加密算法的要求与表列透明加密相同，具体请参考 7.1.3 表列透明加密；

2. HUGE 表不支持半透明加密列；

3. 半透明加密列不能作为索引列，索引列也不能修改为半透明加密列；

4. 半透明加密列不允许添加引用约束；

5. 若表中包含半透明加密列，则不允许创建聚集索引，同时不允许删除该表原有的聚集索引；

6. 不允许修改半透明加密列的列定义；

7. 当增加列、删除列或者修改列定义时，若表中包含半透明加密列，则 INI 参数 ALTER_TABLE_OPT 不能设置为 1，若参数 ALTER_TABLE_OPT 为 1，则系统自动按照参数 ALTER_TABLE_OPT 为 0 时处理，关于参数 ALTER_TABLE_OPT 的详细介绍请参考手册《DM8 系统管理员手册》；

8. 当增加列且新增列的默认值非 NULL 时，若新增列为半透明加密列，则 INI 参数 ALTER_TABLE_OPT 不能设置为 3，若参数 ALTER_TABLE_OPT 为 3，则系统自动按照参数 ALTER_TABLE_OPT 为 0 时处理；

9. 若表中包含半透明加密列，则不允许使用 REBUILD COLUMNS 子句修改表，关于 REBUILD COLUMNS 子句的详细介绍请参考手册《DM8_SQL 语言使用手册》；

10. 物化视图的查询语句中不能涉及半透明加密列；

11. 若表指定了 USING LONG ROW 存储选项，即支持超长记录存储时，如果表中包含半透明加密列，并且该列的数据类型为字符串类型，则系统将默认该列不支持超长记录存储；

12. 半透明列不支持带默认值的快速加列；

13. 半透明加密列支持 UPDATE，具体规则如下：

1） 用户通过 UPDATE 语句修改半透明加密列数据，执行提交或回滚操作前：当前用户可查询更新后表中 UID 与自身用户 ID 相匹配的半透明加密列数据，其他用户可查询更新前表中 UID 与自身用户 ID 相匹配的半透明加密列数据；

2） 用户通过 UPDATE 语句修改半透明加密列数据，执行回滚操作后：表中数据恢复到执行 UPDATE 操作前的状态，当前用户和其他用户均可查询更新前表中 UID 与自身用户 ID 相匹配的半透明加密列数据；

3） 用户通过 UPDATE 语句修改半透明加密列数据，执行提交操作后：当前用户和其他用户均可查询更新后表中 UID 与自身用户 ID 相匹配的半透明加密列数据。

14. <半透明加密选项>用于指定对当前列进行按列加密。和非按列加密的半透明加密相比，使用按列加密后，很多半透明加密规则发生了改变。例如：创建索引，修改列定义等。具体规则如下：

1） <半透明加密选项>用于指定对该列进行按列加密。<半透明加密选项>缺省时，使用用户的<存储加密密钥>对列加密。其中<半透明加密选项 1>使用系统自动生成的秘钥进行列加密。<半透明加密选项 2>和<半透明加密选项 3>使用各自指定的密钥进行列加密。其中<半透明加密选项 2>使用之前需先设置 INI 参数 AUTO_ENCRYPT=1，打开支持列透明加密时用户手动输入<列存储密钥>开关。按列加密的好处是将列秘钥与用户本身的密钥进行区分，可灵活设置不

同的列秘钥。某个用户是否能访问按列加密的列，取决于该用户是否在指定的<可见用户列表>中；

2) <可见用户列表>用于指定该列的可见用户。按列加密的列可见性是列级的，且只对指定用户可见。指定的<用户名>必须存在且不能重复。如果<可见用户列表>缺省或者 USER 指定的用户列表为空，则表示当前列对所有用户都不可见。只有可见用户能看见加密列数据。非可见用户查看到的信息为 NULL，但是非可见用户可以对加密列进行删除、更新和插入操作；

3) 支持将按列加密的列（即指定了<半透明加密选项>的列）作为二级索引的主键，但是不能作为聚集索引、位图索引、空间索引、数组索引和虚索引的主键。当二级索引中包含按列加密的列，且当前会话的用户不可见时，则该用户不能使用此索引进行快速过滤查询；

4) <列存储密钥>必须满足长度大于等于 INI 参数 PWD_MIN_LEN 设置的值，小于等于 128 个字符，且包含大写、小写、数字。

15. 表列半透明加密和按列加密不受过程的 DEFINER、CURRENT_USER 影响，过程中访问加密的表列，可见性由当前会话的用户决定。

例 1 以下是一些对表列进行半透明加密的例子。

```
CREATE TABLE TEST_ENCRYPT6(C1 INT, C2 INT ENCRYPT MANUAL);
CREATE TABLE TEST_ENCRYPT7(C1 INT, C2 INT ENCRYPT WITH DES_ECB MANUAL);
CREATE TABLE TEST_ENCRYPT8(C1 INT, C2 INT ENCRYPT WITH DES_ECB MANUAL HASH WITH MD5 SALT);
CREATE TABLE TEST_ENCRYPT9(C1 INT, C2 INT ENCRYPT MANUAL HASH WITH MD5 SALT);
```

例 2 以下是对半透明加密列进行更新操作的示例。

创建用户 USER1 和 USER2。

```
CREATE USER USER1 IDENTIFIED BY "USER1_psd";
```

```
CREATE USER USER2 IDENTIFIED BY "USER2_psd";
GRANT DBA TO USER1;
GRANT DBA TO USER2;
COMMIT;
```

用户 USER1 创建包含半透明加密列的表 TEST 并插入数据，执行提交操作后，用户 USER1 和 USER2 对表中数据进行查询。

```
CONN USER1/USER1_psd@LOCALHOST
CREATE TABLE TEST(C1 VARCHAR ENCRYPT MANUAL);
INSERT INTO TEST VALUES('AAA');
COMMIT;
SELECT * FROM USER1.TEST;
行号     C1
---------- ---
1        AAA

CONN USER2/USER2_psd@LOCALHOST
SELECT * FROM USER1.TEST;
行号     C1
---------- ----
1        NULL
```

用户 USER2 对表 TEST 中的半透明加密列进行更新，执行提交操作后，用户 USER1 和 USER2 对表中数据进行查询。

```
CONN USER2/USER2_psd@LOCALHOST
UPDATE USER1.TEST SET C1 = 'BBB';
COMMIT;
SELECT * FROM USER1.TEST;
行号     C1
---------- ---
```

```
1       BBB


CONN USER1/USER1_psd@LOCALHOST
SELECT * FROM USER1.TEST;
行号    C1
---------- ----
1       NULL
```

用户 USER1 对表 TEST 中的半透明加密列进行更新，执行回滚操作后，用户 USER1 和 USER2 对表中数据进行查询。

```
CONN USER1/USER1_psd@LOCALHOST
UPDATE USER1.TEST SET C1 = 'CCC';
ROLLBACK;
SELECT * FROM USER1.TEST;
行号    C1
---------- ----
1       NULL


CONN USER2/USER2_psd@LOCALHOST
SELECT * FROM USER1.TEST;
行号    C1
---------- ---
1       BBB
```

例 3 对表 T 的 C1 列进行按列加密。

先创建用户 USER01 和 USER02。

```
CREATE USER USER01 IDENTIFIED BY "USER01_psd";
CREATE USER USER02 IDENTIFIED BY "USER02_psd";
```

创建 T 表，对 C1 列进行按列加密，加密列对用户 USER01 和 USER02 可见。

```
CREATE TABLE T(C1 INT ENCRYPT MANUAL USER (USER01,USER02));
```

7.3 非透明加密

DM 对非透明加密的支持是通过对用户提供加解密接口实现的。用户在使用非透明加密时，需要提供密钥并调用加解密接口。采用非透明加密可以保证个人私密数据不被包括 DBA 在内的其他人获取。

非透明加密通过用户调用存储加密函数来进行，DM 提供了一系列的存储加密函数，还提供了一个数据加密包 DBMS_OBFUSCATION_TOOLKIT。本节主要介绍 DM 的存储加密函数，关于包 DBMS_OBFUSCATION_TOOLKIT 的介绍请参看《DM8 系统包使用手册》的相关章节。

DM 提供了下列存储加密函数：

1. CFALGORITHMSENCRYPT

```
CFALGORITHMSENCRYPT(
    SRC             VARCHAR/TEXT/CLOB,
    ALGORITHM       INT,
    KEY             VARCHAR
)
```

参数说明：

SRC：需要被加密的数据，数据类型可以为 VARCHAR、TEXT 或 CLOB。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

功能说明：

对指定的明文进行加密，并返回密文。src 为 NULL 时，加密结果也为 NULL。

当输入的参数为非法参数进而导致加密失败时，进行报错。

返回值：

加密后的密文，密文数据类型和 SRC 类型相同。

举例说明：

对数据进行加密：

```
CREATE TABLE enc_001(c1 VARCHAR(200));

INSERT INTO enc_001 VALUES(CFALGORITHMSENCRYPT('tt', 514, '仅供测试使用'));
```

这样就将加密后的数据存放到表列中。

2. CFALGORITHMSDECRYPT

```
CFALGORITHMSDECRYPT(
    SRC             VARCHAR/TEXT/CLOB,
    ALGORITHM       INT,
    KEY             VARCHAR
)
```

参数说明：

SRC：需要被解密的数据，数据类型可以为 VARCHAR、TEXT 或 CLOB。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

功能说明：

对密文进行解密，并得到加密前的相同数据类型的明文。

当输入的参数为非法参数进而导致解密失败时，进行报错。

返回值：

解密后的明文，明文数据类型和 SRC 类型相同。

举例说明：

对数据进行解密：

```
SELECT CFALGORITHMSDECRYPT(c1, 514, '仅供测试使用') FROM enc_001;

行号     CFALGORITHMSDECRYPT(C1,514,'仅供测试使用')

---------- ----------------------------------------

1        tt
```

3. SF_ENCRYPT_BINARY

```
SF_ENCRYPT_BINARY(
    SRC                 VARBINARY,
    ALGORITHM           INT,
    KEY                 VARCHAR,
    IV                  VARCHAR
)
```

参数说明：

SRC：需要被加密的 VARBINARY 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：无实际作用。

功能说明：

对 VARBINARY 类型明文进行加密，并返回密文。src 为 NULL 时，加密结果也为 NULL。

当输入的参数为非法参数进而导致加密失败时，进行报错。

返回值：

加密后的密文，数据类型为 VARBINARY。

举例说明：

对数据进行加密：

```
CREATE TABLE enc_002(c1 VARBINARY(200));
```

```
INSERT INTO enc_002 VALUES(SF_ENCRYPT_BINARY(0x12345678EF, 514, '仅供测试使用', NULL));
```

这样就将加密后的数据存放到表列中。

4. SF_DECRYPT_TO_BINARY

```
SF_DECRYPT_TO_BINARY(
    SRC             VARBINARY,
    ALGORITHM       INT,
    KEY             VARCHAR,
    IV              VARCHAR
)
```

参数说明：

SRC：需要被解密的 VARBINARY 类型密文。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：无实际作用。

功能说明：

对密文进行解密，并得到加密前的 VARBINARY 类型明文。

当输入的参数为非法参数进而导致解密失败时，进行报错。

返回值：

解密后的明文，数据类型为 VARBINARY。

举例说明：

对数据进行解密：

```
SELECT SF_DECRYPT_TO_BINARY(c1, 514, '仅供测试使用',NULL) FROM enc_002;

行号     SF_DECRYPT_TO_BINARY(C1,514,'仅供测试使用',NULL)
```

```
---------- --------------------------------------------------
1          0x12345678EF
```

5. SF_ENCRYPT_CHAR

```
SF_ENCRYPT_CHAR(
    SRC                 VARCHAR,
    ALGORITHM           INT,
    KEY                 VARCHAR,
    IV                  VARCHAR
)
```

参数说明：

SRC：需要被加密的 CHAR/VARCHAR 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：无实际作用。

功能说明：

对 VARCHAR 类型明文进行加密，并返回密文。src 为 NULL 时, 加密结果也为 NULL。

当输入的参数为非法参数进而导致加密失败时，进行报错。

返回值：

加密后的密文，数据类型为 VARBINARY。

举例说明：

对数据进行加密：

```
CREATE TABLE enc_003(c1 VARBINARY(200));

INSERT INTO enc_003 VALUES(SF_ENCRYPT_CHAR('测试数据', 514, '仅供测试使用',NULL));
```

这样就将加密后的数据存放到表列中。

6. SF_DECRYPT_TO_CHAR

```
SF_DECRYPT_TO_CHAR(
    SRC             VARBINARY,
    ALGORITHM       INT,
    KEY             VARCHAR,
    IV              VARCHAR
)
```

参数说明：

SRC：需要被解密的 VARBINARY 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：无实际作用。

功能说明：

对密文进行解密，并得到加密前的 VARCHAR 类型明文。

当输入的参数为非法参数进而导致解密失败时，进行报错。

返回值：

解密后的明文，数据类型为 VARCHAR。

举例说明：

对数据进行解密：

```
SELECT SF_DECRYPT_TO_CHAR(c1, 514, '仅供测试使用',NULL) FROM enc_003;

行号     SF_DECRYPT_TO_CHAR(C1,514,'仅供测试使用',NULL)
---------- ----------------------------------------------
1        测试数据
```

7. SF_ENCRYPT_DATE

```
SF_ENCRYPT_DATE(
    SRC             DATE,
    ALGORITHM       INT,
    KEY             VARCHAR,
    IV              VARCHAR
)
```

参数说明：

SRC：需要被加密的 DATE 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：无实际作用。

功能说明：

对 DATE 类型明文进行加密，并返回密文。src 为 NULL 时, 加密结果也为 NULL。

当输入的参数为非法参数进而导致加密失败时，进行报错。

返回值：

加密后的密文，数据类型为 VARBINARY。

举例说明：

对数据进行加密：

```
CREATE TABLE enc_004(c1 VARBINARY(200));
INSERT INTO enc_004 VALUES(SF_ENCRYPT_DATE(cast('2023-1-1' as date), 514, '仅供测试使用
',NULL));
```

这样就将加密后的日期类型数据存放到表列中。

8. SF_DECRYPT_TO_DATE

```
SF_DECRYPT_TO_DATE(
    SRC                 VARBINARY,
    ALGORITHM           INT,
    KEY                 VARCHAR,
    IV                  VARCHAR
)
```

参数说明：

SRC：需要被解密的 VARBINARY 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：无实际作用。

功能说明：

对密文进行解密，并得到加密前的 DATE 类型明文。

当输入的参数为非法参数进而导致解密失败时，进行报错。

返回值：

解密后的明文，数据类型为 DATE。

举例说明：

对数据进行解密：

```
SELECT SF_DECRYPT_TO_DATE(c1, 514, '仅供测试使用',NULL) FROM enc_004;

行号     SF_DECRYPT_TO_DATE(C1,514,'仅供测试使用',NULL)
---------- ----------------------------------------------
1        2023-01-01
```

9. SF_ENCRYPT_DATETIME

```
SF_ENCRYPT_DATETIME(
```

```
    SRC             DATETIME,
    ALGORITHM       INT,
    KEY             VARCHAR,
    IV              VARCHAR
)
```

参数说明：

SRC：需要被加密的 DATETIME 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：无实际作用。

功能说明：

对 DATETIME 类型明文进行加密，并返回密文。src 为 NULL 时, 加密结果也为 NULL。

当输入的参数为非法参数进而导致加密失败时，进行报错。

返回值：

加密后的密文，数据类型为 VARBINARY。

举例说明：

对数据进行加密：

```
CREATE TABLE enc_005(c1 VARBINARY(200));
INSERT INTO enc_005 VALUES(SF_ENCRYPT_DATETIME(cast('2023-12-12 11:11:11' as datetime), 514,
'仅供测试使用',NULL));
```

这样就将加密后的日期时间类型数据存放到表列中。

10. SF_DECRYPT_TO_DATETIME

```
SF_DECRYPT_TO_DATETIME(
    SRC             VARBINARY,
```

```
    ALGORITHM       INT,
    KEY             VARCHAR,
    IV              VARCHAR
)
```

参数说明：

SRC：需要被解密的 VARBINARY 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：无实际作用。

功能说明：

对密文进行解密，并得到加密前的 DATETIME 类型明文。

当输入的参数为非法参数进而导致解密失败时，进行报错。

返回值：

解密后的明文，数据类型为 DATETIME。

举例说明：

对数据进行解密：

```
SELECT SF_DECRYPT_TO_DATETIME (c1, 514, '仅供测试使用',NULL) FROM enc_005;

行号     SF_DECRYPT_TO_DATETIME(C1,514,'仅供测试使用',NULL)
---------- ----------------------------------------------
1        2023-12-12 11:11:11.000000
```

11. SF_ENCRYPT_DEC

```
SF_ENCRYPT_DEC(
    SRC             DEC,
    ALGORITHM       INT,
```

```
    KEY             VARCHAR,
    IV              VARCHAR
)
```

参数说明：

SRC：需要被加密的 DEC 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：无实际作用。

功能说明：

对 DEC 类型明文进行加密，并返回密文。src 为 NULL 时, 加密结果也为 NULL。

当输入的参数为非法参数进而导致加密失败时，进行报错。

返回值：

加密后的密文，数据类型为 VARBINARY。

举例说明：

对数据进行加密：

```
CREATE TABLE enc_006(c1 VARBINARY(200));
INSERT INTO enc_006 VALUES(SF_ENCRYPT_DEC(cast('3.1415900000'as dec(15,10)), 514, '仅供测试使用',NULL));
```

这样就将加密后的 DEC 数据存放到表列中。

12. SF_DECRYPT_TO_DEC

```
SF_DECRYPT_TO_DEC(
    SRC             VARBINARY,
    ALGORITHM       INT,
    KEY             VARCHAR,
```

```
    IV                  VARCHAR
)
```

参数说明：

SRC：需要被解密的 VARBINARY 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：无实际作用。

功能说明：

对密文进行解密，并得到加密前的 DEC 类型明文。

当输入的参数为非法参数进而导致解密失败时，进行报错。

返回值：

解密后的明文，数据类型为 DEC。

举例说明：

对数据进行解密：

```
SELECT SF_DECRYPT_TO_DEC(c1, 514, '仅供测试使用',NULL) FROM enc_006;

行号     SF_DECRYPT_TO_DEC(C1,514,'仅供测试使用',NULL)

---------- ----------------------------------------------

1        3.14159
```

13. SF_ENCRYPT_TIME

```
SF_ENCRYPT_TIME(
    SRC                 TIME,
    ALGORITHM           INT,
    KEY                 VARCHAR,
    IV                  VARCAHR
```

```
)
```

参数说明：

SRC：需要被加密的 TIME 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：无实际作用。

功能说明：

对 TIME 类型明文进行加密，并返回密文。src 为 NULL 时, 加密结果也为 NULL。

当输入的参数为非法参数进而导致加密失败时，进行报错。

返回值：

加密后的密文，数据类型为 VARBINARY。

举例说明：

对数据进行加密：

```
CREATE TABLE enc_007(c1 VARBINARY(200));
INSERT INTO enc_007 VALUES(SF_ENCRYPT_TIME(cast('12:12:12' as time), 514, '仅供测试使用
',NULL));
```

这样就将加密后的时间类型数据存放到表列中。

14. SF_DECRYPT_TO_TIME

```
SF_DECRYPT_TO_TIME(
    SRC             VARBINARY,
    ALGORITHM       INT,
    KEY             VARCHAR,
    IV              VARCHAR
)
```

参数说明：

SRC：需要被解密的 VARBINARY 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：无实际作用。

功能说明：

对密文进行解密，并得到加密前的 TIME 类型明文。

当输入的参数为非法参数进而导致解密失败时，进行报错。

返回值：

解密后的明文，数据类型为 TIME。

举例说明：

对数据进行解密：

```
SELECT SF_DECRYPT_TO_TIME(c1, 514, '仅供测试使用',NULL) FROM enc_007;

行号     SF_DECRYPT_TO_TIME(C1,514,'仅供测试使用',NULL)

---------- ----------------------------------------------

1        12:12:12.000000
```

15. SF_ENCRYPT_BINARY2

```
SF_ENCRYPT_BINARY2(
    SRC                 VARBINARY,
    ALGORITHM           INT,
    KEY                 VARCHAR,
    IV                  VARCHAR
)
```

参数说明：

SRC：需要被加密的 VARBINARY 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：采用的初始化矢量，可以为 NULL。使用需要 IV 的加密算法时，要求 IV 长度不小于块大小。

功能说明：

对 VARBINARY 类型明文进行加密，并返回密文。src 为 NULL 时，加密结果也为 NULL。

当输入的参数为非法参数进而导致加密失败时，进行报错。

返回值：

加密后的密文，数据类型为 VARBINARY。

举例说明：

对数据进行加密：

```
DROP TABLE enc_002;
CREATE TABLE enc_002(c1 VARBINARY(200));
INSERT INTO enc_002 VALUES(SF_ENCRYPT_BINARY2(0x12345678EF, 514, '仅供测试使用',
'1234567812345678'));
```

这样就将加密后的数据存放到表列中。

16. SF_DECRYPT_TO_BINARY2

```
SF_DECRYPT_TO_BINARY2(
    SRC             VARBINARY,
    ALGORITHM       INT,
    KEY             VARCHAR,
    IV              VARCHAR
)
```

参数说明：

SRC：需要被解密的 VARBINARY 类型密文。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：采用的初始化矢量，可以为 NULL。使用需要 IV 的加密算法时，要求 IV 长度不小于块大小。

功能说明：

对密文进行解密，并得到加密前的 VARBINARY 类型明文。

当输入的参数为非法参数进而导致解密失败时，进行报错。

返回值：

解密后的明文，数据类型为 VARBINARY。

举例说明：

对数据进行解密：

```
SELECT SF_DECRYPT_TO_BINARY2(c1, 514, '仅供测试使用', '1234567812345678') FROM enc_002;

行号     SF_DECRYPT_TO_BINARY2C1,514,'仅供测试使用','1234567812345678')
---------- ----------------------------------------------
1        0x12345678EF
```

17. SF_ENCRYPT_CHAR2

```
SF_ENCRYPT_CHAR2(
    SRC                 VARCHAR,
    ALGORITHM           INT,
    KEY                 VARCHAR,
    IV                  VARCHAR
)
```

参数说明：

SRC：需要被加密的 CHAR/VARCHAR 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：采用的初始化矢量，可以为 NULL。使用需要 IV 的加密算法时，要求 IV 长度不小于块大小。

功能说明：

对 VARCHAR 类型明文进行加密，并返回密文。src 为 NULL 时, 加密结果也为 NULL。

当输入的参数为非法参数进而导致加密失败时，进行报错。

返回值：

加密后的密文，数据类型为 VARBINARY。

举例说明：

对数据进行加密：

```
DROP TABLE enc_003;
CREATE TABLE enc_003(c1 VARBINARY(200));
INSERT INTO enc_003 VALUES(SF_ENCRYPT_CHAR2('测试数据', 514, '仅供测试使用',
'1234567812345678'));
```

这样就将加密后的数据存放到表列中。

18. SF_DECRYPT_TO_CHAR2

```
SF_DECRYPT_TO_CHAR2(
    SRC             VARBINARY,
    ALGORITHM       INT,
    KEY             VARCHAR,
    IV              VARCHAR
)
```

参数说明：

SRC：需要被解密的 VARBINARY 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：采用的初始化矢量，可以为 NULL。使用需要 IV 的加密算法时，要求 IV 长度不小于块大小。

功能说明：

对密文进行解密，并得到加密前的 VARCHAR 类型明文。

当输入的参数为非法参数进而导致解密失败时，进行报错。

返回值：

解密后的明文，数据类型为 VARCHAR。

举例说明：

对数据进行解密：

```
SELECT SF_DECRYPT_TO_CHAR2(c1, 514, '仅供测试使用','1234567812345678') FROM enc_003;

行号     SF_DECRYPT_TO_CHAR2(C1,514,'仅供测试使用','1234567812345678')
---------- ----------------------------------------------
1        测试数据
```

19. SF_ENCRYPT_DATE2

```
SF_ENCRYPT_DATE2(
    SRC                 DATE,
    ALGORITHM           INT,
    KEY                 VARCHAR,
    IV                  VARCHAR
```

```
)
```

参数说明：

SRC：需要被加密的 DATE 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：采用的初始化矢量，可以为 NULL。使用需要 IV 的加密算法时，要求 IV 长度不小于块大小。

功能说明：

对 DATE 类型明文进行加密，并返回密文。src 为 NULL 时, 加密结果也为 NULL。

当输入的参数为非法参数进而导致加密失败时，进行报错。

返回值：

加密后的密文，数据类型为 VARBINARY。

举例说明：

对数据进行加密：

```
DROP TABLE enc_004;
CREATE TABLE enc_004(c1 VARBINARY(200));
INSERT INTO enc_004 VALUES(SF_ENCRYPT_DATE2(cast('2023-1-1' as date), 514, '仅供测试使用',
'1234567812345678'));
```

这样就将加密后的日期类型数据存放到表列中。

20. SF_DECRYPT_TO_DATE2

```
SF_DECRYPT_TO_DATE2(
    SRC                 VARBINARY,
    ALGORITHM           INT,
    KEY                 VARCHAR,
```

```
    IV                  VARCHAR
)
```

参数说明：

SRC：需要被解密的 VARBINARY 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：采用的初始化矢量，可以为 NULL。使用需要 IV 的加密算法时，要求 IV 长度不小于块大小。

功能说明：

对密文进行解密，并得到加密前的 DATE 类型明文。

当输入的参数为非法参数进而导致解密失败时，进行报错。

返回值：

解密后的明文，数据类型为 DATE。

举例说明：

对数据进行解密：

```
SELECT SF_DECRYPT_TO_DATE2(c1, 514, '仅供测试使用', '1234567812345678') FROM enc_004;

行号     SF_DECRYPT_TO_DATE2(C1,514,'仅供测试使用', '1234567812345678')
---------- ----------------------------------------------
1        2023-01-01
```

21. SF_ENCRYPT_DATETIME2

```
SF_ENCRYPT_DATETIME2(
    SRC                 DATETIME,
    ALGORITHM           INT,
    KEY                 VARCHAR,
```

```
    IV                  VARCHAR
)
```

参数说明：

SRC：需要被加密的 DATETIME 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：采用的初始化矢量，可以为 NULL。使用需要 IV 的加密算法时，要求 IV 长度不小于块大小。

功能说明：

对 DATETIME 类型明文进行加密，并返回密文。src 为 NULL 时, 加密结果也为 NULL。

当输入的参数为非法参数进而导致加密失败时，进行报错。

返回值：

加密后的密文，数据类型为 VARBINARY。

举例说明：

对数据进行加密：

```
DROP TABLE enc_005;
CREATE TABLE enc_005(c1 VARBINARY(200));
INSERT INTO enc_005 VALUES(SF_ENCRYPT_DATETIME2(cast('2023-12-12 11:11:11' as datetime),
514, '仅供测试使用', '1234567812345678'));
```

这样就将加密后的日期时间类型数据存放到表列中。

22. SF_DECRYPT_TO_DATETIME2

```
SF_DECRYPT_TO_DATETIME2(
    SRC                 VARBINARY,
    ALGORITHM           INT,
```

```
    KEY                 VARCHAR,
    IV                  VARCHAR
)
```

参数说明：

SRC：需要被解密的 VARBINARY 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：采用的初始化矢量，可以为 NULL。使用需要 IV 的加密算法时，要求 IV 长度不小于块大小。

功能说明：

对密文进行解密，并得到加密前的 DATETIME 类型明文。

当输入的参数为非法参数进而导致解密失败时，进行报错。

返回值：

解密后的明文，数据类型为 DATETIME。

举例说明：

对数据进行解密：

```
SELECT SF_DECRYPT_TO_DATETIME2 (c1, 514, '仅供测试使用', '1234567812345678') FROM enc_005;

行号     SF_DECRYPT_TO_DATETIME2(C1,514,'仅供测试使用', '1234567812345678')
---------- ----------------------------------------------------
1        2023-12-12 11:11:11.000000
```

23. SF_ENCRYPT_DEC2

```
SF_ENCRYPT_DEC2(
    SRC                 DEC,
    ALGORITHM           INT,
```

```
    KEY                 VARCHAR,
    IV                  VARCHAR
)
```

参数说明：

SRC：需要被加密的 DEC 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：采用的初始化矢量，可以为 NULL。使用需要 IV 的加密算法时，要求 IV 长度不小于块大小。

功能说明：

对 DEC 类型明文进行加密，并返回密文。src 为 NULL 时, 加密结果也为 NULL。

当输入的参数为非法参数进而导致加密失败时，进行报错。

返回值：

加密后的密文，数据类型为 VARBINARY。

举例说明：

对数据进行加密：

```
DROP TABLE enc_006;
CREATE TABLE enc_006(c1 VARBINARY(200));
INSERT INTO enc_006 VALUES(SF_ENCRYPT_DEC2(cast('3.1415900000'as dec(15,10)), 514, '仅供测试使用', '1234567812345678'));
```

这样就将加密后的 DEC 数据存放到表列中。

24. SF_DECRYPT_TO_DEC2

```
SF_DECRYPT_TO_DEC2(
```

```
    SRC             VARBINARY,
    ALGORITHM       INT,
    KEY             VARCHAR,
    IV              VARCHAR
)
```

参数说明：

SRC：需要被解密的 VARBINARY 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：采用的初始化矢量，可以为 NULL。使用需要 IV 的加密算法时，要求 IV 长度不小于块大小。

功能说明：

对密文进行解密，并得到加密前的 DEC 类型明文。

当输入的参数为非法参数进而导致解密失败时，进行报错。

返回值：

解密后的明文，数据类型为 DEC。

举例说明：

对数据进行解密：

```
SELECT SF_DECRYPT_TO_DEC2(c1, 514, '仅供测试使用', '1234567812345678') FROM enc_006;

行号     SF_DECRYPT_TO_DEC2(C1,514,'仅供测试使用', '1234567812345678')
---------- ----------------------------------------------
1        3.14159
```

25. SF_ENCRYPT_TIME2

```
SF_ENCRYPT_TIME2(
```

```
    SRC             TIME,
    ALGORITHM       INT,
    KEY             VARCHAR,
    IV              VARCAHR
)
```

参数说明：

SRC：需要被加密的 TIME 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：采用的初始化矢量，可以为 NULL。使用需要 IV 的加密算法时，要求 IV 长度不小于块大小。

功能说明：

对 TIME 类型明文进行加密，并返回密文。src 为 NULL 时, 加密结果也为 NULL。

当输入的参数为非法参数进而导致加密失败时，进行报错。

返回值：

加密后的密文，数据类型为 VARBINARY。

举例说明：

对数据进行加密：

```
DROP TABLE enc_007;
CREATE TABLE enc_007(c1 VARBINARY(200));
INSERT INTO enc_007 VALUES(SF_ENCRYPT_TIME2(cast('12:12:12' as time), 514, '仅供测试使用',
'1234567812345678'));
```

这样就将加密后的时间类型数据存放到表列中。

26. SF_DECRYPT_TO_TIME2

```
SF_DECRYPT_TO_TIME2(
    SRC                 VARBINARY,
    ALGORITHM           INT,
    KEY                 VARCHAR,
    IV                  VARCHAR
)
```

参数说明：

SRC：需要被解密的 VARBINARY 类型数据。

ALGORITHM：加密算法 ID。加密算法对应的 ID 可通过查询 V$CIPHERS 得到。

KEY：采用的密钥。

IV：采用的初始化矢量，可以为 NULL。使用需要 IV 的加密算法时，要求 IV 长度不小于块大小。

功能说明：

对密文进行解密，并得到加密前的 TIME 类型明文。

当输入的参数为非法参数进而导致解密失败时，进行报错。

返回值：

解密后的明文，数据类型为 TIME。

举例说明：

对数据进行解密：

```
SELECT SF_DECRYPT_TO_TIME2(c1, 514, '仅供测试使用', '1234567812345678') FROM enc_007;

行号     SF_DECRYPT_TO_TIME2(C1,514,'仅供测试使用', '1234567812345678')
---------- ----------------------------------------------
1        12:12:12.000000
```

27. SF_GET_CIPHER_NAME

```
SF_GET_CIPHER_NAME(
```

```
    CIPHER_ID IN INT
)
```

参数说明：

CIPHER_ID：加密算法 ID。

功能说明：

根据加密算法 ID，获取加密算法的名称。

返回值：

加密算法的名称，最大长度 32767。

举例说明：

```
CREATE TABLE T1(C1 INT ENCRYPT ,C2 INT ENCRYPT );
SELECT ID FROM SYSOBJECTS WHERE NAME = 'T1';
   //设返回 1152
SELECT * FROM SYS.SYSCOLCYT WHERE TID = 1152;
   //SYSCOLCYT 表中 ENC_ID 列对应的值，即为列的加密类型 ID，此处返回 2050
SELECT SF_GET_CIPHER_NAME(2050);
行号      SF_GET_CIPHER_NAME(2050)
---------- ------------------------
1         AES256_CBC
```

7.4 加密算法和散列算法

本章语法中的<加密算法>和<散列算法>既支持使用系统算法，也支持使用第三方算法。下面对支持的<加密算法>和<散列算法>进行介绍：

- **系统加密和散列算法**

系统内置了常用的 DES、DESEDE、AES、RC4、MD5 和 SHA 等类型的加密和散列算法，以此来保护数据的安全性。DM 支持的加密算法和散列算法可通过查询动态视图 V$CIPHERS 得到，表 7.1 列出了 DM 内置的加密和散列算法。由于 DM 安装程序中自带 OPENSSL 的动态库，因此查询 V$CIPHERS 时除表 7.1 中列出的算法外，还会查询到 OPENSSL 动态库的相关算法。

DM 内置的算法中，DES、DESEDE 和 AES 为分组加密算法，RC4 为流加密算法，MD5 和 SHA 为散列算法。分组加密算法和流加密算法都是对称加密算法，加密和解密使用同一个密钥。其中，分组加密算法又称为块加密算法，分组长度又称为块大小。

DESEDE 是基于 DES 改进的加密算法，对数据进行三次加密，与 DES 相比安全性更高。AES 是相较于 DES 和 DESEDE 安全性更高的加密算法，并且支持多种长度的密钥，灵活性更高，但 DES 和 DESEDE 的加解密速度更快，因此也被广泛应用。RC4 是流加密算法，常用于加密通信协议。MD5 和 SHA 都是目前常用的散列算法，两者相比，SHA 算法的安全性较高，MD5 算法的计算速度较快。用户可根据实际应用场景选择合适的加密和散列算法。

DES、DESEDE 和 AES 算法名称中的 ECB、CBC、CFB、OFB 和 NOPAD 为分组加密算法的算法模式， 其中 NOPAD 表示非填充模式，非填充模式的加密算法主要用于数据页分片加密，需要用户保证原始数据长度是分组长度的整数倍，如果无法保证，则建议选择其他加密算法。有关算法模式的更多说明请参考 8.1.1 算法信息相关接口中的表 8.3。

表 7.1 DM 内置的加密和散列算法

算法名称	算法类型	分组长度（单位：字节）	密钥长度（单位：字节）
DES_ECB	分组加密算法	8	8
DES_CBC	分组加密算法	8	8

DES_CFB	分组加密算法	8	8
DES_OFB	分组加密算法	8	8
DESEDE_ECB	分组加密算法	8	16
DESEDE_CBC	分组加密算法	8	16
DESEDE_CFB	分组加密算法	8	16
DESEDE_OFB	分组加密算法	8	16
DESEDE_3KEY_ECB	分组加密算法	8	24
DESEDE_3KEY_CBC	分组加密算法	8	24
DESEDE_3KEY_CFB	分组加密算法	8	24
DESEDE_3KEY_OFB	分组加密算法	8	24
AES128_ECB	分组加密算法	16	16
AES128_CBC	分组加密算法	16	16
AES128_CFB	分组加密算法	16	16
AES128_OFB	分组加密算法	16	16
AES192_ECB	分组加密算法	16	24
AES192_CBC	分组加密算法	16	24
AES192_CFB	分组加密算法	16	24
AES192_OFB	分组加密算法	16	24
AES256_ECB	分组加密算法	16	32
AES256_CBC	分组加密算法	16	32
AES256_CFB	分组加密算法	16	32
AES256_OFB	分组加密算法	16	32
DES_ECB_NOPAD	分组加密算法	8	8
DES_CBC_NOPAD	分组加密算法	8	8
DESEDE_ECB_NOPAD	分组加密算法	8	16
DESEDE_CBC_NOPAD	分组加密算法	8	16
DESEDE_3KEY_ECB_NOPAD	分组加密算法	8	24

DESEDE_3KEY_CBC_NOPAD	分组加密算法	8	24
AES128_ECB_NOPAD	分组加密算法	16	16
AES128_CBC_NOPAD	分组加密算法	16	16
AES192_ECB_NOPAD	分组加密算法	16	24
AES192_CBC_NOPAD	分组加密算法	16	24
AES256_ECB_NOPAD	分组加密算法	16	32
AES256_CBC_NOPAD	分组加密算法	16	32
RC4	流加密算法	-	16
MD5	散列算法	-	-
SHA1	散列算法	-	-
SHA224	散列算法	-	-
SHA256	散列算法	-	-
SHA384	散列算法	-	-
SHA512	散列算法	-	-

- **散列算法函数**

DM 提供了散列算法函数 SF_HASH。

```
SF_HASH (
    ID          INT,
    DATA        VARBINARY/BLOB/VARCHAR/TEXT
);
```

参数说明:

ID：使用的散列算法 ID;

DATA：需要进行散列操作的数据。

功能说明:

对数据 DATA 进行散列，获取散列值。

返回值:

散列值，为 VARBINARY 类型。

举例说明：

```
SQL> SELECT SF_HASH(2432, 0xa);
```

执行结果如下：

```
行号      SF_HASH(2432,0x0A)

---       ------------------

1         0xEC664E889ED6C1B2763CACF7899D95B7F347373EB982E523419FEEA3AA362D891B3BF02

5F292267A5854049091789C3E
```

- **第三方加密和散列算法**

存储加密还支持第三方加密和散列算法，详情请参考 8 加密引擎。第三方加密算法可通过 V$EXTERNAL_CIPHERS 查看。

8 加密引擎

DM 系统中内置了常用的 DES，AES，RC4，SHA 等类型的加密和散列算法供用户使用，以此来保护数据的安全性。然而在有些特殊的环境下，这些算法可能不能满足用户的需求，用户可能希望使用自己特殊的或更高强度的加密和散列算法（例如：国密算法）。DM 的加密引擎功能则可以满足这样的需求。

用户只需要按照 DM 提供的加密引擎 C 语言编程接口，封装自己的加密和散列算法，并编译成第三方加密动态库，即可以在 DM 系统中使用自己的加密和散列算法。


注意：

用户使用自行封装的加密和散列算法前需确保该算法适用于当前应用场景。例如，用户使用自行封装的加密算法进行存储加密时，需要确保该算法适用于存储加密。

第三方加密动态库需要提供如下的对外接口，如下表所示。

表 8.1 第三方加密接口定义

接口名称	功能说明
cipher_get_count	获取实现的算法个数
cipher_get_info	获取算法的基本信息，当实现了 cipher_get_info_ex 时，cipher_get_info 自动失效
cipher_get_info_ex	获取算法的基本信息
cipher_get_para	获取当前加密算法的当前参数的值
cipher_encrypt_init	加密初始化，存在 cipher_encrypt_init_with_iv 时，此函数失效
cipher_encrypt_init_with_iv	携带 iv 参数的加密初始化
cipher_get_cipher_text_size	计算给定长度的明文，加密后得到的密文所占的字节数
cipher_encrypt	加密函数
cipher_cleanup	回收密码算法在处理过程中申请的系统资源

cipher_decrypt_init	解密初始化，存在 cipher_decrypt_init_with_iv 时，此函数失效
cipher_decrypt_init_with_iv	携带 iv 参数的解密初始化
cipher_decrypt	解密函数
cipher_hash_init	散列过程的初始化工作
cipher_hash_update	计算数据的散列值
cipher_hash_final	返回散列值的实际长度
cipher_hash_mac_init	HMAC 算法散列过程的初始化工作
cipher_hash_mac_update	HMAC 算法计算数据的散列值
cipher_hash_mac_final	HMAC 算法返回散列值的实际长度
cipher_asym_sign_ex	用户私钥对数据进行签名
cipher_asym_verify_ex	用户公钥对数据进行验签
crypto_login	登录设备
crypto_logout	退出设备
crypto_read_cert	读取用户证书
cipher_gen_random	产生随机数
cipher_asym_sign	用户私钥对数据进行签名
cipher_asym_verify	用户公钥对数据进行验签
crypto_get_name	获取加密引擎名字
crypto_get_type	获取加密引擎类型。0：第三方软加密；1：第三方硬件加密；2：ukey
crypto_encrypt	加密服务器主密钥。仅当 dminit 工具初始化库时指定了参数 EXTERNAL_CRYPTO_NAME 时才有效
crypto_decrypt	解密服务器主密钥。与 crypto_encrypt 对应
cipher_get_key_id	生成加密 key 的标识符 key_id
cipher_free_key_id	释放加密 key 的标识符 key_id

8.1 编程接口介绍

8.1.1 算法信息相关接口

1. cipher_get_count

```
ulint
cipher_get_count(
);
```

功能说明:

获取实现的算法个数。

返回值:

返回实现的算法个数。

2. cipher_get_info

```
dm_bool
cipher_get_info(
    ulint   seqno,
    ulint* cipher_id,
    byte**          cipher_name,
    byte* type,
    ulint* blk_size,
    ulint* kh_size
);
```

参数说明:

seqno：输入参数，算法的序号，从 1 开始，小于等于算法的个数。

cipher_id：输出参数，算法的 ID，第三方加密算法的 ID 必须大于等于 5000。

cipher_name：输出参数，算法名。

type：输出参数，算法类型。算法类型取值见下表：

表 8.2 算法类型取值

算法类型取值	释义
0 或 CYT_TYPE_UNKNOWN	未知类型
1 或 CYT_TYPE_SYM_BLOCK_ENCRYPT	分组加密算法
2 或 CYT_TYPE_SYM_STREAM_ENCRYPT	流加密算法
3 或 CYT_TYPE_ASYM_ENCRYPT	非对称加密算法
4 或 CYT_TYPE_HASH	散列算法
5 或 CYT_TYPE_HASH_MAC	HMAC 算法（HASH_MAC 算法）

blk_size：输出参数，块大小。

kh_size：输出参数，密钥长度或 hash 长度。在非对称加密算法中，该参数为公钥长度。

功能说明：

获取算法的基本信息，有 cipher_get_info_ex 存在时，cipher_get_info 失效。

返回值：

TRUE/FALSE：获取成功/获取失败。

3. cipher_get_info_ex

```
dm_bool
cipher_get_info_ex(
    ulint       seqno,
    ulint*      cipher_id,
    byte**      cipher_name,
    byte*       type,
    ulint*      blk_size,
```

```
    ulint*          kh_size,
    byte*    work_mode
);
```

参数说明：

seqno：输入参数，算法的序号，从 1 开始，小于等于算法的个数。

cipher_id：输出参数，算法的 ID，第三方加密算法的 ID 必须大于等于 5000。

cipher_name：输出参数，算法名。

type：输出参数，算法类型，具体见表 8.2。

blk_size：输出参数，块大小。

kh_size：输出参数，密钥长度或 hash 长度。在非对称加密算法中，该参数为公钥长度。

work_mode：输出参数，算法模式。算法模式取值见下表：

表 8.3 算法模式取值

模式名取值	释义
0 或 WORK_MODE_ECB	ECB（Electronic CodeBook），电子密码本模式
2 或 WORK_MODE_CBC	CBC（Cipher-Block Chaining），密码块连接模式
4 或 WORK_MODE_CFB	CFB（Cipher FeedBack），密文反馈模式
8 或 WORK_MODE_OFB	OFB（Output FeedBack），输出反馈模式
16 或 WORK_MODE_CBC_NOPAD	CBC 同上。 NOPAD（NO PADDING），非填充模式
32 或 WORK_MODE_ECB_NOPAD	ECB 同上。 NOPAD（NO PADDING），非填充模式
64 或 WORK_MODE_EXTKEY	EXTKEY（EXTERNAL KEY），外部密钥模式，每个数据页一个密钥，数据页不做变长处理，由外部算法单独处理并维护一个校验库。XX 项目专用算法（XX 为项目名称，保密）

67 或 WORK_MODE_KID	KID(KEY ID)，密钥 ID 模式，由加密引擎管理密钥, 数据库仅记录和使用加密引擎所提供的密钥 ID。在 WORK_MODE_KID 模式下, 对于服务器而言, 此时服务器不再存储真实密钥, 而是存储密钥 ID, 在创建密钥时, 调用 cipher_get_key_id 生成密钥 ID, 之后该操作相关的加解密操作, 服务器均使用该密钥 ID 来作为 key 传递给加密引擎, 销毁密钥时调用 cipher_free_key_id 释放密钥 ID。备份还原不支持使用 WORK_MODE_KID 的加密算法，该种加密算法也不支持作为 external_cipher_name 来进行数据库初始化

WORK_MODE_KID 的可选配置文件为 dmkid.ini。dmkid.ini 为 DM 适配信安世纪所使用的加密引擎配置文件，若不使用该加密引擎，则不需要使用 dmkid.ini。dmkid.ini 文件位于 DM 安装目录$DM_HOME/bin 目录下。dmkid.ini 详细参数如下表所示:

表 8.4 dmkid.ini 中控制文件相关参数

参数名	缺省值	说明
InstanceURL	http://127.0.0.1:8214/kmip	加密服务器 URL
InstanceFormat	1	格式：NONE = 1, JSON = 1, XML = 2
InstanceUser	KmipAdmin	用户名
InstancePwd	88888888	密码
InstanceCa	CA 证书	如果使用 HTTPS, 则在这里指定 CA 证书
RETRY_TIMES	5	出现错误时，重试次数
RETRY_SLEEP	1000	每次重试等待时间(MS)
ENC_DEBUG	0	是否生成日志
ENC_LOG	ENC.LOG	日志文件完整路径
CONFIG_LOG_FILE	无	加密服务器生成日志完整路径

功能说明:

获取算法的基本信息，有 cipher_get_info_ex 存在时，cipher_get_info 失效。

返回值:

TRUE/FALSE：获取成功/获取失败。

4. cipher_get_para

```
dmbool cipher_get_para(
        ulint cipher_id,
        ulint      para_id,
        void* value
);
```

参数说明：

cipher_id：输入参数，算法的 ID，第三方加密算法的 ID 必须大于等于 5000。

para_id：输入参数，参数 ID。目前仅支持三个参数：0 表示 WORK_MODE；1 表示 EXTEND_SIZE；2 表示 PRIKEY_SIZE。

value：输出参数，参数值。类型将根据 para_id 变化：para_id 为 0 时，value 为 unsigned char*；para_id 为 1 或 2 时，value 为 unsigned int*。用户须保证不会写越界。

功能说明：

获取当前加密算法的当前参数的值，目前支持获取 WORK_MODE 和 EXTEND_SIZE，PRIKEY_SIZE。WORK_MODE 表示加密算法模式；EXTEND_SIZE 表示加密算法最大扩展长度，即密文相对于明文一次加密最大扩展的长度，通常可以认为是一次加密(密文-明文)的最大值；PRIKEY_SIZE 表示非对称加密算法中的私钥长度，在其他类型的算法中无效。

返回值：

TRUE/FALSE：获取成功/获取失败。



说明： 当获取失败时，将使用参数的默认值。WORK_MODE 默认值为 0；EXTEND_SIZE 默认值为通过 cipher_get_info 获得的 kh_size。

当同时使用 cipher_get_info_ex 和 cipher_get_para 对 work_mode 进行正确赋值时，最终结果为 cipher_get_para 所设置的值。

8.1.2 加密过程相关接口

1. cipher_encrypt_init

```
dm_bool
cipher_encrypt_init(
    ulint       inner_id,
    byte*       key,
    ulint       key_size,
    void**      encrypt_para
);
```

参数说明：

inner_id：输入参数，调用的密码算法在加密引擎中的内部编号。

key：输入参数，加密密钥。如果加密算法模式为 WORK_MODE_KID，则此接口的 key 和 key_size 分别对应 cipher_get_key_id 的 key_id 和 key_id_size。

key_size：输入参数，密钥的字节数。

encrypt_para：输出参数，加密过程的全局参数。该参数为加密过程全程使用的参数，由用户负责分配内存空间，其中可以携带 key、key_size、用户自定义信息等。该参数需要在整个加密过程中保持有效，后续加密相关接口中的 encrypt_para 都是由该接口生成。

功能说明：

加密初始化，存在 cipher_encrypt_init_with_iv 时，此函数失效。

返回值

TRUE/FALSE：初始化成功/初始化失败。

2. cipher_encrypt_init_with_iv

```
dm_bool
cipher_encrypt_init_with_iv(
    ulint       inner_id,
    byte*       key,
    ulint       key_size,
    byte*       iv,
    ulint       iv_len,
    void**      encrypt_para
);
```

参数说明：

inner_id：输入参数，调用的密码算法在加密引擎中的内部编号。

key：输入参数，加密密钥。如果加密算法模式为 WORK_MODE_KID，则此接口的 key 和 key_size 分别对应 cipher_get_key_id 的 key_id 和 key_id_size。

key_size：输入参数，密钥的字节数。

iv：输入参数，初始化向量。

iv_len：输入参数，iv 的字节数。

encrypt_para：输出参数，加密过程的全局参数。该参数为加密过程全程使用的参数，由用户负责分配内存空间，其中可以携带 key、key_size、用户自定义信息等。该参数需要在整个加密过程中保持有效，后续加密相关接口中的 encrypt_para 都是由该接口生成。

功能说明：

携带 iv 参数的加密初始化，存在此函数时，cipher_encrypt_init 函数失效。

返回值

TRUE/FALSE：初始化成功/初始化失败。

3. cipher_get_cipher_text_size

```
lint
cipher_get_cipher_text_size(
    ulint       inner_id,
    void*       encrypt_para,
    lint plain_text_size
);
```

参数说明：

inner_id：输入参数，密码算法在加密引擎中的内部编号。

encrypt_para：输入参数，cipher_encrypt_init 的输出参数。encrypt_para 是 cipher_encrypt_init 时产生的输出参数，如何利用其中信息由用户决定。

plain_text_size：输入参数，待加密的明文所占的字节数。

功能说明

计算给定长度的明文加密后得到的密文所占的字节数。DM 照返回值分配密文缓冲区。

返回值

与明文对应的密文所占的字节数。

4. cipher_encrypt

```
lint
cipher_encrypt(
    ulint       inner_id,
    void*       encrypt_para,
    byte*       plain_text,
    ulint       plain_text_size,
    byte*       cipher_text,
```

```
    ulint       cipher_text_buf_size
);
```

参数说明：

inner_id：输入参数，密码算法在加密引擎中的内部编号。

encrypt_para：输入参数，密码算法的全局变量。encrypt_para 是 cipher_encrypt_init 时产生的输出参数，如何利用其中信息由用户决定。

plain_text：输入参数，明文。

plain_text_size：输入参数，明文所占的字节数。

cipher_text：输出参数，密文。

cipher_text_buf_size：输入参数，密文缓冲区的字节数。

功能说明

加密。用户需要保证根据提供的参数不会出现写越界等问题，密文长度不能超过所提供的空间，这里的密文缓冲区字节数不会小于 cipher_get_cipher_text_size 的返回值，用户还需确保 cipher_get_cipher_text_size 与该接口使用相同参数时，应该返回相同值。

返回值

加密后密文的长度。

5. cipher_cleanup

```
void
cipher_cleanup(
    ulint       inner_id,
    void*       encrypt_para
);
```

参数说明：

inner_id：输入参数，密码算法在加密引擎中的内部编号。

encrypt_para：输入参数，密码算法的全局参数。

功能说明

回收密码算法在处理过程中申请的系统资源。一般情况下，用户需要在这里释放在 cipher_encrypt_init 或 cipher_decrypt_init 中创建的 encrypt_para/decrypt_para。

返回值

无，假定该算法总是成功。


说明：

在使用一个密码算法进行加密之前应首先对该密码算法进行初始化处理，DM 数据库管理系统根据所指定的密码算法准备明文，并为密文分配密文缓冲区。然后调用加密函数进行加密，完成加密动作后回收加密过程中申请的系统资源。

8.1.3 解密过程相关接口

1. cipher_decrypt_init

```
dm_bool
cipher_decrypt_init(
    ulint       inner_id,
    byte*       key,
    ulint       key_size,
    void**      decrypt_para
);
```

参数说明：

inner_id：输入参数，调用的密码算法在加密引擎中的内部编号。

key：输入参数，加密密钥。如果加密算法模式为 WORK_MODE_KID，则此接口的 key 和 key_size 分别对应 cipher_get_key_id 的 key_id 和 key_id_size。

key_size：输入参数，密钥的字节数。

decrypt_para：输出参数，解密过程的全局参数，由用户分配内存。

功能说明

解密初始化。用法与加密的 cipher_encrypt_init 类似。存在 cipher_decrypt_init_with_iv 时，此函数失效。

返回值

TRUE/FALSE：初始化成功/初始化失败。

2. cipher_decrypt_init_with_iv

```
dm_bool
cipher_decrypt_init_with_iv(
    ulint       inner_id,
    byte*       key,
    ulint       key_size,
    byte*       iv,
    ulint       iv_len,
    void**      decrypt_para
);
```

参数说明：

inner_id：输入参数，调用的密码算法在加密引擎中的内部编号。

key：输入参数，加密密钥。如果加密算法模式为 WORK_MODE_KID，则此接口的 key 和 key_size 分别对应 cipher_get_key_id 的 key_id 和 key_id_size。

key_size：输入参数，密钥的字节数。

iv：输入参数，初始化向量。

iv_len：输入参数，iv 的字节数。

decrypt_para：输出参数，解密过程的全局参数，由用户分配内存。

功能说明：

携带 iv 参数的解密初始化。用法与加密的 cipher_encrypt_init_with_iv 类似。

返回值

TRUE/FALSE：初始化成功/初始化失败。

3. cipher_decrypt

```
lint
cipher_decrypt(
    ulint       inner_id,
    void*       decrypt_para,
    byte*       cipher_text,
    ulint       cipher_text_size,
    byte*       plain_text,
    ulint       plain_text_buf_size
);
```

参数说明：

inner_id：输入参数，调用的密码算法在加密引擎中的内部编号。

decrypt_para：输入参数，cipher_decrypt_init 的输出参数。该参数来自于 cipher_decrypt_init 的输出参数，如何利用其中数据由用户决定。

cipher_text：输入参数，密文。

cipher_text_size：输入参数，密文所占的字节数。

plain_text：输出参数，明文。

plain_text_buf_size：输入参数，明文缓冲区的长度。

功能说明

解密。用户需要保证根据提供的参数不会出现写越界等问题，明文长度不能超过所提供的空间。

返回值

明文的实际长度。


说明：

在使用一个密码算法进行解密之前应首先对该密码算法进行初始化处理，然后调用解密函数进行解密，完成解密动作后回收解密过程中申请的系统资源。

8.1.4 散列过程相关接口

1. cipher_hash_init

```
dm_bool
cipher_hash_init(
    ulint       inner_id,
    void**      hash_para
);
```

参数说明：

inner_id：输入参数，调用的密码算法在加密引擎中的内部编号。

hash_para：输出参数，散列过程的全局变量，由用户分配内存。

功能说明

散列过程初始化。

返回值

TRUE/FALSE：初始化成功/初始化失败。

2. cipher_hash_update

```
void
cipher_hash_update(
    ulint       inner_id,
    void*       hash_para,
    byte*       msg,
    ulint       msg_size
);
```

参数说明：

inner_id：输入参数，调用的密码算法在加密引擎中的内部编号。

hash_para：输入参数，散列过程中的全局参数。

msg：输入参数，待散列的数据。

msg_size：输入参数，数据长度。

功能说明

计算 msg 的散列值。

返回值

无。

3. cipher_hash_final

```
lint
cipher_hash_final(
    ulint       inner_id,
    void*       hash_para,
```

```
    byte*       digest,
    ulint       digest_buf_size
);
```

参数说明：

inner_id：输入参数，调用的密码算法在加密引擎中的内部编号。

hash_para：输入参数，散列过程中的全局参数。

digest：输出参数，散列值。

digest_buf_size：输入参数，散列缓冲区的大小。

功能说明

将整个散列过程得到的散列值存放到 digest 中。

返回值

散列值的实际长度。

4. cipher_hash_mac_init

```
dm_bool
cipher_hash_mac_init(
    ulint       inner_id,
    byte*       key,
    ulint       key_len,
    void**      cipher_para
);
```

参数说明：

inner_id：输入参数，调用的密码算法在加密引擎中的内部编号。

key：输入参数，密钥。

key_len：输入参数，密钥长度。

cipher_para：输出参数，散列过程的全局变量，由用户分配内存。

功能说明

HMAC 算法散列过程的初始化工作。

返回值

TRUE/FALSE：初始化成功/初始化失败。

5. cipher_hash_mac_update

```
dm_bool
cipher_hash_mac_update(
    ulint       inner_id,
    byte*       msg,
    ulint       msg_len,
    void*       cipher_para
);
```

参数说明：

inner_id：输入参数，调用的密码算法在加密引擎中的内部编号。

msg：输入参数，待散列的数据。

msg_len：输入参数，数据长度。

cipher_para：输入参数，散列过程中的全局参数。

功能说明

HMAC 算法计算 msg 的散列值。

返回值

TRUE/FALSE：计算成功/计算失败。

6. cipher_hash_mac_final

```
lint
cipher_hash_mac_final(
    ulint       inner_id,
    void*       cipher_para,
    byte*       digest,
    ulint       digest_buf_size
);
```

参数说明：

inner_id：输入参数，调用的密码算法在加密引擎中的内部编号。

cipher_para：输入参数，散列过程中的全局参数。

digest：输出参数，散列值。

digest_buf_size：输入参数，散列缓冲区的大小。

功能说明

HMAC 算法将整个散列过程得到的散列值存放到 digest 中。

返回值

散列值的实际长度。

8.1.5 其他可选相关接口

1. cipher_asym_sign

```
dm_bool
cipher_asym_sign(
    ulint       inner_id,
    byte*       prikey,
    ulint       prikey_size,
    byte*       data,
```

```
    ulint       data_size,
    byte*       signdata,
    ulint*      signdata_buf_size
);
```

参数说明：

inner_id：输入参数，调用的密码算法在加密引擎中的内部编号。

prikey：输入参数，私钥。

prikey_size：输入参数，私钥长度。

data：输入参数，需要进行签名的数据。

data_size：输入参数，数据的长度。

signdata：输出参数，数据的签名值。

signdata_buf_size：输出参数，签名值长度。

功能说明

用户私钥对数据进行签名。

返回值

TRUE/FALSE：签名成功/签名失败。

2. cipher_asym_verify

```
dm_bool
cipher_asym_verify(
    ulint       inner_id,
    byte*       pubkey,
    ulint       pubkey_size,
    byte*       data,
    ulint       data_size,
    byte*       signdata,
```

```
    ulint       signdata_size
);
```

参数说明：

inner_id：输入参数，调用的密码算法在加密引擎中的内部编号。

pubkey：输入参数，公钥。

pubkey_size：输入参数，公钥长度。

data：输入参数，需要进行签名的数据。

data_size：输入参数，数据的长度。

signdata：输出参数，数据的签名值。

signdata_buf_size：输出参数，签名值长度。

功能说明

用户公钥对数据进行验签。

返回值

TRUE/FALSE：验签成功/验签失败。

3. cipher_asym_sign_ex

```
dm_bool
cipher_asym_sign_ex(
    void*   cipher_para,
    ulint       inner_id,
    byte*       prikey,
    ulint       prikey_size,
    byte*       data,
    ulint       data_size,
    byte*       signdata,
    ulint*      signdata_buf_size
);
```

参数说明：

cipher_para：输入参数，登录过程的全局参数。

inner_id：输入参数，调用的密码算法在加密引擎中的内部编号。

prikey：输入参数，私钥。

prikey_size：输入参数，私钥长度。

data：输入参数，需要进行签名的数据。

data_size：输入参数，数据的长度。

signdata：输出参数，数据的签名值。

signdata_buf_size：输出参数，签名值长度。

功能说明

用户私钥对数据进行签名。

cipher_asym_sign_ex 存在时，cipher_asym_sign 失效。

返回值

TRUE/FALSE：签名成功/签名失败。

4. cipher_asym_verify_ex

```
dm_bool
cipher_asym_verify_ex(
    void*   cipher_para,
    ulint       inner_id,
    byte*       pubkey,
    ulint       pubkey_size,
    byte*       data,
    ulint       data_size,
    byte*       signdata,
    ulint       signdata_size
```

```
);
```

参数说明：

cipher_para：输入参数，登录过程的全局参数。

inner_id：输入参数，调用的密码算法在加密引擎中的内部编号。

pubkey：输入参数，公钥。

pubkey_size：输入参数，公钥长度。

data：输入参数，需要进行签名的数据。

data_size：输入参数，数据的长度。

signdata：输出参数，数据的签名值。

signdata_buf_size：输出参数，签名值长度。

功能说明

用户公钥对数据进行验签。

存在 cipher_asym_verify_ex 时，cipher_asym_verify 失效。

返回值

TRUE/FALSE：验签成功/验签失败。

5. crypto_login

```
dm_bool
crypto_login(
    void**      cipher_para,
    byte*       pin,
    ulint       pin_len
);
```

参数说明：

cipher_para：输出参数，登录过程的全局参数，由用户负责内存分配。

pin：输入参数，设备口令。

pin_len：输入参数，口令长度。

功能说明

登录设备。

返回值

TRUE/FALSE：登录成功/登录失败。

6. crypto_logout

```
dm_bool
crypto_logout(
    void*       cipher_para
);
```

参数说明：

cipher_para：输入参数，登录过程时使用的全局参数，此时需要释放。

功能说明

退出设备。

返回值

TRUE/FALSE：退出成功/退出失败。

7. crypto_read_cert

```
dm_bool
crypto_read_cert(
    void*       cipher_para,
    byte*       cert,
    ulint*      cert_len
```

```
);
```

参数说明：

cipher_para：输入参数，登录过程时使用的全局参数。

cert：输出参数，用户证书信息。

cert_len：输出参数，用户证书长度。

功能说明

读取用户证书。

返回值

TRUE/FALSE：读取成功/读取失败。

8. cipher_gen_random

```
dm_bool
cipher_gen_random(
    byte*       random,
    ulint       random_length
);
```

参数说明：

random：输出参数，生成的随机数。

random_length：输入参数，需要产生随机数的长度。

功能说明

产生随机数。

返回值

TRUE/FALSE：随机数生成成功/随机数生成失败。

9. cipher_get_name

```
dm_bool
crypto_get_name(
    byte**      crypto_name,
    ulint*      len
);
```

参数说明：

crypto_name：输出参数，加密引擎的名字。

len：输出参数，加密引擎的名字的长度。

功能说明

获取加密引擎名字。

返回值

TRUE/FALSE：获取成功/获取失败。

10. cipher_get_type

```
dm_bool
crypto_get_type(
    ulint*      crypto_type
);
```

参数说明：

crypto_type：输出参数，加密引擎的类型。

功能说明

获取加密引擎类型。

返回值

TRUE/FALSE：获取成功/获取失败。

11. crypto_encrypt

```
lint
crypto_encrypt(
    byte*       plain_text,
    ulint       plain_text_size,
    byte*       cipher_text,
    ulint       cipher_text_buf_size
);
```

参数说明：

plain_text：输入参数，明文。

plain_text_size：输入参数，明文所占的字节数。

cipher_text：输出参数，密文。

cipher_text_buf_size：输入参数，密文缓冲区的字节数。

功能说明

加密服务器主密钥。仅当 dminit 工具初始化库时指定了参数 EXTERNAL_CRYPTO_NAME 时才有效，专用于服务器 server key 的加密。由第三方动态加密库指定加密的算法和密钥 key，可以使用第三方设备（硬件加密卡、ukey）中的密钥 key。为了安全起见，设备中的密钥 key 可以不加载到内存中，加密的全程都在第三方设备内完成。

返回值

加密后密文的长度。

12. crypto_decrypt

```
lint
crypto_decrypt(
    byte*       cipher_text,
    ulint       cipher_text_size,
    byte*       plain_text,
    ulint       plain_text_buf_size
);
```

参数说明：

cipher_text：输入参数，密文。

cipher_text_buf_size：输入参数，密文所占的字节数。

plain_text：输出参数，明文。

plain_text_size：输入参数，明文缓冲区的字节数。

功能说明

解密服务器主密钥。与 crypto_encrypt 对应。

返回值

明文的实际长度。

13. cipher_get_key_id

```
lint
cipher_get_key_id(
    ulint       cipher_id,
    byte*       key_id,
    ulint       key_id_size,
    ulint*      key_size
);
```

参数说明：

cipher_id：输入参数，算法的 ID，第三方加密算法的 ID 必须大于等于 5000。

key_id：输出参数，key_id。

key_id_size：输入参数，key_id 预设的空间大小，防止出现空间越界问题。

key_size：输出参数，key_id 实际大小。

功能说明

生成加密 key 的标识符 key_id，需要对应加密算法模式为 WORK_MODE_KID。

返回值

是否生成成功。

14. cipher_free_key_id

```
lint
cipher_free_key_id(
    ulint       cipher_id,
    byte*       key_id,
    ulint       key_id_size
);
```

参数说明：

cipher_id：输入参数，算法的 ID，第三方加密算法的 ID 必须大于等于 5000。

key_id：输入参数，key_id。

key_id_size：输入参数，key_id 预设的空间大小，防止出现空间越界问题。

功能说明

释放加密 key 的标识符 key_id，需要对应加密算法模式为 WORK_MODE_KID。

返回值

是否释放成功。

8.2 接口库文件使用说明

想要使用自己的加密算法的用户需要用 C 语言编写实现 8.1 节中介绍的接口（8.1.5 节中的接口可选），并编译生成第三方动态库文件，如：Windows 操作系统下为 external_crypto.dll（LINUX 操作系统下为 libexternal_crypto.so）。

DM 支持同时加载多个第三方动态库文件。但这些 DLL 库文件必须放在 DM 数据库服务器所在 BIN 目录的 external_crypto_libs 子目录下。external_crypto_libs 目录需用户自己创建。如果是 DM 控制台工具，需将这些 DLL 库文件放在 CONSOLE 工具所在 TOOL 目录的 external_crypto_libs 子目录下。

为了保证加密算法有效性，所有第三方库文件中的加密方法名及 ID 不同。DM 提供一个动态视图 V$EXTERNAL_CIPHERS，用户可查看所有的第三方加密算法的 ID、名称、所在的库文件以及是否有效标记。

V$EXTERNAL_CIPHERS 的结构如下表所示。

表 8.5 V$EXTERNAL_CIPHERS 结构

序号	列	数据类型	说明
1	ID	INTEGER	算法 ID
2	NAME	VARCHAR(128)	算法名
3	LIB	VARCHAR(300)	算法所在的 lib 库文件名
4	VALID	CHAR	算法是否有效。'Y'：是；'N'：否

8.3 UKEY 使用说明

8.3.1 使用介绍

DM 支持使用 UKEY 进行身份鉴别和加密。UKEY 用到的 UKEY 标准库（Linux 环境 libe2acsp11.so、Windows 环境 e2acsp11.dll）请用户自行从 UKEY 厂家获取，并放入达梦安装目录 bin 中。

ukey 认证操作说明如下：

1. 编写动态库，分别作用于客户端和服务器。

使用 UKEY 进行身份鉴别时，应按照 8.1 节的介绍编写两个动态库，分别作用于客户端和服务器。

服务器端动态库需要实现：

1） crypto_get_name 接口，用于设置加密引擎名， 服务器与客户端一致；

2） crypto_get_type 接口，返回 1；

3） cipher_asym_verify 或 cipher_asym_verify_ex 接口，用于验签；

4） cipher_gen_random 接口，用于生成随机数。

客户端动态库需要实现：

1） crypto_get_name 接口，用于设置加密引擎名，服务器与客户端一致；

2） crypto_get_type 接口，返回 2，表明为移动设备 ukey；

3） crypto_read_cert 接口，用于从 ukey 中读取客户端证书(DER 编码)，并发送给服务器；

4） cipher_asym_sign 或 cipher_asym_sign_ex 接口，用于签名；

5） crypto_login 接口，用于登录 ukey；

6） crypto_logout 接口，用于退出登录。

编写动态库时应注意：

1） 客户端和服务器端的动态库 crypto_get_name()的返回值必须相同；

2） 客户端动态库的名字*.dll/*.so 中的*必须与 crypto_get_name()返回值相同，否则 JDBC 驱动无法完成 UKEY 鉴别，DPI 接口无此要求；

3） 客户端动态库的 crypto_get_type()应返回 2，表示移动设备 UKEY；服务器端动态库的 crypto_get_type()返回 1，表示软件或硬件加密卡。

编写动态库完成后，分别将服务端动态库和客户端动态库放置于服务端和客户端的 external_crypto_libs 目录下。

2. 生成认证所需证书或使用版本中自带的 SSL 证书文件。

所需文件为 ca-cert.pem 和 client.crt(DER 编码)。

ca-cert.pem 文件放置于服务器端$DM_HOME/bin/server_ukey 目录下，若该目录下没有 ca-cert.pem，服务器将读取$DM_HOME/bin/server_ssl 下的 ca_cert.pem 文件。

使用 openssl 证书时，ca-cert.pem 文件放置于服务器端$DM_HOME/bin/server_ukey/openssl 目录下；若使用 gmssl 证书，则放置于$DM_HOME/bin/server_ukey/gmssl 目录下。若该目录下没有 ca-cert.pem，服务器将在$DM_HOME/bin/server_ukey 目录下查找 ca-cert.pem 文件。若证书依然未找到，使用 openssl 证书时服务器读取$DM_HOME/bin/server_ssl 下的 ca_cert.pem 文件；使用 gmssl 证书时服务器读取$DM_HOME/bin/server_gmssl 下的 cacert.pem 文件。

client.crt 文件需要存储在 ukey 中，用户可以自行导入，仅需保证通过客户端动态库接口 crypto_read_cert 可以获取 DER 编码的客户端证书即可。

3. 尝试连接。

完成以上操作之后，即可尝试连接。

以 disql 为例，假设 SYSDBA 用户的密码为 DMdba_123，加密引擎名为 test，ukey 登录 pin 为 password。

```
./disql "SYSDBA/DMdba_123@LOCALHOST#{UKEY_NAME=test,UKEY_PIN=password}"
```

8.3.2 示例

以渔翁 ukey、证书使用 SYSDBA 的通信加密证书为例。

我们已经提供了对应的服务端和客户端动态库, 并均已放置在对应目录。

ukey 导入交互式工具是 fm_gen，其专为渔翁 ukey 使用，在 $DM_HOME/bin/thirdparty/fisherman 目录下。目前仅有导入 DER 编码的客户端证书功能，无参数。其可以初始化 ukey 文件系统，创建目录及文件，将客户端证书写入文件，如下所示:

```
open device succeed
input password:
12345678
login succeed
write cert(1)/exit(0):
1
Init File System ? 1/0
1
succeed
Create dir ? 1/0
1
succeed
Create File ? 1/0
1
succeed
Write File ? 1/0
```

```
1
input cert file path
/home/dm/bin/client_ssl/SYSDBA/client.crt
```

使用 fm_gen 将 SYSDBA 目录下的 client.crt 导入 ukey 中，若没有对应文件，可以将 client-cert.pem 转换为 DER 编码。

最后启动服务器，通过 disql 进行连接。

假设 SYSDBA 用户的密码为 DMdba_123，加密引擎名为 dmukey_fm，默认 pin 为 12345678

```
./disql "SYSDBA/DMdba_123@LOCALHOST#{UKEY_NAME=dmukey_fm,UKEY_PIN=12345678}"
```

8.4 DM 提供的第三方加密和散列算法

DM 提供了第三方加密和第三方散列算法供用户使用。第三方算法来源分为两类：一 OPENSSL 软件库提供；二 渔翁硬件加密卡提供。

8.4.1 OPENSSL 软件库提供

OPENSSL 软件库提供了 OPENSSL_SM4 系列、OPENSSL_SM3 算法。OPENSSL_SM4 系列为分组加密算法，与 DM 内置的分组加密算法用途一致，可以用于通信加密、存储加密等场景。OPENSSL_SM3 为散列算法，与 DM 内置的散列算法用途一致，可以用于完整性校验。

OPENSSL 动态库提供了调用 OPENSSL 算法的接口。DM 的 OPENSSL 动态库文件为位于 bin/dependencies 文件夹下的 libcrypto-3.so, libssl-3.so（Linux 操作系统）或 libcrypto-3.dll 和 libssl-3.dll（Windows 32 位操作系统）、libcrypto-3-x64.dll 和 libssl-3-x64.dll（Windows 64 位操作系统）。

DM 提供的 OPENSSL_SM3、OPENSSL_SM4 系列算法是由 OPENSSL 软件库提供，DM 进行封装的，方便用户直接使用。因此，如果用户不想使用 DM 提供的 OPENSSL 算法，也可以直接使用 OPENSSL 软件库提供的算法。需要注意，若用户选择使用的 OPENSSL 版本过低，可能会缺失 OPENSSL_SM3、OPENSSL_SM4 系列算法。其中，OPENSSL_SM4_CFB_V1、OPENSSL_SM4_OFB_V1 算法分别是 OPENSSL_SM4_CFB、OPENSSL_SM4_OFB 算法的新版本，与老版本不兼容，因此当用户需要使用上述算法时，建议优先选择新版本。

表 8.6 DM 提供的第三方加密（OPENSSL_SM4）和散列算法（OPENSSL_SM3）

算法名称	算法类型	分组长度（单位：字节）	密钥长度（单位：字节）
OPENSSL_SM4_ECB	分组加密算法	16	16
OPENSSL_SM4_CBC	分组加密算法	16	16
OPENSSL_SM4_CFB	分组加密算法	16	16
OPENSSL_SM4_OFB	分组加密算法	16	16
OPENSSL_SM3	散列算法	-	-
OPENSSL_SM4_ECB_NOPAD	分组加密算法	16	16
OPENSSL_SM4_CBC_NOPAD	分组加密算法	16	16
OPENSSL_SM4_CFB_V1	分组加密算法	16	16
OPENSSL_SM4_OFB_V1	分组加密算法	16	16

OPENSSL_SM3、OPENSSL_SM4 系列算法可以通过动态视图 V$CIPHERS 查看。因为通过 external_crypto_libs 文件夹下的文件进行支持，因此也可以通过查询动态视图 V$external_ciphers 进行查询。

```
select * from V$external_ciphers;


行号     ID          NAME                LIB                 VALID
```

```
---------- ----------- --------------------- ----------------------- -----
1          5201        OPENSSL_SM4_ECB       openssl1.1.1_crypto.dll Y
2          5202        OPENSSL_SM4_CBC       openssl1.1.1_crypto.dll Y
3          5203        OPENSSL_SM4_CFB       openssl1.1.1_crypto.dll Y
4          5204        OPENSSL_SM4_OFB       openssl1.1.1_crypto.dll Y
5          5207        OPENSSL_SM3           openssl1.1.1_crypto.dll Y
6          5205        OPENSSL_SM4_ECB_NOPAD openssl1.1.1_crypto.dll Y
7          5206        OPENSSL_SM4_CBC_NOPAD openssl1.1.1_crypto.dll Y
8          5208        OPENSSL_SM4_CFB_V1    openssl1.1.1_crypto.dll Y
9          5209        OPENSSL_SM4_OFB_V1    openssl1.1.1_crypto.dll Y
```

8.4.2 渔翁硬件加密卡提供

渔翁硬件加密卡提供了 FM_SM1、FM_SM2、FM_SM3、FM_SM4、FM_SM6 算法。只有使用了渔翁加密卡的场景中才适用本节。FM_SM1 系列、 FM_SM4 系列、FM_SM6 系列为分组加密算法，与 DM 内置的分组加密算法用途一致，可以用于通信加密、存储加密等场景。FM_SM3、FM_SM2 为散列算法，与 DM 内置的散列算法用途一致，可以用于完整性校验。FM_SM3_HMAC 为 HMAC 算法，基于哈希的消息认证码算法，可以用于完整性校验， 使用方式与散列算法类似。

渔翁动态库提供了调用渔翁算法的接口。渔翁动态库文件为位于 bin/external_crypto_libs 文件夹下的 libenc_dll.so(linux) 或 card_enc.dll(windows)，此动态库依赖于 bin 文件夹下的 libfmapiv100.so(linux)或 fmapiv100.dll(windows)。

表 8.7 DM 提供的第三方加密和散列算法（渔翁硬件加密卡）

算法名称	算法类型	分组长度（单位：字节）	密钥长度（单位：字节）
FM_SM1_ECB	分组加密算法	16	16

FM_SM1_CBC	分组加密算法	16	16
FM_SM4_ECB	分组加密算法	16	16
FM_SM4_CBC	分组加密算法	16	16
FM_SM6_ECB	分组加密算法	16	16
FM_SM6_CBC	分组加密算法	16	16
FM_SM1_ECB_NOPAD	分组加密算法	16	16
FM_SM1_CBC_NOPAD	分组加密算法	16	16
FM_SM4_ECB_NOPAD	分组加密算法	16	16
FM_SM4_CBC_NOPAD	分组加密算法	16	16
FM_SM6_ECB_NOPAD	分组加密算法	16	16
FM_SM6_CBC_NOPAD	分组加密算法	16	16
FM_SM3	散列算法	-	-
FM_SM2	散列算法	-	-
FM_SM3_HMAC	HMAC算法	-	-

渔翁硬件加密卡算法可以通过动态视图 V$CIPHERS 或 V$external_ciphers 查看。示例使用了渔翁硬件加密卡 5.0 版本。

```
SQL> select * from v$external_ciphers;


行号     ID          NAME            LIB          VALID

---------- ----------- ---------------- ------------ -------------

1          5001        FM_SM1_ECB       libenc_dll.so Y

2          5002        FM_SM1_CBC       libenc_dll.so Y

3          5003        FM_SM4_ECB       libenc_dll.so Y

4          5004        FM_SM4_CBC       libenc_dll.so Y

5          5005        FM_SM6_ECB       libenc_dll.so Y

6          5006        FM_SM6_CBC       libenc_dll.so Y

7          5007        FM_SM3           libenc_dll.so Y

8          5008        FM_SM2           libenc_dll.so Y
```

```
9        5009      FM_SM1_ECB_NOPAD libenc_dll.so Y
10       5010      FM_SM1_CBC_NOPAD libenc_dll.so Y
11       5011      FM_SM4_ECB_NOPAD libenc_dll.so Y
12       5012      FM_SM4_CBC_NOPAD libenc_dll.so Y
13       5013      FM_SM6_ECB_NOPAD libenc_dll.so Y
14       5014      FM_SM6_CBC_NOPAD libenc_dll.so Y
15       5015      FM_SM3_HMAC      libenc_dll.so Y
```

8.5 编程实例

8.5.1 加密引擎接口

本节用一个实例来说明 DM 加密引擎接口的编程方法。

例子中用户自定义加密算法为实现一个简单的异或加密算法和一个 HASH 算法。异或加密算法包括单字节的流加密（xor1、xor2）和 4 字节的块加密方式（xor3），HASH 算法为 MD5（名称为 new_md5）。

第一步，生成动态库。

使用 Microsoft Visual Studio 2008 创建新项目 mycrypto，将 xor.h 和 xor.c、hash.h 和 hash.c、crypto_engine.h 和 my_crypto.c 生成动态库 mycrypto.dll。其中 crypto_engine.h 被放在 DM 安装目录的 include 文件夹中。

涉及到的文件如下：

- xor.h

说明：异或加密算法函数的声明。

```
//xor.h :declare the xor cipher
#ifndef CRYPTO_ENGINE_H
typedef unsigned char byte;
#endif
```

```
int
xor_encrypt(
    byte key,
    byte* plain_text,
    int plain_text_len,
    byte* cipher_text,
    int cipher_text_len);
int
xor_decrypt(
    byte key,
    byte* cipher_text,
    int cipher_text_len,
    byte* plain_text,
    int plain_text_len);
int
xor_data_block_encrypt(
        int key,
        byte* in_text,
        int in_text_len,
        byte* out_text,
        int out_text_len);
int
xor_data_block_decrypt(
        int key,
        byte* in_text,
        int in_text_len,
        byte* out_text,
        int out_text_len);
```

- xor.c

  说明：异或加密算法的实现。

```
// xor.c : implement the xor cipher.
#include <stdio.h>
#include <stdlib.h>
#include "xor.h"
int
xor_data(
    byte key,
    byte* in_text,
    int in_text_len,
    byte* out_text,
    int out_text_len)
{
    int i = 0;
    for (i = 0; i < in_text_len; i++)
    {
        out_text[i] = in_text[i] ^ key;
    }
    return in_text_len;
}
int
xor_data_block_encrypt(
                        int key,
                        byte* in_text,
                        int in_text_len,
                        byte* out_text,
                        int out_text_len)
{
    int i = 0;
    int in_buf = 0;
int out_buf = 0;
```

```
    int out_len = 0;
    int pos = 0;
    int block_len = sizeof(int);
    byte* p = NULL;
    byte* t = NULL;
        int n = 0;
        int m = 0;
        int fill_num = 0;
        n = in_text_len / block_len + 1;
        m = in_text_len % block_len;
        out_len = block_len * n;
        p = (byte*)malloc(out_len);
    memset(p, 0, out_len);
    memcpy(p, in_text, in_text_len);
    fill_num = out_len - in_text_len;
        memset(p + in_text_len, fill_num, out_len - in_text_len);
    for (i = 0; i < out_len; i++)
    {
        p[i] = p[i] ^ key;
    }
        memcpy(out_text, p, out_len);
    free(p);
    return out_len;
}
int
xor_data_block_decrypt(
                        int key,
                        byte* in_text,
                        int in_text_len,
                        byte* out_text,
```

```
                        int out_text_len)
{
   int i = 0;
   int in_buf = 0;
      int out_buf = 0;
   int out_len = 0;
   int pos = 0;
   int block_len = sizeof(int);
   byte* p = NULL;
   byte* t = NULL;
      int n = 0;
      int m = 0;
      int fill_num = 0;
      n = in_text_len / block_len;
      m = in_text_len % block_len;
      if (m != 0)
      {
         return -1;
      }
   p = (byte*)malloc(in_text_len);
      memcpy(p, in_text, in_text_len);
   for (i = 0; i < in_text_len; i++)
   {
      p[i] = p[i] ^ key;
   }
      fill_num = p[in_text_len - 1];
   memcpy(out_text, p, in_text_len - fill_num);
   free(p);
   return in_text_len - fill_num;
}
```

```
int
xor_encrypt(
    byte key,
    byte* plain_text,
    int plain_text_len,
    byte* cipher_text,
    int cipher_text_len)
{
    return xor_data(key, plain_text, plain_text_len, cipher_text, cipher_text_len);
}
int
xor_decrypt(
    byte key,
    byte* cipher_text,
    int cipher_text_len,
    byte* plain_text,
    int plain_text_len)
{
    return xor_data(key, cipher_text, cipher_text_len, plain_text, plain_text_len);
}
```

- hash.h

  说明：hash 算法函数的声明。

```
// hash.h
typedef struct {
    unsigned int state[4];
    unsigned int count[2];
    unsigned char buffer[64];
} MD5Context;
void MD5_Init(MD5Context * context);
void MD5_Update(MD5Context * context, unsigned char * buf, int len);
```

```
void MD5_Final(MD5Context * context, unsigned char digest[16]);
```

- hash.c

  说明：hash 算法实现。

```
#include <string.h>
#include "hash.h"
#define S11 7
#define S12 12
#define S13 17
#define S14 22
#define S21 5
#define S22 9
#define S23 14
#define S24 20
#define S31 4
#define S32 11
#define S33 16
#define S34 23
#define S41 6
#define S42 10
#define S43 15
#define S44 21
static unsigned char PADDING[64] =
{0x80, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
   0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
   0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
#define F(x, y, z) (((x) & (y)) | ((~x) & (z)))
#define G(x, y, z) (((x) & (z)) | ((y) & (~z)))
#define H(x, y, z) ((x) ^ (y) ^ (z))
#define I(x, y, z) ((y) ^ ((x) | (~z)))
#define ROTATE_LEFT(x, n) (((x) << (n)) | ((x) >> (32-(n))))
```

```
#define FF(a, b, c, d, x, s, ac)          \
  {                  \
   (a) += F((b), (c), (d)) + (x) + (unsigned int)(ac); \
   (a) = ROTATE_LEFT((a), (s));          \
   (a) += (b);               \
   }
#define GG(a, b, c, d, x, s, ac)          \
   {                  \
   (a) += G((b), (c), (d)) + (x) + (unsigned int)(ac); \
   (a) = ROTATE_LEFT((a), (s));          \
   (a) += (b);               \
   }
#define HH(a, b, c, d, x, s, ac)          \
   {                  \
   (a) += H((b), (c), (d)) + (x) + (unsigned int)(ac); \
   (a) = ROTATE_LEFT((a), (s));          \
   (a) += (b);               \
   }
#define II(a, b, c, d, x, s, ac)          \
   {                  \
   (a) += I((b), (c), (d)) + (x) + (unsigned int)(ac); \
   (a) = ROTATE_LEFT((a), (s));          \
   (a) += (b);               \
   }
static void MD5_Encode(unsigned char * output, unsigned int * input, int len)
{
   unsigned int i, j;
   for (i = 0, j = 0; j < (unsigned int)len; i++, j += 4)
   {
     output[j] = (unsigned char) (input[i] & 0xff);
```

```
    output[j + 1] = (unsigned char) ((input[i] >> 8) & 0xff);
    output[j + 2] = (unsigned char) ((input[i] >> 16) & 0xff);
    output[j + 3] = (unsigned char) ((input[i] >> 24) & 0xff);
  }
}
static void MD5_Decode(unsigned int * output, unsigned char * input, int len)
{
  unsigned int i, j;
  for (i = 0, j = 0; j < (unsigned int)len; i++, j += 4)
  {
    output[i] = ((unsigned int) input[j]) |
      (((unsigned int) input[j + 1]) << 8) |
      (((unsigned int) input[j + 2]) << 16) |
      (((unsigned int) input[j + 3]) << 24);
  }
}
static void MD5_Transform(unsigned int state[4], unsigned char block[64])
{
  unsigned int a = state[0], b = state[1], c = state[2], d = state[3], x[16];
  MD5_Decode(x, block, 64);
  //Round 1
  FF(a, b, c, d, x[0], S11, 0xd76aa478);   /* 1 */
  FF(d, a, b, c, x[1], S12, 0xe8c7b756);   /* 2 */
  FF(c, d, a, b, x[2], S13, 0x242070db);   /* 3 */
  FF(b, c, d, a, x[3], S14, 0xc1bdceee);   /* 4 */
  FF(a, b, c, d, x[4], S11, 0xf57c0faf);   /* 5 */
  FF(d, a, b, c, x[5], S12, 0x4787c62a);   /* 6 */
  FF(c, d, a, b, x[6], S13, 0xa8304613);   /* 7 */
  FF(b, c, d, a, x[7], S14, 0xfd469501);   /* 8 */
  FF(a, b, c, d, x[8], S11, 0x698098d8);   /* 9 */
```

```
FF(d, a, b, c, x[9], S12, 0x8b44f7af);    /* 10 */
FF(c, d, a, b, x[10], S13, 0xffff5bb1);   /* 11 */
FF(b, c, d, a, x[11], S14, 0x895cd7be);   /* 12 */
FF(a, b, c, d, x[12], S11, 0x6b901122);   /* 13 */
FF(d, a, b, c, x[13], S12, 0xfd987193);   /* 14 */
FF(c, d, a, b, x[14], S13, 0xa679438e);   /* 15 */
FF(b, c, d, a, x[15], S14, 0x49b40821);   /* 16 */
// Round 2
GG(a, b, c, d, x[1], S21, 0xf61e2562);    /* 17 */
GG(d, a, b, c, x[6], S22, 0xc040b340);    /* 18 */
GG(c, d, a, b, x[11], S23, 0x265e5a51);   /* 19 */
GG(b, c, d, a, x[0], S24, 0xe9b6c7aa);    /* 20 */
GG(a, b, c, d, x[5], S21, 0xd62f105d);    /* 21 */
GG(d, a, b, c, x[10], S22, 0x2441453);    /* 22 */
GG(c, d, a, b, x[15], S23, 0xd8a1e681);   /* 23 */
GG(b, c, d, a, x[4], S24, 0xe7d3fbc8);    /* 24 */
GG(a, b, c, d, x[9], S21, 0x21e1cde6);    /* 25 */
GG(d, a, b, c, x[14], S22, 0xc33707d6);   /* 26 */
GG(c, d, a, b, x[3], S23, 0xf4d50d87);    /* 27 */
GG(b, c, d, a, x[8], S24, 0x455a14ed);    /* 28 */
GG(a, b, c, d, x[13], S21, 0xa9e3e905);   /* 29 */
GG(d, a, b, c, x[2], S22, 0xfcefa3f8);    /* 30 */
GG(c, d, a, b, x[7], S23, 0x676f02d9);    /* 31 */
GG(b, c, d, a, x[12], S24, 0x8d2a4c8a);   /* 32 */
// Round 3
HH(a, b, c, d, x[5], S31, 0xfffa3942);    /* 33 */
HH(d, a, b, c, x[8], S32, 0x8771f681);    /* 34 */
HH(c, d, a, b, x[11], S33, 0x6d9d6122);   /* 35 */
HH(b, c, d, a, x[14], S34, 0xfde5380c);   /* 36 */
HH(a, b, c, d, x[1], S31, 0xa4beea44);    /* 37 */
```

```
HH(d, a, b, c, x[4], S32, 0x4bdecfa9);   /* 38 */
HH(c, d, a, b, x[7], S33, 0xf6bb4b60);   /* 39 */
HH(b, c, d, a, x[10], S34, 0xbebfbc70);  /* 40 */
HH(a, b, c, d, x[13], S31, 0x289b7ec6);  /* 41 */
HH(d, a, b, c, x[0], S32, 0xeaa127fa);   /* 42 */
HH(c, d, a, b, x[3], S33, 0xd4ef3085);   /* 43 */
HH(b, c, d, a, x[6], S34, 0x4881d05);        /* 44 */
HH(a, b, c, d, x[9], S31, 0xd9d4d039);   /* 45 */
HH(d, a, b, c, x[12], S32, 0xe6db99e5);  /* 46 */
HH(c, d, a, b, x[15], S33, 0x1fa27cf8);  /* 47 */
HH(b, c, d, a, x[2], S34, 0xc4ac5665);   /* 48 */
// Round 4
II(a, b, c, d, x[0], S41, 0xf4292244);   /* 49 */
II(d, a, b, c, x[7], S42, 0x432aff97);   /* 50 */
II(c, d, a, b, x[14], S43, 0xab9423a7);  /* 51 */
II(b, c, d, a, x[5], S44, 0xfc93a039);   /* 52 */
II(a, b, c, d, x[12], S41, 0x655b59c3);  /* 53 */
II(d, a, b, c, x[3], S42, 0x8f0ccc92);   /* 54 */
II(c, d, a, b, x[10], S43, 0xffeff47d);  /* 55 */
II(b, c, d, a, x[1], S44, 0x85845dd1);   /* 56 */
II(a, b, c, d, x[8], S41, 0x6fa87e4f);   /* 57 */
II(d, a, b, c, x[15], S42, 0xfe2ce6e0);  /* 58 */
II(c, d, a, b, x[6], S43, 0xa3014314);   /* 59 */
II(b, c, d, a, x[13], S44, 0x4e0811a1);  /* 60 */
II(a, b, c, d, x[4], S41, 0xf7537e82);   /* 61 */
II(d, a, b, c, x[11], S42, 0xbd3af235);  /* 62 */
II(c, d, a, b, x[2], S43, 0x2ad7d2bb);   /* 63 */
II(b, c, d, a, x[9], S44, 0xeb86d391);   /* 64 */
state[0] += a;
state[1] += b;
```

```
    state[2] += c;
    state[3] += d;
    memset((char *) x, 0, sizeof(x));
}
void MD5_Init(MD5Context * context)
{
    context->count[0] = context->count[1] = 0;
    context->state[0] = 0x67452301;
    context->state[1] = 0xefcdab89;
    context->state[2] = 0x98badcfe;
    context->state[3] = 0x10325476;
}
void MD5_Update(MD5Context * context, unsigned char * buf, int len)
{
    unsigned int i, index, partLen;
    index = (unsigned int) ((context->count[0] >> 3) & 0x3F);
    if ((context->count[0] += ((unsigned int) len << 3)) < ((unsigned int) len << 3))
    context->count[1]++;
    context->count[1] += ((unsigned int) len >> 29);
    partLen = 64 - index;
    if ((unsigned int)len >= partLen)
    {
      memcpy((char *) &context->buffer[index], (char *) buf, partLen);
      MD5_Transform(context->state, context->buffer);
      for (i = partLen; i + 63 < (unsigned int)len; i += 64)
         MD5_Transform(context->state, &buf[i]);
      index = 0;
    }
    else
    {
```

```
        i = 0;
    }
    memcpy((char *) &context->buffer[index], (char *) &buf[i], len - i);
}
void MD5_Final(MD5Context * context, unsigned char digest[16])
{
    unsigned char bits[8];
    unsigned int index, padLen;
    MD5_Encode(bits, context->count, 8);
    index = (unsigned int) ((context->count[0] >> 3) & 0x3f);
    padLen = (index < 56) ? (56 - index) : (120 - index);
    MD5_Update(context, PADDING, padLen);
    MD5_Update(context, bits, 8);
    MD5_Encode(digest, context->state, 16);
    memset((char *) context, 0, sizeof(*context));
}
```

- my_crypto.c

  说明：封装供 DM 加载的函数接口。

```
//my_crypto.c
#include <stdlib.h>
#include "crypto_engine.h"
#include "xor.h"
#include "hash.h"
#define  MIN_EXTERNAL_CIPHER_ID        5000
#define CYT_TYPE_UNKNOWN                    0           //类型, 未知算法
#define CYT_TYPE_SYM_BLOCK_ENCRYPT          1           //类型, 分组加密算法
#define CYT_TYPE_SYM_STREAM_ENCRYPT         2           //类型, 流加密算法
#define CYT_TYPE_ASYM_ENCRYPT               3           //类型, 非对称加密算法, 保留
```

```
#define CYT_TYPE_HASH                    4              //类型, 散列算法
#define KEY1 ((byte)(0XAA))
#define KEY2 ((byte)(0x55))
#define KEY3 ((int)(0xCCCCCCCC))
DllExport
ulint
cipher_get_count(
)
{
    return 4;
}

DllExport
dm_bool
cipher_get_info(
  ulint   seqno,
  ulint* cipher_id,
  byte**          cipher_name,
  byte* type,
  ulint* blk_size,
  ulint* kh_size
)
{
  switch (seqno)
  {
  case 1:
    *cipher_id   = MIN_EXTERNAL_CIPHER_ID;
    *cipher_name = "xor1";
    *type        = CYT_TYPE_SYM_STREAM_ENCRYPT;
    *blk_size    = 0;
```

```
        *kh_size    =  sizeof(KEY1);
        break;
    case 2:
        *cipher_id   = MIN_EXTERNAL_CIPHER_ID + 1;
        *cipher_name = "xor2";
        *type        =  CYT_TYPE_SYM_STREAM_ENCRYPT;
        *blk_size    =  0;
        *kh_size    =  sizeof(KEY1);
        break;
    case 3:
        *cipher_id   = MIN_EXTERNAL_CIPHER_ID + 2;
        *cipher_name = "xor3";
        *type        =  CYT_TYPE_SYM_BLOCK_ENCRYPT;
        *blk_size    =  4;
        *kh_size    =  sizeof(KEY1);
        break;
    case 4:
        *cipher_id   = MIN_EXTERNAL_CIPHER_ID + 3;
        *cipher_name = "new_md5";
        *type        =  CYT_TYPE_HASH;
        *blk_size    =  0;
        *kh_size    =  16;
        break;
    default:
        return DM_FALSE;
    }
    return DM_TRUE;
}

DllExport
```

```
dm_bool
cipher_get_para(
        ulint     cipher_id,
        ulint     para_id,
        void*     value
){
    switch (cipher_id)
  {
    case MIN_EXTERNAL_CIPHER_ID:
     switch (para_id)
          {
                case CYT_PARA_WORKMODE:
                *(byte*)value = 0;
                break;
              case CYT_PARA_EXTENDSIZE:
                *(int*)value = 0;
                break;
                default:
                        return DM_FALSE;
          }
    break;
  case MIN_EXTERNAL_CIPHER_ID+1:
    switch (para_id)
          {
                case CYT_PARA_WORKMODE:
                *(byte*)value = 0;
                break;
              case CYT_PARA_EXTENDSIZE:
                *(int*)value = 0;
                break;
```

```
                default:
                        return DM_FALSE;
        }
    break;
case MIN_EXTERNAL_CIPHER_ID+2:
    switch (para_id)
        {
                case CYT_PARA_WORKMODE:
                *(byte*)value = 0;
                break;
            case CYT_PARA_EXTENDSIZE:
                *(int*)value = 4;
                break;
                default:
                        return DM_FALSE;
        }
    break;
case MIN_EXTERNAL_CIPHER_ID+3:
    switch (para_id)
        {
                case CYT_PARA_WORKMODE:
                *(byte*)value = 0;
                break;
            case CYT_PARA_EXTENDSIZE:
                *(int*)value = 0;
                break;
                default:
                        return DM_FALSE;
        }
    break;
```

```
    default:
        return DM_FALSE;
}
    return DM_TRUE;
}

DllExport
dm_bool
cipher_encrypt_init(
        ulint           inner_id,
        byte*           key,
        ulint           key_size,
        void**          encrypt_para
)
{
        switch (inner_id)
        {
                case MIN_EXTERNAL_CIPHER_ID:
                        break;
                case MIN_EXTERNAL_CIPHER_ID + 1:
                        break;
                case MIN_EXTERNAL_CIPHER_ID + 2:
                        break;
                default:
                        return DM_FALSE;
        }
        return DM_TRUE;
}

DllExport
```

```
lint
cipher_get_cipher_text_size(
ulint      inner_id,
void*      cipher_para,
lint       plain_text_size
)
{
lint len = 0;
      lint block_len = sizeof(int);
switch (inner_id)
{
          case MIN_EXTERNAL_CIPHER_ID:
                    len = plain_text_size;
                    break;
          case MIN_EXTERNAL_CIPHER_ID + 1:
                    len = plain_text_size;
                    break;
          case MIN_EXTERNAL_CIPHER_ID + 2:
                    len = (plain_text_size / block_len + 1) * block_len;
                    break;
          default:
                    return -1;
}
return len;
}

DllExport
lint
cipher_encrypt(
      ulint          inner_id,
```

```
    void*       cipher_para,
    byte*       plain_text,
    ulint       plain_text_size,
    byte*       cipher_text,
    ulint       cipher_text_buf_size
)
{
    int len = plain_text_size;
    if (plain_text_size > cipher_text_buf_size)
  {
        return -1;
  }
    switch(inner_id)
    {
        case MIN_EXTERNAL_CIPHER_ID:
                len = xor_encrypt(KEY1, plain_text, plain_text_size, cipher_text,
cipher_text_buf_size);
                break;
        case MIN_EXTERNAL_CIPHER_ID + 1:
                len = xor_encrypt(KEY2, plain_text, plain_text_size, cipher_text,
cipher_text_buf_size);
                break;
        case MIN_EXTERNAL_CIPHER_ID + 2:
                len = xor_data_block_encrypt(KEY3, plain_text, plain_text_size, cipher_text,
cipher_text_buf_size);
    break;
        default:
                return -1;
    }
    return len;
```

```
}

DllExport
dm_bool
cipher_decrypt_init(
        ulint           inner_id,
        byte*           key,
        ulint           key_size,
        void**          decrypt_para
)
{
        switch (inner_id)
        {
            case MIN_EXTERNAL_CIPHER_ID:
                        break;
            case MIN_EXTERNAL_CIPHER_ID + 1:
                        break;
            case MIN_EXTERNAL_CIPHER_ID + 2:
                        break;
            default:
                        return DM_FALSE;
        }
    return DM_TRUE;
}

DllExport
lint
cipher_decrypt(
   ulint    inner_id,
   void*  decrypt_para,
```

```
  byte*  cipher_text,
  ulint    cipher_text_size,
  byte*  plain_text,
  ulint    plain_text_buf_size
)
{
      dm_bool    bRet = DM_FALSE;
      int  len = cipher_text_size;
      switch(inner_id)
      {
          case MIN_EXTERNAL_CIPHER_ID:
                    len = xor_decrypt(KEY1, cipher_text, cipher_text_size, plain_text,
plain_text_buf_size);
                    break;
          case MIN_EXTERNAL_CIPHER_ID + 1:
                    len = xor_decrypt(KEY2, cipher_text, cipher_text_size, plain_text,
plain_text_buf_size);
                    break;
          case MIN_EXTERNAL_CIPHER_ID + 2:
             len = xor_data_block_decrypt(KEY3, cipher_text, cipher_text_size, plain_text,
plain_text_buf_size);
                    break;
          default:
                    return -1;
      }
      return len;
}

DllExport
dm_bool
```

```
cipher_hash_init(
        ulint           inner_id,
        void**          hash_para
)
{
        MD5Context* md5_context = NULL;
        switch (inner_id)
        {
            case MIN_EXTERNAL_CIPHER_ID + 3:
                        *hash_para = malloc(sizeof(MD5Context));
        md5_context = (MD5Context*)(*hash_para);
                        MD5_Init(md5_context);
                        break;
            default:
                        return DM_FALSE;
        }
    return DM_TRUE;
}

DllExport
void
cipher_hash_update(
        ulint           inner_id,
        void*           hash_para,
        byte*           msg,
        ulint           msg_size
)
{
        MD5Context* md5_context = NULL;
        switch (inner_id)
```

```
    {
        case MIN_EXTERNAL_CIPHER_ID + 3:
    md5_context = (MD5Context*)(hash_para);
                MD5_Update(md5_context, msg, msg_size);
                break;
        default:
                return;
    }
  return;
}

DllExport
lint
cipher_hash_final(
    ulint       inner_id,
    void*       hash_para,
    byte*       digest,
    ulint       digest_buf_size
)
{
    lint len = 0;
    MD5Context* md5_context = NULL;
  switch (inner_id)
    {
        case MIN_EXTERNAL_CIPHER_ID + 3:
                md5_context = (MD5Context*)(hash_para);
                MD5_Final(md5_context, digest);
    len = 16; // MD5 后为一个 bit 的摘要
                free(md5_context);
                break;
```

```
        default:
                return -1;
    }
  return len;
}

DllExport
void
cipher_cleanup(
    ulint       inner_id,
    void*       cipher_para
)
{
    switch (inner_id)
    {
        case MIN_EXTERNAL_CIPHER_ID:
                break;
        case MIN_EXTERNAL_CIPHER_ID + 1:
                break;
        case MIN_EXTERNAL_CIPHER_ID + 2:
                break;
        case MIN_EXTERNAL_CIPHER_ID + 3:
                break;
        default:
                return;
    }
  return;
}

DllExport
```

```
dm_bool
cipher_asym_sign(
    ulint           inner_id,
    byte*           prikey,         //私钥签名
    ulint           prikey_size,
    byte*           data,
    ulint           data_size,
    byte*           signdata,
    ulint*          signdata_buf_size
)
{
    return DM_FALSE;
}

DllExport
dm_bool
cipher_asym_verify(
    ulint       inner_id,
    byte*       pubkey,             //公钥验签
    ulint       pubkey_size,
    byte*       data,
    ulint       data_size,
    byte*       signdata,
    ulint       signdata_size
)
{
    return DM_FALSE;
}

DllExport
```

```
dm_bool
crypto_login(
    void**      cipher_para,
    byte*   pin,
    ulint       pin_len
)
{
    return DM_FALSE;
}

DllExport
dm_bool
crypto_logout(
    void*       cipher_para
)
{
    return DM_FALSE;
}

DllExport
dm_bool
crypto_read_cert(
    void*               cipher_para,
    byte*               cert,
    ulint *     cert_len
)
{
    return DM_FALSE;
}
```

```
DllExport
dm_bool
cipher_gen_random(
    byte*       random,
    ulint       random_length
)
{
    return DM_FALSE;
}

DllExport
dm_bool
crypto_get_name(
    byte**      crypto_name,
    ulint*      len
)
{
    *crypto_name = "mycrypto";
    *len = 8;
    return DM_TRUE;
}

DllExport
dm_bool
crypto_get_type(
    ulint*      crypto_type
)
{
    *crypto_type = 0;
    return DM_TRUE;
```

```
}
```

- Crypto_engine.h

说明：库接口说明文件。

```
#ifndef CRYPTO_ENGINE_H
#define CRYPTO_ENGINE_H
#ifndef dm_h
typedef int   lint;
typedef short sint;
typedef signed char tint;
typedef unsigned int ulint;
typedef unsigned short usint;
typedef unsigned char byte;
#ifdef _WIN64
#define dm_int3264 __int64
#define dm_uint3264 unsigned __int64
#else
#define dm_int3264 long
#define dm_uint3264 unsigned long
#endif
#endif //#ifndef dm_h
#ifdef WIN32
#define DllImport __declspec( dllimport )
#define DllExport __declspec( dllexport )
#else
#define DllImport
#define DllExport
#endif
#define  MIN_EXTERNAL_CIPHER_ID        5000
#define  CYT_TYPE_UNKNOWN                    0                    //type, unknown arithmetic
```

```
#define CYT_TYPE_SYM_BLOCK_ENCRYPT 1 //type, symmetry
encrypt arithmetic
#define CYT_TYPE_SYM_STREAM_ENCRYPT 2 //type, flow encrypt
arithmetic
#define CYT_TYPE_ASYM_ENCRYPT 3 //type,
unsymmetrical encrypt arithmetic, reserved
#define CYT_TYPE_HASH 4 //type, hash
arithmetic
#ifdef __cplusplus
extern "C" {
#endif
#ifdef WIN32
#define dm_bool long
#else
#define dm_bool unsigned int
#endif
#define DM_FALSE 0
#define DM_TRUE 1
#define CYT_PARA_WORKMODE 0
#define CYT_PARA_EXTENDSIZE 1
DllExport
ulint
cipher_get_count(
);
DllExport
dm_bool
cipher_get_info(
   ulint   seqno,
   ulint* cipher_id,
   byte**  cipher_name,
```

```
    byte* type,
    ulint* blk_size,
    ulint* kh_size
);
DllExport
dm_bool
cipher_encrypt_init(
        ulint     inner_id,
        byte*     key,
        ulint     key_size,
        void**    encrypt_para
);
DllExport
lint
cipher_get_cipher_text_size(
        ulint     inner_id,
        void*     cipher_para,
        lint      plain_text_size
);
DllExport
lint
cipher_encrypt(
        ulint     inner_id,
        void*     encrypt_para,
        byte*     plain_text,
        ulint     plain_text_size,
        byte*     cipher_text,
        ulint     cipher_text_buf_size
);
DllExport
```

```
dm_bool
cipher_decrypt_init(
        ulint     inner_id,
        byte*     key,
        ulint     key_size,
        void**    decrypt_para
);
DllExport
lint
cipher_decrypt(
  ulint   inner_id,
  void*   decrypt_para,
  byte*   cipher_text,
  ulint   cipher_text_size,
  byte*   plain_text,
  ulint   plain_text_buf_size
);
DllExport
dm_bool
cipher_hash_init(
        ulint     inner_id,
        void**    hash_para
);
DllExport
void
cipher_hash_update(
        ulint     inner_id,
        void*     hash_para,
        byte*     msg,
        ulint     msg_size
```

```
);
DllExport
lint
cipher_hash_final(
        ulint     inner_id,
        void*     hash_para,
        byte*     digest,
        ulint     digest_buf_size
);
DllExport
void
cipher_cleanup(
        ulint     inner_id,
        void*     hash_para
);
#ifdef __cplusplus
}
#endif
#endif
```

第二步，在安装目录 bin 下创建一个 external_crypto_libs 文件夹。将第一步中生成的动态库 mycrypto.dll 放入该文件夹中。

第三步，重启服务器。

第四步，至此，可以使用自定义的算法了。也可以通过 V$EXTERNAL_CIPHERS 查看到。

```
SQL> select * from V$EXTERNAL_CIPHERS;
行号     ID          NAME    LIB       VALID
---------- ----------- ------- --------- -----
1        5000        xor1    mycrypto.dll Y
```

```
2       5001      xor2    mycrypto.dll Y
3       5002      xor3    mycrypto.dll Y
4       5003      new_md5 mycrypto.dll Y
```

8.5.2 密码机引擎接口

密码机引擎和常规加密引擎的主要区别在于增加了 cipher_get_key_id 和 cipher_free_key_id 的使用，分别用于生成 key_id 和销毁 key_id。

密码机引擎的主要特点是密钥由密码机生成、销毁。因此当数据库需要生成密钥时，将会调用加密引擎中的 cipher_get_key_id 接口来获取一个新密钥，后续的相关加解密均使用该密钥。当不再需要秘钥，例如和密钥相关的表被删除时，数据库会调用 cipher_free_key_id 接口进行密钥的销毁。

本例实现了可供密码机使用的，通过加密引擎来管理秘钥的两个加密算法 XASJ_SYMM 和 XASJ_ASYMM。 引擎向服务器提供密钥标识符。加解密时服务器提供该标识符, 引擎根据标识符决定使用哪一个密钥。

第一步，生成动态库。

使用 Microsoft Visual Studio 2008 创建新项目 cryptoengine，将头文件 crypto_engine.h 和源文件 crypto_engine.c 生成动态库 cryptoengine.dll。

在“配置属性—常规”中：输出目录设为 D:\xx\tt\；目标文件扩展名选择动态库（.dll）；配置类型选择动态库（.dll）；字符集选择使用多字节字符集。

如果是 Windows 系统，还需要选择“配置属性—C/C++—预处理器—预处理定义”中增加 WIN32。

涉及到的文件如下：

- **头文件 crypto_engine.h**

  crypto_engine.h 为系统自带文件，包含了引擎的接口定义和宏定义。请用户前往 DM 安装目录 include 文件夹中查找。

- **源文件 crypto_engine.c**

  实现加密算法 XASJ_SYMM 和 XASJ_ASYMM。

```
// crypto_engine.c
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <time.h>
#include "crypto_engine.h"
#ifdef WIN32
#include "windows.h"
#include "Wincrypt.h"
#else
#include <pthread.h>
#include <dlfcn.h>
#include <errno.h>
#include <unistd.h>
#include <sys/stat.h>
#endif
#define DM_NULL                 0
#define CRYPTO_NAME             "DMSDB"
#define CIPHER_COUNT            1
#define RET_OK                  0
#define ISEC_ERR_NONE           0X00000000
#define ISEC_ERR_FAIL           0X0A000001
#define CYT_NAME_XASJ_SYMM      "XASJ_SYMM"
#define CYT_NAME_XASJ_ASYMM     "XASJ_ASYMM"
```

```
#define EXTERNAL_CIPHER_ID  5100
//symm
#define CYT_ID_XASJ_SYMM          (EXTERNAL_CIPHER_ID + 1)
//asymm
#define CYT_ID_XASJ_ASYMM         (EXTERNAL_CIPHER_ID + 2)
#define WDIR_CHAR       '\\'
#define WDIR_STR        "\\"
#define UDIR_CHAR       '/'
#define UDIR_STR        "/"
#ifdef WIN32
#define DIR_CHAR        WDIR_CHAR
#define DIR_STR         WDIR_STR
#define DIR_STR_LEN     1
#else
#define DIR_CHAR        UDIR_CHAR
#define DIR_STR         UDIR_STR
#define DIR_STR_LEN     1
#endif

typedef char              pathname_t[256];
typedef unsigned int          ulint;
#define END               0   /* \0 */
#define QUOTATION_MARK      39  /* ' */
#define CYT_TYPE_UNKNOWN              0
#define CYT_TYPE_SYM_BLOCK            1
#define CYT_TYPE_SYM_STREAM           2
#define CYT_TYPE_ASYM_ENCRYPT         3
#define TOKEN_TYPE_ENCRYPT_CARD       1
#define TOKEN_TYPE_USB_KEY            2
#define ENC_LOG_FILE          "enc.log"
```

```
#define CONFIG_LOG_FILE        "/dev/stderr"
#define ENC_EXTENT_SIZE        0       //0 - plain_size == cipher_size.
#define ENC_EXTENT_SIZE_SYMM    0
#define ENC_EXTENT_SIZE_ASYMM   0
#define INVALID_HANDLE_VALUE ((HANDLE)(LONG_PTR)-1)
#ifdef WIN32
typedef struct __stat64     dmstat_t;
#define dm_stat(path, st)   _stat64(path, st)
#else
typedef struct stat         dmstat_t;
#define dm_stat(path, st)   stat(path, st)
#endif


char*          global_msg_buff;
int            global_msg_size;
char           str_time[100] = "";
char           global_log_str[1024] = "";
int            global_enc_debug = 0;
int            global_sleep = 1000;
int            global_retry_time = 5;
char           global_enc_log[512] = {0};
char           global_log_file[512] = {0};


DllExport
ulint
cipher_get_count()
{
   return CIPHER_COUNT;
}
```

```
DllExport
dm_bool
cipher_get_info(
    ulint   seqno,
    ulint*  cipher_id,
    byte**  cipher_name,
    byte*   type,
    ulint*  blk_size,
    ulint*  kh_size
)
{
    if (seqno > CIPHER_COUNT)
    {
        return DM_FALSE;
    }

    *cipher_id = seqno + EXTERNAL_CIPHER_ID;
    switch(*cipher_id)
    {
        case CYT_ID_XASJ_SYMM:
            *cipher_name = CYT_NAME_XASJ_SYMM;
            *type = CYT_TYPE_SYM_BLOCK;
            *blk_size = 16;
            *kh_size = 0;
            break;
        case CYT_ID_XASJ_ASYMM:
            *cipher_name = CYT_NAME_XASJ_ASYMM;
            *type = CYT_TYPE_ASYM_ENCRYPT;
            *blk_size = 16;
            *kh_size = 0;
```

```
            break;
        default:
            return DM_FALSE;
    }
    return DM_TRUE;
}

DllExport
dm_bool
cipher_encrypt_init(
    ulint   cipher_id,
    byte*   key,
    ulint   key_size,
    void**  cipher_para
)
{
    byte*    para;
    para = (byte*)malloc(key_size);
    memcpy(para, key, key_size);
    *cipher_para = para;
    return DM_TRUE;
}

DllExport
lint
cipher_get_cipher_text_size(
    ulint   cipher_id,
    void*   cipher_para,
    lint    plain_text_size
)
```

```
{
    lint        len = 0;
    switch (cipher_id)
    {
    case CYT_ID_XASJ_SYMM:
        len = plain_text_size;
        break;
    case CYT_ID_XASJ_ASYMM:
        len = plain_text_size;
        break;
    default:
        return -1;
    }
    return len;
}


DllExport
lint
cipher_encrypt(
    ulint   cipher_id,
    void*   cipher_para,
    byte*   plain_text,
    ulint   plain_text_size,
    byte*   cipher_text,
    ulint   cipher_text_buf_size
)
{
    ulint           rv = 0;
    ulint           cipher_len;
    char            key[1024]="";
```

```
    ulint               key_len = 0;
    int                 i;

    if (plain_text_size == 0)
        return 0;
    switch(cipher_id)
    {
    case CYT_ID_XASJ_SYMM:
        for (i = 0; i < plain_text_size; ++i)
            cipher_text[i] = plain_text[i] + ((byte*)cipher_para)[0];
        break;
    case CYT_ID_XASJ_ASYMM:
        for (i = 0; i < plain_text_size; ++i)
            cipher_text[i] = plain_text[i] + ((byte*)cipher_para)[0];
        break;
    default:
        return -1;
    }
    if (rv != 0)
    {
        return -1;
    }
    cipher_len = plain_text_size;
    return cipher_len;
}

DllExport
dm_bool
cipher_decrypt_init(
    ulint   cipher_id,
```

```
    byte*   key,
    ulint   key_size,
    void**  cipher_para
)
{
    return cipher_encrypt_init(cipher_id, key, key_size, cipher_para);
}

DllExport
lint
cipher_decrypt(
    ulint   cipher_id,
    void*   cipher_para,
    byte*   cipher_text,
    ulint   cipher_text_size,
    byte*   plain_text,
    ulint   plain_text_buf_size
)
{
    ulint           rv = 0;
    ulint           cipher_len;
    char            key[1024]="";
    ulint           key_len = 0;
    int             i;

    switch(cipher_id)
    {
    case CYT_ID_XASJ_SYMM:
        for (i = 0; i < cipher_text_size; ++i)
            plain_text[i] = cipher_text[i] - ((byte*)cipher_para)[0];
```

```
        break;
    case CYT_ID_XASJ_ASYMM:
        for (i = 0; i < cipher_text_size; ++i)
            plain_text[i] = cipher_text[i] - ((byte*)cipher_para)[0];
        break;
    default:
        return -1;
    }

    if (rv != 0)
    {
        return -1;
    }
    return cipher_text_size;
}

DllExport
dm_bool
crypto_login(
    void**  cipher_para,
    byte*   pin,
    ulint   pin_len
)
{
    return DM_FALSE;
}

DllExport
dm_bool
crypto_logout(
```

```
    void*    cipher_para
)
{
    return DM_FALSE;
}

DllExport
dm_bool
cipher_asym_gen_keypair(
    byte*    pubkey,
    ulint*   pubkey_size,
    byte*    prikey,
    ulint*   prikey_size
)
{
    return DM_FALSE;
}

DllExport
void
cipher_cleanup(
    ulint    cipher_id,
    void*    cipher_para
)
{
    ulint       rc = 0;
    return;
}

DllExport
```

```
dm_bool
crypto_get_name(
    byte**  crypto_name,
    ulint*  len
)
{
    *crypto_name = CRYPTO_NAME;
    *len = (ulint)strlen(CRYPTO_NAME);
    return DM_TRUE;
}

DllExport
dm_bool
crypto_get_type(
    ulint*  crypto_type
)
{
    *crypto_type = 1;
    return DM_TRUE;
}

DllExport
lint
crypto_encrypt(
    byte* plain_text,
    ulint plain_text_size,
    byte* cipher_text,
    ulint cipher_text_buf_size
)
{
```

```
    return -1;
}

DllExport
lint
crypto_decrypt(
    byte* cipher_text,
    ulint cipher_text_size,
    byte* plain_text,
    ulint plain_text_buf_size
)
{
    return -1;
}

DllExport
dm_bool
cipher_get_para(
    ulint cipher_id,
    ulint   para_id,
    void* value
)
{
    switch (para_id)
    {
    case CYT_PARA_WORKMODE:
        *(byte*)value=WORK_MODE_KID;
        break;
    case CYT_PARA_EXTENDSIZE:
        if(cipher_id == CYT_ID_XASJ_SYMM)
```

```
            *(int*)value = 0;
        else
            *(int*)value = 0;
        break;
    default:
        return DM_FALSE;
    }
    return DM_TRUE;
}

DllExport
dm_bool
cipher_hash_init(
    ulint   cipher_id,
    void**  cipher_para
)
{
        return DM_FALSE;
}

DllExport
void
cipher_hash_update(
    ulint   cipher_id,
    void*   hash_para,
    byte*   msg,
    ulint   msg_size
)
{
    return ;
```

```
}

DllExport
lint
cipher_hash_final(
    ulint   cipher_id,
    void*   hash_para,
    byte*   digest,
    ulint   digest_buf_size
)
{
    return -1;
}

DllExport
lint
cipher_get_key_id(
    ulint   cipher_id,
    byte*   key_id,
    ulint   key_id_size,
    ulint*  key_size
)
{
    *key_size = 1;
    key_id[0] = 1;
    return DM_TRUE;
}
DllExport
lint
cipher_free_key_id(
```

```
    ulint cipher_id,
    byte *key_id,
    ulint key_id_size)
{
    return DM_TRUE;
}
```

第二步，在安装目录 bin 下创建一个 external_crypto_libs 文件夹。将第一步中生成的动态库 D:\xx\tt\cryptoengine.dll 放入该文件夹中。

第三步，重启服务器。

第四步，至此，可以使用自定义加密引擎中的算法了。也可以通过 V$EXTERNAL_CIPHERS 查看到。

```
SQL> select * from V$EXTERNAL_CIPHERS;

行号     ID          NAME                 LIB                      VALID

---------- ----------- -------------------- -------------------------- -----

1        5101        XASJ_SYMM            cryptoengine.dll         Y

2        5102        XASJ_ASYMM           cryptoengine.dll         Y
```

9 资源限制

DM 支持在创建和修改用户时指定对用户进行资源限制，具体语法参考本文 2.2 节的<资源限制子句>介绍。

用户的资源限制管理有两种方式:

方式一，将用户关联到某个已存在的 PROFILE 对象上，之后用户的资源限制由关联的 PROFILE 统一管理配置，且无法再使用方式二中的 LIMIT <资源设置>设置资源设置项。

方式二，未关联 PROFILE 的用户使用 LIMIT <资源设置>设置资源设置项。

新建用户默认使用方式二管理资源设置，在关联到 PROFILE 时，之前已设置的资源限制项会立即失效。

资源限制用于限制用户对 DM 数据库系统资源的使用，表 9.1 列出了资源限制所包含的设置项内容。

表 9.1 DM 资源限制一览表

资源设置项	说明	最大值	最小值	缺省值
GLOBAL_SESSION_PER_USER	在所有 DSC 实例中，一个用户可以同时拥有的会话数量	32768	1	系统所能提供的最大值
SESSION_PER_USER	在一个实例中，一个用户可以同时拥有的会话数量	32768	1	安全版本中默认值为 4096;其他版本中默认值为系统所能提供的最大值
CONNECT_TIME	一个会话连接、访问和操作数据	当	1	无限制

	库服务器的时间上限。 单位由 INI 参数 RESOURCE_FLAG 决定。 RESOURCE_FLAG 取值 1、0 时，CONNECT_TIME 单位分别为秒、分钟。 RESOURCE_FLAG 缺省值为 0	RESOURCE_FLAG 取值 1、0 时，缺省值分别为 86400 秒、1440 分钟（1 天）		
CONNECT_IDLE_TIME	会话最大空闲时间。单位由 INI 参数 RESOURCE_FLAG 决定。 RESOURCE_FLAG 取值 1、0 时，CONNECT_IDLE_TIME 单位分别为秒、分钟。 RESOURCE_FLAG 缺省值为 0	当 RESOURCE_FLAG 取值 1、0 时，缺省值分别为 86400 秒、1440 分钟（1 天）	1	无限制
FAILED_LOGIN_ATTEMPTS	将引起一个账户被锁定的连续注册失败的次数	100	1	3，系统预设管理员用户默认无限制
CPU_PER_SESSION	一个会话允许使用的 CPU 时间上限（单位：秒）	31536000（365 天）	1	无限制
CPU_PER_CALL	用户的一个请求能够使用的 CPU 时间上限（单位：秒）	86400（1 天）	1	无限制
READ_PER_SESSION	会话能够读取的总数据页数上限	2147483646	1	无限制
READ_PER_CALL	每个请求能够读取的数据页数	2147483646	1	无限制
MEM_SPACE	会话占有的私有内存空间上限（单位：MB）	2147483647	1	无限制
PASSWORD_LIFE_TIME	口令有效期（单位：天）。过了有效期，会进入过期口令宽限	365	1	无限制

	期。过期口令宽限期由 PASSWORD_GRACE_TIME 设置			
PASSWORD_REUSE_TIME	重用同一个口令的时间间隔（单位：天）。一个口令过了宽限期之后，将禁止使用。如果想再次使用该口令，须同时满足 PASSWORD_REUSE_TIME 和 PASSWORD_REUSE_MAX 的规定	365	1	无限制
PASSWORD_REUSE_MAX	重用同一个口令之前，需要修改口令的次数	32768	1	无限制
PASSWORD_LOCK_TIME	如果超过 FAILED_LOGIN_ATTEMPTS 设置值，一个用户将被锁定的分钟数	1440（1 天）	1	1，系统预设管理员用户默认无限制
PASSWORD_GRACE_TIME	过期口令宽限期（单位：天）。宽限期仍可正常访问数据库。但在宽限期中，必须对口令进行修改，修改之后使用新口令可以继续登录系统。如果宽限期未修改口令，超过该宽限期后，将禁止执行除修改口令以外的其他操作	30	1	10，系统预设管理员用户默认无限制
INACTIVE_ACCOUNT_TIME	指定用户的非活跃时间，用户超过指定时间未登录将被自动锁定	24855	15	无限制
 注意：	当 PASSWORD_GRACE_TIME 为无限制时不能设置 PASSWORD_LIFE_TIME。			

除上述资源设置项外，DM 还支持用户 IP 地址限制和用户时间段限制。

用户 IP 地址设置项包括 ALLOW_IP（允许的 IP 地址）和 NOT_ALLOW_IP（不允许的 IP 地址），具体的限制规则如下表所示。

表 9.2 IP 地址限制规则

IP 地址限制项		集合状态			
IP 地址集合	不允许的 IP 地址	空	N	空	N
	允许的 IP 地址	Y	空	空	Y
IP 限制结果		只允许属于集合 Y 的 IP 地址	只允许属于集合 N 以外的所有 IP 地址	允许所有 IP，即无任何 IP 地址限制	只允许属于 Y-N （差集）集合的 IP 地址

允许 IP 和禁止 IP 用于控制此登录是否可以从某个 IP 访问数据库，其中禁止 IP 优先。在设置 IP 时，设置的允许 IP 和禁止 IP 需要用双引号括起来，中间用逗号隔开，如"192.168.0.29","192.168.0.30"；也可以利用*或 CIDR 格式来设置网段，如"192.168.0.*"或"192.168.0.0/16"；需要注意，*和 CIDR 格式不能在同一 IP 中一起设置。

用户时间段设置项包括 ALLOW_DATETIME（允许的时间段）和 NOT_ALLOW_DATETIME（不允许的时间段），具体的限制规则如下表所示:

表 9.3 用户时间段限制规则

时间段限制项		集合状态			
时间段集合	不允许的时间段	空	N	空	N
	允许的时间段	Y	空	空	Y
时间段限制结果		只允许属于集合 Y 的时间段	只允许属于集合 N 以外的所有时间段	允许所有时间段，即无任何时间段限制	只允许属于 Y-N （差集）集合的时间段

允许时间段和禁止时间段用于控制此登录是否可以在某个时间段访问数据库，其中禁止时间段优先。设置的时间段中的日期和时间要分别用双引号括起来。在设置时间段时，有两种方式：

- 具体时间段，如 2023 年 1 月 1 日 8：30 至 2023 年 2 月 1 日 17：00；
- 规则时间段，如每周一 8：30 至每周五 17：00。

例 1 创建对失败登录次数进行控制的用户，如果用户失败的登录次数达到 3 次，这个用户账号将被锁定。

```
CREATE USER USER1 IDENTIFIED BY "USER1_psd"
LIMIT FAILED_LOGIN_ATTEMPTS 3, PASSWORD_LOCK_TIME 5;
```

例 2 创建对修改口令进行限制的用户，要求用户在 30 天内必须把口令修改过 5 次后，才能使用过去用过的口令。

```
CREATE USER USER2 IDENTIFIED BY "USER2_psd"
LIMIT PASSWORD_REUSE_TIME 30, PASSWORD_REUSE_MAX 5;
```

例 3 创建对允许 IP 和允许时间段进行限制的用户，要求用户在 192.168.0.*网段进行登录，且访问时间必须为工作时段。

```
CREATE USER USER3 IDENTIFIED BY "USER3_psd"
ALLOW_IP "192.168.0.*" ALLOW_DATETIME MON "8:30:00" TO FRI "17:30:00";
```

例 4 创建对允许 IP 和允许时间段进行限制的用户，要求用户在 192.168.0.29 或 192.168.0.30 IP 进行登录，且访问时间段为 2023 年 1 月 1 日 8：30 至 2023 年 12 月 31 日 17：30。

```
CREATE USER USER4 IDENTIFIED BY "USER4_psd"
ALLOW_IP "192.168.0.29", "192.168.0.30" ALLOW_DATETIME  "2023-1-1" "8:30:30" to "2023-12-31"
"17:30:00";
```

10 客体重用

在普通的环境下，数据库客体（主要指数据库对象、数据文件、缓存区）回收后不做处理，直接分配给新来的请求，但是有些窃密者会利用这一点编写特殊的非法进程通过数据库管理系统的内存泄露来获取数据库系统的信息。

DM 提供了客体重用安全功能。为防止非法进程利用数据库客体的内存泄露来攻击数据库，DM 主要从内存和文件两个方面进行了处理：

- ✓ 内存重用：DM 从系统分配内存及释放内存时均对内存内容进行清零，以保证不利用内存中前一进程所残留内容，且不泄漏 DM 的内容给其他进程。
- ✓ 文件重用：DM 在系统生成、扩展及删除文件时，对文件内容也进行了清零。

DM 提供了 INI 参数 ENABLE_OBJ_REUSE 用来控制是否启用客体重用功能，将该参数置为 1 表示启用客体重用，0 表示不启用，缺省为 0。

系统管理员可通过查询 V$PARAMETER 动态视图查询 ENABLE_OBJ_REUSE 的当前值。

```
SELECT * FROM V$PARAMETER WHERE NAME='ENABLE_OBJ_REUSE';
```

ENABLE_OBJ_REUSE 为静态参数，管理员可通过使用 DM 系统过程 SP_SET_PARA_VALUE 来修改该参数值，也可以使用 DM 控制台 Console 工具对该参数进行配置，不过配置后都需要重新启动 DM 数据库服务器才能生效。

11 登录用户名密码增强加密

非通信加密的情况下，为了实现对登录用户名、密码的安全管理，达梦对登录消息中的用户名密码进行了增强加、解密处理。

具体实现分为四步：一是通过 dmkey 工具生成公钥文件和私钥文件。二是配置服务器参数，使用私钥对客户端发送过来的登录名和密码进行解密。三是配置客户端参数，使用公钥文件对登录用户名和密码进行加密。四，启动数据库服务器。

11.1 dmkey 工具的使用

dmkey 是一个 RSA 非对称加解密的公钥文件、私钥文件生成工具。公钥文件和私钥文件分别用于对登录消息中的用户名、密码进行加密和解密。

11.1.1 启动 dmkey

安装好 DM 数据库管理系统后，在安装目录的“bin”子目录下可找到 dmkey 执行文件。

启动操作系统的命令行窗口，进入“dmkey”所在目录，可以准备启动 dmkey 工具了。

dmkey 的使用需要指定必要的参数。为 dmkey 指定参数的格式为：

```
dmkey [ keyword=value[ keyword=value ...] ]
```

常见的使用搭配有两种：

一，生成公钥和私钥。

```
dmkey.exe PATH=path PUBKEY=pubkey
```

PATH：指定生成公钥和私钥文件的目标目录（缺省为 dmkey.exe 所在的目录）；

PUBKEY：指定生成公钥的文件名（缺省为 dm_login.pubkey），私钥为固定文件名 dm_login.prikey。公钥文件为文本方式存储，私钥文件则为二进制存储。

例如：

```
>dmkey PATH=D:\aa pubkey=test.pubkey
DMKEY V8
generate public key file D:\aa\test.pubkey success!
generate private key file D:\aa\dm_login.prikey success!
```

二，显示私钥的相关信息（文件版本号、私钥 Guid、私钥的生成时间、dmkey 工具的版本号）。

该信息主要用于校验公钥、私钥是否匹配。同一套公钥文件、私钥文件的 Guid 一样的。文件版本号只有私钥文件有，公钥文件没有。

```
dmkey.exe PRIKEY=prikey
```

PRIKEY：私钥文件 dm_login.prikey 所在路径。

例如：

```
>dmkey PRIKEY=D:\aa\dm_login.prikey
DMKEY V8

File version          : 0
Guid                  : 1C2CF0AAC51A4B7F507790C6992ED187
Generation time       : 2023-08-15 17:00:29
Generation tool version: V8.1.3.26
```

11.1.2 查看 dmkey 参数

dmkey 使用较为灵活，参数较多，用户可以使用“dmkey help”查看 dmkey 版本信息以及各参数的简单信息。

```
./dmkey HELP
dmkey V8
version: 03134284389-20250411-268655-10000
书写格式: ./dmkey KEYWORD=<value> {KEYWORD=<value>}
```

```
        其中{}表示大括号内的关键词可为 0~N 个，N 为正整数，{}本身无需出现在语句中

使用示例: ./dmkey path=/opt/dmdbms/data/DAMENG pubkey=dm_login.pubkey

使用示例: ./dmkey prikey=/opt/dmdbms/data/DAMENG/dm_login.prikey

------------------------------------------------------------------------------

关键词（KEYWORD）        说明                          缺省值                是否必选项

------------------------------------------------------------------------------

PATH                  生成公钥和私钥文件的目标目录       无                    否

PUBKEY                要生成的公钥文件名                 dm_login.pubkey       否

PRIKEY                要打印信息的私钥文件路径名         无                    否

HELP                  打印帮助信息                       无                    否

------------------------------------------------------------------------------

注意:指定 PRIKEY 时仅打印所指私钥文件的相关信息，

      否则生成公钥文件和名为 dm_login.prikey 的私钥文件
```

11.2 服务器端配置

首先，将私钥文件拷贝到 ini 参数 SYSTEM_PATH 指定的目录下。

其次，配置 INI 参数 FORCE_CERTIFICATE_ENCRYPTION=1，即非加密通讯的情况下，开启用户名密码强制证书加密。该参数有效值为 0、1，缺省值为 0，运行中无法更改。

该项配置成功后，启动数据库服务器时，服务器会对登录的用户名密码强制使用 SYSTEM_PATH 目录下的私钥文件进行解密。

11.3 客户端配置

客户端配置有两种方式：一种是通过 dm_svc.conf 文件进行配置，供所有用户使用；另一种是通过 DPI 进行配置，供 DPI 编程用户使用。使用 DPI 编程的用户可以使用第一种方式设置，也可以使用第二种方式设置，当两种方式不一致时，第二种方式优先。

11.3.1 dm_svc.conf

使用配置项 LOGIN_CERTIFICATE 指定登录加密用户名密码公钥所在的路径。该配置项可全局配置也可在某个服务名下配置，一旦配置即认为开启了客户端的证书加密用户名密码模式。

11.3.2 DPI

指定 con 上属性 DSQL_ATTR_LOGIN_CERTIFICATE，用于指定加密登录用户名密码的公钥所在的路径。该属性和 dm_svc.conf 中配置项 LOGIN_CERTIFICATE 功能一样。两者不一致时，DSQL_ATTR_LOGIN_CERTIFICATE 设置优先。

11.4 启动数据库服务器

全部配置完成后，启动数据库服务器。服务器启动时自动加载 SYSTEM_PATH 目录下的私钥文件，对登录的用户名密码使用私钥文件 dm_login.prikey 进行解密。

11.5 应用实例

下面通过一个具体的实例，展示如何配置登录用户名和密码的增强加、解密功能。

第一步，通过 dmkey 生成公钥文件、私钥文件。

```
>dmkey PATH=D:\DAMENG pubkey=test.pubkey
```

```
DMKEY V8
generate public key file D:\DAMENG\test.pubkey success!
generate private key file D:\DAMENG\dm_login.prikey success!
```

第二步，首先将私钥文件 dm_login.prikey 拷贝到数据库 SYSTEM PATH 目录下，然后配置服务器 ini 参数 FORCE_CERTIFICATE_ENCRYPTION=1。

第三步，配置 dm_svc.conf 文件的 LOGIN_CERTIFICATE 配置项，指定公钥文件所在位置。dm_svc.conf 文件内容如下：

```
TIME_ZONE=(480)
LANGUAGE=(cn)
LOGIN_CERTIFICATE=(D:\DAMENG\test.pubkey)
```

第四步，启动数据库服务器。

至此，配置完毕。

12 登录用户名密码外部存储

为了避免用户登录数据库时直接接触数据库登录密码，DM 提供了一种登录用户名密码外部存储方式，系统管理员可将用户名和密码存储在外部加密的密码文件（wallet 文件）中，用户登录数据库时只需提供配置好的服务名便可成功登录数据库。

具体实现分为两步：一是创建 wallet 文件，用于存储服务名、用户名和登录密码。二是配置 dm_svc.conf 文件，设置 wallet 文件路径以及服务名对应的数据库连接地址（IP 和 PORT）。配置完成后用户便可直接通过服务名成功登录数据库。

以上数据库登录方式存在以下限制：

- LOGIN 命令不支持利用 wallet 文件登录数据库。
- JDBC 支持利用 wallet 文件登录数据库。JDBC 支持使用的加密算法包括 DES_ECB、AES128_ECB、AES128_CBC、AES128_CFB、AES128_OFB、AES192_ECB、AES192_CBC 、 AES192_CFB 、 AES192_OFB 、 AES256_ECB 、 AES256_CBC 、 AES256_CFB 和 AES256_OFB，即创建 wallet 文件时-cipherName 参数需要指定为以上加密算法；JDBC 支持使用的杂凑算法为 SHA512，即创建 wallet 文件时-hashName 参数缺省或指定为 SHA512。

DM 提供了一个数据库密码管理工具 dmmkstore，系统管理员可通过 dmmkstore 工具创建、访问或修改 wallet 文件。

12.1 dmmkstore 工具的使用

dmmkstore 是 DM 数据库密码管理工具。系统管理员可以通过 dmmkstore 工具创建、访问或修改 wallet 文件，wallet 文件中存储有数据库登录信息。支持跨平台或跨编码下发 wallet 文件。

数据库登录信息在 wallet 文件中以凭据（credential）的格式存储。每个凭据包含三个条目（entry）：服务名、用户名、密码。系统管理员可以利用 dmmkstore 工具增加、删除或修改 wallet 文件中的凭据。

12.1.1 启动 dmmkstore

安装好 DM 数据库管理系统后，在安装目录的“bin”子目录下可找到 dmmkstore 执行文件。

启动操作系统的命令行窗口，进入 dmmkstore 所在目录，可以准备启动 dmmkstore 工具了。

dmmkstore 的使用需要指定必要的参数。为 dmmkstore 指定参数的格式为：

```
dmmkstore -wrl <wallet_location> {keyword [<value>]}
或
dmmkstore -help
```

-wrl：指定 wallet 文件路径

keyword：参数关键字。关键字不区分大小写，多个参数之间使用空格间隔

<value>：参数取值。参数关键字与参数取值之间使用空格间隔

-help：显示帮助信息

例如：

在/home/test/dmdbms/wallet 目录下创建一个 wallet 文件，文件密码为 Wallet_123。

```
./dmmkstore -wrl /home/test/dmdbms/wallet -create

输入口令: Wallet_123

再次输入口令: Wallet_123
```

如果创建成功，则/home/test/dmdbms/wallet 目录下会生成一个加密的 dmwallet.prikey 文件。

在 wallet 文件中创建凭据:

```
./dmmkstore -wrl /home/test/dmdbms/wallet -createCredential dm_user01 USER01
USER01_psd

输入 Wallet 口令: Wallet_123

Create credential DM.security.client.connect_string1
```

查看 wallet 文件中的凭据:

```
./dmmkstore -wrl /home/test/dmdbms/wallet -listCredential

输入 Wallet 口令: Wallet_123

List credential (index: connect_string username)

1: dm_user01 USER01
```

12.1.2 查看 dmmkstore 参数

dmmkstore 使用较为灵活，参数较多。用户可使用"-help"命令快速查看 dmmkstore 版本信息以及各参数信息。

```
./dmmkstore -help
DM Secret Store Tool: V8


version: 05134284311-20240822-240257-10000
```

```
dmmkstore [-wrl wrl] [-create] [-list] [-viewEntry alias] [-modifyEntry alias secret] [-createCredential connect_string username password] [-listCredential] [-modifyCredential connect_string username password] [-deleteCredential connect_string] [-cipherName name] [-hashName name] [-help]
```

12.1.3 dmmkstore 参数详解

下面将详细介绍 dmmkstore 工具各参数的功能以及使用方法。

12.1.3.1 -wrl

指定 wallet 文件路径。

语法如下：

```
dmmkstore -wrl <wallet_location>
```

<wallet_location>：wallet 文件路径

该参数单独使用时无任何实际效果，必须搭配 dmmkstore 工具的其他参数（除-help 参数）一起使用。

当-wrl 参数与其他参数一起使用时，-wrl 参数必须作为第一个参数，否则报错。

12.1.3.2 -create

创建 wallet 文件。

语法如下：

```
dmmkstore -wrl <wallet_location> -create
```

创建 wallet 文件时，需要指定 wallet 文件的密码，该密码长度范围为 9~48，并且必须同时包含大写字母、小写字母、数字以及特殊字符。

创建成功后将在<wallet_location>指定路径下生成一个加密的 dmwallet.prikey 文件。若指定路径下已存在 dmwallet.prikey 文件，则报错。

例 在/home/test/dmdbms/wallet 目录下创建一个 wallet 文件，文件密码为 Wallet_123。

```
./dmmkstore -wrl /home/test/dmdbms/wallet -create

输入口令: Wallet_123

再次输入口令: Wallet_123
```

12.1.3.3 -createCredential

在 wallet 文件中创建凭据。

语法如下：

```
dmmkstore -wrl <wallet_location> -createCredential <server_name> <username> <password>
```

<server_name>：服务名。必须与 dm_svc.conf 文件中配置的服务名相匹配

<username>：用户名

<password>：密码

若 wallet 文件中已存在服务名相同的凭据，则报错。

wallet 文件最大为 64M，若创建凭据时 wallet 文件大小超过 64M，则报错。

-createCredential 在没有输入密码时，支持交互式输入。其使用限制如下：

1．暂时考虑命令中只包含一个-createCredential/-modifyCredential 的情况，不考虑后面的命令错误当口令使用的情况；

2．暂时只考虑没有口令的情况。


注意：

由于修改 wallet 文件过程中可能发生错误导致 wallet 文件受到损坏，因此修改 wallet 文件之前系统会自动在 wallet 文件路径下生成一个备

份文件 dmwallet.prikey.bak，若 wallet 文件修改失败，用户可以利用备份文件手动恢复 wallet 文件；若 wallet 文件修改成功，则系统自动删除备份文件。

例 在 wallet 文件中创建凭据，服务名为 dm_user01，用户名为 USER01，登录密码为 USER01_psd。

```
./dmmkstore -wrl /home/test/dmdbms/wallet -createCredential dm_user01 USER01 USER01_psd
```

12.1.3.4 -listCredential

显示 wallet 文件中的凭据。

语法如下：

```
dmmkstore -wrl <wallet_location> -listCredential
```

执行结果中会显示凭据的序号、服务名、用户名。

例 查看/home/test/dmdbms/wallet 路径下 wallet 文件中的凭据。

```
./dmmkstore -wrl /home/test/dmdbms/wallet -listCredential
输入 Wallet 口令: Wallet_123
List credential（index: connect_string username）
1: dm_user01 USER01
```

12.1.3.5 -modifyCredential

修改 wallet 文件中指定凭据的用户名和密码。

语法如下：

```
dmmkstore -wrl <wallet_location> -modifyCredential <server_name> <username> <password>
```

<server_name>：待修改凭据的服务名

<username>：修改后的用户名

<password>：修改后的密码

若 wallet 文件中不存在服务名为<server_name>的凭据，则报错。

-modifyCredential 在没有输入密码时，支持交互式输入。其使用限制如下：

1．暂时考虑命令中只包含一个-createCredential/-modifyCredential 的情况，不考虑后面的命令错误当口令使用的情况；

2．暂时只考虑没有口令的情况。

例 将 wallet 文件中服务名为 dm_user01 的凭据的用户名改为 SYSDBA，密码改为 DMdba_123。

```
./dmmkstore -wrl /home/test/dmdbms/wallet -modifyCredential dm_user01 SYSDBA
DMdba_123
输入 Wallet 口令: Wallet_123
Modify credential
Modify 1
```

12.1.3.6 -deleteCredential

删除 wallet 文件中指定的凭据。

语法如下：

```
dmmkstore -wrl <wallet_location> -deleteCredential <server_name>
```

<server_name>：待删除凭据的服务名

若 wallet 文件中不存在服务名为<server_name>的凭据，则报错。

例 删除 wallet 文件中服务名为 dm_user01 的凭据。

```
./dmmkstore -wrl /home/test/dmdbms/wallet -deleteCredential dm_user01
输入 Wallet 口令: Wallet_123
```

```
Delete credential

Delete 1
```

12.1.3.7 -list

显示 wallet 文件中所有凭据条目的别名。

语法如下：

```
dmmkstore -wrl <wallet_location> -list
```

凭据中各条目的别名如下表所示：

表 12.1 凭据中各条目的别名

条目	别名
服务名	DM.security.client.connect_string%d
用户名	DM.security.client.username%d
密码	DM.security.client.password%d

其中，%d 表示该条目所在凭据的序号。

例 查看 wallet 文件中的所有凭据。

```
./dmmkstore -wrl /home/test/dmdbms/wallet -listCredential

输入 Wallet 口令: Wallet_123

List credential (index: connect_string username)

1: dm_user01 USER01

2: dm_admin SYSDBA
```

可以看到，此时 wallet 文件中共有两条凭据，服务名分别为 dm_user01 和 dm_admin，凭据序号分别为 1 和 2。

查看 wallet 文件中所有凭据条目的别名。

```
./dmmkstore -wrl /home/test/dmdbms/wallet -list

输入 Wallet 口令: Wallet_123

DM 密钥存储条目:
```

```
DM.security.client.connect_string1
DM.security.client.connect_string2
DM.security.client.password1
DM.security.client.password2
DM.security.client.username1
DM.security.client.username2
```

12.1.3.8 -viewEntry

显示凭据中指定条目的值。

语法如下：

```
dmmkstore -wrl <wallet_location> -viewEntry <alias>
```

<alias>：指定条目的别名。条目别名不区分大小写，关于条目别名的详细介绍请参考[12.1.3.7 -list](#)。

若指定的条目为密码，则将以明文形式显示密码。

若指定的条目不存在，则报错。

例 查看别名为 DM.security.client.password1 的条目的值。

```
./dmmkstore -wrl /home/test/dmdbms/wallet -viewEntry DM.security.client.password1
输入 Wallet 口令: Wallet_123
DM.security.client.password1 = USER01_psd
```

12.1.3.9 -modifyEntry

修改凭据中指定条目的值。

语法如下：

```
dmmkstore -wrl <wallet_location> -modifyEntry <alias> <value>
```

<alias>：指定条目的别名。条目别名不区分大小写，关于条目别名的详细介绍请参考 12.1.3.7 -list

<value>：修改后条目的值

若指定的条目不存在，则报错。

例如：

将别名为 DM.security.client.password1 的条目的值修改为“USER01_newpsd”。

```
./dmmkstore -wrl /home/test/dmdbms/wallet -modifyEntry DM.security.client.password1
USER01_newpsd
输入 Wallet 口令: Wallet_123
```

查看别名为 DM.security.client.password1 的条目的值。

```
./dmmkstore -wrl /home/test/dmdbms/wallet -viewEntry DM.security.client.password1
输入 Wallet 口令: Wallet_123
DM.security.client.password1 = USER01_newpsd
```

12.1.3.10 -cipherName

设置加密算法。

语法如下：

```
dmmkstore -wrl <wallet_location> -cipherName <cipher_name> -create
```

<cipher_name>：加密算法名称，默认使用 DESEDE_CFB 加密算法

参数-cipherName 和-create 必须一起使用，且-cipherName 必须位于-create 的前面。

若-cipherName 参数指定的不是加密算法，或指定的加密算法名称中包含 NOPAD 或 EXTKEY，则报错。

例 指定使用 AES256_CFB 加密算法创建 wallet 文件。

```
./dmmkstore -wrl /home/test/dmdbms/wallet -cipherName AES256_CFB -create

输入口令: Wallet_123

再次输入口令: Wallet_123
```

12.1.3.11 -hashName

设置杂凑算法（哈希算法）。

语法如下：

```
dmmkstore -wrl <wallet_location> -hashName <hash_name> -create
```

<hash_name>：杂凑算法名称，默认使用 SHA512 杂凑算法

参数-hashName 和-create 必须一起使用，且-hashName 必须位于-create 的前面。

若-hashName 参数指定的不是杂凑算法，则报错。

例 指定使用 SHA512 杂凑算法创建 wallet 文件。

```
./dmmkstore -wrl /home/test/dmdbms/wallet -hashName SHA512 -create

输入口令: Wallet_123

再次输入口令: Wallet_123
```

12.1.3.12 -help

显示帮助信息。

语法如下：

```
dmmkstore -help
```

12.2 dm_svc.conf 文件配置

使用全局配置项 WALLET_LOCATION 指定 wallet 文件所在路径，并配置服务名对应的数据库连接地址（IP 和 PORT）。仅当全局配置项 WALLET_LOCATION 非空时，才可通过 wallet 文件登录数据库。

12.3 应用实例

下面列出一个完整的 dmmkstore 工具应用实例以供参考。

1. 用户准备

数据库中新建用户 user01 和 user02。

```
CREATE USER USER01 IDENTIFIED BY "USER01_psd";
CREATE USER USER02 IDENTIFIED BY "USER02_psd";
COMMIT;
```

2. 配置 dm_svc.conf 文件

dm_svc.conf 文件中的配置内容如下：

```
WALLET_LOCATION=(/home/test/dmdbms/wallet)
dm_user01=(192.168.100.163:5254)
dm_user02=(192.168.100.163:5254)
```

3. 创建 wallet 文件。

在 /home/test/dmdbms/wallet 路径下创建一个 wallet 文件，文件密码为 Wallet_123。

```
./dmmkstore -wrl /home/test/dmdbms/wallet -create
输入口令: Wallet_123
再次输入口令: Wallet_123
```

4. 创建凭据

在 wallet 文件中创建两条凭据，服务名分别为 dm_user01 和 dm_user02。

```
./dmmkstore -wrl /home/test/dmdbms/wallet -createCredential dm_user01 USER01
USER01_psd
输入 Wallet 口令: Wallet_123
Create credential DM.security.client.connect_string1
./dmmkstore -wrl /home/test/dmdbms/wallet -createCredential dm_user02 USER02
USER02_psd
输入 Wallet 口令: Wallet_123
Create credential DM.security.client.connect_string2
```

5． 查看凭据

查看 wallet 文件中的凭据。

```
./dmmkstore -wrl /home/test/dmdbms/wallet -listCredential
输入 Wallet 口令: Wallet_123
List credential (index: connect_string username)
1: dm_user01 USER01
2: dm_user02 USER02
```

6． 使用客户端工具连接数据库

disql 客户端工具通过 dm_user01 服务名登录数据库。

```
./disql /@dm_user01
服务器[192.168.100.163:5254]:处于普通打开状态
登录使用时间：9.232(ms)
disql V8
SQL> SELECT USER();

行号     USER()
---------- ------
1        USER01
```

关于各客户端工具登录数据库方法的详细介绍请参考各客户端工具使用手册。

12.4 dmmkstore 错误码汇编

运行 DM 数据库密码管理工具 dmmkstore 时，可能会就使用过程中的一些错误进行报错提示。

表 12.2 dmmkstore 错误码

代码	解释
-86500	未指定 wallet 文件的位置
-86501	命令无效
-86502	命令中缺少参数
-86503	无法创建目录
-86504	系统找不到指定的路径
-86505	系统指定的路径过长
-86506	系统找不到指定的文件
-86507	系统指定路径下已存在 wallet 文件
-86508	口令无效
-86509	口令不匹配
-86513	内存空间不足
-86514	写文件失败
-86515	无法创建文件
-86516	无法打开文件
-86517	无法删除文件
-86518	无法拷贝文件
-86519	无法截断文件
-86520	加密模块加载失败
-86521	生成杂凑值失败
-86522	杂凑值校验失败

-86523	加密失败
-86524	解密失败
-86525	命令中参数值无效
-86526	缺少必要命令
-86527	命令重复
-86528	无法关闭文件
-86529	文件校验失败
-86530	需要检查加密模块
-86531	wallet 文件验证失败
-86532	凭据已存在
-86533	凭据不存在
-86534	-cipherName 和-hashName 命令只能在-create 命令之前指定
-86535	wallet 文件最大为 64M
-86537	指定的别名不存在
-86538	转码失败

附录 1 “三权分立”预设角色权限列表

说明：每种角色仅对同类型的数据库对象拥有相应权限。如 AUDITOR 类型的预设角色拥有创建用户权限时仅能创建审计用户，DBA 类型的预设角色拥有查询表记录权限时仅能查询非安全且非审计相关的表记录。

预设角色	预设数据库权限	说明
	ALTER DATABASE	修改数据库
	RESTORE DATABASE	还原数据库
	CREATE USER	创建用户
	ALTER USER	修改用户
	DROP USER	删除用户
	CREATE ROLE	创建角色
	CREATE SCHEMA	创建模式
	CREATE TABLE	创建表
	CREATE VIEW	创建视图
	CREATE PROCEDURE	创建存储过程
	CREATE SEQUENCE	创建序列
	CREATE TRIGGER	创建触发器
	CREATE INDEX	创建索引
	CREATE CONTEXT INDEX	创建全文索引
	BACKUP DATABASE	备份数据库
	CREATE LINK	创建外部链接

CREATE PACKAGE	创建包规范
CREATE SYNONYM	创建同义词
CREATE PUBLIC SYNONYM	创建全局同义词
DROP ROLE	删除角色
ADMIN ANY ROLE	转授任意角色
ADMIN ANY DATABASE PRIVILEGE	转授任意数据库权限
GRANT ANY OBJECT PRIVILEGE	授予任意对象权限
CREATE ANY SCHEMA	创建任意模式
DROP SCHEMA	删除模式
DROP ANY SCHEMA	删除任意模式
CREATE ANY TABLE	在任意模式下创建表
ALTER TABLE	修改表
ALTER ANY TABLE	修改任意表
DROP TABLE	删除表
DROP ANY TABLE	删除任意表
INSERT TABLE	在表中插入表记录
INSERT ANY TABLE	在任意表中插入表记录
UPDATE TABLE	更新表记录
UPDATE ANY	更新任意表的记录

TABLE	
DELETE TABLE	删除表记录
DELETE ANY TABLE	删除任意表记录
SELECT TABLE	查询表记录
SELECT ANY TABLE	查询任意表记录
REFERENCES TABLE	引用表
REFERENCES ANY TABLE	引用任意表
DUMP TABLE	导出表
DUMP ANY TABLE	导出任意表
GRANT TABLE	向其他用户进行表上权限的授权
GRANT ANY TABLE	向其他用户进行任意表上权限的授权
FLASHBACK TABLE	闪回表
FLASHBACK ANY TABLE	闪回任意表
COMMENT ANY TABLE	为任意表、视图或列创建注释信息
CREATE ANY VIEW	创建任意视图
ALTER VIEW	修改视图
ALTER ANY VIEW	修改任意视图
DROP VIEW	删除视图
DROP ANY VIEW	删除任意视图
INSERT VIEW	向视图插入记录
INSERT ANY VIEW	向任意视图插入记录

UPDATE VIEW	更新视图
UPDATE ANY VIEW	更新任意视图
DELETE VIEW	删除视图的记录
DELETE ANY VIEW	删除任意视图中的记录
SELECT VIEW	查询视图
SELECT ANY VIEW	查询任意视图
GRANT VIEW	向其他用户进行视图上权限的授权
GRANT ANY VIEW	向其他用户进行任意视图上权限的授权
CREATE ANY PROCEDURE	创建任意存储过程
DROP PROCEDURE	删除存储过程
DROP ANY PROCEDURE	删除任意存储过程
EXECUTE PROCEDURE	执行存储过程
EXECUTE ANY PROCEDURE	执行任意存储过程
GRANT PROCEDURE	向其他用户进行存储过程上权限的授权
GRANT ANY PROCEDURE	向其他用户进行任意存储过程上权限的授权
CREATE ANY SEQUENCE	创建任意序列
ALTER SEQUENCE	修改序列
ALTER ANY SEQUENCE	修改任意序列
DROP SEQUENCE	删除序列
DROP ANY	删除任意序列

SEQUENCE	
SELECT SEQUENCE	查询序列
SELECT ANY SEQUENCE	查询任意序列
GRANT SEQUENCE	向其他用户进行序列上权限的授权
GRANT ANY SEQUENCE	向其他用户进行任意序列上权限的授权
CREATE ANY TRIGGER	创建任意触发器
ALTER TRIGGER	修改触发器
ALTER ANY TRIGGER	修改任意触发器
DROP TRIGGER	删除触发器
DROP ANY TRIGGER	删除任意触发器
CREATE ANY INDEX	创建任意索引
ALTER INDEX	修改索引
ALTER ANY INDEX	修改任意索引
DROP INDEX	删除索引
DROP ANY INDEX	删除任意索引
CREATE ANY CONTEXT INDEX	创建任意全文索引
ALTER CONTEXT INDEX	修改全文索引
ALTER ANY CONTEXT INDEX	修改任意全文索引
DROP CONTEXT	删除全文索引

INDEX	
DROP ANY CONTEXT INDEX	删除任意全文索引
CREATE ANY PACKAGE	创建任意包规范
DROP PACKAGE	删除包规范
DROP ANY PACKAGE	删除任意包规范
EXECUTE PACKAGE	执行包规范
EXECUTE ANY PACKAGE	执行任意包规范
GRANT PACKAGE	向其他用户进行包规范上权限的授权
GRANT ANY PACKAGE	向其他用户进行任意包规范上权限的授权
CREATE ANY LINK	创建任意外部链接
CREATE PUBLIC LINK	创建公共外部链接
DROP LINK	删除外部链接
DROP ANY LINK	删除任意外部链接
DROP PUBLIC LINK	删除公共外部链接
CREATE ANY SYNONYM	创建任意非全局同义词
DROP SYNONYM	删除同义词
DROP ANY SYNONYM	删除任意非全局同义词
DROP PUBLIC SYNONYM	删除全局同义词
ADMIN JOB	管理作业权限

CREATE TABLESPACE	创建表空间
ALTER TABLESPACE	修改表空间
DROP TABLESPACE	删除表空间
CREATE MATERIALIZED VIEW	创建物化视图
CREATE ANY MATERIALIZED VIEW	创建任意物化视图
DROP MATERIALIZED VIEW	删除物化视图
DROP ANY MATERIALIZED VIEW	删除任意物化视图
ALTER MATERIALIZED VIEW	修改物化视图
ALTER ANY MATERIALIZED VIEW	修改任意物化视图
SELECT MATERIALIZED VIEW	查询物化视图
SELECT ANY MATERIALIZED VIEW	查询任意物化视图
CREATE ANY	创建任意数据域

DOMAIN	
DROP DOMAIN	删除数据域
DROP ANY DOMAIN	删除任意数据域
CREATE DOMAIN	创建数据域
GRANT ANY DOMAIN	向其他用户进行任意数据域上权限的授权
GRANT DOMAIN	向其他用户进行数据域上权限的授权
USAGE ANY DOMAIN	使用任意数据域
USAGE DOMAIN	使用数据域
CREATE ANY CONTEXT	创建任意上下文
DROP ANY CONTEXT	删除任意上下文
GRANT ANY CONTEXT	向其他用户进行任意上下文上权限的授权
CREATE ANY DIRECTORY	创建任意目录
DROP ANY DIRECTORY	删除任意目录
CREATE PROFILE	创建 PROFILE
ALTER PROFILE	修改 PROFILE
DROP PROFILE	删除 PROFILE
CREATE PARTITION GROUP	创建分区组
CREATE ANY PARTITION GROUP	创建任意分区组

DROP ANY PARTITION GROUP	删除任意分区组
USAGE PARTITION GROUP	使用分区组
USAGE ANY PARTITION GROUP	使用任意分区组
CREATE OPERATOR	创建自定义运算符
CREATE ANY OPERATOR	创建任意自定义运算符
DROP OPERATOR	删除自定义运算符
DROP ANY OPERATOR	删除任意自定义运算符
SELECT ANY DICTIONARY	查询任意系统表或系统视图的记录
DROP PARTITION GROUP	删除分区组
CREATE LIBRARY	创建 library
ALTER LIBRARY	修改 library
EXECUTE LIBRARY	执行 library
DROP LIBRARY	删除 library
GRANT LIBRARY	向其他用户进行 library 上权限的授权
CREATE ANY LIBRARY	在任意模式下创建 library
ALTER ANY LIBRARY	在任意模式下修改 library
EXECUTE ANY	在任意模式下执行 library

	LIBRARY	
	DROP ANY LIBRARY	在任意模式下删除 library
	GRANT ANY LIBRARY	向其他用户进行任意 library 上权限的授权
	UNLIMITED TABLESPACE	用户对所有表空间均无配额限制
	SUBSCRIBE LOG	手动获取订阅归档日志
RESOURCE	CREATE SCHEMA	创建模式
	CREATE TABLE	创建表
	CREATE VIEW	创建视图
	CREATE PROCEDURE	创建存储过程
	CREATE SEQUENCE	创建序列
	CREATE TRIGGER	创建触发器
	CREATE INDEX	创建索引
	CREATE CONTEXT INDEX	创建全文索引
	CREATE LINK	创建外部链接
	CREATE PUBLIC LINK	创建公共外部链接
	CREATE PACKAGE	创建包规范
	CREATE SYNONYM	创建同义词
	CREATE PUBLIC SYNONYM	创建全局同义词
	INSERT TABLE	在表中插入表记录
	UPDATE TABLE	更新表记录

DELETE TABLE	删除表记录
SELECT TABLE	查询表记录
REFERENCES TABLE	引用表
DUMP TABLE	导出表
GRANT TABLE	向其他用户进行表上权限的授权
CREATE LIBRARY	创建 library
ALTER LIBRARY	修改 library
EXECUTE LIBRARY	执行 library
DROP LIBRARY	删除 library
GRANT LIBRARY	向其他用户进行 library 权限的授权
INSERT VIEW	向视图插入记录
UPDATE VIEW	更新视图
DELETE VIEW	删除视图的记录
SELECT VIEW	查询视图中的记录
GRANT VIEW	向其他用户进行视图上权限的授权
EXECUTE PROCEDURE	执行存储过程
GRANT PROCEDURE	向其他用户进行存储过程上权限的授权
SELECT SEQUENCE	查询序列
GRANT SEQUENCE	向其他用户进行序列上权限的授权
EXECUTE PACKAGE	执行包规范
GRANT PACKAGE	向其他用户进行包规范上权限的授权
CREATE MATERIALIZED VIEW	创建物化视图

SELECT MATERIALIZED VIEW	查询物化视图
CREATE DOMAIN	创建数据域
GRANT DOMAIN	向其他用户进行数据域上权限的授权
USAGE DOMAIN	使用数据域
CREATE OPERATOR	创建自定义运算符
FLASHBACK TABLE	闪回表
DROP TABLE	删除表
DROP VIEW	删除视图
DROP SCHEMA	删除模式
DROP PROCEDURE	删除存储过程
DROP PACKAGE	删除包规范
DROP SEQUENCE	删除序列
DROP LINK	删除外部链接
DROP CONTEXT INDEX	删除全文索引
DROP INDEX	删除索引
DROP TRIGGER	删除触发器
DROP DOMAIN	删除数据域
DROP PARTITION GROUP	删除分区组
DROP MATERIALIZED VIEW	删除物化视图
DROP OPERATOR	删除自定义运算符
DROP SYNONYM	删除同义词

	ALTER TABLE	修改表
	ALTER VIEW	修改视图
	ALTER MATERIALIZED VIEW	修改物化视图
	ALTER SEQUENCE	修改序列
	ALTER TRIGGER	修改触发器
	ALTER INDEX	修改索引
	CREATE PARTITION GROUP	创建分区组
	USAGE PARTITION GROUP	使用分区组
	ALTER CONTEXT INDEX	修改全文索引
PUBLIC	INSERT TABLE	在表中插入表记录
	UPDATE TABLE	更新表记录
	DELETE TABLE	删除表记录
	SELECT TABLE	查询表记录
	REFERENCES TABLE	引用表
	GRANT TABLE	向其他用户进行表上权限的授权
	INSERT VIEW	向视图插入记录
	UPDATE VIEW	更新视图
	DELETE VIEW	删除视图的记录
	SELECT VIEW	查询视图中的记录
	GRANT VIEW	向其他用户进行视图上权限的授权
	EXECUTE PROCEDURE	执行存储过程的权限

GRANT PROCEDURE	向其他用户进行存储过程上权限的授权
ALTER LIBRARY	修改 library
EXECUTE LIBRARY	执行 library
DROP LIBRARY	删除 library
GRANT LIBRARY	向其他用户进行 library 权限的授权
SELECT SEQUENCE	查询序列
GRANT SEQUENCE	向其他用户进行序列上权限的授权
EXECUTE PACKAGE	执行包规范
GRANT PACKAGE	向其他用户进行包规范上权限的授权
SELECT MATERIALIZED VIEW	查询物化视图
GRANT DOMAIN	向其他用户进行数据域上权限的授权
USAGE DOMAIN	使用数据域
DUMP TABLE	导出表
DROP TABLE	删除表
DROP VIEW	删除视图
DROP SCHEMA	删除模式
DROP PROCEDURE	删除存储过程
DROP PACKAGE	删除包规范
DROP SEQUENCE	删除序列
DROP LINK	删除外部链接
DROP CONTEXT INDEX	删除全文索引
DROP INDEX	删除索引

	DROP TRIGGER	删除触发器
	DROP DOMAIN	删除数据域
	DROP PARTITION GROUP	删除分区组
	DROP MATERIALIZED VIEW	删除物化视图
	DROP OPERATOR	删除自定义运算符
	DROP SYNONYM	删除同义词
	ALTER TABLE	修改表
	ALTER VIEW	修改视图
	ALTER MATERIALIZED VIEW	修改物化视图
	ALTER SEQUENCE	修改序列
	ALTER TRIGGER	修改触发器
	ALTER INDEX	修改索引
	ALTER CONTEXT INDEX	修改全文索引
DB_AUDIT_ADMIN	CREATE USER	创建用户
	ALTER USER	修改用户
	DROP USER	删除用户
	CREATE PROFILE	创建 PROFILE
	ALTER PROFILE	修改 PROFILE
	DROP PROFILE	删除 PROFILE
	AUDIT DATABASE	执行审计
DB_AUDIT_OPER	AUDIT DATABASE	执行审计
DB_AUDIT_PUBLIC	无	无
DB_POLICY_ADMIN	CREATE USER	创建用户

	ALTER USER	修改用户
	DROP USER	删除用户
	CREATE PROFILE	创建 PROFILE
	ALTER PROFILE	修改 PROFILE
	DROP PROFILE	删除 PROFILE
	LABEL DATABASE	强制访问控制
DB_POLICY_OPER	LABEL DATABASE	强制访问控制
DB_POLICY_PUBLIC	无	无
VTI	无	具有系统动态视图的查询权限。VTI 默认授权给 DBA 角色且可转授，不支持从 DBA 中回收 VTI。可以为 SYSDBA 授予和回收 VTI
SOI	无	具有非审计/安全系统表的查询权限。SOI 默认授权给 SYSDBA 用户且可转授，不支持从 SYSDBA 回收 SOI
SVI	无	具有基础 V 视图的查询权限。SVI 默认授权给 PUBLIC 且可转授，不支持从 PUBLIC 中回收 SVI。可以为 SYSDBA 授予和回收 SVI
DB_AUDIT_VTI	无	具有系统动态视图的查询权限，DB_AUDIT_VTI 默认授权给 DB_AUDIT_ADMIN 且可转授
DB_AUDIT_SOI	无	具有基础类系统表和与审计操作相关的系统表的查询权限
DB_AUDIT_SVI	无	具有基础 V 视图和审计 V 视图的查询权限。DB_AUDIT_SVI 默认授权给 DB_AUDIT_PUBLIC 且可转授
DB_POLICY_VTI	无	具有系统动态视图的查询权限，DB_POLICY_VTI 默认授权给

		DB_POLICY_ADMIN 且可转授
DB_POLICY_SOI	无	具有基础类系统表和与安全操作相关的系统表的查询权限
DB_POLICY_SVI	无	具有基础 V 视图和安全 V 视图的查询权限
SYS_ADMIN	无	系统预留角色，暂未启用
SELECT_CATALOG_ROLE	无	METADATA.GET_DDL 专用，查询相同类型其他用户的对象、或 SYSDBO 用户（包含创建的用户）下的对象、或 SYS 模式下的对象。此处的相同类型其他用户是指 SYSDBA 或 SYSDBA 创建的用户。SELECT_CATALOG_ROLE 默认授权给 DBA 角色且可转授
DB_AUDIT_SELECT_CATALOG_ROLE	无	METADATA.GET_DDL 专用，查询相同类型其他用户或不同类型用户的对象（包含 SYS 模式）。DB_AUDIT_SELECT_CATALOG_ROLE 默认授权给 DB_AUDIT_ADMIN 角色且可转授
DB_POLICY_SELECT_CATALOG_ROLE	无	METADATA.GET_DDL 专用，查询相同类型其他用户或不同类型用户的对象（包含 SYS 模式）。DB_POLICY_SELECT_CATALOG_ROLE 默认授权给 DB_POLICY_ADMIN 角色且可转授

附录 2 “四权分立”预设角色权限列表

说明：每种角色仅对同类型的数据库对象拥有相应权限。如 AUDITOR 类型的预设角色拥有创建用户权限时仅能创建审计用户，DBA 类型的预设角色拥有查询表记录权限时仅能查询非安全且非审计相关的表记录。

预设角色	预设数据库权限	说明
	ALTER DATABASE	修改数据库
	BACKUP DATABASE	备份数据库
	RESTORE DATABASE	还原数据库
	CREATE USER	创建用户
	ALTER USER	修改用户
	DROP USER	删除用户
	CREATE ROLE	创建角色
	DROP ROLE	删除角色
	ADMIN ANY ROLE	转授任意角色
	CREATE TABLESPACE	创建表空间
	ALTER TABLESPACE	修改表空间
	DROP TABLESPACE	删除表空间
	ADMIN JOB	管理作业权限
	CREATE PROFILE	创建 PROFILE

	ALTER PROFILE	修改 PROFILE
	DROP PROFILE	删除 PROFILE
	SUBSCRIBE LOG	手动获取订阅归档日志
RESOURCE	CREATE ROLE	创建角色
	DROP ROLE	删除角色
PUBLIC	无	无
DB_OBJECT_ADMIN	CREATE USER	创建用户
	ALTER USER	修改用户
	DROP USER	删除用户
	CREATE ROLE	创建角色
	DROP ROLE	删除角色
	ADMIN ANY ROLE	转授任意角色
	CREATE SCHEMA	创建模式
	CREATE TABLE	创建表
	INSERT TABLE	在表中插入表记录
	UPDATE TABLE	更新表记录
	DELETE TABLE	删除表记录
	SELECT TABLE	查询表记录
	REFERENCES TABLE	引用表
	DUMP TABLE	导出表
	GRANT TABLE	向其他用户进行表上权限的授权
	CREATE VIEW	创建视图
	INSERT VIEW	向视图插入记录
	UPDATE VIEW	更新视图
	DELETE VIEW	删除视图的记录

SELECT VIEW	查询视图中的记录
GRANT VIEW	向其他用户进行视图上权限的授权
CREATE DOMAIN	创建数据域
GRANT DOMAIN	向其他用户进行数据域上权限的授权
USAGE DOMAIN	使用数据域
CREATE PROCEDURE	创建存储过程
EXECUTE PROCEDURE	执行存储过程的权限
GRANT PROCEDURE	向其他用户进行存储过程上权限的授权
CREATE SEQUENCE	创建序列
SELECT SEQUENCE	查询序列
GRANT SEQUENCE	向其他用户进行序列上权限的授权
CREATE TRIGGER	创建触发器
CREATE INDEX	创建索引
CREATE CONTEXT INDEX	创建全文索引
CREATE PACKAGE	创建包规范
EXECUTE PACKAGE	执行包规范

GRANT PACKAGE	向其他用户进行包规范上权限的授权
CREATE SYNONYM	创建同义词
CREATE PUBLIC SYNONYM	创建全局同义词
DROP PUBLIC SYNONYM	删除全局同义词
CREATE LINK	创建外部链接
CREATE PUBLIC LINK	创建公共外部链接
CREATE ANY CONTEXT	创建任意上下文
DROP ANY CONTEXT	删除任意上下文
GRANT ANY CONTEXT	向其他用户进行任意上下文上权限的授权
COMMENT ANY TABLE	为任意表、视图或列创建注释信息
CREATE ANY DIRECTORY	创建任意目录
DROP ANY DIRECTORY	删除任意目录
CREATE PROFILE	创建 PROFILE
ALTER PROFILE	修改 PROFILE
DROP PROFILE	删除 PROFILE
CREATE OPERATOR	创建自定义运算符
DROP TABLE	删除表

DROP VIEW	删除视图
DROP SCHEMA	删除模式
DROP PROCEDURE	删除存储过程
DROP PACKAGE	删除包规范
DROP SEQUENCE	删除序列
DROP LINK	删除外部链接
DROP PUBLIC LINK	删除公共外部链接
DROP CONTEXT INDEX	删除全文索引
DROP INDEX	删除索引
DROP TRIGGER	删除触发器
DROP DOMAIN	删除数据域
DROP PARTITION GROUP	删除分区组
DROP MATERIALIZED VIEW	删除物化视图
DROP OPERATOR	删除自定义运算符
DROP SYNONYM	删除同义词
ALTER TABLE	修改表
ALTER VIEW	修改视图
ALTER MATERIALIZED	修改物化视图

VIEW	
ALTER SEQUENCE	修改序列
ALTER TRIGGER	修改触发器
ALTER INDEX	修改索引
ALTER CONTEXT INDEX	修改全文索引
CREATE MATERIALIZED VIEW	创建物化视图
SELECT MATERIALIZED VIEW	查询物化视图
CREATE PARTITION GROUP	创建分区组
USAGE PARTITION GROUP	使用分区组
DUMP ANY TABLE	导出任意表
FLASHBACK ANY TABLE	闪回任意表
FLASHBACK TABLE	闪回表
ADMIN JOB	管理作业权限
UNLIMITED TABLESPACE	用户对所有表空间均无配额限制
SUBSCRIBE LOG	手动获取订阅归档日志

DB_OBJECT_OPER	CREATE SCHEMA	创建模式
	CREATE TABLE	创建表
	INSERT TABLE	在表中插入表记录
	UPDATE TABLE	更新表记录
	DELETE TABLE	删除表记录
	SELECT TABLE	查询表记录
	REFERENCES TABLE	引用表
	DUMP TABLE	导出表
	GRANT TABLE	向其他用户进行表上权限的授权
	CREATE VIEW	创建视图
	INSERT VIEW	向视图插入记录
	UPDATE VIEW	更新视图
	DELETE VIEW	删除视图的记录
	SELECT VIEW	查询视图中的记录
	GRANT VIEW	向其他用户进行视图上权限的授权
	CREATE DOMAIN	创建数据域
	CREATE PROCEDURE	创建存储过程
	EXECUTE PROCEDURE	执行存储过程的权限
	GRANT PROCEDURE	向其他用户进行存储过程上权限的授权
	CREATE SEQUENCE	创建序列
	SELECT SEQUENCE	查询序列
	GRANT	向其他用户进行序列上权限的授权

SEQUENCE	
CREATE TRIGGER	创建触发器
CREATE INDEX	创建索引
CREATE CONTEXT INDEX	创建全文索引
CREATE PACKAGE	创建包规范
EXECUTE PACKAGE	执行包规范
GRANT PACKAGE	向其他用户进行包规范上权限的授权
CREATE SYNONYM	创建同义词
CREATE PUBLIC SYNONYM	创建全局同义词
CREATE LINK	创建外部链接
CREATE PUBLIC LINK	创建公共外部链接
CREATE OPERATOR	创建自定义运算符
DROP TABLE	删除表
DROP VIEW	删除视图
DROP SCHEMA	删除模式
DROP PROCEDURE	删除存储过程
DROP PACKAGE	删除包规范
DROP	删除序列

SEQUENCE	
DROP LINK	删除外部链接
DROP CONTEXT INDEX	删除全文索引
DROP INDEX	删除索引
DROP TRIGGER	删除触发器
DROP DOMAIN	删除数据域
DROP PARTITION GROUP	删除分区组
DROP MATERIALIZED VIEW	删除物化视图
DROP OPERATOR	删除自定义运算符
DROP SYNONYM	删除同义词
ALTER TABLE	修改表
ALTER VIEW	修改视图
ALTER MATERIALIZED VIEW	修改物化视图
ALTER SEQUENCE	修改序列
ALTER TRIGGER	修改触发器
ALTER INDEX	修改索引
ALTER CONTEXT INDEX	修改全文索引
CREATE	创建物化视图

	MATERIALIZED VIEW	
	SELECT MATERIALIZED VIEW	查询物化视图
	CREATE PARTITION GROUP	创建分区组
	USAGE PARTITION GROUP	使用分区组
	FLASHBACK TABLE	闪回表
	CREATE ROLE	创建角色
	DROP ROLE	删除角色
	CREATE LIBRARY	创建 library
	ALTER LIBRARY	修改 library
	EXECUTE LIBRARY	执行 library
	DROP LIBRARY	删除 library
	GRANT LIBRARY	向其他用户进行 library 权限的授权
	GRANT DOMAIN	向其他用户进行数据域上权限的授权
	USAGE DOMAIN	使用数据域
DB_OBJECT_PUBLIC	INSERT TABLE	在表中插入表记录
	UPDATE TABLE	更新表记录
	DELETE TABLE	删除表记录
	SELECT TABLE	查询表记录

REFERENCES TABLE	引用表
DUMP TABLE	导出表
GRANT TABLE	向其他用户进行表上权限的授权
INSERT VIEW	向视图插入记录
UPDATE VIEW	更新视图
DELETE VIEW	删除视图的记录
SELECT VIEW	查询视图中的记录
GRANT VIEW	向其他用户进行视图上权限的授权
EXECUTE PROCEDURE	执行存储过程的权限
GRANT PROCEDURE	向其他用户进行存储过程上权限的授权
SELECT SEQUENCE	查询序列
GRANT SEQUENCE	向其他用户进行序列上权限的授权
EXECUTE PACKAGE	执行包规范
GRANT PACKAGE	向其他用户进行包规范上权限的授权
DROP TABLE	删除表
DROP VIEW	删除视图
DROP SCHEMA	删除模式
DROP PROCEDURE	删除存储过程
DROP PACKAGE	删除包规范
DROP SEQUENCE	删除序列

DROP LINK	删除外部链接
DROP CONTEXT INDEX	删除全文索引
DROP INDEX	删除索引
DROP TRIGGER	删除触发器
DROP DOMAIN	删除数据域
DROP PARTITION GROUP	删除分区组
DROP MATERIALIZED VIEW	删除物化视图
DROP OPERATOR	删除自定义运算符
DROP SYNONYM	删除同义词
ALTER TABLE	修改表
ALTER VIEW	修改视图
ALTER MATERIALIZED VIEW	修改物化视图
ALTER SEQUENCE	修改序列
ALTER TRIGGER	修改触发器
ALTER INDEX	修改索引
SELECT MATERIALIZED VIEW	查询物化视图
GRANT DOMAIN	向其他用户进行数据域上权限的授权

	USAGE DOMAIN	使用数据域
	ALTER LIBRARY	修改 library
	EXECUTE LIBRARY	执行 library
	DROP LIBRARY	删除 library
	GRANT LIBRARY	向其他用户进行 library 权限的授权
	ALTER CONTEXT INDEX	修改全文索引
DB_AUDIT_ADMIN	CREATE USER	创建用户
	ALTER USER	修改用户
	DROP USER	删除用户
	AUDIT DATABASE	执行审计
	CREATE PROFILE	创建 PROFILE
	ALTER PROFILE	修改 PROFILE
	DROP PROFILE	删除 PROFILE
DB_AUDIT_OPER	AUDIT DATABASE	执行审计
DB_AUDIT_PUBLIC	无	无
DB_POLICY_ADMIN	CREATE USER	创建用户
	ALTER USER	修改用户
	DROP USER	删除用户
	LABEL DATABASE	强制访问控制
	CREATE PROFILE	创建 PROFILE
	ALTER PROFILE	修改 PROFILE

	DROP PROFILE	删除 PROFILE
DB_POLICY_OPER	LABEL DATABASE	强制访问控制
DB_POLICY_PUBLIC	无	无
VTI	无	具有系统动态视图的查询权限。VTI 默认授权给 DBA 角色且可转授，不支持从 DBA 中回收 VTI。可以为 SYSDBA 授予和回收 VTI
SOI	无	具有非审计/安全系统表的查询权限。SOI 默认授权给 SYSDBA 用户且可转授，不支持从 SYSDBA 回收 SOI
SVI	无	具有基础 V 视图的查询权限。SVI 默认授权给 PUBLIC 且可转授，不支持从 PUBLIC 中回收 SVI。可以为 SYSDBA 授予和回收 SVI
DB_AUDIT_VTI	无	具有系统动态视图的查询权限，DB_AUDIT_VTI 默认授权给 DB_AUDIT_ADMIN 且可转授
DB_AUDIT_SOI	无	具有基础类系统表和与审计操作相关的系统表的查询权限
DB_AUDIT_SVI	无	具有基础 V 视图和审计 V 视图的查询权限。DB_AUDIT_SVI 默认授权给 DB_AUDIT_PUBLIC 且可转授
DB_POLICY_VTI	无	具有系统动态视图的查询权限，DB_POLICY_VTI 默认授权给 DB_POLICY_ADMIN 且可转授
DB_POLICY_SOI	无	具有基础类系统表和与安全操作相关的系统表的查询权限
DB_POLICY_SVI	无	具有基础 V 视图和安全 V 视图的查询权限

DB_OBJECT_VTI	无	具有系统动态视图的查询权限，DB_OBJECT_VTI 默认授权给 DB_OBJECT_ADMIN 且可转授
DB_OBJECT_SOI	无	具有非审计/安全系统表的查询权限
DB_OBJECT_SVI	无	和 SVI 权限一样
SYS_ADMIN	无	系统内部角色，具有 CREATE ANY TRIGGER 和 DROP ANY TRIGGER 权限。四权分立时，由于没有 CREATE ANY TRIGGER 和 DROP ANY TRIGGER 权限，系统自动将 SYS_ADMIN 授权给 SYSDBA，只有具备 SYS_ADMIN 时 SYSDBA 才可以创建库级的事件触发器。SYS_ADMIN 既不可转授，也不可从 SYSDBA 回收
SELECT_CATALOG_ROLE	无	METADATA.GET_DDL 专用，查询相同类型其他用户的对象、或 SYSDBO 用户（包含创建的用户）下的对象、或 SYS 模式下的对象。此处的相同类型其他用户是指 SYSDBA 或 SYSDBA 创建的用户。SELECT_CATALOG_ROLE 默认授权给 DBA 角色且可转授
DB_AUDIT_SELECT_CATALOG_ROLE	无	METADATA.GET_DDL 专用，查询相同类型其他用户或不同类型用户的对象（包含 SYS 模式）。DB_AUDIT_SELECT_CATALOG_ROLE 默认授权给 DB_AUDIT_ADMIN 角色且可转授
DB_POLICY_SELECT_CATALOG_ROLE	无	METADATA.GET_DDL 专用，查询相同类型其他用户或不同类型用户的对象（包含

		SYS 模式）。DB_POLICY_SELECT_CATALOG_ROLE 默认授权给 DB_POLICY_ADMIN 角色且可转授
DB_OBJECT_SELECT_CATALOG_ROLE	无	METADATA.GET_DDL 专用，查询相同类型其他用户的对象、或 SYSDBA 用户（包含创建的用户）下的对象、或 SYS 模式下的对象。此处的相同类型其他用户是指 SYSDBO 或 SYSDBO 创建的用户。DB_OBJECT_SELECT_CATALOG_ROLE 默认授权给 DB_OBJECT _ADMIN 角色且可转授

DM 技术支持

如果您在安装或使用 DM 及其相应产品时出现了问题，请首先访问我们的 Web 站点 http://www.dameng.com/。在此站点我们收集整理了安装使用过程中一些常见问题的解决办法，相信会对您有所帮助。

您也可以通过以下途径与我们联系，我们的技术支持工程师会为您提供服务。

武汉达梦数据库股份有限公司

地址：武汉市东湖新技术开发区甲铺岭街 39 号达梦中国数据库产业基地

电话：(+86)027-87588000

传真：(+86)027-87588810

武汉达梦数据技术有限公司

地址：武汉市东湖新技术开发区甲铺岭街 39 号达梦中国数据库产业基地 13-14 层

电话：(+86)027-87588000

传真：(+86)027-87588810

北京达梦数据库技术有限公司

地址：北京市海淀区中关村南大街 2 号数码大厦 B 座 1003

电话：(+86)010-51727900

传真：(+86)010-51727983

上海达梦数据库有限公司

地址：上海市张江高科技园区亮秀路 112 号 A 座 401

电话：（+86）021-50111338

四川蜀天梦图数据科技有限公司

地址：成都市天府新区湖畔西路 99 号 B7 栋（天府英才中心）6 层

电话：（+86）028-61458838

达梦数据技术（江苏）有限公司

地址：苏州吴中经济开发区越溪街道吴中大道 1431 号苏州（太湖）软件产业园智慧谷园区 1601 室 03-04 号

电话：（+86）0512-65285955

传真：（+86）0512-65286955

重庆达梦大数据有限公司

地址：重庆市沙坪坝区重庆师范大学（沙坪坝校区）第二教学楼 1002

电话：（+86）023-68165682

达梦数据技术（深圳）有限公司

地址：深圳市龙岗区园山街道荷坳社区龙岗大道 8288 号大运软件小镇 17 栋 101

技术服务：

咨询热线：400-991-6599

技术支持：dmtech@dameng.com

商务咨询：400@dameng.com

手册版本

版本号：V8

发版日期：2026 二季度版-26876



武汉达梦数据库股份有限公司
官方网站：www.dameng.com

达梦数据版权所有，侵权必究©